

Paper 4 – Research Design and Evidence Base

Cross-provider cryptographic conformance for heterogeneous multiplatform SDKs

Version 0.6 – 2 September 2026 – PREEXPERIMENTAL DESIGN FREEZE

Contents

1 Purpose and working rule	5
2 Scientific gap and positioning	5
2.1 Neighbouring literature that does not eliminate the gap	5
3 Research framing	6
3.1 Research questions	6
3.2 Hypotheses	7
3.3 Refutation criteria (pre-registered)	7
4 Formal contract model	7
4.1 Six conformance relations	8
4.2 Applicability vs dependency	8
5 Shared clause taxonomy	8
5.1 Two-level coverage	9
6 Pre-registration and diagnostic machinery	9
6.1 Attribution rule	10
7 Experimental protocol	10
7.1 Baselines	10
7.2 Threats to validity already identified	10
8 Backends and feasibility	11
9 Candidate operation set and current status	11
10 HKDF-SHA-256 – closed pilot contract	12
10.1 HKDF pre-registered mutation classes	13

11 AES-GCM – frozen portable profile (C_{pre}^{GCM})	14
11.1 Frozen portable profile	15
11.2 Frozen Γ_0^{GCM}	18
11.3 Backend verification: Crypto++ (initial verification)	22
11.4 Backend verification: Crypto++ IV, AAD, and tag-length capability (confirmed against source)	23
11.5 Backend verification: Bouncy Castle (confirmed)	24
11.6 Backend verification: Bouncy Castle nonce, AAD, and tag-length capability (confirmed against source)	25
12 RSA-OAEP – normative fiche (RFC 8017 §7.1)	27
12.1 Backend verification: WebCrypto (initial inventory)	28
12.2 Backend verification: Crypto++ (complete)	29
12.3 Backend verification: Bouncy Castle (complete)	31
12.4 Three-way OAEP comparison (P_{OAEP}^{common} candidate observations, still not frozen) . .	33
12.5 Frozen E_{OAEP}^{SDK} error model	33
12.6 Frozen C_{pre}^{OAEP} fragment: OAEP digest coupling	35
12.7 Frozen C_{pre}^{OAEP} fragment: label policy	36
12.8 Frozen C_{pre}^{OAEP} fragment: hash profile	36
12.9 Frozen C_{pre}^{OAEP} fragment: RSA modulus and derived message-length boundary . . .	37
12.10 Frozen C_{pre}^{OAEP} fragment: randomness	37
12.11 Frozen C_{pre}^{OAEP} fragment: ciphertext representation	38
13 RSA-OAEP – frozen contract clauses	39
14 RSA-OAEP – frozen Γ_0^{OAEP}	40
15 RSA-PSS – normative fiche (RFC 8017 §8.1/§9.1)	46
15.1 Three-backend inventory: four geometry-determining dimensions	46
15.2 Emerging geometry (not yet frozen)	49
15.3 Two causal layers, not one: pre- C_{pre}^{PSS} frontier vs. cryptographic verification result . .	50
15.4 Consolidated ten-dimension inventory	52
15.5 Formal derivation of P_{PSS}^{common}	53
15.6 Formal derivation of $P_{PSS}^{portable}$	54
15.7 Formal derivation of C_{pre}^{PSS}	55
15.8 Resolution: signature-length normalization (D-046)	57
15.9 Frozen E_{PSS}^{SDK} (D-047)	57
16 RSA-PSS – frozen contract clauses	58

17 RSA-PSS – frozen Γ_0^{PSS}	60
18 Relation-spectrum diagnostic equivalence	68
19 RSA Key Serialization – normative fiche	69
19.1 Backend inventory: WebCrypto RSA (closed, v0.5.3)	71
19.2 Backend inventory: Crypto++ RSA (closed)	73
19.3 Backend inventory: Bouncy Castle RSA (closed)	75
19.4 Three-backend RSA serialization intersection and portable profile	77
19.5 Admission-time validation semantics	79
19.6 Precondition contract $C_{pre}^{RSA-ser}$	80
19.7 SDK error semantics	82
20 RSA Key Serialization – frozen contract clauses	86
21 RSA Key Serialization – frozen $\Gamma_0^{RSA-ser}$	88
21.1 RSA Key Serialization: accumulated audit and closure	93
22 EC P-256 Key Serialization – normative fiche	95
22.1 Normatively established	95
22.2 Explicitly OPEN – provider inventory required, not decided here	96
22.3 Backend inventory: WebCrypto EC P-256 (closed)	97
22.4 Backend inventory: Crypto++ EC P-256 (closed)	100
22.5 Backend inventory: Bouncy Castle EC P-256 (closed)	103
22.6 Three-backend EC P-256 common intersection P_{EC-ser}^{common}	106
22.7 EC P-256 portable profile $P_{EC-ser}^{portable}$	107
22.8 EC P-256 mathematical validity: Membership and $Q = dG$	109
22.9 Precondition contract C_{pre}^{EC-ser}	111
22.10 EC P-256 SDK error semantics and common→portable normalization	112
22.11 EC P-256 frozen contract clauses	114
22.12 EC P-256 causal classes and Γ_0^{EC-ser}	116
22.13 Cross-operation audit: RSA key serialization vs. EC P-256 key serialization	118
23 Necessity decision rule	119
23.1 Formal Coverage/Marginal computation, stages 1–2	119
23.2 Stages 3–4 protocol: detection and diagnostic gain, pre-registered	121
23.3 The O_{min} decision boundary, closed preexperimentally	123
24 Decision log	124

25 Evidence and reference register	158
26 Explicit non-claims / out of scope	160
27 Open items – pick up here next session	160
28 Preexperimental design freeze	162

Status. This is not the final manuscript. It is the cumulative, versioned source of truth for the scientific design, formal model, experimental protocol, evidence, decisions, exclusions, and operation-level contracts. Closed decisions are recorded here before research proceeds. This file supersedes the earlier Markdown version of the same document; LaTeX is now the primary working format.

1 Purpose and working rule

The paper investigates observable cryptographic conformance across multiple realizations of one multiplatform client SDK when the realizations intentionally rely on heterogeneous cryptographic backends. The motivating architectural pattern is a shared SDK/core deployed across native and web targets, while low-level cryptographic operations may be delegated to different providers.

Iberbox is architectural motivation only. The paper must remain scientifically complete if Iberbox is removed from the empirical evaluation. No claim will state that the real Iberbox SDK has been verified or audited. The empirical validation will use a reference SDK/framework owned and implemented for the research.

Workflow rule: research $\rightarrow$ close decision $\rightarrow$ document decision/evidence $\rightarrow$ continue. Important reasoning must not exist only in chat.

2 Scientific gap and positioning

The target problem is not compiler semantic preservation and not conventional differential testing between independent cryptographic libraries. It is the intermediate architectural case in which one product exposes one SDK-level contract but its platform realizations use heterogeneous low-level cryptographic backends by design.

The central question is therefore whether the public cryptographic contract remains observably conformant across targets even when implementation mechanisms, provider APIs, parameter exposure, serialization conventions, and error models differ.

Gap statement (deliberately hedged; no “first” claim):

No general framework has been found that formalizes the observable conformance obligations of a multi-platform client cryptographic SDK and materializes them systematically as a family of distinct oracles – byte equality, cross-provider interoperability, serialization, validation/error, and capability – against cryptographic providers that are heterogeneous by design.

2.1 Neighbouring literature that does not eliminate the gap

WasmChecker. Compiler/translation semantic preservation between native x86-64 and WebAssembly (Emscripten). This is the *closest* neighbour and must be explicitly differentiated in Related Work: it asks whether compilation changes behaviour, not whether heterogeneous cryptographic providers satisfy one SDK contract. Found 11 real Emscripten bugs; not crypto-specific.

Cryptofuzz / CryptoFuzz++ and related differential cryptographic testing. Compare independent implementations of the same primitives/standards. Methodologically relevant, but the unit of comparison is *algorithm* $\rightarrow$ *library*, not *product SDK contract* $\rightarrow$ *platform realization*.

FHE backend/scheme mutation work (“HERTA”). Keeps a program fixed while mutating backend/scheme (CPU/GPU, OpenFHE/SEAL, BFV/BGV) and checks invariant outputs, with an

exact-vs-approximate (CKKS) oracle distinction. Strong methodological precedent in a different cryptographic domain (FHE, not client SDKs).

EverCrypt / Project Everest. Verified-provider approach: heterogeneity is reduced by construction (*Implementation* $\models$ *Specification*). Useful conceptual contrast, not a competitor: it assumes control over the implementations being verified, whereas our problem arises precisely when providers (WebCrypto, BoringSSL, platform keystores) are *not* controlled.

Google package:webcrypto. Production evidence that cross-platform cryptographic API parity requires explicit handling of backend differences: same API surface over BoringSSL (native) and WebCrypto (browser), documented discrepancies (formats, curves, AES-CTR, import/export), and an explicit admission that error/exception equivalence is not yet systematically tested. Best real-world motivating evidence found.

Intent-Based Cryptographic API Design for Cryptographic Agility (Rameshan & Messmer, IBM Research, 2026; Eurocrypt 2026 MAgiCS workshop, with companion Assessment Framework paper). Relevant abstraction/provider-agility neighbour – addresses *how* crypto-agile APIs should be designed, not *how to verify empirically* that heterogeneous realizations of such an API actually conform. Complementary, not overlapping.

ACVP / ML-KEM cross-implementation conformance work (2026). Recent evidence for capability-boundary discipline and primitive-level conformance (byte equality, validation verdict, unsupported operations, adapter errors kept distinct); scoped to a single primitive/standard, not a composed SDK workflow with serialization and cross-platform consumption.

X.509 post-quantum linting work. Includes an isolated encode/decode-identity conformance rule for ML-KEM in SPKI. Narrow, structural, primitive-level; not product-level.

Conclusion of the gap-killing exercise. Retired claims: “nobody tests crypto cross-backend” (false – Cryptofuzz does); “nobody tests native and WebCrypto under the same API” (false – package:webcrypto does); “nobody uses relational oracles across backends” (false – HERTA does, in FHE). What survives: no general framework formalizes and systematizes the *typed family* of conformance obligations for a *product-level multi-platform SDK contract*.

3 Research framing

Provisional framing: a methodology and reference framework for cross-provider cryptographic conformance of multiplatform SDKs. The contribution is a typed observational contract, a differential/metamorphic testing methodology, and an empirical evaluation using a synthetic/reference SDK with deliberate fault seeding.

The paper does not claim equivalence of all security guarantees. In particular it does not audit browser isolation, side channels, memory handling, hardware keystores, CSPRNG quality, or provider-internal nonce-generation policies.

3.1 Research questions

RQ1 (main). How can cryptographic conformance be defined and systematically verified across heterogeneous platform realizations of the same client-side SDK contract?

RQ2. Which observational relations are required to capture cryptographic equivalence when byte-level equality is not an appropriate oracle?

RQ3. Which classes of cross-provider semantic divergence can each conformance relation detect?

RQ4. To what extent does a typed conformance framework expose discrepancies that conventional known-answer or same-output testing would miss?

3.2 Hypotheses

H1. A single output-equality relation is insufficient to characterize cryptographic conformance across heterogeneous providers.

H2 (revised). Typed conformance relations provide complementary *detection and diagnostic* power for cross-provider semantic divergences beyond conventional known-answer and undifferentiated interoperability testing.

H3. Serialization, validation, error, and capability discrepancies constitute a substantial part of the observable divergence space of a multi-platform SDK. This hypothesis is genuinely falsifiable and may turn out false; the design must survive that outcome.

3.3 Refutation criteria (pre-registered)

The project should be abandoned or substantially reconsidered if, during operation-by-operation design or implementation, any of the following occurs:

1. The six relations collapse to one or two because the rest add no independent detection/diagnostic value.
2. Known-answer plus interoperability testing alone detects essentially all seeded mutations.
3. The contract model $\mathcal{C}$ turns out to be merely descriptive and cannot derive test obligations automatically.
4. Real inter-provider differences are so trivial that the “SDK contract” abstraction adds nothing over normal library testing.
5. The framework requires so much backend-specific adaptation that no reusable methodology remains.

4 Formal contract model

Current model:

$$\mathcal{C} = (O, T, P, R, A)$$

O is the operation set; T captures types/representations; P captures contractual parameters; R is the family of typed conformance relations; A is the pre-registered provider capability manifest.

Each platform/backend implements a realization $I_p : \mathcal{C} \rightarrow B_p$, where B_p is a concrete provider (e.g. WebCrypto, Crypto++, Bouncy Castle). Conformance between two realizations is

$$I_p \equiv_{\mathcal{C}} I_q \iff \forall o, x, \theta : R_o(I_p(o, x, \theta), I_q(o, x, \theta)) = 1.$$

Common contract parameters are the intersection, not the union, of what each backend externally exposes:

$$P_o^{\text{common}} = \bigcap_p P_{p,o}.$$

Cross-provider conformance obligations apply only over P_o^{common} . Backend-specific extra control (e.g. Bouncy Castle allowing an explicit PSS salt where WebCrypto only exposes `saltLength`) is a *declared capability extension*, never a conformance failure.

Adapter model. What is actually observed is $I_p = API_p \circ Adapter_p$, so

$$\text{ObservedDivergence} = \text{BackendDivergence} \vee \text{AdapterDivergence}.$$

This underlies the adapter-induced divergence threat (§7.2).

4.1 Six conformance relations

R_byte – Deterministic byte equality when all inputs affecting the output are contractually fixed.

R_interop – Directional cross-provider interoperability. It is evaluated $p \rightarrow q$ and $q \rightarrow p$ because interoperability may be asymmetric. It absorbs cross-round-trip; local round-trip is not a cross-provider relation.

R_ser – Conformance of a contractually defined portable representation/serialization.

R_val – Conformance of acceptance and rejection semantics for valid and invalid inputs, parameters, and artifacts.

R_err – Conformance of observable contractual error classes, kept distinct from validation.

R_cap – Conformance between actual supported behaviour and the capability boundary declared and frozen before execution.

R_{round} was removed as an independent relation. A local property $\text{Inverse}_p(\text{Operation}_p(x)) = x$ is an internal correctness/sanity check (Phase A). The cross-provider form $\text{Inverse}_q(\text{Operation}_p(x)) = x$ is an instance of directional R_{interop} .

4.2 Applicability vs dependency

A relation belongs to an operation only if that operation’s own contract exposes the corresponding obligation. Using a serialized key produced by another operation does not make serialization a property of RSA-OAEP or RSA-PSS.

$$\text{Dep}(o_i, o_j) = 1$$

means that testing operation o_i requires an artifact produced/validated by o_j . Dependencies do not inherit the conformance relations of the dependency. Failures traceable to a dependency are attributed to the owning contract clause/operation, not to the operation where the symptom was observed.

5 Shared clause taxonomy

Local contract clauses retain operation-specific identifiers for fine-grained attribution, but necessity analysis must operate over a shared semantic taxonomy. Otherwise local namespaces are disjoint by construction and cross-operation Coverage/Marginal analysis becomes meaningless.

Frozen first-level taxonomy:

$$K_1 = \{\text{Key, Parameter, Input, Output, Authentication, Encoding, Validation, Membership, Error, Capability}\}.$$

Membership remains at K_1 , separate from Validation. This prevents premature fusion of mathematical/structural validity (e.g. EC point membership) with API acceptance/rejection semantics.

Current K_2 subtypes (extended only when a real clause requires it):

- Encoding.{ASN1, KeyPoint, AEADArtifact, Textual}
- Validation.{Syntactic, Semantic}
- Membership.{CurveMembership, SubgroupMembership}
- Parameter.{AlgorithmChoice, Length, DomainParameter, ExternalRandomnessControl}

- Authentication.{AEADTag, SignatureVerification}

The subtype `Parameter.ExternalRandomnessControl` covers only externally observable or controllable randomness-related interface parameters (e.g. a supplied GCM IV or a PSS `saltLength`). It explicitly excludes CSPRNG quality, entropy sources, reseeding, uniqueness policy, persistence, restart behaviour, and hardware RNG.

5.1 Two-level coverage

For operation o :

$$\text{Coverage}_{\text{local}}(o) = \bigcup_{c \in \text{Mut}(o)} \Gamma_0(c), \quad \text{Coverage}_{\kappa}(o) = \{\text{kind}(g) : g \in \text{Coverage}_{\text{local}}(o)\}.$$

Marginal coverage is calculated over shared kinds/subtypes, not local clause IDs:

$$\text{Marginal}_{\kappa}(o) = \text{Coverage}_{\kappa}(o) - \bigcup_{o' \neq o} \text{Coverage}_{\kappa}(o').$$

An empty marginal kind set does not automatically eliminate an operation: necessity also considers subtype coverage, detection gain, and diagnostic gain (§23).

6 Pre-registration and diagnostic machinery

Γ_0 is a pre-registered causal map. For each mutation class c it contains the local contract clauses that could, in principle, explain that class: $\Gamma_0(c) \subseteq \text{Clauses}(\mathcal{C})$. It is frozen before executing the mutation campaign and must not be constructed retrospectively from observed failures. One clause corresponds to one stable contractual obligation; mutations attack clauses, they do not generate new ones.

Diagnostic inference uses the complete result spectrum

$$r(c) = (R_{\text{byte}}, R_{\text{interop}}, R_{\text{ser}}, R_{\text{val}}, R_{\text{err}}, R_{\text{cap}})$$

with pass/fail/not-applicable per component, rather than conditioning on one relation in isolation. This is conceptually aligned with spectrum-based fault localization: the joint pass/fail pattern carries localization information.

Γ_0 is causal ground truth, not the diagnostic hypothesis space (D-052). During the mutation campaign, the injected class c is known by construction; $\Gamma_0(c)$ records which clauses that known cause genuinely violates, for attribution and coverage purposes (§6.1 below) – it is not a set of alternative hypotheses for r to narrow. The elements of $\Gamma_0(c)$ are jointly violated by the same single cause, not competing explanations r could adjudicate between: no observable relation can coherently “eliminate” one clause from $\Gamma_0(c)$ while retaining another, since both are already-known consequences of the one cause under test, not rival candidates.

The genuine diagnostic question – and the one D-051’s cross-operation collisions concretely demonstrate – is different: given only the observed spectrum r , which *pre-registered causal classes*, among the full candidate set M_o for operation o , remain consistent with it? Define the diagnostic hypothesis set

$$H(c \mid r) = \{c' \in M_o : r(c') = r(c)\},$$

which is exactly the diagnostic-equivalence class $[c]_r$ (§18). The marginal diagnostic contribution of relation R_j is the reduction in this hypothesis set obtained by including R_j ’s component, relative to the spectrum with that component masked (r_{-j}):

$$\Delta_j^{\text{diag}}(c) = |H(c \mid r_{-j})| - |H(c \mid r)|, \quad DG_j(c) = 1 - \frac{|H(c \mid r)|}{|H(c \mid r_{-j})|} \in [0, 1].$$

Since $c \in H(c \mid r)$ always (by reflexivity of $r(c) = r(c)$), $|H(c \mid r)| \geq 1$ and $DG_j(c)$ is always well-defined. R_j has diagnostic uniqueness for c iff $\Delta_j^{\text{diag}}(c) > 0$. Detection uniqueness and diagnostic uniqueness remain distinct: a relation may detect c (some component of $r(c)$ fails, distinguishing it from conforming behavior) while contributing $\Delta_j^{\text{diag}}(c) = 0$ if it fails to shrink the hypothesis set any further than the rest of the spectrum already does. $|H(c \mid r)| = 1$ denotes diagnostic identifiability; $|H(c \mid r)| > 1$ denotes residual diagnostic equivalence – exactly the situation found for 13 of PSS’s 15 classes (D-051).

6.1 Attribution rule

Every observed divergence records three levels: (1) detection site, (2) attributed pre-registered contract clause, and (3) owning operation. If $\text{Dep}(o_i, o_j) = 1$ and the causal clause belongs to o_j , experimental value is attributed to o_j rather than inflated for o_i .

7 Experimental protocol

Phase A – Internal correctness. KATs and local round-trips establish baseline correctness of each primitive/provider wrapper. This reduces, but cannot eliminate, adapter-induced divergence.

Phase B – Cross-provider conformance. Evaluate applicable typed relations across provider pairs/directions on genuine conforming implementations. Measure false-positive behaviour (specificity) *before* mutation sensitivity is assessed – detection rate alone is insufficient, since an oracle that always reports “divergent” scores 100% recall trivially.

Phase C – Mutation sensitivity. Inject pre-registered divergence classes and measure detection rate/recall, specificity/false-positive rate, detection uniqueness, and marginal diagnostic gain.

7.1 Baselines

Baseline A – known-answer testing (KAT) only. **Baseline B** – undifferentiated byte-level differential testing, where applicable. **Proposed** – the full typed $\mathcal{R}$ family. Target finding: $\text{Detection}_{\text{typed}} > \text{Detection}_{\text{KAT}}$, and specifically *where* (which relations, which operations) – ideally including cases where two backends pass identical KATs yet are not SDK-contract-conformant (e.g. both encrypt correctly, but one cannot import the other’s exported key).

7.2 Threats to validity already identified

- **Adapter-induced divergence.** Wrapper/marshaling code can itself introduce discrepancies not attributable to the underlying provider.
- **Fault-seeding external validity.** Mutation sensitivity demonstrates sensitivity to seeded classes, not exhaustive coverage of all real provider discrepancies.
- **Externally controlled randomness / ecological-validity threat.** Reproducible experiments may control IV/nonce-related inputs, but this does not evaluate provider-internal RNG or nonce-generation policy: $\text{Conformance}_c \not\equiv \text{Conformance}_{\text{RNGPolicy}}$.
- **Capability circularity.** Provider capability manifests must be derived from documentation/specification and frozen before execution.
- **Causal circularity.** Γ_0 must be frozen before mutation execution.

8 Backends and feasibility

Current reference lanes: WebCrypto (materialized by a concrete browser/runtime version), Crypto++, and Bouncy Castle. WebCrypto must never be treated as an abstract provider independent of a real implementation/version.

The feasibility pass established that SHA-256, HKDF-SHA-256, PBKDF2-HMAC-SHA-256, AES-GCM, RSA-OAEP, RSA-PSS, and SPKI/PKCS#8 import/export are implementable across the three lanes, with important differences in exposed parameters.

9 Candidate operation set and current status

Operation	Status	Primary scientific role	Notes
HKDF-SHA-256	Closed pilot	Deterministic R_{byte} case; validation/error/capability boundaries	Template operation.
AES-256-GCM	Closed	Rich case activating all six relations (five confirmed applicable, R_{err} provisional pending necessity)	Common profile: 96-bit IV, 128-bit tag; canonical AEADArtifact; Γ_0^{GCM} frozen at 12 causal classes.
RSA-OAEP	Closed	Probabilistic encryption; R_{interop} without R_{byte} ; R_{ser} inapplicable	Key serialization is a dependency, not OAEP R_{ser} . RSA-3072/SHA-256 profile; Γ_0^{OAEP} frozen at 12 causal classes.
RSA-PSS	Closed	Probabilistic sign/verify; new Authentication.SignatureVerification family; R_{byte} inapplicable, no decryption_error-equivalent	RSA-3072/SHA-256/ $sLen = 32$ profile; Γ_0^{PSS} frozen at 15 causal classes; first operation-level derivation of relation-spectrum diagnostic equivalence (D-051).
RSA key serialization	Closed contract / O_{pending} (empirical necessity, D-067)	ASN.1 SPKI/PKCS#8 serialization	Zero Coverage marginal against the full operation set (D-065); survival in O_{min} depends entirely on stages 3–4 ($H_{\text{RSA-ser}}$, D-066).

Operation	Status	Primary scientific role	Notes
EC P-256 key import/export/validation	Closed contract / O_{fixed} (structurally necessary, D-065/D-067)	KeyPoint encoding, capability and curve membership	$\text{Marginal}_{K_1} = \{\text{Membership}\}$, the only nonempty K_1 marginal in the full operation set; no need to reintroduce full ECDSA.
PBKDF2	Secondary / likely removable	Extra parameter richness	Potentially redundant with HKDF.
SHA-256	Control only	Simple deterministic KAT	Not expected to carry a major contribution.

10 HKDF-SHA-256 – closed pilot contract

HKDF was deliberately specified first because it has a small deterministic contract surface. The pilot validated the clause $\rightarrow \text{kind} \rightarrow \Gamma_0$ workflow and exposed inconsistencies cheaply before moving to AES-GCM/EC.

Normative rule: where an external standard fixes semantics, C_{pre} adopts the standard rather than inventing a framework convention. RFC 5869 fixes absent-salt semantics and the upper bound $L \leq 255 \cdot \text{HashLen}$. The RFC does not explicitly impose $L > 0$; the strict positive lower bound is fixed as an explicit portable-profile restriction (D-068), not a normative RFC requirement: $1 \leq L \leq 255 \cdot \text{HashLen}$, i.e. $1 \leq L \leq 8160$ octets for HKDF-SHA-256 ($\text{HashLen} = 32$).

ID	K1	K2	Contract clause
hkdf.ikm	Input	—	IKM is an opaque byte sequence with no implicit normalization. Absent salt is interpreted as HashLen zero octets per RFC 5869; an explicitly supplied zero-length salt is a distinct valid input.
hkdf.salt	Input	—	
hkdf.info	Input	—	
hkdf.hash	Parameter	AlgorithmChoice	info is an opaque externally supplied byte sequence with no implicit normalization. The common profile uses HKDF-SHA-256.

ID	K1	K2	Contract clause
hkdf.length	Parameter	Length	Requested OKM length L is externally controlled and MUST satisfy $1 \leq L \leq 255 \cdot \text{HashLen}$. For HKDF-SHA-256 this yields $1 \leq L \leq 8160$ octets. The upper bound is inherited from RFC 5869; the strict positive lower bound is a portable SDK-profile restriction (D-068).
hkdf.output	Output	—	Identical contractual inputs produce identical OKM bytes.
hkdf.validation	Validation	Semantic	Inputs/parameter combinations outside the common contractual domain are accepted/rejected according to the contract.
hkdf.error	Error	—	Rejected requests map to the contractually defined error class.
hkdf.cap	Capability	—	Declared HKDF support and parameter bounds match the pre-registered capability manifest.

10.1 HKDF pre-registered mutation classes

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected detecting relations
HKDF-NULL-VS-EMPTY-SALT	hkdf.salt; hkdf.output	Input; Output	R_byte; R_val only if a backend rejects one case
HKDF-INFO-TAMPER	hkdf.info; hkdf.output	Input; Output	R_byte
HKDF-IKM-NORMALIZATION	hkdf.ikm; hkdf.output	Input; Output	R_byte
HKDF-HASH-MISMATCH	hkdf.hash; hkdf.output; hkdf.cap	Parameter. AlgorithmChoice; Output; Capability	R_byte; R_cap

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected detecting relations
HKDF-LENGTH-WITHIN-DOMAIN- PERTURBATION	hkdf.length; hkdf.output	Parameter.Length; Output	R_byte
HKDF-LENGTH- BOUNDARY- CROSSING	hkdf.length; hkdf.validation; hkdf.error	Parameter.Length; Validation.Semantic; Error	R_val; R_err. Stimuli: $L = 0$ and $L = 8161$ (the exact bilateral boundary under D-068's $1 \leq L \leq 8160$) – two instances of one causal class, not two. hkdf.cap removed from Γ_0 (D-068): a syntactically valid but out-of-portable- profile length is invalid_parameter, not unsupported – hkdf.cap belongs only to HKDF-CAPABILITY-BOUNDARY-MISM. which attacks the manifest declaration itself, not to every parameter-boundary crossing merely because a manifest also happens to exist. R_cap; R_val; R_err
HKDF- UNSUPPORTED- HASH- DECLARATION	hkdf.hash; hkdf.cap; hkdf.validation; hkdf.error	Parameter. AlgorithmChoice; Capability; Validation.Semantic; Error	
HKDF- CAPABILITY- BOUNDARY- MISMATCH	hkdf.cap; hkdf.validation; hkdf.error	Capability; Validation.Semantic; Error	R_cap; R_val; R_err

Coverage note for `hkdf.ikm`: only one dedicated mutation is retained deliberately. RFC 5869 does not provide a useful IKM validation/error/capability boundary for this profile that warrants an artificial mutation merely for symmetry. Its experimental role is therefore input/output semantics via normalization sensitivity.

11 AES-GCM – frozen portable profile (C_{pre}^{GCM})

AES-GCM is the first rich operation, and its portable profile is now closed: all three backends are verified at source/primary-specification level (§11.3 below), and C_{pre}^{GCM} is frozen accordingly.

Normative grounding (SP 800-38D). Permitted tag lengths are $t \in \{128, 120, 112, 104, 96, 64, 32\}$

bits; 64 and 32 bits are subject to additional Appendix C conditions. Crucially, the standard also requires a *fixed* t per key, and imposes a coupling rule: *if $|IV| \neq 96$, then t must equal 128*. Our profile ($|IV| = 96$, $t = 128$) satisfies this coupling cleanly, and the rule itself yields a normatively-grounded mutation, **IV-LENGTH-TAG-LENGTH-COUPPLING-VIOLATION** ($IV \neq 96$ combined with $t \neq 128$), independent of our own profile choice.

Timely context. NIST has decided to revise SP 800-38D to remove support for authentication tags shorter than 96 bits; a Rev.1 draft’s public comment period closed 31 July 2026 – i.e. this revision is actively in progress as of this writing. Our decision to fix $t = 128$ in the portable profile is therefore aligned with where the standard itself is heading, not an arbitrary restriction. This connects directly to the crypto-agility framing of the JISA line: a provider that still offers 32/64-bit tags today is exactly the kind of agility debt that line studies.

WebCrypto specifics. `additionalData` is optional and may be omitted without compromising security, so the common contract adopts $AAD_{\text{absent}} \equiv AAD_{\emptyset}$. Note the deliberate contrast with HKDF: for `hkdf.salt`, `absent` $\neq$ empty (RFC 5869), while for `gcm.aad`, `absent` $\equiv$ empty (WebCrypto, and now confirmed for Bouncy Castle and Crypto++ as well – see §11.3) – good evidence that a generic framework cannot impose one universal null/empty convention; it must be decided per operation, grounded in the applicable normative source. Separately, the Web Crypto API specification allows `tagLength` $\in \{32, 64, 96, 104, 112, 120, 128\}$ unconditionally, which is *more permissive* than NIST’s own recommendation (96–128 recommended; 32/64 acceptable only in some applications per Appendix C) – a clean illustration of P_o^{common} as an intersection versus a backend-specific capability extension.

Confirmed against the normative specification text. The Web Cryptography API Level 2 Editor’s Draft (`encrypt` operation, AES-GCM) specifies that *ciphertext* C and authentication tag T are the outputs of the SP 800-38D Authenticated Encryption Function, and the immediately following step states: “Let `ciphertext` be equal to $C \parallel T$, where ‘ $\parallel$ ’ denotes concatenation,” with that concatenated value returned as the operation’s result. The `decrypt` operation confirms the inverse convention explicitly: “Let `tag` be the last `tagLength` bits of `ciphertext`”, with the preceding bits treated as the actual ciphertext. Therefore, for WebCrypto:

$$\text{WebCryptoGCMOutput} = C \parallel T, \quad \text{Decrypt} : C \parallel T \mapsto \{C = \text{all but final } t \text{ bits}, T = \text{final } t \text{ bits}\}.$$

The `decrypt` algorithm also confirms `additionalData` is taken from the corresponding member “if present or an empty byte sequence otherwise,” normatively closing $AAD_{\text{absent}} \equiv AAD_{\emptyset}$ for WebCrypto. This was verified through convergent evidence: the W3C bug tracker (Bug 22570, *RESOLVED FIXED*, “AES-GCM should provide distinct inputs/outputs for the tag” – resolution: “the tag is appended to the result”) independently corroborates the same convention documented in the current Editor’s Draft algorithm steps.

11.1 Frozen portable profile

$$C_{\text{pre}}^{\text{GCM}} = \text{AES-256-GCM}, \quad |K| = 256 \text{ bits}, \quad |IV| = 96 \text{ bits}, \quad |T| = 128 \text{ bits}.$$

Plaintext M and AAD are opaque byte strings, with $AAD_{\text{absent}} \equiv AAD_{\emptyset}$ (confirmed for all three backends, §11.3). The IV is externally supplied by the harness/SDK contract for this operation; this implies no claim about any backend’s internal nonce-generation policy, which remains out of scope per the ecological-validity threat already declared.

Formally, three nested capability sets:

$$P_{\text{GCM}}^{\text{portable}} \subseteq P_{\text{GCM}}^{\text{common}} \subseteq P_{p,\text{GCM}} \quad (\text{for each backend } p).$$

$P_{\text{GCM}}^{\text{common}}$ is what every backend *could* express; $P_{\text{GCM}}^{\text{portable}}$ is what the reference SDK actually offers. Concretely, $P_{\text{GCM}}^{\text{portable}}.\text{tagLength} = \{128\}$, even though $P_{\text{GCM}}^{\text{common}}$ likely contains more than one tag

length shared by all three backends. This distinction lets the framework test both provider-only capability drift (a value in some $P_{p,GCM}$ but not P_{GCM}^{common}) and below-standard-floor drift (a value outside the NIST-recognized set entirely) as two structurally different mutation classes.

AEADArtifact – concrete, not abstract. Given the frozen profile ($|IV| = 12$ bytes, $|T| = 16$ bytes):

$$\text{AEADArtifact} = \text{version} \| IV_{12} \| C \| T_{16}$$

with a single version byte for explicit format evolution. AAD is not stored in the artifact – it is external contextual input to the contract, not part of the portable representation. The parser’s obligation is now falsifiable and concrete: byte 0 is the version; the next 12 bytes are the IV; the last 16 bytes are the tag; everything in between is ciphertext. This replaces the earlier abstract $\text{Encode}_C(IV, C, T)$ placeholder, which could have silently hidden any representation.

Frozen SDK error model (pre-registered classification order, not post-hoc adaptation).

$$E_{GCM}^{SDK} = \{\text{invalid_parameter}, \text{malformed_artifact}, \text{unsupported}, \text{authentication_failure}\}.$$

No individual backend is required to natively produce these four classes – WebCrypto’s undifferentiated **OperationError** does not block this model. Classification happens in the adapter, in a fixed, pre-registered order:

1. The adapter first checks the operation/parameter against the pre-registered capability manifest A_p^{pre} and the portable profile boundary $P_{GCM}^{\text{portable}}$. **Disambiguation rule (frozen):** a parameter value outside $P_{GCM}^{\text{portable}}$ but syntactically well-formed (e.g. $t = 80$, a value some backend can execute internally but the portable contract does not offer) is classified *invalid_parameter* – the SDK’s own boundary, not a provider limitation. *unsupported* is reserved for operations or capabilities that a specific provider’s manifest declares genuinely unavailable, independent of the portable profile. This distinction is decided *before* the backend is ever invoked.
2. The adapter structurally validates the **AEADArtifact**. If it cannot contain a well-formed IV/ciphertext/tag split, this is *malformed_artifact*.
3. The adapter validates remaining contractual parameters against C_{pre}^{GCM} – *invalid_parameter* if violated.
4. Only after (1)–(3) have excluded every structural or parametric cause is a backend decryption or authentication failure normalized as *authentication_failure*.

This order is frozen now, before any backend is executed against mutants – exactly the same discipline already applied to A_p^{pre} and Γ_0 , and specifically designed to prevent R_{err} from being rescued post hoc by loosening the classification rules after seeing which backend fails where. R_{err} remains applicable to AES-GCM under this model, but – consistent with the necessity cascade in §23 – it must still earn its place through detection/diagnostic gain in the necessity analysis, not by definitional fiat.

Capability is split into two local obligations. R_{cap} conformance is not violated merely because a backend supports more than the portable profile offers – $\text{Observed}_{BC}(t=80) = A_{BC}^{pre}(t=80) = \text{supported}$ is a legitimate provider extension, not a failure. Failure occurs only if (a) observed behavior diverges from the pre-registered manifest, or (b) the adapter lets a provider-only capability leak into the portable surface. Both remain $K_1 = \text{Capability}$, split at the local-clause level for diagnostic granularity:

- **gcm.cap.provider:** observed backend behavior matches A_p^{pre} .
- **gcm.cap.portable:** the SDK exposes only $P_{GCM}^{\text{portable}}$; any provider extension not explicitly re-labeled as non-portable must not leak through.

ID	K1	K2	Frozen contract clause
gcm.key	Key	—	Portable profile uses a 256-bit AES key with identical key bytes across realizations.
gcm.plaintext	Input	—	Plaintext is an opaque byte sequence without normalization.
gcm.aad	Input	—	AAD is an opaque byte sequence; $AAD_{\text{absent}} \equiv AAD_{\emptyset}$, confirmed for WebCrypto (W3C spec), Bouncy Castle (source: null-to-empty-array guard), and Crypto++ (source: AAD_CHANNEL routing architecture).
gcm.iv	Parameter	External Randomness Control	Portable profile fixes an externally supplied 96-bit (12-byte) IV; wider ranges are supported by all three backends but not offered portably ($P_{GCM}^{\text{portable}} \subsetneq P_{GCM}^{\text{common}}$).
gcm.tagLength	Parameter	Length	Portable profile fixes the authentication tag to 128 bits (16 bytes); coupled to IV length per SP 800-38D. Real backend capability is wider and asymmetric (see tag-length capability matrix above).
gcm.ciphertext	Output	—	Equal K, IV, AAD and M produce identical ciphertext bytes.
gcm.authentication	Authen- tication	AEADTag	Decryption succeeds only for a valid tag over the exact contractual inputs; any modification to IV/AAD/C/T must not authenticate.
gcm.artifact	Encoding	AEADArtifact	Portable SDK representation is $\text{version} \ IV_{12} \ C \ T_{16}$ (concrete, not abstract). WebCrypto’s own provider representation is normatively $C \ T$ (W3C spec); Bouncy Castle and Crypto++ (selected filter configuration) match. The adapter normalizes each provider representation into the canonical artifact.
gcm.validation	Validation	Semantic	Inputs/parameter combinations outside the portable domain are accepted/rejected according to the contract.
gcm.error	Error	—	Frozen four-class SDK error model E_{GCM}^{SDK} with pre-registered adapter classification order and disambiguation rule (above); no individual backend need natively distinguish all four classes.

ID	K1	K2	Frozen contract clause
gcm.cap.provider	Capability	—	Observed backend behavior matches the frozen provider manifest A_p^{pre} .
gcm.cap.portable	Capability	—	The SDK exposes only $P_{GCM}^{portable} \subseteq P_{GCM}^{common} \subseteq P_{p,GCM}$; no unlabeled provider extension leaks through the adapter.

11.2 Frozen Γ_0^{GCM}

Before pre-registering mutation classes, a naming discipline is enforced: a *mutation class* names the seeded defect (a causal claim about which clause(s) it violates); a *stimulus* is a concrete test input used to trigger and detect it. Several previously-listed items (TAG-TAMPER, AAD-TAMPER, CIPHERTEXT-TAMPER, IV-TAMPER, TAG-TRUNCATION) are stimuli, not mutation classes in their own right – they become concrete instances used to exercise GCM-AUTH-BYPASS or GCM-ARTIFACT-STRUCTURE-CORRUPTION depending on how the corrupted artifact is constructed (structurally invalid vs. structurally valid but cryptographically wrong). This keeps Γ_0 from degenerating into one clause per stimulus (D-017).

Causality first, detection second – a self-correction. An earlier draft of this table assigned Γ_0 (AAD-ABSENT-EMPTY-DIVERGENCE) and Γ_0 (AAD-IGNORED) different clause sets partly to make the two experimentally distinguishable. That reasoning is backwards: $\Gamma_0(c)$ must record which clauses a defect *could plausibly violate*, decided independently of which relation we hope will detect it; distinguishability is then an empirical question answered later by $H(c \mid r)$ (§6, D-052), not something pre-registration should engineer in advance. The corrected assignments below fix this. The same audit revealed a second, more consequential error: a mutation described as “take a correctly-produced ciphertext and flip a bit afterward to test whether authentication catches it” does not violate `gcm.ciphertext` – the encryption computation was correct; the artifact was corrupted *after* the fact. That defect belongs entirely to `gcm.authentication`, and is in fact already covered as a CIPHERTEXT-TAMPER stimulus of GCM-AUTH-BYPASS (below) – keeping it as a separate row with $\Gamma_0 = \{\text{gcm.ciphertext}, \text{gcm.authentication}\}$ would have made it a strict, redundant subset of GCM-AUTH-BYPASS. It is removed as a standalone class. `gcm.ciphertext` still needs its own genuine causal coverage – a defect where the *computation itself* diverges across backends – which is restored below as GCM-CIPHERTEXT-COMPUTATION-DIVERGENCE. The same ambiguity affected the IV mutation (post-hoc IV tampering vs. a backend misinterpreting/normalizing the IV during encryption); it is resolved the same way.

The 12 causal classes are organized into four families for clarity, though necessity analysis (§23) treats each independently:

Family 1 – Computation/semantic divergence (defects in what a backend computes, detectable primarily via $R_{\text{byte}}/R_{\text{interop}}$):

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
GCM-PLAINTEXT-NORMALIZATION	gcm.plaintext; gcm.ciphertext	Input; Output	$R_{\text{byte}}; R_{\text{interop}}$

Mutation class	$\Gamma_0(c)$	$\mathbf{Kinds}_0(c)$	Expected relations
GCM-AAD-ABSENT-EMPTY-DIVERGENCE	<code>gcm.aad</code>	Input	R_{byte} (both absent and empty are individually valid inputs; the defect is in <code>gcm.aad</code> 's own <code>absent</code> $\equiv$ <code>empty</code> semantics, not in output computation per se)
GCM-AAD-IGNORED	<code>gcm.aad</code> ; <code>gcm.authentication</code>	Input; <code>Authentication.AEADTag</code>	R_{interop} expected valuable (cross-decrypt with tampered AAD should fail but wrongly succeeds) – a genuinely distinct causal claim from the row above, not a relabeling of it
GCM-IV-INTERPRETATION-DIVERGENCE	<code>gcm.iv</code> ; <code>gcm.ciphertext</code>	<code>Parameter.ExternalRandomnessControl</code> ; Output	R_{byte} (a backend misinterprets/normalizes the IV during encryption, diverging from the others on identical input)
GCM-CIPHERTEXT-COMPUTATION-DIVERGENCE	<code>gcm.ciphertext</code>	Output	R_{byte}

Family 2 – Authentication / adversarial stimuli (a single causal class, exercised by multiple stimuli):

Mutation class	$\Gamma_0(c)$	Kinds₀(c)	Expected relations
GCM-AUTHENTICATION-BYPASS	gcm.authentication; gcm.validation	Authentication.AEADTag; Validation.Semantic	R_{val} ; R_{interop} applicable when the stimulus is instantiated through cross-provider authenticated decryption. Stimuli: TAG-TAMPER, AAD-TAMPER, IV-TAMPER, CIPHERTEXT-TAMPER (post-hoc corruption of an otherwise-correct artifact) – four instances of one causal class, not four classes.

Family 3 – Representation / validation:

Mutation class	$\Gamma_0(c)$	Kinds₀(c)	Expected relations
GCM-ARTIFACT-C-T-SWAP	gcm.artifact	Encoding.AEADArtifact	R_{ser} ; downstream
GCM-ARTIFACT-STRUCTURE-CORRUPTION	gcm.artifact; gcm.validation	Encoding.AEADArtifact; Validation.Semantic	R_{interop} ; R_{ser} ; R_{val} . Stimuli include TAG-TRUNCATION when it breaks artifact structure (vs. the same stimulus used against GCM-AUTH-BYPASS when it produces a structurally valid but cryptographically wrong artifact – which row applies depends on how the concrete test is constructed).

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
GCM-KEY-PROFILE-BOUNDARY-BYPASS	gcm.key; gcm.validation; gcm.cap.portable	Key; Validation.Semantic; Capability	R_{val} ; R_{cap}
GCM-IV-PROFILE-BOUNDARY-BYPASS	gcm.iv; gcm.validation; gcm.cap.portable	Parameter.External RandomnessControl; Validation.Semantic; Capability	R_{val} ; R_{cap}
GCM-TAGLENGTH-PROFILE-BOUNDARY-BYPASS	gcm.tagLength; gcm.validation; gcm.cap.portable	Parameter.Length; Validation.Semantic; Capability	R_{val} ; R_{cap} . Stimuli: $t = 80$ (provider-valid, non-portable) and $t = 16$ or $t = 0$ (provider- permitted, below the NIST-recognized floor) – two instances of one causal class.

Family 4 – Capability / error semantics:

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
GCM-PROVIDER-CAPABILITY-MISMATCH	gcm.cap.provider	Capability	R_{cap}
GCM-ERROR-MISCLASSIFICATION	gcm.error	Error	R_{err}

GCM-ERROR-

MISCLASSIFICATION is the key case for $R_{\text{val}} \neq R_{\text{err}}$. It instantiates the scenario $R_{\text{val}} = 1 \wedge R_{\text{err}} = 0$: the SDK correctly rejects a bad tag (validation succeeds) but classifies it as *invalid_parameter* instead of *authentication_failure* (misclassification). This is the experimental case that can demonstrate R_{val} and R_{err} are genuinely independent relations, not one relation observed twice – though, per the necessity cascade, R_{err} must still earn detection/diagnostic gain to survive.

IV-LENGTH-TAG-LENGTH-COUPLING-VIOLATION excluded from the minimal set. With the frozen profile ($|IV| = 96$, $|T| = 128$), no portable-domain combination can violate SP 800-38D’s coupling rule – reaching $|IV| \neq 96 \wedge |T| \neq 128$ simultaneously requires already having violated two separate portable boundaries (already covered independently by GCM-IV-PROFILE-BOUNDARY-BYPASS and GCM-TAGLENGTH-PROFILE-BOUNDARY-BYPASS). It is retained in the Evidence/Mutation Register as a *compound normative stress case*, runnable optionally against raw provider extensions, but excluded from the minimal mutation set used for necessity analysis. This keeps the core experiment free of redundant compound cases.

Coverage check.

$$\text{Coverage}_{K_1}(\text{AES-GCM}) = \{\text{Key, Parameter, Input, Output, Authentication, Encoding, Validation, Error, Capability}\}.$$

Thus, AES-GCM covers 9 of the 10 K_1 kinds. The only absent kind is **Membership**, which has no natural instantiation in AES-GCM and is exactly the kind EC key validation exists to cover (curve-point membership). This is a useful independent check on the operation set design: AES-GCM earns its status as the richest operation, while EC retains a structural, non-redundant reason to remain in O_{cand} – precisely the kind of cross-operation comparison the shared K_1/K_2 taxonomy (§5) was built to support.

11.3 Backend verification: Crypto++ (initial verification)

Findings below record the initial Crypto++ verification against the official Crypto++ Wiki (primary project documentation), addressing output format, tag length, error behaviour, and verification-failure policy. Subsequent source-level verification for Crypto++ and Bouncy Castle is recorded in Sections 11.4–11.6; the current frozen AES-GCM contract incorporates those later findings.

Output format.

For brevity, we denote Crypto++’s `AuthenticatedEncryptionFilter` as `AESFilter` and `AuthenticatedDecryptionFilter` as `ADFilter`. Crypto++’s GCM output is documented as “a single cipher text message which has two components: the concatenated pair {Encrypted, Tag}” – i.e. $C||T$, matching the WebCrypto convention independently confirmed above – but this is the *default* behavior of `AESFilter`, not an inherent property of GCM itself. `ADFilter` supports both `MAC_AT_BEGIN` and `MAC_AT_END` flags, meaning the same library can expose different observable representations depending on filter configuration. This does not weaken $C||T$ as the confirmed default; it strengthens the case for R_{ser} as a genuinely observable relation rather than a coincidence, since a single backend’s own representation is itself a configurable capability, not a fixed constant.

Tag length. Crypto++ produces tags of 128, 120, 112, 104, or 96 bits, with 64 and 32 bits “available but not recommended”; the default is 128 bits. This matches the SP 800-38D value set exactly. Tag size is configured via the `truncatedDigestSize` parameter of `AESFilter` and `ADFilter`. The parameter is specified in bytes rather than bits, unlike WebCrypto’s `tagLength`; the adapter must therefore normalize this unit difference, which is itself a candidate source of adapter-induced divergence.

Error model – key finding. Crypto++ distinguishes, via distinct exception types, at least two of the classes provisionally listed in E_{GCM} : `InvalidKeyLength` is thrown when the key size is not 16/24/32 bytes (*invalid_parameter*), while `HashVerificationFilter::HashVerificationFailed` is thrown specifically on authentication tag verification failure (*authentication_failure*). This is a materially different error model from WebCrypto, which collapses invalid `tagLength`, an undersized ciphertext, and authentication failure into a single undifferentiated `OperationError`. Whether R_{err} survives necessity analysis for AES-GCM now depends on whether Bouncy Castle also preserves this distinction, or collapses it as WebCrypto does; if the latter, the common contract’s E_{GCM} must be defined at the coarsest granularity shared by all three backends, per the normative-first and no-post-hoc-adaptation rules already in force (D-016, D-017).

Verification-failure policy is itself a capability dimension. Crypto++ exposes a `THROW_EXCEPTION` flag: verification failures can either raise `HashVerificationFailed` or fail silently, at the caller’s discretion. This is not noise – it means *how* a backend reports authentication failure is itself a configurable behavior, which belongs in the capability manifest A_p^{pre} (does the reference SDK’s Crypto++ adapter fix this flag, and to which value?) rather than being left implicit.

11.4 Backend verification: Crypto++ IV, AAD, and tag-length capability (confirmed against source)

Confirmed directly against Crypto++’s official GitHub source (weidai11/cryptopp), addressing the IV-length and tag-length capability gaps identified during backend verification.

IV length – confirmed. `gcm.h` defines `GCM_Base` with

```
IVSize()=12, MinIVLength()=1, and MaxIVLength()=UINT_MAX // (261 – 1) in the
standard
```

matching Bouncy Castle’s pattern: 12 bytes is the default/recommended size, not the only valid one; the standard’s true upper bound is documented in a comment but not enforced by the API type. Our common profile’s fixed 96-bit IV remains a legitimate narrowing of a much wider real capability, exactly as for Bouncy Castle.

AAD null-vs-empty – confirmed by architecture, via a different code path than originally sought. Source (`cryptlib.cpp`, `filters.cpp`) confirms `AAD_CHANNEL` is a named channel ("AAD"), and `AuthenticatedEncryptionFilter::ChannelPut2` only forwards bytes to the underlying `HashFilter` when the caller explicitly writes to `AAD_CHANNEL`:

```
if (channel == AAD_CHANNEL) return m_hf.Put2(begin, length, 0, blocking);
```

If the application never writes to `AAD_CHANNEL` – the case of absent AAD – zero bytes reach the GHASH computation, which is architecturally indistinguishable from explicitly writing a zero-length AAD. This confirms $AAD_{\text{absent}} \equiv AAD_{\emptyset}$ by the channel-routing design itself, not by a defensive null/zero-length guard inside a single function. This is a different verification mechanism than the `if(!input || !len) return;` guard originally sought in `authenc.cpp` – that specific line was not located – but it is evidence of comparable strength for the same conclusion, and is recorded as source-confirmed via channel architecture, with the note that the specific early-return guard remains unlocated.

Tag length – key finding, fully source-confirmed. `AEFilter`’s `truncatedDigestSize` parameter is passed to the underlying `HashFilter`, which sets `m_digestSize` directly with no GCM-specific floor. The only generic guard in the call chain is `TruncatedFinal()`, which calls `ThrowIfInvalidTruncatedSize()`. It is implemented as:

```
void HashTransformation::ThrowIfInvalidTruncatedSize(size_t size) const {
    if (size > DigestSize()) throw InvalidArgument(...); }
```

i.e. the *only* enforced constraint is $t \leq \text{DigestSize}() = 16$ bytes (128 bits) – there is no lower bound at all in this generic path. This is a stronger and more surprising result than the SP 800-38D-aligned $\{32, \dots, 128\}$ figures reported by the Crypto++ Wiki (a usage recommendation, not the implementation’s actual constraint): via this API route, Crypto++ would accept tag sizes far below WebCrypto’s or Bouncy Castle’s floors, including values with no cryptographic authentication value whatsoever.

Updated tag-length capability matrix:

Backend	Observed tag-length capability
WebCrypto	$\{32, 64, 96, 104, 112, 120, 128\}$ bits (discrete set)
Bouncy Castle	$32 \leq t \leq 128, t \bmod 8 = 0$
Crypto++ (generic API path)	$0 \leq t \leq 128$ bits, $t \bmod 8 = 0$, no GCM-specific floor enforced (equivalently $0 \leq d \leq 16$ bytes, $d = t/8$)

giving $P_{GCM}^{\text{WebCrypto}} \subset P_{GCM}^{\text{BC}} \subseteq P_{GCM}^{\text{Crypto++}}$ for tag length. Since the common profile fixes $t = 128$ deliberately, the reference SDK does not need to adopt the full intersection as its offered surface – it only needs 128 to belong to it, which it does for all three. But this asymmetry is valuable evidence in its own right: it demonstrates a case where *provider capability* $\neq$ *standard-conformant common capability*, motivating why R_{cap} cannot be defined merely as “what the library permits.” It also yields a second capability-only mutation, **GCM-TAGLENGTH-BELOW-STANDARD-FLOOR** (e.g. $t = 16$ bits: acceptable to the Crypto++ API, rejected by Bouncy Castle and WebCrypto, and outside any NIST-recognized GCM tag length) – more aggressive than **GCM-TAGLENGTH-PROVIDER-ONLY**, since it falls outside every backend’s *recommended* range and outside the standard itself, not merely outside WebCrypto’s discrete set.

AES-GCM backend verification is now complete. Confirmed across all three backends, each at source or primary-specification level: output format (with the Crypto++ filter-configurability caveat), tag-length capability (including the Crypto++ no-lower-bound asymmetry), IV length ranges, AAD absent/empty semantics (WebCrypto via W3C spec text; Bouncy Castle via a direct null-to-empty-array guard; Crypto++ via channel-routing architecture in `filters.cpp/cryptlib.cpp`), and observable error granularity. This completed the evidence required to freeze C_{pre}^{GCM} and construct Γ_0^{GCM} .

11.5 Backend verification: Bouncy Castle (confirmed)

Findings below are confirmed against bc-java source and, for the authentication-failure exception, independently corroborated across multiple unrelated real-world bug reports spanning 2014–2024 (bc-java, bc-csharp, and third-party projects), which is stronger evidence than a single documentation page.

Output format – three-way convergence (with one caveat). In Bouncy Castle, the `doFinal()` method of `GCMBlockCipher` appends the computed tag immediately after the ciphertext on encryption, and recovers the tag from the end of the message on decryption – i.e. $C||T$, matching WebCrypto’s normative behavior. WebCrypto and Bouncy Castle expose $C||T$ as their direct, unconfigurable behavior; the selected Crypto++ encryption-filter configuration (`AEFilter`, default `MAC_AT_END`-equivalent behavior) also emits $C||T$, although Crypto++ additionally supports alternative MAC-position configurations (`MAC_AT_BEGIN`) not available in the other two backends. This gives, for the specific configurations selected by the reference SDK’s adapters:

$$\text{WebCrypto} = \text{Bouncy Castle} = \text{Crypto++}_{\text{selected config}} = C||T.$$

This convergence does *not* make R_{ser} redundant: the contract must not assume agreement in advance, and the framework’s job is precisely to be capable of detecting a fourth realization – or a different Crypto++ filter configuration – that used a different representation (e.g. $T||C$, or separate fields). The convergence is a genuine empirical/documentary finding worth reporting, and it means the reference SDK’s $C||T$ -based **AEADArtifact** normalization is cheap across all three lanes, while the fact that one backend’s own representation is itself a configurable capability (not a fixed constant) strengthens rather than weakens the case for treating R_{ser} as genuinely observable.

Error model – distinguishes at least three causes. Bouncy Castle compares the received and computed tags in constant time and, on mismatch, throws `InvalidCipherTextException` with message “mac check in GCM failed” (*authentication_failure*) – confirmed verbatim across independent bug reports in bc-java, bc-csharp, and third-party consumers over a decade. If the input is too short to even contain the MAC, it also throws `InvalidCipherTextException`, but with cause “data too short” (*malformed_artifact*) – the same exception *class* as authentication failure, but a distinguishable *cause*. Output-buffer sizing problems raise a separate `OutputLengthException`.

Backend	Observable GCM failure granularity
WebCrypto	single <code>OperationError</code> for multiple causes
Crypto++	distinguishable exception types (<code>InvalidKeyLength</code> vs. <code>HashVerificationFailed</code>)
Bouncy Castle	distinguishable exception types/causes (<code>InvalidCipherTextException</code> w/ cause; <code>OutputLengthException</code>)

WebCrypto is the bottleneck for R_{err} granularity, not Bouncy Castle.

Conceptual refinement – provider exception $\neq$ SDK error class. This does not yet mean R_{err} must die for AES-GCM. There are two levels:

$$E_p^{\text{provider}} \xrightarrow{\text{Adapter}_p} E^{\text{SDK}}.$$

Some causes are classifiable by the adapter *before* calling the backend at all – e.g. an out-of-profile `tagLength` can be rejected by the contract/adaptor itself as *invalid_parameter* without ever reaching WebCrypto. Others – an incorrect tag – can only be discovered by executing the backend. Under this view, R_{err} conformance does not require “the three providers throw the same exception”; it requires “the three SDK realizations produce the same abstract observable error class,” which matches the paper’s actual object of study (SDK-contract realizations, not literal provider-API parity). The risk is symmetric, however: if the adapter is allowed to perform *all* classification arbitrarily, R_{err} risks becoming a property we construct rather than one we observe. Error-normalization rules therefore belong in C_{pre} and must be frozen before the experiment, exactly like A and Γ_0 – not decided post hoc per backend.

Bouncy Castle also supplies a second methodological point: “too-short artifact” and “bad MAC” are distinct *causes* internally even though both raise `InvalidCipherTextException`. This reinforces that R_{err} must compare *contractual error class*, not exception class name – exception-name matching alone would conflate two genuinely different failure modes.

Threat to note (not a mutation, not in-scope for the core experiment): CVE-2026-8149.

Published May 2026, this Bouncy Castle vulnerability documents that GCM decryption chunked at certain boundaries could incorrectly raise a bad-tag exception on specific BC-LTS versions (2.73.0 before 2.73.11); the documented mitigation is passing the full message to `doFinal()` in one call rather than chunking. This is valuable evidence, not a mutation to add to scope: it demonstrates empirically that *same primitive, same backend* does not imply *same observable behavior across versions or execution paths* ($\text{SamePrimitive} \wedge \text{SameBackend} \not\Rightarrow \text{SameObservableBehavior}$), which is exactly why the harness’s backend version-pinning and fixed execution path (single `doFinal()` call, not incremental `update()`) were already required design decisions, not incidental implementation details.

Resolved in §below. At the time of the findings above, tag-length acceptance and nonce/MAC constraints for `GCMBlockCipher/init()` were still open; these are closed against source in the next subsection.

11.6 Backend verification: Bouncy Castle nonce, AAD, and tag-length capability (confirmed against source)

The following are confirmed directly against `GCMBlockCipher.java` source (bc-java, main branch), not documentation.

Nonce length. The source enforces

```
if (nonce == null || nonce.length < 1)
    throw new IllegalArgumentException("IV must be at least 1 byte");
```

with a dedicated optimized path when `nonce.length == 12` (matching the SP 800-38D-recommended 96-bit IV, $J_0 = \text{nonce} \parallel 0x00000001$) and a general GHASH-based computation of J_0 for any other valid length. This confirms Bouncy Castle accepts a much wider nonce-length range than our common profile requires; the common profile’s fixed 96-bit IV remains a legitimate narrowing, not an assumption about what BC can do.

AAD null-vs-empty. The source normalizes `A == null` to `A = new byte[0]` before use. This confirms $AAD_{\text{absent}} \equiv AAD_{\emptyset}$ for Bouncy Castle by direct source inspection, matching the WebCrypto convention (and, again, the opposite convention from HKDF’s `hkdf.salt`).

Tag length – a genuine capability divergence. The source validates

```
if (macSizeBits < 32 || macSizeBits > 128 || macSizeBits % 8 != 0)
    throw new IllegalArgumentException(...);
```

i.e. Bouncy Castle accepts *any* multiple of 8 bits in $[32, 128]$ – not the discrete WebCrypto set $\{32, 64, 96, 104, 112, 120, 128\}$. Values such as 40, 48, 56, 72, 80, or 88 bits are valid for Bouncy Castle but meaningless for the portable contract, since WebCrypto cannot realize them. This is exactly the class of divergence R_{cap} exists to capture, and it sharpens the common-profile rule already stated in §7.2:

$$P_{GCM}^{\text{common}} = P_{GCM}^{\text{WebCrypto}} \cap P_{GCM}^{\text{Crypto++}} \cap P_{GCM}^{\text{BC}}.$$

General rule (applies beyond AES-GCM): common capability $\neq$ offered capability. P_o^{common} (what every backend *could* support) and the portable contract’s actually-offered surface are distinct by design:

$$P_o^{\text{portable}} \subseteq P_o^{\text{common}}.$$

The reference SDK deliberately fixes $P_{GCM}^{\text{portable}} = \{t = 128\}$, a strict subset of P_{GCM}^{common} (which likely still contains multiple valid tag lengths). “Every backend can do X” does not imply “the portable contract must offer X” – this distinction is expected to recur for RSA-OAEP and RSA-PSS, where individual backends may expose more parameter flexibility (hash/MGF1 combinations, explicit PSS salt bytes) than the common profile chooses to offer. Bouncy Castle’s acceptance of $t = 80$ does not enter the portable contract merely because BC supports it; the framework must actively reject a realization that silently widens the portable contract on the strength of one backend’s extra capability. This yields a genuine capability-specific mutation, **GCM-TAGLENGTH-PROVIDER-ONLY** (e.g. $t = 80$: valid for BC, outside the common capability manifest) – now registered as a stimulus of **GCM-TAGLENGTH-PROFILE-BOUNDARY-BYPASS** (§11), distinct from an output-correctness or validation-boundary defect because it targets R_{cap} specifically, not $R_{\text{byte}}/R_{\text{val}}$.

Two distinct, traceable reasons for strictness (extension, not a redesign). $P_o^{\text{portable}} \subsetneq P_o^{\text{common}}$ may hold for two categorically different reasons, and every future narrowing decision must be attributed to exactly one:

1. **Profile-selection reasons.** Several options are technically *and* normatively valid – fully covered by the governing specification for the operation – and the portable SDK intentionally narrows to a subset for reproducibility, simplicity, or comparability. AES-GCM’s $t = 128$ is this category: SP 800-38D itself defines $t \in \{32, \dots, 128\}$ as a normatively valid set, and the SDK picks one value from within it.
2. **Normative-scope reasons.** A behavior belongs to the common *technical* capability of all providers, but falls outside the scope actually covered by the governing normative specification for the operation, and is therefore excluded unless separately justified by another normative basis.

Safeguard. A normative-scope exclusion must cite the exact scope limitation of the governing specification; it cannot be inferred merely from the absence of examples or from observed provider

behavior. This prevents “the standard doesn’t say much about this” from being used as an ad hoc justification for narrowing whenever convenient.

A second, unplanned finding: the tag-length floor itself is not stable across Bouncy Castle history. Search results surfaced at least three distinct historical validations of the same `init()` check across different bc-java versions/forks: the current `main` branch and several published releases enforce $t \geq 32$; some older release snapshots enforce $t \geq 64$; at least one fork (kerryjiang/BouncyCastle.Crypto) enforces $t \geq 96$. This means the capability boundary of “Bouncy Castle” is itself not a fixed constant across versions – the same provider name has offered different tag-length floors over time. This is, if anything, stronger motivation for mandatory backend version-pinning than the CVE-2026-8149 case above: it shows capability drift, not just behavioral drift, within a single named provider across its own release history. This belongs in the paper’s motivation alongside the CVE finding.

AES-GCM is currently the richest candidate operation: R_{byte} , R_{interop} , R_{ser} , R_{val} , and R_{cap} are applicable under the current contract; R_{err} remains applicable under the frozen SDK error model, but its retention as an independent relation remains subject to the subsequent necessity analysis.

12 RSA-OAEP – normative fiche (RFC 8017 §7.1)

Status: RSA-OAEP methodologically closed. Normative fiche closed; all three backends fully inventoried; $C_{\text{pre}}^{\text{OAEP}}$ fully frozen across all six fragments (D-027, D-028, D-030 through D-034, §12.5–§12.11); thirteen `oaep.*` clauses frozen (D-035, §13; `oaep.ciphertext` split into semantic-correctness and length obligations by D-037); Γ_0^{OAEP} frozen at 12 causal classes across 5 families (D-039, §14). Two retroactive corrections to already-frozen Γ_0^{GCM} content were made during this process (D-036, D-038), both found via cross-auditing OAEP against GCM. The final mechanical consistency pass over the whole document has been completed for this checkpoint (v0.4).

Primary source: RFC 8017 (PKCS #1 v2.2), §7.1 (RSAES-OAEP), confirmed directly against IETF Datatracker/RFC Editor text.

- **Message length bound.** $mLen \leq k - 2hLen - 2$, where k is the RSA modulus length in octets and $hLen$ the hash output length. Violation yields the normative error “message too long.”
- **Label.** Optional; default is the empty string if not provided. **Important scope restriction, confirmed verbatim:** “In this version of PKCS #1, L is the empty string; other uses of the label are outside the scope of this document.” The ASN.1 syntax (`RSAES-OAEP-params`, Appendix A.2.1/A.2.3) nonetheless defines `pSourceAlgorithm/id-pSpecified` with an `EncodingParameters` `OCTET STRING`, technically permitting a non-empty label – a real tension between what the ASN.1 syntax allows and what the RFC body normatively scopes. Whether $P_{\text{OAEP}}^{\text{portable}}$.label is $\{\emptyset\}$ or wider is deliberately left open pending the three-backend inventory.
- **Ciphertext length.** Exactly k octets.
- **Hash and MGF are normatively distinct dimensions.** `RSAES-OAEP-params` defines separate `hashAlgorithm` and `maskGenAlgorithm` fields (default: SHA-1, MGF1-SHA1, empty label – confirmed against the ASN.1 text). This is a normative basis for treating `oaep.hash` and `oaep.mgfHash` as conceptually distinct clauses, even where a given backend’s API couples them.
- **Randomness.** EME-OAEP generates an internal seed of $hLen$ octets. Encryption is therefore probabilistic even with $(K, M, Hash, MGF, L)$ fully fixed – R_{byte} is not a valid oracle for the OAEP ciphertext itself; R_{interop} ($Dec_q(Enc_p(M, L), L) = M$, both directions) is the natural relation.

- **Decryption error model – security-motivated, not implementation convenience.** Ciphertext-length mismatch, $k < 2hLen + 2$, RSA representative out of range, and all internal OAEP-decoding failures (bad lHash, missing separator, incorrect leading byte) collapse to a single generic “decryption error.” Critically, RFC 8017 states this collapse exists specifically “to avoid implementation weaknesses related to the way errors are handled within the decoding operation (see [BLEICHENBACHER] and [MANGER]).” **This is a security requirement, not a convenience default:** distinguishing internal decoding-failure causes at an observable level (message or timing) would reconstruct a chosen-ciphertext padding-oracle attack. Consequently, E_{OAEP}^{SDK} must be designed deliberately *flatter* than E_{GCM}^{SDK} in the decryption-failure region – the four-class GCM model (with its adapter-driven pre-classification of *invalid_parameter* vs. *unsupported* vs. *malformed_artifact*) is not automatically transferable; API-misuse/contract-violation classification (rejected *before* the cryptographic operation runs) remains legitimate, but no sub-classification of decoding failure itself should be introduced.

12.1 Backend verification: WebCrypto (initial inventory)

Findings below are confirmed against the W3C Web Cryptography API Editor’s Draft algorithm steps (github.com/w3c/webcrypto) unless noted otherwise. Confidence is marked per item, following the same discipline as the AES-GCM verification: **[confirmed]** against primary spec/algorithm text; **[pending]** plausible but not yet independently verified to the same standard.

- **Hash/MGF1 coupling – [confirmed, primary algorithm text].** The `encrypt/decrypt` algorithm steps state the operation is performed “with... the hash function specified by the `RsaHashedKeyAlgorithm/hash` attribute of the `CryptoKey`’s internal slot as the Hash option and MGF1... as the MGF option.” A single hash attribute, stored on the key at generation/import time, drives both roles; no independent MGF1-hash parameter exists in `RsaOaepParams` or anywhere in the `encrypt/decrypt` call. Therefore $H_{OAEP} = H_{MGF1}$ for WebCrypto, structurally. **Minor citation note:** the WebCrypto algorithm steps reference RFC 3447 (obsoleted by, but textually identical in §7.1 to, RFC 8017) – worth preserving this exact traceability when citing WebCrypto’s own normative basis in the manuscript.
- **Label – [confirmed].** `RsaOaepParams.label` accepts arbitrary bytes; if omitted, treated as an empty sequence (*AAD*-style *absent* $\equiv$ *empty*, confirmed via MDN/spec). WebCrypto exposes a non-empty label even though RFC 8017 scopes that usage outside the document – a clean provider-capability-vs-normative-scope divergence, candidate R_{cap} material.
- **Message-too-long $\rightarrow$ `OperationError` – [confirmed, real-world corroboration].** Confirmed structurally by the generic `OperationError` pattern, and independently corroborated by a Mozilla Bugzilla thread (Bug 1235768) where an engineer encounters exactly this RSA-OAEP length-limit failure in Firefox and confirms no further diagnostic information is given by design.
- **OAEP decoding failures $\rightarrow$ `OperationError` – [confirmed].** All causes internal to RSAES-OAEP-DECRYPT collapse to `OperationError`, preserving the RFC’s anti-oracle collapse.
- **Randomness – [confirmed by API surface].** No parameter exists to supply or influence the internal OAEP seed; encryption is probabilistic and not externally controllable, consistent with the ecological-validity threat already declared for AES-GCM’s IV.
- **RSA modulus sizes – [confirmed structurally, no closed set].** `generateKey()` accepts an open `modulusLength`; the specification does not itself enumerate a closed portable set. A concrete WebCrypto implementation (browser/runtime version) will be required to determine what is actually accepted – consistent with the standing rule that WebCrypto is never treated as an abstract backend independent of a concrete runtime.

- **OAEP hash set** $\{SHA-1, SHA-256, SHA-384, SHA-512\}$ – **[pending]**. Widely and consistently reported across secondary/tutorial sources, but not yet pulled from primary normative enumeration text. Not to be used to freeze `oaep.hash`'s portable domain until confirmed to the same standard as the items above.
- **API-misuse error codes** (`InvalidAccessError`, `SyntaxError`, `DataError`) – **[pending]**. Plausible by analogy with WebCrypto's general parameter-validation pattern (confirmed for AES-family algorithms via W3C Bug 25741's explicit validation-error table), but no RSA-OAEP-specific entry was found in that table. Not to be used to construct E_{OAEP}^{SDK} until confirmed.
- **Serialization – dependency, not an OAEP-owned relation**. SPKI/PKCS#8/JWK import/export belongs to Key Serialization (a separate operation); OAEP only requires the imported key carry the correct hash and usages – same **Dep**-based treatment already established for AES-GCM/RSA relationships.

12.2 Backend verification: Crypto++ (complete)

Findings below are confirmed directly against Crypto++ source (`weidai11/cryptopp: oaep.h`, `oaep.cpp`, `pubkey.h`, `pubkey.cpp`), not documentation, except where noted.

- **Hash/MGF1 coupling** – **[confirmed, source-level, three escape routes checked and closed]**. The standard template is `OAEP<H, MGF=P1363_MGF1>`; the second parameter selects the mask-generation *algorithm family*, not a second digest. `OAEP_Base::Pad()/Unpad()` each instantiate a single `HashTransformation` via `NewHash()` and pass that same object to `MaskGeneratingFunction::GenerateAndMask()` for both applications of the mask. No alternative derived class of `OAEP_Base`, no alternative runtime configuration of `RSAPES<...>`, and no alternative OAEP API distinct from the standard template were found in the public source tree. **Precisely stated:** no independent MGF1 digest is exposed by Crypto++'s standard OAEP path, nor by any alternative route found; $H_{OAEP} = H_{MGF1}$ for Crypto++ as shipped. This corrects the initial working hypothesis (WebCrypto coupled, Crypto++/Bouncy Castle decoupled) – the coupling holds for *two* of three backends, pending Bouncy Castle.
- **Label/encoding parameters** – **[confirmed, source-level; absent-vs-explicit-empty equivalence yellow-green]**. `OAEP_Base::ParameterSupported()` returns true only for `Name::EncodingParameters()`. `Pad()/Unpad()` both retrieve a `ConstByteArrayParameter` `encodingParameters` and hash it into *DB* (functionally equivalent to RFC 8017's *L/pSource*). Architecturally, an absent parameter routes to zero effective bytes – the same channel-non-use pattern already confirmed for Crypto++ AES-GCM's AAD – but the exact behavior of `GetValue()` when the parameter is entirely absent has not been independently traced to the same certainty as `MinIVLength/ThrowIfInvalidTruncatedSize` were for GCM. Recorded as high-confidence, not maximum-confidence.
- **Message length bound** – **[confirmed, source-level]**. `OAEP_Base::MaxUnpaddedLength()` computes the bound from padded block size and `DigestSize()`; `TF_EncryptBase::Encrypt()` checks `plaintextLength > FixedMaxPlaintextLength()` *before* padding and throws `InvalidArgument` with a message stating both the received and maximum length. The bound is derived from the RSA modulus and OAEP digest size, not an arbitrary provider policy – consistent with RFC 8017's $mLen \leq k - 2hLen - 2$.
- **Ciphertext length** – **[confirmed]**. Fixed, derived from the RSA trapdoor function (`FixedCiphertextLength()`).
- **Decryption** – **two observable tiers, confirmed source-level** (`pubkey.cpp` lines 140–160). `TF_DecryptBase::Decrypt()` first checks `ciphertextLength != FixedCiphertextLength()`

and throws `InvalidArgument` with an explicit descriptive message *before* any RSA/OAEP processing. Only after this structural check passes does it invoke the trapdoor function and call `Unpad()`, which accumulates internal decode failures (missing separator, non-zero padding, incorrect `lHash`) into a single internal `invalid` flag and returns an opaque `DecodingResult` – no sub-cause is exposed. Notably, the integer-out-of-range case is handled by zeroing rather than early-returning, with the source comment “don’t return false here to prevent timing attack,” evidencing timing-attack awareness consistent with RFC 8017’s Bleichenbacher/Manger concern. **This is a genuine, real error-model divergence from WebCrypto:** WebCrypto collapses ciphertext-length mismatch and all OAEP-decode failures into one undifferentiated `OperationError`; Crypto++ exposes a structural pre-check (`InvalidArgument`) as observably distinct from the cryptographic decode failure itself (`DecodingResult`). This distinction occurs at the structural length-check stage that precedes OAEP decoding in RSAES-OAEP-DECRYPT; however, RFC 8017 ultimately specifies a generic *decryption error* outcome and explicitly warns against distinguishable failure behavior. The SDK’s error model must therefore decide conservatively whether this provider-level distinction can be exposed safely or should be normalized away. **Working position (not yet frozen):** treat this the same as GCM’s adapter-first classification – ciphertext-length mismatch can be legitimately classified before decryption is attempted (an *invalid_parameter/malformed_artifact*-class check), but no further sub-classification within the OAEP decode-failure region should be introduced, regardless of what any individual backend’s API allows.

- **Randomness – [confirmed].** `Pad()` takes an explicit `RandomNumberGenerator&` and calls `rng.GenerateBlock(maskedSeed, seedLen)` – the caller supplies the RNG object generating the OAEP seed, a capability WebCrypto does not expose at all. Per the ecological-validity threat already declared, this does not enter P_{OAEP}^{common} : $OAEP \text{ seed control} \in P_{Crypto++,OAEP} \setminus P_{OAEP}^{common}$, a clean provider-only capability extension, not a cross-provider conformance failure.
- **RSA sizes / OAEP hash set – [confirmed to have no closed enumeration; correcting an earlier framing].** Crypto++ exposes no closed whitelist analogous to a high-level API’s fixed set: hash selection is a compile-time template parameter (`OAEP<H>` over any `HashTransformation`-derived class), and key size derives from the RSA modulus actually constructed, not a declared list. Framing this as “Crypto++ supports $\{SHA-1, SHA-256, SHA-384, SHA-512\}$ ” (by analogy with WebCrypto) would misrepresent how the capability is actually expressed. The practical consequence for the experiment: confirm only that the specific hash(es) and key size(s) chosen for the portable profile (most likely SHA-256 at minimum) genuinely exist across all three backends, rather than attempting to enumerate Crypto++’s full OAEP capability surface.

WebCrypto vs. Crypto++ comparison:

Dimension	WebCrypto	Crypto++
OAEP hash MGF	bound to <code>CryptoKey</code> MGF1 (fixed)	compile-time template <code>H</code> MGF1 (configurable as a family, default <code>P1363_MGF1</code>)
Independent MGF1 digest Label Absent label	No arbitrary bytes empty (confirmed)	No arbitrary <code>EncodingParameters</code> architecturally empty (yellow-green)
OAEP seed	internal, uncontrollable	caller-supplied <code>RandomNumberGenerator</code>
Message too long Ciphertext length mismatch	<code>OperationError</code> folded into <code>OperationError</code>	<code>InvalidArgument</code> , descriptive <code>InvalidArgument</code> , descriptive, pre-decode
OAEP decode/label failure Cross-provider ciphertext bytes	<code>OperationError</code> not comparable (R_{byte} invalid)	opaque <code>DecodingResult</code> can be made locally deterministic via RNG injection, but not portably comparable

Methodological consequence. Crypto++ demonstrates that a provider offering *more* observability (caller-controlled RNG, a pre-decode length error distinct from decode failure) does not obligate the portable SDK contract to preserve that observability. Both remain legitimate provider capabilities that the adapter is free to not expose portably: RNG control → provider-only capability (not in $P_{\text{OAEP}}^{\text{portable}}$); detailed pre-decode error → candidate for normalization into the same coarse *mal-formed_artifact/invalid_parameter* treatment used for GCM, pending Bouncy Castle confirmation and final $E_{\text{OAEP}}^{\text{SDK}}$ design. Neither position is yet frozen.

12.3 Backend verification: Bouncy Castle (complete)

Findings below are confirmed directly against `bc-java` source (`OAPEncoding.java`, `main` branch), cross-corroborated where noted by real-world bug reports and BC’s own test suite.

- **Hash/MGF1 coupling – [confirmed, source-level, field and constructor].** `OAPEncoding` declares a distinct `private final Digest mgf1Hash` field alongside `hash`. The public constructor `OAPEncoding(cipher, hash, mgf1Hash, encodingParams)` accepts them as genuinely independent parameters, confirmed both in `bc-java` and `bc-csharp`. Official Javadoc (current release) lists this constructor explicitly; the 1.37-era Javadoc does not – **this capability was added in a later BC release**, a second, independent instance (after AES-GCM’s tag-length floor) of intra-provider capability drift across versions, reinforcing the mandatory version-pinning requirement. A BC maintainer’s own comment on a related `bc-csharp` issue confirms the design intent: “The BC one, in line with the recommended practice with OAEP, defaults to the same mask generation digest as the base one” – i.e. coupled by default, decoupled by explicit override. **This is Bouncy Castle’s sole confirmed hash/MGF1-independence capability among the three backends;** WebCrypto and Crypto++ are both structurally coupled.
- **Ciphertext-length failure vs. internal decode failure – [confirmed, source-level, architecturally distinct from Crypto++].** `decodeBlock()` does *not* branch early on length mismatch. It accumulates a `wrongMask` via arithmetic right-shift (`getOutputBlockSize()` » 31, then `(block.length - data.length)` » 31) – a constant-time masking idiom – and continues executing the identical code path regardless of whether the length was wrong, folding

the length check into the same constant-time flow used for internal OAEP-decode validation. This is a genuine architectural contrast with Crypto++, which throws an immediate, distinct `InvalidArgument` for ciphertext-length mismatch *before* any RSA/OAEP processing. **On this specific dimension, Bouncy Castle’s design is closer to RFC 8017’s anti-oracle spirit than Crypto++’s:** Bouncy Castle folds the structural length condition into the same constant-time failure path as internal OAEP validation, whereas Crypto++ exposes an early, structurally distinguishable provider-level outcome. RFC 8017 performs the length check before OAEP decoding but ultimately specifies the same generic *decryption error* outcome and warns against distinguishable failure behaviour. Real-world corroboration: bug report #1381 (bcgit/bc-java) shows the observable decrypt-failure exception as `InvalidCipherTextException: data wrong` originating in `OAPEncoding.decodeBlock` – a single generic message, not differentiated by internal cause.

- **Message too long (encrypt) – [confirmed, source-level].** `encodeBlock()` throws `DataLengthException("input data too long")` – a third distinct exception type, different from both WebCrypto’s `OperationError` and Crypto++’s `InvalidArgument`.
- **Label/encoding parameters, absent-vs-empty – [confirmed, source-level].** The constructor does: `if (encodingParams != null) { hash.update(encodingParams, 0, encodingParams.length); }` – no `else` branch. An absent (`null`) label contributes zero bytes to the hash, architecturally identical to an explicit empty label. This closes $AAD_{\text{absent}} \equiv AAD_{\emptyset}$ -style equivalence for Bouncy Castle’s OAEP label at the same confidence tier as the GCM AAD findings.
- **Randomness – [confirmed, source-level, with practical corroboration].** `init(boolean, CipherParameters)` extracts a caller-supplied `SecureRandom` when the parameter is a `ParametersWithRandom` instance. Bouncy Castle’s own test suite uses exactly this mechanism with a deterministic test RNG (`new ParametersWithRandom(pubParameters, new VecRand(seed))`) to produce reproducible OAEP known-answer test vectors – i.e. RNG injection is not merely possible in principle but is the project’s own mechanism for deterministic OAEP testing. Matches Crypto++’s capability; absent in WebCrypto. Per the ecological-validity threat already declared, this remains a provider-only capability extension, not part of P_{OAEP}^{common} .
- **Hash set / RSA sizes – [no closed enumeration, same discipline as Crypto++].** Hash selection is via runtime `Digest` objects (`SHA1Digest`, `SHA256Digest`, etc., BC’s own digest class hierarchy), not a fixed enum; RSA key size derives from the `RSAPublicKeyParameters` supplied, not a declared whitelist. As with Crypto++, no artificial closed list will be fabricated; the experiment will instead confirm that the specific candidate(s) chosen for the portable profile exist across all three backends.

12.4 Three-way OAEP comparison (P_{OAEP}^{common} candidate observations, still not frozen)

Dimension	WebCrypto	Crypto++	Bouncy Castle
OAEP hash $\leftrightarrow$ MGF1 hash	coupled	coupled	independently configurable (coupled by default)
Label/pSource	arbitrary bytes	arbitrary EncodingParameters	arbitrary encodingParams
Absent label	empty (confirmed)	architecturally empty (source-confirmed)	empty (source-confirmed, no else branch)
OAEP seed/RNG	internal, uncontrollable	caller-supplied RandomNumberGenerator	caller-supplied SecureRandom via ParametersWithRandom
Message too long	OperationError	InvalidArgument (descriptive)	DataLengthException (descriptive)
Ciphertext-length mismatch	folded into OperationError	InvalidArgument , early, structurally distinct from decode failure	folded into the same constant-time path as decode validation (wrongMask)
OAEP decode/label failure	OperationError	opaque DecodingResult	InvalidCipherTextException (generic message, e.g. “data wrong”)
Hash/key-size capability surface	closed set in practice (browser- dependent), open modulusLength	open (template/runtime HashTransformation , no enum)	open (runtime Digest objects, no enum)

Emerging picture, not yet a decision. All three backends converge on absent and explicitly empty OAEP labels having the same effective semantics, $L_{\text{absent}} \equiv L_{\emptyset}$ (mirroring, not contradicting, the opposite HKDF-salt convention – confirming again that null/empty semantics are operation-specific, never framework-universal). All three diverge in decrypt-failure exception type and structural granularity, in three genuinely different ways. These differences are likely to provide especially informative stimuli for evaluating the eventual OAEP error-model clauses and R_{err} , once the contract and Γ_0^{OAEP} are frozen. Bouncy Castle alone offers independent OAEP/MGF1 digest selection, a clean single-provider capability extension. Caller-controlled RNG is shared by Crypto++ and Bouncy Castle but absent from WebCrypto, meaning it cannot enter P_{OAEP}^{common} regardless of two-out-of-three support – consistent with the intersection rule (D-021) applied throughout. No **oaep.*** clauses, no C_{pre}^{OAEP} , and no Γ_0^{OAEP} are frozen yet.

12.5 Frozen E_{OAEP}^{SDK} error model

The first piece of C_{pre}^{OAEP} to be closed is the error model, since it conditions **oaep.validation**, **oaep.error**, and the eventual applicability/necessity of R_{err} before any other clause is written.

The `invalid_key` audit. Before adding a fourth class beyond the three-class GCM-inherited candidate (`{unsupported, invalid_parameter, decryption_error}`), the key-role obligation (encrypt $\Rightarrow$ public key, decrypt $\Rightarrow$ private key) was audited across all three backends specifically to test whether it constitutes a semantic obligation distinct from **Parameter**, rather than assumed by taxonomic convenience:

- **WebCrypto – runtime-enforced.** Confirmed against the current Editor’s Draft algorithm steps: algorithm-name mismatch and missing required `usages` both throw `InvalidAccessError`, *before* the algorithm-specific encrypt/decrypt steps execute. Public/private role mismatch is not confirmed verbatim for RSA-OAEP specifically, but is strongly evidenced by a real developer-reported issue (w3c/webcrypto #315: a generated RSA-OAEP private key only permits `decrypt`, the public key only `encrypt`, “regardless of the usages I specify”) and by the identical `[[type]]`-check pattern confirmed verbatim for newer WebCrypto algorithms (ML-KEM, Ed448).
- **Crypto++ – compile-time-enforced.** `RSAS<OAEP<H>::Encryptor` accepts only `RSA::PublicKey` in its constructor; `::Decryptor` accepts only `RSA::PrivateKey`. This is a C++ type-system constraint, not a runtime check – role misuse does not compile under normal use, and consequently can never surface as a runtime-observable error from this backend at all.
- **Bouncy Castle (low-level API: `OAEPEncoding`, `RSAEngine`, `RSAPublicKeyParameters`) – not enforced.** `RSAPublicKeyParameters` is a bare (`isPrivate`, `modulus`, `exponent`) tuple; `RSAPublicKeyParameters/RSAPublicKeyParameters` `init(forEncryption, param)` does not check that `isPrivate` is consistent with `forEncryption`. Confirmed by a real code example that initializes for encryption while passing `isPrivate=true` on public-key material without error, and by an open, unresolved bcgit/bc-java issue (#317) titled “How can I use this to encrypt with private key,” confirming this is a known, available usage pattern at this API surface, not a hypothetical edge case. **Reference-surface decision:** the low-level API (already used throughout this section’s audits, and required for `mgf1Hash/RNG/encodingParams` control) is retained as the audited BC surface; the higher-level JCA/JCE Cipher wrapper was checked only far enough to confirm it throws `InvalidKeyException` for unrecognized key *types*, with no confirmed evidence either way for role-based (encrypt-requires-public) validation specifically – not pursued further, as the low-level surface is the one already governing every other confirmed finding in this section, and **backend capability = provider + version + API surface** (an extension of the version-pinning principle, D-024/D-025, now confirmed to include *API surface* as a third axis alongside provider and version).

Decision: `invalid_key` is retained, but on a stronger basis than shared provider behavior – the three backends enforce the obligation through three genuinely different mechanisms (runtime, compile-time, not-at-all), which is precisely why the obligation must live in the abstract SDK contract rather than being inferred from any single provider’s error surface. Where a provider does not enforce it (Bouncy Castle, at the audited surface), the adapter must – a direct, concrete instance of `ObservedConformance = Provider + Adapter`, previously only a declared threat, now demonstrated as load-bearing for a specific contractual decision.

Frozen model:

$$E_{OAEP}^{SDK} = \{\text{unsupported}, \text{invalid_key}, \text{invalid_parameter}, \text{decryption_error}\}$$

with a fixed, pre-registered adapter classification order:

1. **Capability/manifest.** The requested operation or profile is not available per the frozen provider manifest $A_p^{pre} \rightarrow \text{unsupported}$.
2. **Key contract.** The key’s role does not match the operation (encrypt requires a public key, decrypt a private key) $\rightarrow \text{invalid_key}$. Checked before `RSAS-OAEP` is invoked, regardless of whether the specific backend would itself have caught it.
3. **Portable parameter/input contract.** Message exceeds $k - 2hLen - 2$; requested hash/MGF1 combination, key size, or label falls outside $P_{OAEP}^{portable} \rightarrow \text{invalid_parameter}$. Fully computable by the adapter in advance from modulus size, hash, and message length, without invoking the backend.

4. **RS_AES-OAEP-DECRYPT domain.** Once an input is admitted as an RSA-OAEP ciphertext and execution enters RSAES-OAEP-DECRYPT, **no provider-specific distinction is allowed to escape the adapter**: every failure governed by that procedure maps to *decryption_error*, with no further sub-classification, regardless of what any individual backend’s API exposes. Per RFC 8017 §7.1.2, this explicitly includes: ciphertext length $\neq k$; $k < 2hLen + 2$; RSA representative out of range; incorrect 1Hash; invalid padding/separator; label mismatch – all of which the RFC’s own pseudocode already labels with the identical generic outcome, not merely a design choice this framework adopts by analogy.

This order is frozen now, before any backend is executed against mutants, mirroring the discipline already applied to A_p^{pre} , Γ_0^{GCM} , and E_{GCM}^{SDK} (D-024, D-025). Note the key structural contrast with GCM: GCM’s four-class model was a design choice balancing adapter classification against R_{err} richness; OAEP’s collapse at stage 4 is not discretionary – it is the literal reading of RFC 8017’s own error pseudocode, motivated by the Bleichenbacher/Manger anti-oracle requirement already established in §12. R_{err} remains applicable to OAEP under this model but, as with GCM, must still earn detection/diagnostic gain in the necessity analysis (§23) to survive as an independent relation for this operation.

Illustrative classification (not exhaustive): backend does not support the requested profile $\rightarrow$ *unsupported*; encrypt attempted with a private key, or decrypt with a public key $\rightarrow$ *invalid_key*; message too long $\rightarrow$ *invalid_parameter*; Bouncy Castle’s $H_{OAEP} = \text{SHA-256}$, $H_{MGF1} = \text{SHA-1}$ requested when the portable profile requires them equal $\rightarrow$ *invalid_parameter* (same disambiguation principle as GCM’s $t = 80$ case, D-025); ciphertext of incorrect length presented at decrypt $\rightarrow$ *decryption_error*, *not invalid_parameter*, even though the adapter could in principle detect it in advance – deliberately not exploiting that foreknowledge, to avoid reconstructing exactly the observable distinction RFC 8017 prohibits; incorrect label at decrypt $\rightarrow$ *decryption_error*.

12.6 Frozen C_{pre}^{OAEP} fragment: OAEP digest coupling

This is a capability-intersection deduction, not a profile preference – it follows necessarily from evidence already confirmed, independent of which specific hash is later chosen for the portable profile.

Derivation. $P_{OAEP}^{\text{WebCrypto}}$ and $P_{OAEP}^{\text{Crypto++}}$ both expose *only* coupled digest selection ($H_{OAEP} = H_{MGF1}$, confirmed structurally for both – §12). Bouncy Castle additionally supports independent selection ($H_{OAEP} \neq H_{MGF1}$ permitted) but defaults to coupled. Since

$$P_{OAEP}^{\text{common}} = P_{OAEP}^{\text{WebCrypto}} \cap P_{OAEP}^{\text{Crypto++}} \cap P_{OAEP}^{\text{BC}},$$

independent digest selection cannot belong to P_{OAEP}^{common} regardless of what value is eventually chosen for the digest itself – two of three backends structurally cannot realize it. Combined with the already-frozen hierarchy $P_{OAEP}^{\text{portable}} \subseteq P_{OAEP}^{\text{common}}$ (§12), this closes automatically:

$$P_{OAEP}^{\text{portable}} \Rightarrow H_{OAEP} = H_{MGF1}.$$

Frozen decision (D-028). The common and portable RSA-OAEP profiles require the OAEP digest and the MGF1 digest to be identical. Bouncy Castle’s independent-digest capability (e.g. $H_{OAEP} = \text{SHA-256}$, $H_{MGF1} = \text{SHA-1}$) remains a valid, legitimate provider-only capability – it is not *unsupported* by Bouncy Castle – but lies outside P_{OAEP}^{common} and $P_{OAEP}^{\text{portable}}$. Consequently, a portable-SDK request for decoupled digests is classified *invalid_parameter* under E_{OAEP}^{SDK} (§12.5), *not unsupported* – the same disambiguation principle already applied to GCM’s $t = 80$ case (D-025): the request violates the portable contract’s boundary, not a genuine capability gap of the specific backend being asked.

This does not yet fix which digest is used – SHA-256, RSA modulus size, and the rest of the profile remain open – only that whichever digest is chosen, it is shared between both roles.

12.7 Frozen C_{pre}^{OAEP} fragment: label policy

This is the first case in the design where $P_o^{\text{portable}} \subsetneq P_o^{\text{common}}$ for **normative-scope reasons** (§12’s general-rule extension), not profile-selection reasons – distinct in kind from the digest-coupling case (D-028), which was capability-forced.

P_{OAEP}^{common} **technically includes non-empty labels**. All three backends support arbitrary label bytes and confirm $L_{\text{absent}} \equiv L_{\emptyset}$ (§12); non-empty labels are common *technical* capability.

Exact scope citation (required by the safeguard). RFC 8017 states, verbatim: “In this version of PKCS #1, L is the empty string; other uses of the label are outside the scope of this document.” This is not an absence of examples or an inference from provider silence – it is an explicit scope disclaimer in the normative text itself. Note the precise nuance: RFC 8017 is not silent on non-empty labels – Appendix A.2.1 defines the `pSourceAlgorithm/id-pSpecified` ASN.1 syntax that syntactically accommodates them – but the RFC body explicitly disclaims scope over their *semantic/operational use*. The exclusion rests on this disclaimer of behavioral scope, not on absence of syntax.

Frozen decision (D-030). $P_{OAEP}^{\text{portable}}.\text{label} = \{\emptyset\}$. Non-empty labels remain a valid, common *technical* capability across all three backends but are excluded from the portable profile: including them would assert cross-provider interoperability for a behavior RFC 8017 does not claim to govern, which the portable profile is not entitled to assert on the RFC’s authority alone. A portable-SDK request for a non-empty label is classified *invalid_parameter* under E_{OAEP}^{SDK} (§12.5) – not *unsupported*, following the same disambiguation already applied to GCM’s $t = 80$ and OAEP’s decoupled-digest case (D-025, D-028), and grounded in the normative-scope category just established (D-029).

12.8 Frozen C_{pre}^{OAEP} fragment: hash profile

Common capability, confirmed across all three backends. WebCrypto’s base specification supports exactly $\{SHA-1, SHA-256, SHA-384, SHA-512\}$ as digest algorithms (converging secondary-source evidence plus Node.js’s own compliance documentation, which explicitly distinguishes these base-spec algorithms from SHA-3/ML-KEM as separate, not-yet-standardized WICG extensions – §12). Crypto++ confirms all four as available hash classes (`SHA1`, `SHA224`, `SHA256`, `SHA384`, `SHA512` per `cryptlib.h`), with `OAEP<H>` generic over any of them and dedicated `RSAES_OAEP_SHA256_Encryptor/Decryptor` typedefs since v8.7/8.8. Bouncy Castle’s `OAEPParameterSpec/MGF1P` confirm the same four as standard digest options. No candidate needed to be invented; all four intersect genuinely:

$$P_{OAEP}^{\text{common}}.\text{hash} = \{SHA-1, SHA-256, SHA-384, SHA-512\}$$

subject to the already-frozen $H_{OAEP} = H_{MGF1}$ constraint (D-028).

Frozen decision (D-031). $P_{OAEP}^{\text{portable}}.\text{hash} = \{SHA-256\}$, classified as **profile-selection** (§12’s D-029 category, not normative-scope): SHA-256 sits within the common capability of all three backends, is fully covered by RFC 8017/PKCS #1, and no framework dimension requires exercising SHA-384/512 or (a deliberately avoided) SHA-1 in the portable profile. SHA-1, SHA-384, and SHA-512 remain valid P_{OAEP}^{common} capability, explicitly *not* declared *unsupported*; a portable-SDK request for e.g. OAEP-SHA512 classifies *invalid_parameter* under E_{OAEP}^{SDK} (§12.5), on the same disambiguation basis as GCM’s $t = 80$, OAEP’s decoupled-digest case, and OAEP’s non-empty label (D-025, D-028, D-030).

12.9 Frozen C_{pre}^{OAEP} fragment: RSA modulus and derived message-length boundary

Capability membership, not a closed common set. Unlike the hash case, $P_{OAEP}^{common}.modulus$ is not asserted as a closed enumeration – all three backends expose modulus size through an open parameter (`modulusLength` in WebCrypto’s `generateKey()`, `GenerateRandomWithKeySize()` in Crypto++, an integer `strength` in Bouncy Castle’s RSA key generator), none imposing a fixed whitelist. What is established, and sufficient for profile-selection, is membership: $3072 \in P_{OAEP}^{common}.modulus$, confirmed by direct usage in all three backends’ own documentation/examples (Crypto++ notably uses 3072 in its own official key-size guidance, not merely as a possible value).

Frozen decision (D-032). $P_{OAEP}^{portable}.modulus = \{3072 \text{ bits}\}$, classified as **profile-selection** (D-029), on two converging grounds:

1. **Security-level alignment with the frozen hash.** Per NIST SP 800-57 Part 1’s security-strength table, RSA-2048 provides approximately 112 bits of security and RSA-3072 approximately 128 bits; SHA-256 (D-031) supports up to 128 bits. RSA-2048 combined with SHA-256 would be bottlenecked by the RSA component at 112 bits; RSA-3072 aligns both components at approximately 128 bits.
2. **Forward-looking normative alignment, not arbitrary preference.** NIST SP 800-131Ar3 discourages signature/encryption operations below 128-bit security after 31 December 2030, requiring $len(n) \geq 3072$ for that tier – the same style of argument already used for AES-GCM’s $t = 128$ (SP 800-38D’s active revision, D-022): the portable profile is chosen to align with where the relevant NIST guidance is heading, not merely with today’s minimum floor. Crypto++’s own wiki states this almost verbatim: “current best practices (from NIST) dictate we use a security level of 128 ... this means we should use an RSA modulus size of 3072 bits.”

RSA-2048 and RSA-4096 remain valid P_{OAEP}^{common} capability, not *unsupported*; a portable-SDK request for a different modulus size classifies *invalid_parameter* under E_{OAEP}^{SDK} (§12.5), on the same disambiguation basis as the prior four instances (D-025, D-028, D-030, D-031).

Derived (not chosen): maximum plaintext length. With $k = 3072/8 = 384$ bytes and $hLen = 32$ bytes (SHA-256):

$$mLen_{max} = k - 2hLen - 2 = 384 - 64 - 2 = 318 \text{ bytes.}$$

This is a direct consequence of D-031 and D-032, not an independent decision – it follows the same discipline already applied to AES-GCM (where the artifact structure was derived from the frozen IV/tag lengths, not chosen separately). It gives `oaep.validation/oaep.message` an exact, falsifiable boundary for future mutation stimuli: 317 bytes valid, 318 bytes valid, 319 bytes $\rightarrow$ *invalid_parameter*.

12.10 Frozen C_{pre}^{OAEP} fragment: randomness

Capability asymmetry, already established. Crypto++ (`RandomNumberGenerator&` passed to `Pad()`) and Bouncy Castle (`SecureRandom` via `ParametersWithRandom`) both accept caller-supplied randomness for the OAEP seed (§12); WebCrypto exposes no such parameter at all. By the intersection rule (D-021), external seed/RNG control cannot belong to P_{OAEP}^{common} , and therefore cannot belong to $P_{OAEP}^{portable}$ either.

Frozen decision (D-033). The portable RSA-OAEP profile does not expose the OAEP seed or an externally supplied RNG; the abstract encryption operation is conceptually `encrypt(publicKey, plaintext)`, with no seed/RNG parameter. Crypto++ and Bouncy Castle’s RNG-injection facilities are recorded as genuine provider capabilities (`gcm.cap`-style manifest entries: `external_rng_control = true`

for Crypto++/Bouncy Castle, `false` for WebCrypto) but are not offered portably – the same $P_p \neq P_{\text{common}} \neq P_{\text{portable}}$ pattern already established for AES-GCM’s tag length and OAEP’s digest coupling/label.

Consequence for R_{byte} . OAEP encryption remains probabilistic under the portable contract: $\text{Enc}(K, M) \neq \text{Enc}(K, M)$ across repeated calls, even within a single backend, is expected and correct. R_{byte} is therefore not applicable to the OAEP ciphertext as a normal cross-provider oracle – R_{interop} ($\text{Dec}_q(\text{Enc}_p(M)) = M$, both directions) remains the operative relation, as already established in §12. This gives the paper one of its cleanest illustrations of why cryptographic conformance cannot be reduced to output equality.

Deterministic RNG use is permitted only as internal instrumentation, never as portable evidence. A deterministic RNG may legitimately be used within Crypto++ or Bouncy Castle individually for adapter unit tests, encoding validation, or reproducing a specific execution – but seeding the same value across backends to compare ciphertext bytes would test a surface WebCrypto does not offer, and any such result must be documented as *provider-specific instrumentation*, never as evidence of portable equivalence.

Scope boundary, reaffirmed. Consistent with `Parameter.ExternalRandomnessControl`’s frozen definition (§5) and the ecological-validity threat (§7.2), this decision does not evaluate CSPRNG quality, entropy source, reseeding, post-restart behavior, or hardware RNG for any backend. The claim is limited to: OAEP encryption is probabilistic in the portable contract; provider-specific facilities for externally supplying the randomness source are recorded as capabilities but are not exposed by the portable profile.

12.11 Frozen $C_{\text{pre}}^{\text{OAEP}}$ fragment: ciphertext representation

No artificial wrapper is introduced. Unlike AES-GCM, where R_{ser} became applicable only because the SDK contract chose to define an explicit portable `AEADArtifact` (D-014), RSA-OAEP is not given an equivalent wrapper here – and for a reason stronger than convenience. RFC 8017 fixes the ciphertext at exactly k octets via the I2OSP (integer-to-octet-string) encoding, a specific, unambiguous, big-endian, fixed-length representation with no interpretive freedom – unlike GCM’s $C\|T$ vs. $T\|C$, which was a genuine representational choice different backends could plausibly make differently. There is no representational degree of freedom here for R_{ser} to meaningfully capture.

Frozen decision (D-034). The portable RSA-OAEP contract uses the raw, fixed-length RSAES-OAEP ciphertext as the operation’s output, with no additional SDK-level wrapper: $C \in \{0, 1\}^{3072}$, i.e. $|C| = 384$ bytes under the frozen RSA-3072 profile (D-032). No `oaep.artifact` clause of kind `Encoding` is introduced, and R_{ser} is not applicable to RSA-OAEP. RSA key serialization (SPKI/PKCS#8) remains entirely owned by the separate, dependent Key Serialization operation (Dep, §4.2) and is not inherited by OAEP. A ciphertext presented at decrypt with length $\neq 384$ bytes is not a new *malformed_artifact* category – it falls under the already-frozen stage-4 collapse of $E_{\text{OAEP}}^{\text{SDK}}$ (D-027): ciphertext length $\neq k$ is explicitly one of RFC 8017 §7.1.2’s own enumerated causes of the generic *decryption_error*, requiring no new machinery.

Coverage note (parallel to AES-GCM/Membership, §11). Because R_{ser} has no instantiation in OAEP, `Encoding` is expected to be absent from $\text{Coverage}_{K_1}(\text{RSA-OAEP})$ once computed – mirroring how AES-GCM lacked `Membership` coverage until EC supplied it. Here, `Encoding` is expected to be supplied by RSA Key Serialization as a dependent operation, not by OAEP itself. This is a candidate cross-check for the eventual necessity analysis (§23), not a decision made now.

$C_{\text{pre}}^{\text{OAEP}}$ is now fully frozen across all six fragments: error model (D-027), digest coupling (D-028), label (D-030), hash (D-031), modulus/message-length (D-032), randomness (D-033), and ciphertext representation (D-034). The next steps are writing the `oaep.*` clause table (K1/K2, mirroring HKDF/AES-GCM’s format), deriving OAEP’s causal mutation classes, constructing and auditing

Γ_0^{OAEP} , and a final redundancy/circularity pass – the same sequence that closed AES-GCM.

13 RSA-OAEP – frozen contract clauses

Thirteen clauses, each a stable contractual obligation (D-016) rather than a mutation or test, derived directly from C_{pre}^{OAEP} (§12–§12.11). Three clause pairs were deliberately kept separate rather than merged, since each pair answers a causally distinct question: `oaep.key`/`oaep.modulus` (key role vs. key size), `oaep.hash`/`oaep.mgfCoupling` (digest choice vs. digest coupling), and `oaep.ciphertext`/`oaep.ciphertextLength` (cryptographic correctness of the output vs. its I2OSP-encoded length – added by D-037 after auditing revealed both can fail independently: a mathematically correct RSA integer can be encoded with missing zero-padding, and a structurally correct 384-octet string can carry an incorrect ciphertext). Collapsing any of these three pairs would have degraded future diagnostic precision – the same discipline that motivated separating GCM-AAD-ABSENT-EMPTY-DIVERGENCE from GCM-AAD-IGNORED. `oaep.label` is classified **Input**, not **Parameter**, for direct structural comparability with `gcm.aad`: both are opaque, caller-supplied, message-associated byte sequences consumed by the algorithm, not algorithm configuration choices – required for $\text{Coverage}_\kappa/\text{Marginal}_\kappa$ comparisons across operations to mean the same thing.

ID	K1	K2	Frozen contract clause
<code>oaep.key</code>	Key	—	Encryption uses an RSA public key; decryption uses the corresponding RSA private key.
<code>oaep.modulus</code>	Parameter	Length	Portable OAEP fixes the RSA modulus length at 3072 bits; other supported modulus lengths lie outside the portable profile.
<code>oaep.message</code>	Input	—	Plaintext is an opaque byte sequence without normalization and must lie within the frozen RSA-OAEP input domain, $0 \leq mLen \leq 318$ octets.
<code>oaep.ciphertext</code>	Output	—	Successful encryption produces a valid RSAES-OAEP ciphertext for the declared plaintext, key, and frozen OAEP parameters.
<code>oaep.ciphertextLength</code>	Output	—	Under the frozen RSA-3072 profile, every successful RSAES-OAEP ciphertext consists of exactly $k = 384$ octets.
<code>oaep.hash</code>	Parameter	AlgorithmChoice	Portable OAEP uses SHA-256 as the OAEP digest.
<code>oaep.mgfCoupling</code>	Parameter	DomainParameter	Portable OAEP requires $H_{\text{OAEP}} = H_{\text{MGF1}}$; independent MGF1 digest selection remains a provider-specific capability outside the portable profile.
<code>oaep.label</code>	Input	—	L is an opaque byte sequence associated with the message; the portable profile fixes $L = \emptyset$.

ID	K1	K2	Frozen contract clause
<code>oaep.randomness</code>	Parameter	ExternalRandomnessControl	The portable contract exposes neither the OAEP seed nor a caller-supplied RNG; provider-specific external randomness control remains capability-only.
<code>oaep.validation</code>	Validation	Semantic	Pre-OAEP requests outside the portable contractual domain (key role, modulus, hash, label, message length) are rejected according to the frozen classification boundary.
<code>oaep.error</code>	Error	—	Observable failures map to the frozen four-class SDK error model $E_{\text{OAEP}}^{\text{SDK}} = \{\text{unsupported}, \text{invalid_key}, \text{invalid_parameter}\}$ according to the frozen classification order; all failures governed by RSAES-OAEP-DECRYPT collapse to <code>decryption_error</code> .
<code>oaep.cap.provider</code>	Capability	—	Observed backend behaviour matches the frozen provider/version/API-surface capability manifest A_p^{pre} .
<code>oaep.cap.portable</code>	Capability	—	The SDK exposes only $P_{\text{OAEP}}^{\text{portable}} \subseteq P_{\text{OAEP}}^{\text{common}} \subseteq P_{p,\text{OAEP}}$; no unlabeled provider extension leaks through the adapter.

On the absence of `oaep.messageLength`. The 318-octet message bound is retained within `oaep.message` rather than represented as a separate `Parameter.Length` clause. Unlike HKDF’s output length L , which is an independently selected algorithm parameter the caller chooses, OAEP’s message length is not a configuration choice – it is a property of the input byte sequence, whose admissible upper bound is derived from the already-frozen modulus and digest profile. This distinction is pre-registered now to avoid fabricating a clause solely to anticipate later boundary mutations (D-016).

Coverage. $\text{Coverage}_{K_1}(\text{RSA-OAEP}) = \{\text{Key}, \text{Parameter}, \text{Input}, \text{Output}, \text{Validation}, \text{Error}, \text{Capability}\}$. **Authentication, Encoding, and Membership** are absent by contractual inapplicability, not omission: OAEP has no authentication tag (unlike GCM), no independent portable serialization layer (§12.11), and no structural membership obligation of its own (RSA has no elliptic-curve-style point validation). This is consistent with the general rule that a relation or K_1 kind belongs to an operation only when that operation’s own contract exposes the corresponding obligation.

With $C_{\text{pre}}^{\text{OAEP}}$ and the clause table both frozen, the remaining work before OAEP closes was constructing OAEP’s causal mutation classes and Γ_0^{OAEP} , applying from the outset the discipline already learned from AES-GCM (causality before detection; mutation distinct from stimulus; redundant classes eliminated before, not after, freezing).

14 RSA-OAEP – frozen Γ_0^{OAEP}

Twelve candidate causal classes were audited one at a time against the frozen `oaep.*` clause table (§13), following exactly the same causality-first discipline established for GCM: $\Gamma_0(c)$ records only clauses a defect could plausibly violate, independent of which relation is expected to detect it.

Two classes did not survive this audit as originally conceived, and one clause discovered mid-audit (`oaep.ciphertextLength`, D-037) required a new class to give it any causal coverage at all.

Class-count reconciliation. 12 original candidates -1 (OAEP-PORTABLE-CAPABILITY-LEAK, eliminated as causally redundant with five more specific classes) $+1$ (OAEP-CIPHERTEXT-LENGTH-DIVERGENCE, added to cover `oaep.ciphertextLength`, otherwise the only one of thirteen clauses with no Γ_0 coverage) = 12 final classes.

Family A – Computation/semantic divergence (detectable primarily via R_{interop} , since R_{byte} does not apply to OAEP’s probabilistic ciphertext, D-033):

Mutation class	$\Gamma_0(c)$	Kinds ₀ (c)	Expected relations
OAEP-MESSAGE-NORMALIZATION	<code>oaep.message;</code> <code>oaep.ciphertext</code>	Input; Output	R_{interop}
OAEP-CIPHERTEXT-COMPUTATION-DIVERGENCE	<code>oaep.ciphertext</code>	Output	R_{interop}
OAEP-CIPHERTEXT-LENGTH-DIVERGENCE	<code>oaep.ciphertextLength</code>	Output	R_{interop} . Stimulus deliberately targets the I2OSP zero-padding case (an RSA integer whose natural encoding is shorter than k octets), not an arbitrary short ciphertext – the specific cause D-037 separated from <code>oaep.ciphertext</code> .

Family B – Profile/capability leakage bypass (a portable-excluded but genuinely provider-supported value is silently accepted; domain clause + validation + `cap.portable`):

Mutation class	$\Gamma_0(c)$	Kinds ₀ (c)	Expected relations
OAEP-MODULUS-PROFILE-BYPASS	<code>oaep.modulus;</code> <code>oaep.validation;</code> <code>oaep.cap.portable</code>	Parameter.Length; Validation.Semantic; Capability	R_{val} ; R_{cap} . Stimulus: RSA-2048/4096 – provider-valid, non-portable.

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
OAEP-HASH- PROFILE-BYPASS	oaep.hash; oaep.validation; oaep.cap.portable	Parameter.AlgorithmChoice; Validation.Semantic; Capa- bility	R_{val} ; R_{cap} . Stimulus: SHA- 512 (coupled, i.e. $H_{\text{OAEP}} =$ $H_{\text{MGF1}} = \text{SHA-}$ 512) – isolates hash choice from coupling.
OAEP-MGF- COUPLING-BYPASS	oaep.mgfCoupling; oaep.validation; oaep.cap.portable	Parameter.DomainParameter; Validation.Semantic; Capa- bility	R_{val} ; R_{cap} . Stimulus: $H_{\text{OAEP}} = \text{SHA-}$ 256, $H_{\text{MGF1}} = \text{SHA-1}$ – isolates coupling from hash choice. Materially ex- ecutable only against Bouncy Castle, the sole backend with a native API surface for de- coupled digests; this does not alter Γ_0 or class identity (see methodological note below).
OAEP-LABEL- PROFILE-BYPASS	oaep.label; oaep.validation; oaep.cap.portable	Input; Validation.Semantic; Capability	R_{val} ; R_{cap} . Stimulus: non- empty L (e.g. $L = 0x01$).

Family C – Intrinsic/derived or semantic-role boundary bypass (domain clause + validation only, *no cap.portable*: the boundary is either a mathematical consequence identical across all three backends, or a structural role obligation with no associated capability space at all – see methodological note below):

Mutation class	$\Gamma_0(c)$	$\mathbf{Kinds}_0(c)$	Expected relations
OAEP-MESSAGE-BOUNDARY-BYPASS	oaep.message; oaep.validation	Input; Validation.Semantic	R_{val} . Stimulus: $mLen = 319$ (the exact $mLen_{\text{max}} + 1$ boundary under the frozen RSA-3072/SHA-256 profile), not an arbitrarily large message.
OAEP-KEY-ROLE-BYPASS	oaep.key; oaep.validation	Key; Validation.Semantic	R_{val} . Stimuli: encrypt-with-private-key, decrypt-with-public-key – two instances of one causal class, not two.

Family D – Interface capability leak (`cap.portable` alone, no validation obligation – the defect is in the portable interface’s own surface, not in accepting an out-of-domain input value):

Mutation class	$\Gamma_0(c)$	$\mathbf{Kinds}_0(c)$	Expected relations
OAEP-RANDOMNESS-INTERFACE-LEAK	oaep.randomness; oaep.cap.portable	Parameter.ExternalRandomness; Capability	R_{cap} . Control; Likely verified as an API-surface property rather than a per-request runtime outcome (see methodological note below). Materially executable only against Crypto++/Bouncy Castle.

Family E – Error/capability (R_{err} or R_{cap} in isolation; no domain or validation clause):

Mutation class	$\Gamma_0(c)$	$\mathbf{Kinds}_0(c)$	Expected relations
OAEP-DECRYPT- ERROR- DISCLOSURE	oaep.error	Error	R_{err} . The adapter exposes distinguish-able internal RSAES-OAEP-DECRYPT failure causes (ci-phertext length, lHash mismatch, padding, RSA-representative range) instead of collapsing all of them to <code>decryption_error</code> . $R_{\text{val}} = 1$ throughout – the ciphertext is correctly re-jected; the defect is disclosing <i>why</i> . Multiple internal-failure stimuli exercise one class, not several. Candidate for later merging with a general <code>OAEP-ERROR-MISCLASSIFICATION</code> class, pending necessity analy-sis (§23) – not decided now.

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
OAEP-PROVIDER-CAPABILITY-MISREPORT	oaep.cap.provider	Capability	R_{cap} . Detectable by comparing declared vs. observed provider+version+API-surface capability alone, without executing any OAEP operation; $R_{\text{val}} = R_{\text{err}} = R_{\text{interop}} = 1$ can coexist with $R_{\text{cap}} = 0$. Structurally identical to the now-corrected GCM-PROVIDER-CAPABILITY- (D-038).

Methodological note 1 – three structurally distinct reasons for a boundary bypass. This audit surfaced a taxonomy that determines whether `cap.portable` belongs in a boundary-bypass class’s Γ_0 , orthogonal to *why* the excluded value is excluded (profile-selection, normative-scope, or capability-forced, D-029/§12.6):

- **Profile/capability leakage** (Family B): a genuine capability dimension exists with real cross-provider variation (some backends support the excluded value, at least one does not) – `cap.portable` applies, since something a provider actually offers is escaping the portable boundary.
- **Intrinsic/derived boundary** (OAEP-MESSAGE-BOUNDARY-BYPASS): the boundary is a mathematical consequence of already-frozen parameters ($mLen_{\text{max}} = k - 2hLen - 2$), identical for all three backends under the same profile – no capability variation exists to leak, so `cap.portable` is inapplicable by construction, not by omission.
- **Semantic-role boundary** (OAEP-KEY-ROLE-BYPASS): the obligation (encrypt $\Rightarrow$ public, decrypt $\Rightarrow$ private) is not a configuration axis of RSAES-OAEP at all – there is no capability space to intersect, matching why this obligation required its own error class (`invalid_key`, D-027) rather than fitting inside `invalid_parameter`.

The diagnostic question is always the same: *does a real capability dimension exist here, with observable cross-provider variation, under the frozen profile?* If yes, `cap.portable` applies; if the boundary is instead a shared mathematical consequence or a structural precondition outside OAEP’s parameter space, it does not.

Methodological note 2 – stimulus executability is backend-specific; class identity and Γ_0 are not. Several classes (OAEP-MGF-COUPPLING-BYPASS, OAEP-RANDOMNESS-INTERFACE-LEAK, and the multi-backend-sourced stimuli of OAEP-DECRYPT-ERROR-DISCLOSURE) can only be materially exercised against a subset of the three backends, because only those backends expose the underlying capability being tested against the portable boundary. This is a property of stimulus design and

execution planning, not of the causal class: the contractual obligation being tested is identical and portable-wide regardless of which backend can concretely attempt to violate it.

Coverage check. All thirteen `oaep.*` clauses (§13) now have Γ_0 coverage, closing the gap identified when auditing `oaep.ciphertextLength` against the original twelve candidates. Coverage $_{K_1}$ (RSA-OAEP) over these twelve classes spans {Key, Parameter, Input, Output, Validation, Error, Capability} – unchanged from the clause-level coverage already established (§13), as expected: mutation classes exercise clauses, they do not introduce new K_1 kinds beyond what the clause table already defines.

With Γ_0^{OAEP} frozen, RSA-OAEP is methodologically closed to the same standard as AES-GCM: normative fiche, three-backend inventory, C_{pre}^{OAEP} , frozen clauses, and audited Γ_0 . The final mechanical pass over the whole document (D-XXX references, 13/12 counts, clause/class names, Kinds $_0$, expected relations, and residual text describing an earlier intermediate state) has been completed, closing v0.4 as a checkpoint with HKDF, AES-GCM, and RSA-OAEP all methodologically closed. The next scientific work is RSA-PSS, not further OAEP revision – unless a later operation surfaces another cross-cutting contradiction, as happened twice with GCM (D-036, D-038).

15 RSA-PSS – normative fiche (RFC 8017 §8.1/§9.1)

Status: normative fiche closed; three-backend inventory of the four geometry-determining dimensions closed (WebCrypto, Crypto++, Bouncy Castle). Secondary dimensions (exact sign/verify error taxonomy, message-length limits, capability manifest specifics) remain open. No `pss.*` clauses, no $P_{PSS}^{common}/P_{PSS}^{portable}$, no C_{pre}^{PSS} , and no Γ_0^{PSS} are frozen yet.

Scheme. RSASSA-PSS combines RSASP1/RSAPV1 with EMSA-PSS encoding (§9.1). Parameters: Hash, MGF (typically MGF1), and *sLen* (salt length, octets) – unlike OAEP’s internally fixed *hLen*-byte seed, *sLen* is itself a scheme parameter. Constraint: $emBits \geq 8hLen + 8sLen + 9$ (equivalently $emLen \geq hLen + sLen + 2$). Salt is randomly generated of length *sLen* (empty if *sLen* = 0) – signature generation is probabilistic, so R_{byte} is not a valid oracle for the signature value itself; $R_{interop}$ (verify-what-another-signed, both directions) is the natural relation, materializing here as *signature verification* rather than decrypt. Sign-side errors: “message too long” (hash input limit, practically unreachable) and “encoding error” (*emLen* too small for the chosen hash+salt+modulus combination). Verify-side output: abstract “consistent”/“inconsistent” verdict.

Critical divergence from OAEP’s anti-oracle framing. Unlike OAEP’s decryption error (explicitly justified in RFC 8017 by citing Bleichenbacher/Manger), **no equivalent security rationale was found for collapsing PSS verification sub-causes.** RFC 8017 frames “consistent”/“inconsistent” as simply what a verification operation naturally returns, not as a stated anti-oracle requirement. Real-world evidence a PSS error model may legitimately be richer than OAEP’s: W3C WebCrypto Bug 27603 documents that the spec itself is *inconsistent across signature algorithms* in how verify failures are reported (ECDSA collapses all failures to **false**; RSASSA-PKCS1-v1_5 rejects with **OperationError** “if performing the operation results in an error”), with the reporter explicitly uncertain whether real libraries distinguish RSA-PSS failures uniformly. E_{PSS}^{SDK} **must therefore be discovered independently, not cloned from E_{OAEP}^{SDK} .**

15.1 Three-backend inventory: four geometry-determining dimensions

Following the discipline that determined OAEP’s shape (resolve dimensions capable of changing the intersection’s geometry before completing the full ten-question inventory), four questions were prioritized: (1) *sLen* configurability and domain; (2) H_{PSS} vs. H_{MGF1} coupling; (3) RNG-to-salt determinism; (4) verify-failure behavior.

WebCrypto. Confirmed against `RsaPssParams` and the W3C Editor’s Draft algorithm steps

(github.com/w3c/webcrypto): `saltLength` is a *required, per-operation* member – `sLen` is genuinely runtime-selectable, unlike OAEP’s fixed internal seed. Maximum domain confirmed via MDN’s exact formula, matching RFC 8017’s own bound: $sLen_{\max} = \lceil (keySizeInBits - 1)/8 \rceil - digestSizeInBytes - 2$ (e.g. 222 for a 2048-bit key with a 32-byte digest). Hash is bound to the `CryptoKey` exactly as in OAEP (`RsaHashedKeyAlgorithm/hash`, single attribute driving both Hash and MGF1) – $H_{PSS} = H_{MGF1}$, structurally coupled. No salt-bytes or RNG control exposed. Key role: `sign` requires "private", `verify` requires "public", both throwing `InvalidAccessError` on mismatch (confirmed verbatim in algorithm steps – no OAEP-style evidentiary gap here). Sign failure: `OperationError`. **Verify failure: pure boolean** – confirmed via MDN (“fulfills with a boolean value: true if the signature is valid, false otherwise”) and the official example code (`result ? "valid" : "invalid"`), with no exception-based collapse for the ordinary invalid-signature case.

Crypto++. Confirmed against `pssr.h/pssr.cpp` (`weidai11/cryptopp`). `sLen` is a **compile-time template parameter** (`template <bool ALLOW_RECOVERY, class MGF=P1363_MGF1, int SALT_LEN=-1, ...> class PSSR_MEM`), not runtime-configurable; the standard PSS type (`PSSR_MEM<false>`) defaults to `SALT_LEN=-1` → `SaltLen() = hashLen`, i.e. $sLen = hLen$ fixed by default. This is a documented, unresolved limitation of the library: GitHub issue `weidai11/cryptopp#1121` (2022) states plainly that runtime salt-length selection is not supported through this API, referencing an identical unanswered question dating to 2005. $H_{PSS} = H_{MGF1}$: confirmed via `pssr.cpp`, `GetMGF().GenerateAndMask(hash, ...)` – the single `hash` object is passed to MGF1, same architecture as OAEP’s `oaep.cpp`. RNG-to-salt: high confidence via consistent `RandomNumberGenerator&` threading through `SignMessage(rng, ...)/SignerFilter(rng, ...)`, but the literal salt-generation call was not independently pinpointed to the same precision as OAEP’s `rng.GenerateBlock`.

Bouncy Castle. Confirmed against `PSSSigner.java` (`bcgit/bc-java`, current + version history). `sLen` is a **genuine runtime constructor parameter**: `public PSSSigner(AsymmetricBlockCipher cipher, Digest digest, int sLen)`. H_{PSS} and H_{MGF1} **independently configurable**, confirmed via source (present since API 1.48): `PSSSigner(cipher, Digest contentDigest, Digest mgfDigest, int sLen, byte trailer)`, with separate `hLen/mgfhLen` fields – the same decoupling pattern as `OAEP_Encoding`, now confirmed for a second, independently-implemented PSS class, not a coincidence specific to OAEP. RNG-to-salt: confirmed with full literal precision via `generateSignature(): if (sLen != 0) { random.nextBytes(salt); ...}` – matching OAEP’s evidentiary tier exactly. Verify: `boolean verifySignature(byte[] signature)`, no checked exception declared (matching WebCrypto’s boolean-verdict pattern); internal unchecked-exception behavior for malformed inputs not independently confirmed.

Additional Bouncy Castle capability, beyond the three-way comparison: newer API versions (documented from 1.84) add constructors accepting an *explicit* `byte[] salt` directly, not merely its length: `PSSSigner(cipher, Digest digest, byte[] salt)`. This is a capability tier beyond RNG injection – three genuinely distinct randomness-related capabilities exist, not one: `saltLength` control $\neq$ external RNG control $\neq$ explicit salt bytes control. The 1.33/1.37 API exposed none of the `mgfDigest`-separated or `byte[]`-salt constructors – a **third independent instance** of Bouncy Castle capability drift across versions (after AES-GCM’s tag-length floor and OAEP’s digest independence), reinforcing `backend capability = provider + version + API surface` as an empirically repeated regularity, not a one-off precaution.

Bouncy Castle message-length/encoding-error check, confirmed at `init()` time. `PSSSigner.init()` computes `emBits = kParam.getModulus().bitLength() - 1` and checks if `(emBits < 8*hLen + 8*sLen + 9)` throw `new IllegalArgumentException("key too small for specified hash and salt lengths")` – matching RFC 8017 §9.1.1’s $emBits \geq 8hLen + 8sLen + 9$ constraint exactly, thrown cleanly before any signing occurs. This is Bouncy Castle’s analogue of RFC 8017’s “encoding error,” occurring at initialization rather than at sign time.

15.2 Emerging geometry (not yet frozen)

Dimension	WebCrypto	Crypto++	Bouncy Castle
$sLen$	runtime, $[0, sLen_{\max}]$ per RFC formula	compile-time template constant , default $hLen$	runtime, constructor <code>int</code>
$H_{PSS} \leftrightarrow H_{MGF1}$	coupled	coupled	independently configurable
Salt bytes / RNG	neither exposed	RNG injectable (high confidence)	RNG injectable (confirmed) <i>and</i> explicit salt bytes (recent versions)
Key role (sign/verify)	<code>InvalidAccessError</code>	compile-time, confirmed: <code>RSASS<PSS,H>::Signer(PSSSignatureKey(k))</code> same type-system pattern as OAEP	confirmed unenforced: <code>RSASSignerKey(k)::Verifier(RSASignatureKey(k))</code> extracts <code>RSAPublicKeyParameters</code> only to compute <code>emBits</code> , never checks <code>isPrivate()</code> against <code>forSigning</code> – matches OAEP’s <code>RSAPublicKeyParameters</code> pattern exactly. Notably, <code>RSADigestSigner</code> (BC’s PKCS#1-v1.5 signer, a different class on the same low-level surface) <i>does</i> enforce this check explicitly – Bouncy Castle is inconsistent with itself across <code>Signer</code> implementations, not just across versions or API layers
Verify failure	boolean, no exception for ordinary invalid signatures; <code>InvalidAccessError</code> for key/algorithm mismatch (pre-verification)	surface-dependent, and layered: <code>Verifier::VerifyMessage()</code> returns plain <code>bool</code> for ordinary invalid signatures; the filter-pipeline route (<code>SignatureVerificationFilter</code> with <code>THROW_EXCEPTION</code>) can throw; <i>additionally</i> , <code>TF_VerifierBase::InputSignature()</code> shares the same <code>MessageRepresentativeBitLength</code> < <code>MinRepresentativeBitLength</code> guard as the sign path, backed by a dedicated named exception (“key too short to sign or verify”) reachable	boolean for ordinary invalid signatures, confirmed source-complete: <code>cipher.processBlock()</code> wrapped in <code>try{...}catch(Exception e){return false;}</code> , and every subsequent EMSA-PSS-VERIFY internal check (trailer signature padding, separator) uniformly <code>return false</code> with no distinguishable cause – RFC 8017 requires for PSS, confirmed by source rather than assumed by analogy with OAEP

BC-FIPS surface drift (evidence only, not part of the reference surface). `org.bouncycastle.crypto.fip` – a distinct build variant, not a version of the standard `org.bouncycastle.crypto.signers.PSSSigner` already fixed as the audited surface – breaks the boolean contract at the identical code point: `catch (Exception e) { throw new InvalidSignatureException(...); }` instead of `return false`. This is recorded as a fourth independent instance of `backend_capability = provider + version + API surface` (surface meaning FIPS vs. standard build here, not version or layer), not as a finding about the reference surface itself.

15.3 Two causal layers, not one: $\text{pre-}C_{pre}^{PSS}$ frontier vs. cryptographic verification result

The Crypto++ guard finding above does not demonstrate that the three backends disagree on what an invalid PSS signature *is*. It demonstrates that Crypto++’s verifier can raise an exception *before* entering verification semantics proper, because the verifier’s own configuration (modulus/hash/salt combination) is unrealizable – structurally the same class of pre-operation failure as Bouncy Castle’s `IllegalArgumentException` in `PSSSigner.init()` and WebCrypto’s `InvalidAccessError` for key/algorithm mismatch. All three backends exhibit this same two-layer structure; they differ only in *where* the boundary between the layers is drawn architecturally, not in whether it exists.

The stronger, still-open claim – not yet refuted, not yet confirmed – is:

$I_p \equiv_{C_{pre}^{PSS}} I_q \implies$ cryptographic validity of σ is represented as a $\{\text{true}, \text{false}\}$ verdict, uniformly across all three

This is consistent with everything confirmed so far: Bouncy Castle’s exception boundary sits entirely at `init()` (before verification), with `verifySignature()` itself fully boolean once past it; Crypto++’s exception boundary is confirmed to exist at the same conceptual point (module/hash/salt realizability) but it is not yet established whether signature-byte-length mismatch or an EMSA-PSS-inconsistent-but-correctly-sized signature can also escape as an exception *within* `VerifyMessage()` rather than being absorbed into `false` – this remains the open item.

Provisional shape for E_{PSS}^{SDK} (explicitly not frozen). The evidence suggests a three-branch verify contract, structurally different from OAEP’s single-collapse model:

$$\text{Verify}_{PSS}(K_{pub}, M, \sigma, P) \rightarrow \begin{cases} \text{SDK error} & C_{pre}^{PSS} \text{ not satisfied (key role, profile, parameters)} \\ \text{true} & \sigma \text{ valid} \\ \text{false} & \sigma \text{ cryptographically inconsistent} \end{cases}$$

No `invalid_signature` class is being introduced: `false` is a functional verification outcome, not an SDK error, in the same spirit as the $R_{val} = 1 \wedge R_{err} = 0$ distinction already established for GCM and OAEP – except here there may be no error classification to get wrong in the first place, since a correctly-rejected signature need not be classified at all. E_{PSS}^{SDK} is *not* assumed to mirror OAEP’s $\{\text{unsupported}, \text{invalid_key}, \text{invalid_parameter}, \text{decryption_error}\}$; a fourth “operational failure” class is a live question, not a default, pending completion of the remaining secondary inventory. The OAEP-style single-collapse model must not be imposed on PSS by analogy: OAEP’s collapse was grounded in an explicit RFC security rationale (Bleichenbacher/Manger, §12) that does not exist for PSS verification (§15).

C_{pre}^{PSS} , P_{PSS}^{common} , $P_{PSS}^{portable}$, and E_{PSS}^{SDK} remain jointly open. Hash, modulus, and message-boundary (or its absence) are now closed as inventory dimensions – see below – narrowing what remains open to the manifest consolidation and the final $C_{pre}^{PSS}/E_{PSS}^{SDK}$ derivation.

15.4 Consolidated ten-dimension inventory

Dimension	WebCrypto	Crypto++	Bouncy Castle (std.)	Provisional reading
Hash	SHA-1/256/384/512	SHA-1/256/384/512	SHA-1/256/384/512	common, wide
Modulus	open; 3072 supported	3072 supported (official example)	3072 supported (official example)	3072 a portable candidate
$H_{PSS} \leftrightarrow H_{MGF1}$	forced coupled	forced coupled	independently configurable	common forces coupling
$saltLength$	runtime, $[0, sLen_{\max}]$	compile-time; default $hLen$ (audited surface)	runtime	common likely $\{hLen\}$
External RNG	no	yes – high confidence from <code>SignMessage(rng, ...)</code> architecture; the literal salt-generation call has not been independently pinpointed	yes (confirmed, <code>random.nextBytes(salt)</code>)	not common
Explicit salt bytes	no	not confirmed / standard surface does not expose it	yes, recent versions	not common
Key role	runtime enforcement (<code>InvalidAccessError</code>)	compile-time enforcement (<code>System</code>)	<i>not</i> enforced in <code>PSSSigner</code> (contrast: enforced in <code>RSADigestSigner</code>)	adapter obligation, not a capability
Verify-invalid after C_{pre} satisfied	false	ordinary boolean; structural edges (signature-length mismatch, correctly-sized-but-inconsistent encoding) still partially open	false (source-confirmed, all internal causes)	candidate for a pure functional result, not an SDK error
Original message	no practical profile boundary	52/162 idem	idem	no <code>messageLength</code> clause

Three clarifying notes, required to read this table correctly.

1. **Crypto++’s *saltLength* row describes the audited surface, not the library’s absolute capability.** PSSR_MEM’s template does permit other SALT_LEN values at compile time; the standard surface fixed as the reference (§15) does not offer runtime selection. This distinction is precisely why the intersection can legitimately collapse to $\{hLen\}$ – a statement about the audited surface, not a claim that Crypto++ is incapable of variable salt length under any configuration.
2. **External RNG and explicit salt bytes are two distinct capabilities, not one “randomness” column.** Bouncy Castle may offer both; Crypto++ has only the former confirmed; WebCrypto has neither. Collapsing them would have obscured a real three-way capability gradient.
3. **Absence of a message-length boundary does not mean absence of `pss.message`.** It means the eventual clause will read something like “message is an opaque byte sequence with no normalization,” without an $mLen_{max}$ -style contractual bound the way OAEP required – because EMSA-PSS bounds the encoded message via modulus/hash/salt, never the original message length itself (RFC 8017 §9.1, §15).

15.5 Formal derivation of P_{PSS}^{common}

Each dimension is classified as *intersection-forced* (the concrete capability intersection admits only one value, regardless of preference), *profile-selection candidate* (multiple values remain available in the intersection; a portable choice will be needed later, per D-029), or *contractual semantic obligation* (not a capability dimension at all – an obligation the SDK imposes regardless of what any backend enforces natively, per the `invalid_key`/OAEP-KEY-ROLE precedent, D-027).

- $H_{PSS} = H_{MGF1}$ – **intersection-forced.** $P_{PSS}^{WebCrypto}$ and $P_{PSS}^{Crypto++}$ both structurally admit only coupled digest selection; Bouncy Castle’s independent-digest capability cannot enter P_{PSS}^{common} regardless of preference. Structurally identical derivation to OAEP’s D-028, arising independently from a different implementation (PSSSigner vs. OAEP_Encoding) – not a coincidence carried over from OAEP, a second confirming instance.
- *saltLength* – **intersection-forced (pending final confirmation of Bouncy Castle’s range, but already directionally clear).** $P_{PSS}^{Crypto++}.saltLength = \{hLen\}$ on the audited surface; since $P_{PSS}^{common} \subseteq P_{PSS}^{Crypto++}$ by definition of intersection, $P_{PSS}^{common}.saltLength \subseteq \{hLen\}$ regardless of how flexible WebCrypto and Bouncy Castle are. Two flexible backends do not outvote one inflexible one – the same structural lesson as OAEP’s digest coupling, now arising from a completely different capability (salt length, not hash selection) and a different single backend (Crypto++, not WebCrypto/Crypto++ jointly).
- **External RNG control, explicit salt bytes – intersection-forced to absent.** Neither is exposed by WebCrypto; by D-021’s intersection rule, neither enters P_{PSS}^{common} regardless of Crypto++/Bouncy Castle support. Consistent with OAEP’s D-033 treatment of randomness.
- **Hash, Modulus – profile-selection candidates, not yet decided.** $P_{PSS}^{common}.hash \supseteq \{SHA-1, SHA-256, SHA-384, SHA-512\}$ and $3072 \in P_{PSS}^{common}.modulus$, confirmed genuinely common across all three backends – unlike *saltLength* and digest coupling, no single backend forces a narrower intersection here. Which specific hash/modulus the portable profile offers remains an open profile-selection decision (D-029 category), not yet made; SHA-256/RSA-3072 are plausible by analogy with D-031/D-032, but are not derived from the intersection the way digest coupling and *saltLength* are, and must not be presented as though they were.

- **Key role – contractual semantic obligation, not a capability dimension.** As with OAEP’s D-027, this does not participate in P_{PSS}^{common} at all: it is an obligation the SDK contract imposes (encrypt/sign $\Rightarrow$ appropriate key type) independent of whether any given backend enforces it natively. Bouncy Castle’s `PSSSigner` not enforcing it (while its own `RSADigestSigner` does) is evidence *for* treating this as an adapter-level contractual obligation, not evidence that the obligation is optional.
- **Message boundary – not a dimension at all.** No $P_{PSS}^{common}.messageLength$ entry exists to derive, for the structural reason given in note 3 above.

Summary. Two dimensions are intersection-forced by a single backend each ($H_{PSS} = H_{MGF1}$ by WebCrypto/Crypto++ jointly; $saltLength = \{hLen\}$ by Crypto++ alone) – portability constrained by the most restrictive concrete API surface, not by consensus, exactly as anticipated. Two dimensions (hash, modulus) remain genuinely open profile-selection choices, not yet frozen. One dimension (key role) is not a capability question at all. One apparent dimension (message boundary) does not exist for PSS the way it did for OAEP. This is a substantially richer and more differentiated geometry than OAEP produced, and it was derived, not assumed – the explicit purpose of keeping this derivation separate from $P_{PSS}^{portable}$ selection.

15.6 Formal derivation of $P_{PSS}^{portable}$

$P_{PSS}^{portable} \subseteq P_{PSS}^{common}$. What follows separates properties inherited without new choice from the two genuine profile-selection decisions, each frozen with its own Decision Log entry – mirroring OAEP’s discipline of dedicating a separate D-XXX to each intersection-forced finding (D-028, D-033) rather than folding them into the surrounding derivation prose.

D-040 – RSA-PSS digest coupling. $H_{PSS} = H_{MGF1}$ in $P_{PSS}^{portable}$, intersection-forced (§15.5) by WebCrypto and Crypto++ against Bouncy Castle’s decoupling capability. Structural parallel to D-028, arising independently from a second, differently-implemented operation.

D-041 – RSA-PSS salt-length intersection. $sLen = hLen$ in $P_{PSS}^{portable}$, intersection-forced by Crypto++’s audited standard surface (§15). Not a security preference and not a universal property of PSS – a consequence of one backend’s concrete API surface, exactly as $t = 128$ coupling was forced by SP 800-38D’s rule for GCM, but arising here from an implementation limitation rather than a normative rule.

D-042 – RSA-PSS portable randomness-control surface. Neither external RNG injection nor explicit salt bytes are exposed by $P_{PSS}^{portable}$; both remain valid, distinct provider-only capabilities (Crypto++: RNG only, high confidence; Bouncy Castle: RNG confirmed and explicit salt bytes in recent versions) excluded by the intersection rule (D-021), since WebCrypto offers neither. True parallel to OAEP’s D-033, kept as its own decision rather than merged with D-041: `saltLength` geometry and randomness-control capability are different contractual dimensions (`Parameter.Length` vs. `Parameter.ExternalRandomnessControl`) that will generate different causal mutation classes once Γ_0^{PSS} is built, exactly as `oaep.mgfCoupling` was kept distinct from `oaep.hash` for the same forward-looking reason.

D-043 – RSA-PSS portable digest selection. $H_{PSS} = H_{MGF1} = SHA-256$, via **profile-selection** (D-029), not intersection-forcing. SHA-256 sits within $P_{PSS}^{common}.hash$ alongside SHA-1/384/512; chosen for interoperability, eliminating legacy SHA-1, parsimony, and alignment with the rest of the frozen cryptographic profile. The coincidence with OAEP’s D-031 value does not make this the same contractual decision – PSS earns its own entry because the choice is independently justified for this operation, not inherited.

D-044 – RSA-PSS portable modulus selection. $modBits = 3072$, via profile-selection, within $P_{PSS}^{common}.modulus$. Aligned with the security target already adopted for RSA-OAEP (D-032), for

the same NIST SP 800-57/SP 800-131Ar3 128-bit-alignment reasoning. **Derived, not chosen:** with $H_{PSS} = SHA-256$ (D-043) fixing $hLen = 32$, and $sLen = hLen$ already forced by D-041, $sLen = 32$ bytes follows automatically – it is not a third independent decision. Verification: $emBits = modBits - 1 = 3071$, $emLen = \lceil emBits/8 \rceil = 384$, and $emLen \geq hLen + sLen + 2$ becomes $384 \geq 66$, satisfied with wide margin – the portable profile sits nowhere near a structural boundary.

Frozen $P_{PSS}^{portable}$:

$$P_{PSS}^{portable} = \begin{cases} \text{RSA modulus} = 3072 \text{ bits} & (\text{D-044}) \\ H_{PSS} = SHA-256 & (\text{D-043}) \\ H_{MGF1} = SHA-256 & (\text{D-040, D-043}) \\ sLen = 32 \text{ bytes} & (\text{D-041, derived via D-043}) \\ \text{no external RNG parameter} & (\text{D-042}) \\ \text{no explicit salt parameter} & (\text{D-042}) \end{cases}$$

Key role, message semantics, and the error/true/false model are deliberately *not* included here: they belong to the contract/validation layer, input semantics, and E_{PSS}^{SDK} respectively, not to the cryptographic parameter profile – keeping these layers separate is precisely what avoided conflating causally distinct obligations throughout GCM and OAEP, and the same discipline is preserved here before C_{pre}^{PSS} is constructed.

15.7 Formal derivation of C_{pre}^{PSS}

$$C_{pre}^{PSS} = C_{manifest} \wedge C_{key} \wedge C_{profile} \wedge C_{request}$$

with $C_{profile} = C_{profile-values} \wedge C_{realizable}$, kept as two conceptually distinct sub-conditions: $C_{profile-values}$ requires the request match the frozen $P_{PSS}^{portable}$ (RSA-3072, $H_{PSS} = H_{MGF1} = SHA-256$, $sLen = 32$); $C_{realizable}$ is the normative encoding-viability condition $emLen \geq hLen + sLen + 2$, which for the frozen portable profile is always satisfied ($384 \geq 66$) but is retained conceptually because it is precisely what the confirmed `KeyTooShort()` guards (Crypto++) and `IllegalArgumentException` (Bouncy Castle `init()`) are checking – a property of the verifier/signer configuration, not of any specific σ .

This four-stage structure preserves the same precedence already established for E_{OAEP}^{SDK} (D-027): manifest/capability, then key, then portable parameter domain, then operation semantics – a backend or surface that does not declare RSA-PSS support at all must never be evaluated against hash/modulus/salt/key correctness.

Condition	sign	verify	Consequence if violated
Backend+version+API surface declares RSA-PSS support ($C_{manifest}$)	yes	yes	unsupported
RSA key is structurally valid (C_{key})	yes	yes	invalid_key
Key role correct (C_{key})	private	public	invalid_key
RSA modulus = 3072 ($C_{profile-values}$)	yes	yes	invalid_parameter
$H_{PSS} = SHA-256$ ($C_{profile-values}$)	yes	yes	invalid_parameter
$H_{MGF1} = SHA-256$, coupled ($C_{profile-values}$)	yes	yes	invalid_parameter

Condition	sign	verify	Consequence if violated
$sLen = 32 = hLen$ ($C_{profile-values}$)	yes	yes	<code>invalid_parameter</code>
No external RNG parameter exposed ($C_{profile-values}$)	yes	yes	<code>invalid_parameter</code> if the request attempts to supply one
No explicit salt parameter exposed ($C_{profile-values}$)	yes	yes	<code>invalid_parameter</code> if the request attempts to supply one
Message is an opaque byte sequence ($C_{request}$)	yes	yes	<code>invalid_parameter</code> only if structurally malformed at the type/request level; no portable length boundary exists (§15, note 3)
Module/hash/salt combination is encoding-realizable, $emLen \geq hLen + sLen + 2$ ($C_{realizable}$)	yes	yes	<code>invalid_parameter</code> (or <code>invalid_key</code> if attributed to key size) – confirmed evidence : Crypto++ throws <code>PK_SignatureScheme::KeyTooShort()</code> from the identical guard shared by <code>TF_SignerBase::SignAndRestart</code> and <code>TF_VerifierBase::VerifyAndRestart</code> ; Bouncy Castle throws <code>IllegalArgumentException("key too small for specified hash and salt lengths")</code> in <code>PSSSigner.init()</code> . For the frozen portable profile this condition is always satisfied ($384 \geq 66$) and never actually fails, but the mechanism is real and normatively grounded (RFC 8017 §9.1.1)
$ \sigma = 384$ octets	—	false	not C_{pre}^{PSS} : RFC 8017 §8.1.2 Step 1 classifies this as part of the <code>Verify</code> result domain (§15.8)
EMSA-PSS encoding of σ is consistent	—	no (given C_{pre}^{PSS})	false
Signature is cryptographically valid	—	no (given C_{pre}^{PSS})	false

The open item, stated precisely (superseded by D-046, §15.8). At the point this section was first drafted, $|\sigma| = 384$ had not yet been classified: whether Crypto++’s `VerifyMessage()` raises an exception or returns `false` when a syntactically-presented signature byte array has the wrong length, given an otherwise fully valid verifier configuration, was unconfirmed empirically (the relevant code path, `TF_VerifierBase::InputSignature(...)`, was not located despite repeated targeted attempts). This empirical gap remains – Crypto++’s exact native behavior is still not confirmed – but it no longer determines the contract: RFC 8017 §8.1.2 Step 1 classifies signature-length mismatch as part of `Verify`’s result domain directly, independent of any backend’s implementation. This is explicitly *not* the same guard as `KeyTooShort()` (which concerns verifier/signer configuration realizability, correctly retained in $C_{profile}$) – conflating the two would repeat the class of error already corrected via D-036/D-038 and the `oaep.ciphertext/oaep.ciphertextLength` split (D-037); the correction here is the mirror case, having initially conflated $|\sigma| \neq k$ with a $C_{request}$ precondition instead of recognizing it as part of `Verify`’s own normative output domain.

Design hypothesis, now normatively grounded rather than merely hoped for:

$$C_{pre}^{PSS} \implies \text{Verify}_{PSS}(K_{pub}, M, \sigma) \in \{\text{true}, \text{false}\} \quad \forall \sigma \in \{0, 1\}^*$$

This is the property the SDK design aims for – a clean, total boolean verification function once the four pre-conditions are satisfied. Bouncy Castle’s standard `PSSigner` already demonstrates it (source-confirmed, §15). WebCrypto’s boolean-only `verify()` is consistent with it. For $|\sigma| \neq k$ specifically, RFC 8017 §8.1.2 Step 1 settles the question directly (§15.8): the adapter normalizes to `false` regardless of what any individual backend does natively, because the RFC already defines this as part of `Verify`’s result, not as an implementation choice to be made once E_{PSS}^{SDK} is designed. Crypto++’s exact native `InputSignature()` behavior remains an open empirical question, but it is now a threats-to-validity/adapter-divergence data point (§7.2), not a contractual unknown.

C_{pre}^{PSS} is **provisionally frozen** with this one open item explicitly flagged, not silently assumed resolved.

15.8 Resolution: signature-length normalization (D-046)

Correction to an earlier draft of this decision. An initial version of D-046 added $|\sigma| = k$ to $C_{request}$ as an adapter-enforced precondition, classifying its violation as `invalid_parameter`. This was wrong, and RFC 8017 says so directly: §8.1.2 (RSASSA-PSS-VERIFY) defines Step 1, *before* any other verification step, as “Length checking: If the length of the signature S is not k octets, output ‘invalid signature’ and stop” – textually adjacent to, and treated identically to, RSAVP1’s “signature representative out of range” and EMSA-PSS-VERIFY’s inconsistency outcome, all folded into the same two-valued output {valid signature, invalid signature}. $|\sigma| \neq k$ is not a request/configuration precondition failure; it is normatively the *first step of the verification result itself*. Treating it as `invalid_parameter` would have repeated, inside PSS, the exact class of causal-layer error already corrected twice for GCM (D-036, D-038).

Corrected resolution. The portable adapter normalizes $|\sigma| \neq k$ to `false` – not by adapter-level design preference (as the key-role case was, D-027), but because RFC 8017 itself already classifies this outcome as part of `Verify`’s result domain. This is a materially stronger foundation than the earlier “adapter closes the gap” framing: for $|\sigma| = k$, we no longer need to know what Crypto++’s `InputSignature()` does natively to define the portable contract – the RFC already tells us what a conformant observer should see. If Crypto++ throws internally, the adapter catches and normalizes to `false`; if it already returns `false` natively, the adapter’s behavior coincides without any correction needed. Crypto++’s exact native behavior remains unconfirmed and is retained as a standing adapter-induced-divergence-threat data point (§7.2), but it is no longer a contractual unknown – only an implementation detail of how the adapter’s already-fixed obligation gets satisfied on that specific backend.

Recovered property, now normatively grounded rather than merely hypothesized.

$$C_{pre}^{PSS} \implies \text{Verify}_{PSS}(K_{pub}, M, \sigma) \in \{\text{true}, \text{false}\} \quad \forall \sigma \in \{0, 1\}^*$$

For the $|\sigma| \neq k$ case specifically, this is no longer a design hypothesis pending empirical confirmation (contrast the weaker framing in D-045) – it is a direct consequence of RFC 8017 §8.1.2 Step 1.

15.9 Frozen E_{PSS}^{SDK} (D-047)

$$E_{PSS}^{SDK} = \{\text{unsupported}, \text{invalid_key}, \text{invalid_parameter}\}$$

with the classification order inherited directly from C_{pre}^{PSS} ’s four-stage precedence (§15.7): $C_{manifest} \rightarrow \text{unsupported}$; $C_{key} \rightarrow \text{invalid_key}$; $C_{profile} \rightarrow \text{invalid_parameter}$; $C_{request}$ (message structural validity only – *not* signature length, corrected §15.8) $\rightarrow \text{invalid_parameter}$.

The central structural asymmetry with OAEP – primary justification, corrected. E_{PSS}^{SDK} has *three* classes, not OAEP’s four – there is no `decryption_error`-equivalent. The primary reason is now the direct normative text just established, not an argument from absence: RFC 8017 §8.1.2

defines **RSASSA-PSS-VERIFY**’s entire output domain as {valid signature, invalid signature}, explicitly folding signature-length mismatch, out-of-range representative, and EMSA-PSS inconsistency into that same two-valued result. Once C_{pre}^{PSS} is satisfied, every one of these causes is already, by the RFC’s own definition, a **Verify** result – not a distinguishable SDK error to classify. **false** therefore comprises, once the genuine preconditions are satisfied: $|\sigma| \neq k$; “signature representative out of range”; EMSA-PSS-inconsistent encoding (bad trailer, bad padding, salt/hash/trailer mismatch); and a cryptographically-correct-but-non-matching signature. The earlier contrastive observation – that OAEP’s single-collapse model rests on an explicit Bleichenbacher/Manger security rationale absent for PSS verification (§15) – remains true and worth keeping as a secondary, explanatory contrast (it explains *why* the two operations differ in kind), but it is not, and should not have been treated as, the primary argument for PSS’s three-class model. The primary argument is simply that RFC 8017 already defines **Verify**’s output domain this way.

sign-side residual. RFC 8017’s “message too long” (the hash function’s own input-length limit, $2^{61} - 1$ octets for SHA-1/SHA-256) is folded into $C_{request}$ ’s existing “message is an opaque byte sequence” clause rather than given a dedicated class: the limit is practically unreachable for any realistic portable input and does not warrant fabricated granularity (D-016).

With E_{PSS}^{SDK} frozen, the next step is writing the **pss.*** clause table, following the same K1/K2 discipline used for HKDF, AES-GCM, and RSA-OAEP.

16 RSA-PSS – frozen contract clauses

Fifteen clauses, each a stable contractual obligation (D-016) derived directly from $C_{pre}^{PSS}/E_{PSS}^{SDK}$ (§15–§15.9). Two structural decisions frozen jointly as D-048: (1) **pss.rngControl** and **pss.saltBytes** are kept as separate local clauses despite both instantiating **Parameter.ExternalRandomnessControl** at K_2 – external RNG control and explicit salt-byte control are genuinely distinct capabilities (§15.4), and local-clause granularity is preserved independently of shared-taxonomy projection, exactly as established for GCM/OAEP; (2) **pss.verifiCation** is the first real materialization of **Authentication.SignatureVerification**, a K_2 subtype reserved in the shared taxonomy (§5) since the framework’s design but not previously instantiated by any operation.

Relation columns deliberately omitted. Following the historical format (HKDF, AES-GCM, RSA-OAEP), this table lists ID/K1/K2/Contract clause only. An earlier draft proposed fixed clause-to-relation assignments (e.g. “**pss.hash** $\rightarrow R_{val} + R_{cap}$ ”) and these were explicitly rejected: R_{cap} activates only when declared and observed capability diverge for a specific causal class, not as an intrinsic property of a parameter clause – assigning it prematurely would have repeated the class of error already corrected via D-036/D-038. Detecting relations are derived exclusively downstream, per causal mutation class and its $\Gamma_0(c)$, once Γ_0^{PSS} is constructed.

ID	K1	K2	Frozen contract clause
pss.key	Key	—	Signature generation uses an RSA private key; verification uses the corresponding RSA public key.
pss.modulus	Parameter	Length	Portable RSA-PSS fixes the RSA modulus length at 3072 bits.
pss.message	Input	—	Message is an opaque byte sequence without normalization; no portable length boundary exists (D-029/§15).
pss.hash	Parameter	AlgorithmChoice	Portable RSA-PSS uses SHA-256 as the message digest.

ID	K1	K2	Frozen contract clause
<code>pss.mgfCoupling</code>	Parameter	DomainParameter	Portable RSA-PSS requires $H_{PSS} = H_{MGF1}$; independent MGF1 digest selection remains a provider-specific capability outside the portable profile.
<code>pss.saltLength</code>	Parameter	Length	Portable RSA-PSS fixes the salt length at $sLen = 32$ octets ($= hLen$).
<code>pss.rngControl</code>	Parameter	ExternalRandomnessControl	The portable SDK exposes no caller-supplied RNG parameter for PSS salt generation.
<code>pss.saltBytes</code>	Parameter	ExternalRandomnessControl	The portable SDK exposes no caller-supplied explicit salt-byte parameter for signature generation.
<code>pss.signature</code>	Output	—	A successful sign operation produces a valid RSASSA-PSS signature for the declared message, key, and frozen profile.
<code>pss.signatureLength</code>	Output	—	A successful sign operation produces a signature of exactly $k = 384$ octets.
<code>pss.verifiication</code>	Authentication	SignatureVerification	$\text{Verify}_{PSS}(K_{pub}, M, \sigma)$ correctly returns true or false for every byte sequence σ admitted as a verification argument; in particular, $ \sigma \neq k$ maps to false under D-046.
<code>pss.validation</code>	Validation	Semantic	Requests violating C_{pre}^{PSS} are rejected according to the frozen acceptance/rejection semantics; cryptographic inconsistency of a supplied verification signature, including $ \sigma \neq k$, is excluded from this clause and belongs to pss.verifiication .
<code>pss.error</code>	Error	—	Observable failures map to the frozen three-class SDK error model $E_{PSS}^{SDK} = \{\text{unsupported}, \text{invalid_key}, \text{invalid_parameter}\}$ according to the D-047 classification order.
<code>pss.cap.provider</code>	Capability	—	Observed backend behaviour matches the frozen provider/version/API-surface capability manifest A_p^{pre} .
<code>pss.cap.portable</code>	Capability	—	The SDK exposes only $P_{PSS}^{portable} \subseteq P_{PSS}^{common} \subseteq P_{p,PSS}$; no unlabeled provider extension leaks through the adapter.

On σ 's domain. The abstract portable interface admits any $\sigma \in \{0, 1\}^*$ as the verification argument; entries violating RFC 8017 §8.1.2 Step 1's length requirement are mapped to **false**, not excluded at the interface level. This is a statement about the abstract interface's admitted argument domain, not a redefinition of what formally constitutes an RSA-PSS signature – RFC 8017 itself defines S as a signature of length k ; the portable **Verify** interface simply does not pre-filter its input argument on that basis, consistent with RFC 8017 §8.1.2 Step 1 folding length mismatch into the verification

result itself.

Coverage. $\text{Coverage}_{K_1}(\text{RSA-PSS}) = \{\text{Key, Parameter, Input, Output, Authentication, Validation, Error, Capa} - 8 \text{ of the } 10 \text{ } K_1 \text{ kinds, one more than AES-GCM (9, missing only Membership) and one more than RSA-OAEP (7, missing Authentication, Encoding, Membership): PSS is the first operation to materialize Authentication. Encoding and Membership remain absent by contractual inapplicability, not omission: the signature has a fixed-length representation via I2OSP with no representational degree of freedom (same argument as OAEP’s D-034), and RSA has no point/curve membership obligation.}$

Separation preserved for the next phase. Four causally distinct layers are now cleanly separated, matching the causal boundaries this design has repeatedly needed to keep apart: cryptographic output correctness (`pss.signature`) $\neq$ structural output obligation (`pss.signatureLength`) $\neq$ cryptographic verification semantics (`pss.verify`) $\neq$ pre-operation acceptance/rejection (`pss.validate`) $\neq$ SDK error normalization (`pss.error`). The next step is constructing PSS’s causal mutation classes and auditing them against these fifteen clauses – the strong test of whether each clause is genuinely independent, or whether some prove redundant once subjected to minimal causal mutations, exactly as happened with OAEP-PORTABLE-CAPABILITY-LEAK.

17 RSA-PSS – frozen Γ_0^{PSS}

Fifteen causal classes across six families (D-049), derived directly from the frozen `pss.*` clauses (§16) rather than copied and renamed from GCM/OAEP – patterns are reused only where the causality is genuinely equivalent (profile-bypass, capability misreport), while PSS introduces genuinely new structure: two distinct randomness-control capabilities, and above all `pss.verify`, the first materialization of `Authentication.SignatureVerification`.

On verify returning false for an invalid signature. This is correct behavior, not a mutation. Every Family E class represents a *defective implementation of verification semantics itself* – accepting a signature that should normatively be rejected, or rejecting one that should normatively be accepted – never “invalid signature” as such.

Family A – Computation/semantic divergence (R_{byte} inapplicable throughout – probabilistic signature, D-033/§15; R_{interop} is the primary detector):

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
PSS-SIGN-MESSAGE-NORMALIZATION	<code>pss.message;</code> <code>pss.signature</code>	Input; Output	R_{interop} . The adapter/backend silently alters M before <code>sign</code> ; the resulting signature is not valid for the declared M . Parallel to GCM-PLAINTEXT-NORMALIZATION (<code>gcm.plaintext;</code> <code>gcm.ciphertext</code>).

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
PSS-VERIFY-MESSAGE-NORMALIZATION	pss.message; pss.verification	Input; Authentication.SignatureVerification	R_{interop} ; R_{val} N/A (M remains a valid input, per the same reasoning applied to OAEP-MESSAGE-NORMALIZATION D-050 – the defect is misinterpretation, not out-of-domain acceptance). The adapter/backend silently alters M before <code>verify</code> ; a signature valid for the declared M is evaluated against a different M' .
PSS-SIGNATURE-COMPUTATION-DIVERGENCE	pss.signature	Output	R_{interop} . <code>sign</code> produces a cryptographically incorrect signature of correct length ($k = 384$) for the declared M /key/profile.
PSS-SIGNATURE-LENGTH-DIVERGENCE	pss.signatureLength	Output	R_{interop} . I2OSP zero-padding omitted despite a mathematically correct underlying RSA signature representative – same class of defect as OAEP-CIPHERTEXT-LENGTH-D

Family B – Profile/capability leakage bypass (domain clause + validation + `cap.portable`; genuine cross-provider capability variation exists for each):

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
PSS-MODULUS-PROFILE-BYPASS	pss.modulus; pss.validation; pss.cap.portable	Parameter.Length; Validation.Semantic; Capability	R_{val} ; R_{cap} . Stimulus: RSA-2048/4096 – provider-valid, non-portable.

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
PSS-HASH-PROFILE-BYPASS	pss.hash; pss.validation; pss.cap.portable	Parameter.AlgorithmChoice; Validation.Semantic; Capability	R_{val} ; R_{cap} . Stimulus: SHA-384/512, coupled – isolates hash choice from coupling.
PSS-MGF-COUPLING-BYPASS	pss.mgfCoupling; pss.validation; pss.cap.portable	Parameter.DomainParameter; Validation.Semantic; Capability	R_{val} ; R_{cap} . Stimulus: $H_{\text{PSS}} = \text{SHA-256}$, $H_{\text{MGF1}} = \text{SHA-1}$ – materially executable only against Bouncy Castle (D-040).
PSS-SALTLENGTH-PROFILE-BYPASS	pss.saltLength; pss.validation; pss.cap.portable	Parameter.Length; Validation.Semantic; Capability	R_{val} ; R_{cap} . No OAEP analogue: OAEP never had a caller-selectable length parameter of this kind. Real cross-provider capability variation confirmed (D-041): WebCrypto/Bouncy Castle runtime-flexible, Crypto++ fixed at $hLen$ on the audited surface – same structural pattern as GCM-TAGLENGTH-PROFILE-BO

Family C – Intrinsic/semantic-role boundary bypass (domain + validation, no cap.portable: no capability space to leak):

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
PSS-KEY-ROLE-BYPASS	pss.key; pss.validation	Key; Validation.Semantic	R_{val}. Stimuli: sign-with-public-key, verify-with-private-key – two instances of one causal class. No cap.portable: key role is a structural obligation of the scheme itself, not a configuration axis with cross-provider capability variation (same argument as OAEP-KEY-ROLE-BYPASS). No pss.message analogue to OAEP-MESSAGE-BOUNDARY-BY exists: pss.message has no portable length boundary to bypass (D-029/§15), so this causal class simply does not exist for PSS – a correct absence, not a gap.

Family D – Interface capability leak (**cap.portable** alone; the defect is in the portable interface’s own surface):

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
PSS-RNG-INTERFACE-LEAK	pss.rngControl; pss.cap.portable	Parameter.ExternalRandomnessCap Capability	cap.Control, Materially executable only against Crypto++/Bouncy Castle.

Mutation class	$\Gamma_0(c)$	Kinds₀(c)	Expected relations
PSS-SALT-BYTES- INTERFACE-LEAK	pss.saltBytes; pss.cap.portable	Parameter.ExternalRandomnessCap Capability	– SSControl is materially executable only against Bouncy Castle (recent versions, D-042). Kept as a class distinct from PSS-RNG-INTERFACE-LEAK – two genuinely different capabilities (D-048), not one “randomness” leak.

Family E – Verification semantics (new to PSS; each represents a defective implementation of verification itself, never a correctly-returned **false/true**):

Mutation class	$\Gamma_0(c)$	$\mathbf{Kinds}_0(c)$	Expected relations
PSS-VERIFICATION- FALSE-ACCEPT	pss.verifcation	Authentication.SignatureVerification	R_{val} A backend accepts (true) a σ that is normatively invalid (bad trailer, bad padding, salt/hash inconsistency, or $\sigma \neq k$ mishandled instead of applying RFC 8017 §8.1.2 Step 1) – the most security-critical class in PSS: a defective verifier that accepts forged signatures. R_{interop} deliberately <i>not</i> listed: the normal interoperability oracle is positive ($\text{Sign}_p(M) \rightarrow \text{Verify}_q(M, \sigma) = \text{true}$); an invalid signature is not a legitimately-produced artifact, and forcing R_{interop} to cover this would blur its boundary with R_{val}.

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
PSS-VERIFICATION- FALSE-REJECT	pss.verification	Authentication.SignatureVerification	$R_{\text{val.}}^{\text{Interp.}}$ $\sigma = \text{Sign}_p(M)$ is genuinely valid, but $\text{Verify}_q(M, \sigma) =$ false – the literal case RSA-PSS was chosen to exercise as a probabilistic cross-provider interoperability operation. Re- quires a stimulus where σ was already estab- lished valid (e.g. cross-signed by another backend); a signature that was itself defec- tively produced and then cor- rectly rejected is PSS-SIGNATURE-COMPUTATION- not this class.

Diagnostic-separability note (not causal equivalence). Two class pairs are intentionally retained despite potentially overlapping observable symptoms: PSS-SIGN-MESSAGE-NORMALIZATION vs. PSS-SIGNATURE-COMPUTATION-DIVERGENCE, and PSS-VERIFY-MESSAGE-NORMALIZATION vs. PSS-VERIFICATION-FAILURE. Their Γ_0 sets are distinct and therefore encode genuinely different semantic causes (input misinterpretation vs. cryptographic computation/verification given a correct input). Later necessity analysis shall assess whether the relation family provides independent diagnostic signal ($\Delta_j^{\text{diag}}(c)$, §6, D-052) for distinguishing these causes; a lack of such signal is a limitation of diagnostic power in the current relation family, *not* evidence that the two causes are equivalent. Causal coverage, detection, and diagnosis remain three distinct levels (§23), and these two pairs are a clean illustration of why: a relation may detect both causes perfectly while remaining unable to say which one occurred.

Family F – Error/capability:

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
PSS-ERROR-MISCLASSIFICATION	pss.error	Error	R_{err} . A request correctly rejected under $C_{\text{pre}}^{\text{PSS}}$ is assigned the wrong $E_{\text{PSS}}^{\text{SDK}}$ class (e.g. <code>invalid_parameter</code> instead of <code>invalid_key</code>).
PSS-PROVIDER-CAPABILITY-MISREPORT	pss.cap.provider	Capability	R_{cap} . Detectable by comparing declared vs. observed provider+version+API-surface capability alone, without executing any PSS operation. Structurally identical to the corrected GCM-PROVIDER-CAPABILITY- (D-038) and OAEP-PROVIDER-CAPABILITY-

No separate profile-realizability class. Unlike the theoretical possibility considered during $C_{\text{pre}}^{\text{PSS}}$'s design, no dedicated causal class is introduced for $C_{\text{realizable}}$ (§15.7): under the frozen portable profile (RSA-3072/SHA-256/ $sLen = 32$), the combination is always realizable ($384 \geq 66$); non-realizable combinations only arise by first abandoning a parameter boundary already covered by PSS-MODULUS-PROFILE-BYPASS, PSS-HASH-PROFILE-BYPASS, or PSS-SALTLENGTH-PROFILE-BYPASS. A fifth generic class would be redundant with these three.

Coverage check. All fifteen `pss.*` clauses have Γ_0 coverage; no orphaned clause. Coverage $_{K_1}$ (RSA-PSS) over these fifteen classes spans {Key, Parameter, Input, Output, Authentication, Validation, Error, Capability} – unchanged from the clause-level coverage already established (§16), as expected. `pss.verification` is now covered by three distinct causes (sign/verify-message-normalization split, false-accept, false-reject) rather than one generic class – a more convincing outcome than introducing `Authentication.SignatureVer` and covering it with a single catch-all.

Cross-operation audit PSS $\leftrightarrow$ HKDF/GCM/OAEP completed after propagation of D-050. Five axes were checked (causal-layer consistency; input/output consistency; profile-bypass consistency; relation consistency; granularity consistency), surfacing two genuine cross-cutting corrections (D-050). A final mechanical propagation pass confirmed: no residual `gcm.error`/ R_{err} in GCM-AUTHENTICATION-BYPASS or GCM-ARTIFACT-STRUCTURE-CORRUPTION; OAEP-MESSAGE-NORMALIZATION consistently reflects its amended Γ_0 everywhere it appears; no remaining text describes D-036 as having exhaustively resolved `gcm.error` contamination (D-036's own entry was, and remains, correctly scoped to the three classes it actually touched). No additional structural inconsistencies were identified in profile-boundary semantics, relation applicability, or causal granularity. This audit is a verification activity, not a source of new contractual decisions – D-050 already records the only modifications it produced.

18 Relation-spectrum diagnostic equivalence

Formal fact. The frozen relation spectrum

$$r(c) = (R_{\text{byte}}, R_{\text{interop}}, R_{\text{ser}}, R_{\text{val}}, R_{\text{err}}, R_{\text{cap}})$$

with each component evaluated as pass, fail, or not-applicable (§6), is **not injective** over the causal mutation classes. Distinct causal classes may therefore induce the same observable relation vector. Define

$$c_1 \sim_r c_2 \iff r(c_1) = r(c_2), \quad [c]_r = \{c' : r(c') = r(c)\}.$$

Evidence. A full applicability/detection audit of all fifteen Γ_0^{PSS} classes (§17) against $r(\cdot)$ yields exactly six diagnostic-equivalence classes:

- $|[c]_r| = 4$: PSS-SIGN-MESSAGE-NORMALIZATION, PSS-VERIFY-MESSAGE-NORMALIZATION, PSS-SIGNATURE-CO-PSS-SIGNATURE-LENGTH-DIVERGENCE – all $r(c) = (-, \text{fail}, -, -, -, -)$.
- $|[c]_r| = 4$: PSS-MODULUS, PSS-HASH, PSS-MGF-COUPPLING, PSS-SALTLENGTH-PROFILE-BYPASS – all $r(c) = (-, -, -, \text{fail}, -, \text{fail})$.
- $|[c]_r| = 2$: PSS-KEY-ROLE-BYPASS, PSS-VERIFICATION-FALSE-ACCEPT – both $r(c) = (-, -, -, \text{fail}, -, -)$.
- $|[c]_r| = 3$: PSS-RNG-INTERFACE-LEAK, PSS-SALT-BYTES-INTERFACE-LEAK, PSS-PROVIDER-CAPABILITY-MIS – all $r(c) = (-, -, -, -, -, \text{fail})$.
- $|[c]_r| = 1$: PSS-VERIFICATION-FALSE-REJECT – $r(c) = (-, \text{fail}, -, \text{fail}, -, -)$, distinguishable from every other class by jointly failing R_{interop} and R_{val} .
- $|[c]_r| = 1$: PSS-ERROR-MISCLASSIFICATION – $r(c) = (-, -, -, -, \text{fail}, -)$, the only class in any operation currently firing R_{err} (a direct consequence of D-050 removing R_{err} from GCM’s boundary-bypass classes).

Only 2 of 15 PSS classes are individually identifiable by $r(\cdot)$ alone; 13 belong to non-unitary equivalence classes.

The phenomenon is structural, not RSA-PSS-specific. RSA-PSS is the operation in which non-injectivity was first identified through exhaustive relation-spectrum partitioning; a directed cross-operation audit, testing four hypotheses generated from the PSS partition, subsequently confirmed the phenomenon already occurs structurally in AES-GCM and RSA-OAEP – it did not originate in PSS, it was merely first made fully visible there.

- **Profile-boundary-bypass classes collapse under $\{R_{\text{val}}, R_{\text{cap}}\}$** in every operation that uses the pattern: GCM’s three (KEY/IV/TAGLENGTH-PROFILE-BOUNDARY-BYPASS, post-D-036), OAEP’s four (MODULUS/HASH/MGF-COUPPLING/LABEL-PROFILE-BYPASS), and PSS’s four – present since the pattern was first designed for GCM.
- **Semantically distinct validation failures collapse under R_{val} alone:** confirmed within OAEP itself (OAEP-MESSAGE-BOUNDARY-BYPASS $\sim_r$ OAEP-KEY-ROLE-BYPASS), and again in PSS (PSS-KEY-ROLE-BYPASS $\sim_r$ PSS-VERIFICATION-FALSE-ACCEPT) – absent from GCM, which has no Family-C-style intrinsic/semantic-role-boundary class (GCM’s key/IV/tagLength boundaries all carry genuine capability variance and were assigned Family B treatment instead).
- **Distinct computation/input-semantics failures collapse under the deterministic relation available to the operation:** R_{byte} for GCM (GCM-AAD-ABSENT-EMPTY-DIVERGENCE, GCM-IV-INTERPRETATION-DIVERGENCE, GCM-CIPHERTEXT-COMPUTATION-DIVERGENCE – 3-way), R_{interop} for the probabilistic operations (OAEP: OAEP-MESSAGE-NORMALIZATION, OAEP-CIPHERTEXT-COMPUTATION-DIVERGENCE).

OAEP-CIPHERTEXT-LENGTH-DIVERGENCE – 3-way; PSS’s 4-way cluster above). R_{byte} vs. R_{interop} tracks R_{byte} ’s applicability (D-033): the same phenomenon, manifesting through whichever relation actually applies.

- **PSS additionally exhibits an operation-specific R_{cap} -only equivalence** (the 3-way cluster above) **not present in GCM or OAEP**: both of those operations have exactly one pure- R_{cap} class (*-PROVIDER-CAPABILITY-MIS*, post-D-038), with nothing to collide against – OAEP-PORTABLE-CAPABILITY-LEAK was eliminated at design time (D-039) as redundant with the profile-bypass classes. PSS’s cluster arises specifically because PSS is the only operation with *two* distinct portable randomness-control surfaces (`rngControl`, `saltBytes`, D-048) plus the provider-misreport class – a consequence of PSS’s own capability structure, not a general framework property.

Correct interpretation. Relation-spectrum equality does *not* imply causal equivalence:

$$r(c_1) = r(c_2) \implies \Gamma_0(c_1) = \Gamma_0(c_2).$$

It establishes only that the frozen relation spectrum cannot distinguish those causes at the diagnostic level $r(\cdot)$ represents. Causal coverage, detection, and diagnostic identifiability remain three distinct levels (D-049, §6): a relation may detect a defect (some component of $r(c)$ fails, distinguishing it from conforming behavior) without the overall vector being able to identify *which* causal class, among several sharing that vector, actually occurred. Causal classes remain separate in Γ_0 whenever their clause sets encode genuinely distinct contractual causes, regardless of which diagnostic-equivalence class they fall into under r .

What is deliberately not claimed here. This decision freezes the finding, not a remedy. It does not assert that the framework requires a seventh relation, that causally-distinct classes sharing $r(c)$ should be merged, or that all future diagnosis must be limited to $r(\cdot)$ as currently defined. Whether the framework’s diagnostic goal requires only detection, a coarser pre-registered diagnostic partitioning, or a richer diagnostic mechanism using additional pre-registered observational information (without ad hoc invention of new relations) is a separate methodological question, deliberately left open for the next phase.

19 RSA Key Serialization – normative fiche

Status (v0.5.16): RSA Key Serialization CLOSED. WebCrypto, Crypto++, and Bouncy Castle backend inventories (§19.1–§19.3); three-backend intersection and portable profile (D-053, §19.4); admission-time validation semantics (D-054, §19.5); $C_{\text{pre}}^{\text{RSA-ser}}$ (D-055, §19.6); directional SDK error semantics (D-056, §19.7); thirteen `rsa-ser.*` clauses (D-057, §20); seventeen causal classes and $\Gamma_0^{\text{RSA-ser}}$ (D-058, §21); accumulated cross-decision audit passed with five residual stale fragments corrected (§21.1). Not reopened during EC P-256 construction absent a genuine cross-operation contradiction.

Three layers, kept explicitly distinct throughout. RSA key material $\rightarrow$ ASN.1 container $\rightarrow$ encoding. Confusing `RSAPublicKey` with SPKI, `RSAPrivateKey` with PKCS#8/`OneAsymmetricKey`, or DER with PEM would collapse causally distinct obligations before any contract is even written.

Layer 1 – RSA key material (RFC 8017 Appendix A.1). Confirmed against the full PKCS#1 genealogy (RFC 2313 $\rightarrow$ 2437 $\rightarrow$ 3447 $\rightarrow$ 8017), identical structure preserved throughout:

`RSAPublicKey ::= SEQUENCE{modulus INTEGER, publicExponent INTEGER}`

`RSAPrivateKey ::= SEQUENCE{version,  $n$ ,  $e$ ,  $d$ ,  $p$ ,  $q$ ,  $d_P$ ,  $d_Q$ ,  $q_{\text{Inv}}$ , otherPrimeInfos OPTIONAL}`

version is 0 for two-prime RSA, 1 for multi-prime (required when **otherPrimeInfos** is present). **Two-prime vs. multi-prime RSA support is now resolved by the three-backend intersection:** the portable profile is restricted to two-prime RSA; multi-prime RSA is outside $P_{\text{RSA-ser}}^{\text{common}}$ and therefore outside $P_{\text{RSA-ser}}^{\text{portable}}$ (D-053).

Layer 2a – public container, SPKI (RFC 5280).

$\text{SubjectPublicKeyInfo} ::= \text{SEQUENCE}\{\text{algorithm AlgorithmIdentifier}, \text{subjectPublicKey BIT STRING}\}$

For RSA: $\text{SPKI}_{\text{RSA}} = (\text{AlgorithmIdentifier}(\text{rsaEncryption}, \text{NULL}), \text{BIT STRING}(\text{DER}(\text{RSAPublicKey}(n, e))))$.

Confirmed: the **rsaEncryption** OID (1.2.840.113549.1.1.1) requires **AlgorithmIdentifier.parameters** to be explicit **NULL** – confirmed not only normatively but with a real-world byte-level example (Microsoft Certificate Enrollment documentation shows the literal two-byte encoding 05 00 in an actual certificate). **SPKI** is not the same object as **RSAPublicKey**: the latter is the RSA key material; the former is the algorithm-tagged container that identifies it as RSA.

Layer 2b – private container, PKCS#8 (RFC 5208, obsoleted) / RFC 5958. Precise relationship, not merely “generalizes”: RFC 5958 formally *obsoletes* RFC 5208 (IETF Standards Track, confirmed via the RFC’s own **Obsoletes:** 5208 header) – not a parallel, coexisting alternative. Backward compatibility is preserved explicitly: **PrivateKeyInfo** (RFC 5208) is retained as exactly **OneAsymmetricKey** with **version=v1** and no **publicKey** field.

$\text{PrivateKeyInfo} ::= \text{SEQUENCE}\{\text{version}, \text{privateKeyAlgorithm}, \text{privateKey OCTET STRING}, \text{attributes OPTIC}\}$

$\text{OneAsymmetricKey} ::= \text{SEQUENCE}\{\text{version}, \text{privateKeyAlgorithm}, \text{privateKey OCTET STRING}, \text{attributes [0]}$

For RSA, **privateKey** is an **OCTET STRING** whose content is **DER(RSAPrivateKey)**. RFC 7468 recognizes both variants under the same PEM label **PRIVATE KEY**. **Normative succession did not by itself determine the portable profile;** the subsequent three-backend inventory resolved that empirical question. The frozen RSA serialization profile uses the RFC 5208-compatible **PrivateKeyInfo** shape common to the selected backend surfaces (D-053), while RFC 5958 remains the current normative successor and is not conflated with the provider intersection. The provider inventory subsequently resolved the portable outer private-key shape as the RFC 5208-compatible **PrivateKeyInfo** form frozen by D-053. Optional RFC 5958-only **publicKey** material does not widen the portable profile. Any residual provider-specific tolerance for optional fields is an import-behaviour question outside the frozen portable writer shape, not an open definition of the portable container itself.

Layer 3 – binary vs. textual encoding. ASN.1 permits multiple BER encodings of the same abstract value; DER eliminates that multiplicity, yielding a canonical binary form – the natural candidate for making serialization comparison scientifically meaningful. **DER is now frozen as the portable binary encoding by D-053.** DER is accepted by all three selected backends; the complete intersection of additional non-DER BER forms remains uncharacterized and therefore does not widen the portable import domain. RFC 7468 defines PEM-style textual wrapping (**---BEGIN/END PUBLIC KEY---**, **---BEGIN/END PRIVATE KEY---**) as a base64 encoding of the same DER content; PEM is **initially out of scope** for the portable contract – it wraps the structures we care about, it is not itself the representation being measured. **R_{byte} applicability is not yet decided:** if all three backends’ DER export of the same key is byte-identical, R_{byte} could apply to serialization for the first time in this design; but optional fields (**attributes**, **publicKey**) could alter bytes without altering semantics, so this must be confirmed against provider evidence, not assumed from DER’s canonical-form property alone.

The obligation under study, stated precisely. Not “can the three libraries read a PEM file” – too superficial. The obligation is structurally:

$$K \xrightarrow{\text{Export}_p} S_p \xrightarrow{\text{Import}_q} K'_q$$

with simultaneous properties required over $\text{structure}(S_p)$, $\text{semantics}(K'_q)$, and $\text{re-export}(K'_q)$. This is expected to exercise R_{ser} , R_{interop} , R_{val} , and possibly R_{byte} jointly – plausibly the operation where this framework’s relation family shows its clearest complementarity, since no prior operation (HKDF, GCM, OAEP, PSS) simultaneously engaged more than two or three of these relations for the same clause.

19.1 Backend inventory: WebCrypto RSA (closed, v0.5.3)

Confirmed against the current W3C Editor’s Draft (Web Cryptography API Level 2). "spki" = DER encoding of SubjectPublicKeyInfo (RFC 5280); "pkcs8" = DER encoding of PrivateKeyInfo (RFC 5208, not RFC 5958) – resolving one of the open items from §19 for WebCrypto specifically. The generic ASN.1 parsing primitives (*parse a subjectPublicKeyInfo*, *parse a PrivateKeyInfo*) are defined with `exactData=true`: if any bytes remain unconsumed after DER parsing, `DataError` is thrown.

OID question resolved: current spec uses the generic `rsaEncryption` OID uniformly, not an algorithm-specific one. For RSA-PSS and RSA-OAEP, both "spki" import (checking the `AlgorithmIdentifier` of SubjectPublicKeyInfo) and "pkcs8" import (checking `privateKeyAlgorithm.algorithm` of PrivateKeyInfo) require the OID to equal `rsaEncryption`; neither `id-RSASSA-PSS` nor `id-RSAES-OAEP` is accepted in the current text. Export mirrors this exactly: RSA-OAEP/RSA-PSS "spki" export constructs SubjectPublicKeyInfo with `algorithm=rsaEncryption`, `parameters=NULL`; "pkcs8" export constructs PrivateKeyInfo with `version=0`, `rsaEncryption`, `NULL`, and a DER-encoded RSAPrivateKey as the inner OCTET STRING. Appendices B/C of the spec confirm this uniformly for RSASSA-PKCS1-v1_5, RSA-PSS, and RSA-OAEP alike, noting explicitly that these OIDs carry no algorithm-specific or hash-specific information.

Historical convergence, two independent rounds – context, not current behavior. An early design (2014, W3C Bugzilla #23786/#27255) debated whether algorithm-specific OIDs (`sha256WithRSAEncryption`, `id-RSASSA-PSS`, `id-RSAES-OAEP`) should be required; real libraries (OpenSSL, BoringSSL, NSS, CAPI/CNG) uniformly rejected them, and the spec resolved toward generic `rsaEncryption`. A *second*, independent convergence occurred later (w3c/webcrypto issue #297, 2021): draft text for RSA-PSS import still validated a full RSASSA-PSS-params ASN.1 structure (hash/MGF/saltLength/trailerField) as a leftover of the earlier design; this was resolved via issue #305 toward the same simplified, generic-OID-only approach, confirmed as the current state by both the resolved spec text (above) and independent library documentation (e.g. the actively-maintained `jose` library’s source comments: “the OID `id-RSASSA-PSS`... is not supported in Web Cryptography API, use the OID `rsaEncryption`... instead for all RSA algorithms”). Two independent instances of the same convergence pattern – normative design yielding to cross-implementation interoperability reality – reinforce that **WebCrypto serialized RSA identity is generic (`rsaEncryption`)**, while **WebCrypto runtime identity is algorithm-specific** (RSA-PSS/RSA-OAEP/RSASSA-PKCS1-v1_5 + `CryptoKey.algorithm`, carried entirely outside the serialized bytes). This is a genuine candidate finding for how `rsa-ser`’s eventual contract should separate material/representation semantics from provider-object semantics, rather than attempting to encode full algorithmic identity inside the portable artifact.

DER, reinforced as a strong candidate. RSA export is confirmed to use canonical DER encoding, not merely “ASN.1-compatible” encoding, for both the outer container and the inner RSAPublicKey/RSAPrivateKey DER content.

extractable – confirmed. `[[extractable]]` is object-local `CryptoKey` state, not part of the serialized bytes. `exportKey()/wrapKey()` check `[[extractable]]=false` → `InvalidAccessError` *before* any algorithm-specific export logic runs; `importKey()`’s `extractable` argument only sets the resulting key’s slot and is not an admission condition on the serialized material. Error triage confirmed distinct: unsupported export operation → `NotSupportedError`; non-extractable key →

`InvalidAccessError`; underlying material inaccessible $\rightarrow$ `OperationError`.

keyUsages – confirmed, three rejection layers. (1) Algorithm-specific: RSA-OAEP restricts public/private usages to `{encrypt,wrapKey}/{decrypt,unwrapKey}`; RSA-PSS to `{verify}/{sign}`; an out-of-set entry $\rightarrow$ `SyntaxError`, checked inside the algorithm-specific `Generate/Import Key` steps. (2) Generic: if `[[type]]` is `secret/private` and `usages` is empty $\rightarrow$ `SyntaxError`, checked by the wrapping `SubtleCrypto` method *after* the algorithm-specific construction succeeds – a public key may legitimately have `usages=[]`. (3) Use-time: a usage absent from `[[usages]]` at `encrypt/sign/etc.` $\rightarrow$ `InvalidAccessError`, orthogonal to construction. `keyUsages` is object state, not serialized material; its only representational contact point is JWK’s `key_ops/use`, documented by the WG itself as inconsistently enforced between the two fields – deferred to the JWK inventory.

RSA mathematical-consistency validation – confirmed not normatively mandated. Four converging lines of evidence: (i) the `Import Key` algorithm text for RSA (`"pkcs8"/"spki"`) proceeds directly from successful ASN.1 parsing to `CryptoKey` construction, with no interposed step checking $n = pq$, $ed \equiv 1 \pmod{\lambda(n)}$, or the CRT relations (d_P, d_Q, q_{Inv}) – confirmed against the RSA private-key import step text; (ii) the current spec’s own table of contents places a dedicated *RSA key generation parameter validation* subsection only under `generateKey`, with no analogous subsection for `importKey` – structural asymmetry corroborating (i); (iii) a W3C WG bug (#27717, “Require RSA key import to validate the key parameters”) explicitly records that RSA import mandates no validity test (e.g. $n = pq$), by direct analogy with EC import’s point-on-curve requirement, which RSA has no equivalent of; (iv) implementer testimony (Chrome, public-webcrypto list, 2016) confirms that rejecting RSA private keys with incomplete CRT parameters is an implementation choice driven by underlying-library limitations, not a traceable spec requirement. Conclusion: `Import Key (RSA)  $\models$  ParseDER/ASN.1  $\wedge$  OID = rsaEncryption`, but `Import Key (RSA)  $\not\models$  CRT/modular coherence`. Structural validity and mathematical-semantic validity are confirmed as causally separable at the normative level; this split was subsequently adopted as a stable K_2 distinction, materialized in D-057’s frozen clauses as `Validation.Syntactic` (exact-consumption) versus `Validation.Semantic` (role-container, public-validity, private-domain, private-relations) – structural well-formedness itself (DER/container parsing) was further separated into its own `Encoding.ASN1` kind (`der-syntax, container`) rather than folded into `Validation`, once the full clause set was built.

Two-prime vs. multi-prime RSA – confirmed asymmetric by format. JWK: *explicitly supported by the normative algorithm* – `exportKey("jwk")` contains a conditional step (“if the underlying RSA private key... is represented by more than two primes, set the member named `oth`”), and `importKey("jwk")` references JWA §6.3.2.7 for `oth` without exclusion. PKCS#8: *no normative exclusion located; the full RSAPrivateKey grammar (RFC 3447 Appendix A.1.2, including otherPrimeInfos) was adopted by deliberate WG design choice* (2012 mailing-list record, explicitly motivated by not wanting a “lossy” private-key format), but no explicit per-multi-prime branching step equivalent to JWK’s was located in the `exportKey("pkcs8")/importKey("pkcs8")` algorithm text itself. WebCrypto cannot be characterized as two-prime-only at the normative level; the earlier working hypothesis to that effect is retracted. Multi-prime exclusion from $P_{RSA-ser}^{portable}$ was subsequently settled by provider intersection, not by any WebCrypto-side prohibition: Crypto++ cannot represent multi-prime material at all, and Bouncy Castle parses but discards it on materialization (D-053, §19.4).

AlgorithmIdentifier.parameters during RSA import – confirmed: checked for OID equality only, not inspected. The `importKey("spki")/("pkcs8")` steps for RSA parse the container, then check *only* that the algorithm object identifier equals the `rsaEncryption` OID ($\rightarrow$ `DataError` otherwise), then proceed directly to parsing the inner RSA key material – no step inspects or constrains the `parameters` field’s content. This contrasts directly, within the same specification family, with X25519 (WICG Secure Curves) and ML-KEM/ML-DSA (WICG Modern Algorithms), each of which has an explicit additional step immediately after the OID check: “if the `parameters` field... is present, then throw a `DataError`.” That clause exists in the same normative tradition

and was not applied to RSA’s `rsaEncryption` path. Classification: *ignored/not inspected* at the RSA import admission level – not *underspecified* (the comparison to X25519/ML-KEM is a positive contrastive confirmation, not silence) and not *NULL-required* (RFC 8017’s NULL requirement is a constraint on the serialized-profile side, confirmed for *export*; WebCrypto’s *import* algorithm does not itself expose that constraint as a rejection rule). A serialized artifact with non-NULL or absent `parameters` therefore lies outside our eventual portable profile without necessarily being rejected by every provider – relevant later for R_{val} .

This closes the WebCrypto-specific normative inventory for RSA Key Serialization admission/object semantics. The later Crypto++ and Bouncy Castle inventories provided the evidence subsequently used to resolve the transversal admission-time-vs-use-time question as D-054 (§19.5); no WebCrypto-specific evidence dependency remains open.

19.2 Backend inventory: Crypto++ RSA (closed)

Two Crypto++ surfaces, kept explicitly distinct. `Save()/Load()` (on `X509PublicKey/PKCS8PrivateKey`, inherited via `ASN1CryptoMaterial`) operate on the full interoperable container (SPKI/PKCS#8) and are the surface comparable to WebCrypto’s `"spki"/"pkcs8"`. `DEREncode*Key/BERDecode*Key` operate only on the inner RSA key material (`RSAPublicKey/RSAPrivateKey`), without the outer container, and inform implementation understanding but are not the comparison surface. `Initialize(n,e,d,p,q,dp,dq,u)` is a separate programmatic-construction path, deliberately excluded from the serialized-import capability inventory.

Container and encoding – confirmed by official documentation. `RSA::PrivateKey` $\equiv$ `PKCS#8 PrivateKeyInfo` (OID `rsaEncryption`, version, `RSAPrivateKey`); `RSA::PublicKey` $\equiv$ `X.509 SubjectPublicKeyInfo`, both on the `Save/Load` surface. Crypto++ follows a declared “write DER, read BER” philosophy (strict producer, permissive consumer) – confirmed documentally, not yet fully characterized empirically (exact BER dialect tolerated by the reader remains open, deliberately not assumed).

OID – confirmed by source. `RSAFunction::GetAlgorithmID()` returns `ASN1::rsaEncryption()` on export. Import enforces the OID strictly via `OID::BERDecodeAndCheck()`; a real-world WG-adjacent issue (weidai11/cryptopp#1181) documents that presenting `id-RSASSA-PSS` instead of `rsaEncryption` throws on import. Same generic-OID behavior as WebCrypto’s current text, reached independently (Crypto++’s code has never round-tripped through the algorithm-specific-OID debate WebCrypto’s WG had).

AlgorithmIdentifier.parameters – confirmed by source, current master: divergent from WebCrypto. `PKCS8PrivateKey::BERDecode` (confirmed against weidai11/cryptopp/blob/master/asn.cpp) computes `parametersPresent` via `algorithm.EndReached()`: if no bytes follow the OID, `parameters` is treated as legitimately absent and `BERDecodeAlgorithmParameters` is not even called; if bytes are present, `BERDecodeAlgorithmParameters` is invoked, whose inherited default (not overridden by RSA) is `{BERDecodeNull(bt); return false;}` – i.e. whatever is present must decode as ASN.1 NULL or an exception is thrown. **Rule: absence tolerated; presence must be NULL; any other value rejected.** This is a genuine, code-confirmed admission-semantics divergence from WebCrypto’s RSA import (which does not inspect `parameters` at all, §19.1) – not a difference this design should paper over.

Mathematical-consistency validation – confirmed not automatic, by three independent lines of evidence. (i) `ASN1CryptoMaterial::Load(bt)` is a one-line wrapper, `{BERDecode(bt);}` – no call to `Validate()` anywhere in its definition. (ii) Official Doxygen documentation for `InvertibleRSAFunction::Load()` states verbatim: “Load() generally does not check that the key is valid.” (iii) A real-world usage example (weidai11/cryptopp#324) shows the idiomatic pattern: `BERDecodePublicKey(...)` followed by a *separate*, manual call to `Validate(prng, level)`, which returns `bool` rather than throwing. **Load $\Rightarrow$ Validate**, confirmed independently of WebCrypto’s

own confirmed non-mandate – a transversal finding (successful serialized-key admission does not imply mathematical-consistency validation) reached by two architecturally unrelated systems, not a coincidence of one implementation’s laxity.

Trailing-data / input-consumption – confirmed not enforced at the Load() level. Tracing `PKCS8PrivateKey::BERDecode` end to end (`BERSequenceDecoder privateKeyInfo(bt) → version → nested algorithm sequence → octetString → key.BERDecodePrivateKey(...)` → successive `MessageEnd()` calls) reveals no assertion, at any point, that the original outer `bt` is fully drained after decoding. Corroborated independently by `weidai11/cryptopp#324`’s example, where the caller must manually assert `queue.IsEmpty()` because the library does not. **Confirmed for the outermost input buffer (Layer 3) and for the inner RSAPublicKey/RSAPrivateKey material (Layer 1, prior turn); the intermediate nested-SEQUENCE consumption semantics (Layer 2, exact behavior of BERSequenceDecoder::MessageEnd() on unconsumed bytes within its own scope) remain architecturally plausible but not textually verified.** Direct contrast with WebCrypto’s `exactData=true` (`DataError` on any unconsumed byte).

Two-prime vs. multi-prime – confirmed two-prime only by internal representation, the strongest form of evidence available. `InvertibleRSAFunction`’s canonical reconstruction signature, `Initialize(n,e,d,p,q,dp,dq,u)`, and its random-generation path (via `RSAPrimeSelector`, `MakeParametersForTwoPrimesOfEqualSize`) both operate over exactly the members `m_n,m_e,m_d,m_p,m_q,m_dp` – the classical two-prime CRT parameters. No container field for additional primes exists anywhere in the class. Consequently, whatever `BERDecodePrivateKey` does when confronted with `version=1+otherPrimeInfos` (reject vs. silently discard – not traced, deliberately low priority per D-discipline: the answer cannot change the conclusion), the class is structurally incapable of *preserving* multi-prime RSA. **Multi-prime RSA is excluded from $P_{RSA-ser}^{common}$ by empirical intersection with Crypto++, not by any WebCrypto-side prohibition** (WebCrypto’s own surface admits multi-prime, §19.1) – exactly the kind of profile-narrowing the D-021/D-029 intersection rule is designed to capture without an arbitrary design choice.

Observable error surface. Structural/admission failures (OID mismatch, malformed ASN.1, non-NULL parameters) converge on `CryptoPP::BERDecodeErr` – confirmed for OID mismatch (documentation + #1181) and for general malformed ASN.1 (a production crash trace in `weidai11/cryptopp#744`: `terminate called after throwing an instance of 'CryptoPP::BERDecodeErr'`); inferred with high confidence, by architectural consistency, for non-NULL parameters (routed through the same `BERDecodeNull` primitive), not independently traced empirically. By sharp contrast, mathematically-inconsistent keys and trailing bytes produce *no exception at all* at `Load()` – the former is only observable via a separate, non-throwing `Validate()` call returning `bool`; the latter is not observable through the public `Load/Save` surface at all. The programmatic-construction surface (`Initialize()`) has its own, separate error surface (e.g. `InvalidArgument` for an invalid public exponent) that must not be conflated with the serialized-import surface. **Same causal event (malformed input) does not map to a uniform observable-error kind:** structural failure throws, semantic/completeness failure is silent – an asymmetry with direct consequences for how R_{err} should be scoped for this operation, distinct from the asymmetry already found and corrected for GCM/OAEP/PSS (D-036/D-038/D-050), since here the asymmetry is architectural (exception vs. silence) rather than a misclassification within a uniformly-thrown error taxonomy.

Round-trip and canonicalization – confirmed for the output rule, plausible-but-unexecuted for byte-stability, by explicit code path. `X509PublicKey::DEREncode` calls `DEREncodeAlgorithmParameters` *unconditionally* on every export, with no branch on whether the original material had parameters absent, NULL, or (impossible, since rejected at import) anything else; the inherited default is `{DEREncodeNull(bt); return false;}`. **Confirmed: Crypto++ emits NULL for parameters on every export, regardless of the admitted input’s absent/NULL distinction** – a clean liberal-reader/canonical-writer pattern, confirmed for the public-key path by literal source; assumed by architectural symmetry, not independently verified line-by-line, for the private-key path. This yields a corrected joint admission/export table against WebCrypto (the naive three-row symmetric

table first proposed was wrong on the “other” row and is superseded here):

parameters in input	Import WebCrypto	Import Crypto++	Export WebCrypto	Export Crypto++
absent	accept	accept	NULL	NULL
NULL	accept	accept	NULL	NULL
other value	accept (not inspected)	reject (BERDecodeErr)	NULL	N/A (key never

Admission semantics and canonical-export semantics are confirmed as separable: providers can converge on output canonicalization while genuinely diverging on input admission – directly relevant to not over-specifying R_{ser} as requiring literal input-representation preservation where the contract permits canonicalization. Byte-for-byte stability of `Save(Load(S))` for Crypto++-originated and WebCrypto-originated canonical DER, and the exact BER dialect tolerated by the reader, remain architecturally plausible from the confirmed code paths above but were not executed in this environment (no network access to build Crypto++ here) – explicitly flagged as inference from confirmed source, not empirical byte-level observation, and left open for the empirical/execution phase of the project rather than for further textual tracing.

Crypto++ RSA Key Serialization backend inventory closed. The subsequently completed Bouncy Castle inventory enabled the three-backend intersection and portable-profile derivation frozen in D-053, and the admission-time-vs-use-time question was subsequently resolved in D-054 (§19.5).

19.3 Backend inventory: Bouncy Castle RSA (closed)

Two surfaces, deliberately not conflated. The lightweight API (core module: `PublicKeyFactory/PrivateKeyFactory` for import, `SubjectPublicKeyInfoFactory/PrivateKeyInfoFactory` for export) is taken as the primary comparison surface, semantically closest to WebCrypto’s `importKey/exportKey` and Crypto++’s `Save/Load`. The JCA provider surface (prov module: `KeyFactory.RSA + X509EncodedKeySpec/PKCS8EncodedKeySpec`) is treated as a contrast surface, checked specifically for divergence rather than assumed identical.

Export – confirmed by source. `SubjectPublicKeyInfoFactory/PrivateKeyInfoFactory` construct `new AlgorithmIdentifier(PKCSObjectIdentifiers.rsaEncryption, DERNull.INSTANCE)` – explicit ASN.1 NULL, not mere absence – with the private path building an `RSAPrivateKey` from exactly $n, e, d, p, q, d_P, d_Q, q_{inv}$. Converges with WebCrypto and Crypto++ on `rsaEncryption+NULL` output despite three architecturally independent implementations.

OID domain – confirmed wider than WebCrypto/Crypto++, and *not* uniform across BC’s own surfaces (no symmetry assumed). Public lightweight (core: `PublicKeyFactory`, confirmed by literal `converters.put` static block): `{rsaEncryption, id-RSASSA-PSS, id-ea-rsa}`, all dispatched to the same `RSAConverter`. JCA layer (prov: `RSAPrivateKeyFactory`, confirmed by literal array): the same three plus `id-RSAES-OAEP` (four total). Private lightweight (core: `PrivateKeyFactory`): plausibly the same four as the JCA layer, inferred with good but not literal-Java confidence from the C# port’s explicit cross-reference comment (`// TODO See RSAPrivateKeyFactory.isRsaOid in Java build`) – a genuine public/private asymmetry candidate within BC’s own lightweight surface, left flagged rather than assumed. Unrecognized OID → `IOException` (checked exception), confirmed by source for the public path.

AlgorithmIdentifier.parameters on import – confirmed not inspected, for all three confirmed public OIDs. `RSAConverter.getPublicKeyParameters` reads only `keyInfo.parsePublicKey()` (the key material) and never touches `keyInfo.getAlgorithm().getParameters()` – confirmed literal, multiple independent mirrors. Notably, this holds even for `id-RSASSA-PSS`: BC’s lightweight converter does not parse or validate the associated `RSASSA-PSS-params` structure (hash/MGF/salt-length/trailer-field) that RFC 4055/8017 defines specifically for that OID – it is dispatched to the same OID-agnostic, parameter-blind logic as `rsaEncryption`. More permissive than Crypto++

(which requires NULL if anything is present) on a wider OID domain than WebCrypto (which ignores parameters only for the single `rsaEncryption` OID).

Trailing-data admission – confirmed, and asymmetric by call-site within the same class, not merely by backend. `ASN1Primitive.fromByteArray(byte[])` – used by `PublicKeyFactory.createKey(b` – checks `aIn.available() != 0` after parsing and throws `IOException("Extra data detected in stream")` if so; this is documented contractually in the method's own Javadoc ("parsing the stream did not exhaust the available data"), functionally equivalent to WebCrypto's `exactData=true` though implemented at a generic ASN.1-utility layer, not an RSA-specific one. By contrast, `createKey(InputStream)` calls `new ASN1InputStream(inStr).readObject()` directly, with no subsequent consumption check – trailing bytes tolerated silently. **Classification depends on which overload is used**, not on "Bouncy Castle" as an undifferentiated whole – a genuine within-backend surface distinction.

Admission-time validation – confirmed partial, not full CRT consistency, and this is the operative finding for the framework's admission-time-vs-use-time distinction. `RSAKeyParameters`'s constructor unconditionally validates the modulus (odd; coprime with the product of the 131 smallest odd primes up to 743) via an automatic, non-bypassable-at-the-object-level check – confirmed by source (`core.main`), corroborated by a real production stack trace (JCA path via `KeyFactory.generatePublic`) and by a community issue (`bcgit/bc-java#449`) documenting this as a genuine interoperability cost, with an environmental escape hatch (`org.bouncycastle.rsa.allow_unsafe_mo` explicitly treated here as an environmental configuration variable, not a normal contract variant). `RSAPrivateCrtKeyParameters`, confirmed via the C# port's parallel structure, adds only per-field scalar sanity checks (p, q, d_p, d_q, q_{inv} , `publicExponent` individually non-null/non-negative) – **no cross-field arithmetic check** ($n = pq$, $ed \equiv 1 \pmod{\lambda(n)}$, $d_p = d \bmod (p-1)$, etc.) was found anywhere in the constructor. Formal result: automatic partial security validation at admission $\wedge$ no demonstrated full CRT consistency validation at admission – distinct from both WebCrypto (no validation at all) and Crypto++ (validation exists but is a separate, manually-invoked, non-automatic call).

Two-prime vs. multi-prime – confirmed as a third, distinct category: parse-but-collapse, not full support and not rejection. Three layers, kept separate: (i) *ASN.1 parser capability* – `org.bouncycastle.asn1.pkcs.RSAPrivateKey` has a dedicated `private ASN1Sequence otherPrimeInfos = null` field and documents the full PKCS#1 v2.1 grammar (`Version ::= INTEGER {two-prime(0), multi(1)}`, constrained so `version` must be `multi` iff `otherPrimeInfos` is present) – confirmed by source, BC *can* parse and hold multi-prime material, unlike Crypto++. (ii) *Internal representation capability* – the confirmed `PrivateKeyFactory` conversion path extracts only the eight classical two-prime fields into `RSAPrivateCrtKeyParameters` (whose constructor has no slot for additional primes); `otherPrimeInfos` is parsed but never read at this stage. (iii) *Export capability* – a necessary corollary of (ii), not independently traced: since `RSAPrivateCrtKeyParameters` never retained `otherPrimeInfos`, `PrivateKeyInfoFactory` cannot reproduce it. **Multi-prime RSA is excluded from $P_{RSA-ser}^{common}$ by empirical intersection with both Crypto++ and BC, for two causally distinct reasons:** Crypto++ cannot even parse it; BC parses it and discards it during materialization – a "silent semantic loss on accepted input" case that belongs to R_{ser} (representation preservation), not R_{err} (error surface), since no exception is raised at any point.

Observable error surface – confirmed heterogeneous in exception kind, with a notable API-design asymmetry. Structural/dispatch failures use checked `IOException` (unrecognized OID; trailing data on the `byte[]` overload). RSA-specific admission failures use unchecked `IllegalArgumentException` – both for security validation (insecure modulus, confirmed) and, critically, for malformed/mistyped ASN.1 structure at the `RSAPrivateKey.getInstance(...)` level (confirmed by a real-world production exception, `bcgit/bc-java#317: IllegalArgumentException: unknown object in getInstance: ...`, combined with confirmed absence of any `try/catch` around this call inside `PrivateKeyFactory.createKey`, whose signature declares only `throws IOException`). **This is a genuine API-surface hazard, not just a taxonomy note:** a caller

catching only the declared `IOException` will not catch this class of failure. CRT-incoherent-but-well-formed keys and successfully-parsed-but-collapsed multi-prime keys produce *no exception at all* – silent admission, differing causal event from silent admission in Crypto++ (which is silent for a different reason: no automatic validation call at all, versus BC’s validation call that simply does not check cross-field coherence).

Closing mini-audit (v0.5.6) of three items initially compressed out of the checklist without independent disposition. (i) *Lightweight vs. JCA contrast*: confirmed as a genuine architectural distinction, not merely assumed – the JCA public-key path (`org.bouncycastle.jcajce.provider.asymmetric`

dispatches via `BouncyCastleProvider.getPublicKey(info)`, a *third* OID-dispatch mechanism distinct from both `core.PublicKeyFactory’s converters` map and `prov.RSAUtil.rsaOids`; its own OID registry was not independently traced and is left as explicit remaining scope, not silent ambiguity. Trailing-data behavior on this path is inferred with high confidence (not line-verified) from BC’s consistent `getInstance(Object)` idiom, which routes `byte[]` input through the same `fromByteArray`-equivalent exact-consumption parsing confirmed for the lightweight `byte[]` surface. (ii) *Round-trip/canonicalization*: closed by direct analogy to the already-confirmed export behavior (`PrivateKeyInfoFactory/SubjectPublicKeyInfoFactory` build `AlgorithmIdentifier(rsaEncryption, DERNull.INSTANCE)` unconditionally from `RSAPrivateCrtKeyParameters’s` eight scalar fields) – deterministic DER encoding of the same values implies byte-stable `Save(Load(S))` for the canonical case, at the same confidence level (architecturally derived, not executed) as the equivalent Crypto++ finding; this does not warrant further code tracing. (iii) *Internal ASN.1 consumption, Layers 1/2*: retained as explicitly low priority, not silently dropped – Layer 3 (outer buffer) is already confirmed and is the layer that determines the observable behavior relevant to $R_{ser}/C_{pre}^{RSA-ser}$; a Layer-1/2 divergence would in most cases be masked by Layer 3 from the public API’s perspective.

Bouncy Castle RSA Key Serialization backend inventory closed. The v0.5.6 closing mini-audit also closes round-trip/canonicalization at architectural confidence and confirms the lightweight-vs-JCA distinction as a genuine API-surface distinction. The exact OID registry reached through `BouncyCastleProvider.getPublicKey(info)` and Layer-1/Layer-2 internal ASN.1 consumption remain explicitly lower-priority scope; neither blocks the three-backend intersection work that follows.

19.4 Three-backend RSA serialization intersection and portable profile

Following D-021, RSA key serialization is modeled as a directional import/export surface rather than as a flat Cartesian product of independent parameters. The common RSA material domain is the tagged union

$$M_{RSA}^{common} = \text{Public}(n, e) \uplus \text{PrivateCRT2}(n, e, d, p, q, d_P, d_Q, qInv).$$

The corresponding common containers are structurally coupled to key role:

$$\text{Public}(n, e) \leftrightarrow \text{SubjectPublicKeyInfo}, \quad \text{PrivateCRT2}(\cdot) \leftrightarrow \text{PrivateKeyInfo}.$$

The common serialization surface is directional:

$$P_{RSA-ser}^{common} = \left(P_{export}^{common}, D_{import}^{common} \right).$$

All three backends converge on the same emitted representation:

$$P_{export}^{common} : \begin{cases} \text{Public}(n, e) \mapsto \text{DER}(\text{SPKI}(\text{rsaEncryption}, \text{NULL}, \text{RSAPublicKey}(n, e))), \\ \text{PrivateCRT2}(\cdot) \mapsto \text{DER}(\text{PrivateKeyInfo}(\text{rsaEncryption}, \text{NULL}, \text{RSAPrivateKey}_{v0}(\cdot))). \end{cases}$$

For import, the fully characterized common dimensions are

$$\text{OID} = \text{rsaEncryption}, \quad \text{parameters} \in \{\text{absent}, \text{NULL}\},$$

with SPKI for public keys, PrivateKeyInfo for private keys, and two-prime RSA material. Multi-prime RSA is outside the common domain because Crypto++ cannot represent it and Bouncy Castle discards additional primes when materializing its operational RSA key object.

DER is accepted by all three backends and is therefore a demonstrated common import encoding:

$$\{\text{DER}\} \subseteq D_{\text{encoding}}^{\text{common,import}}.$$

The complete intersection of non-DER BER forms is not characterized and is not inferred from partial backend permissiveness. This absence of characterization is recorded as *unestablished*, not as a negative property of P^{common} – the remaining BER domain is unknown, not excluded.

The reference SDK therefore freezes the portable import profile as

$$P_{\text{RSA-ser}}^{\text{portable}} = (P_{\text{export}}^{\text{common}}, D_{\text{import}}^{\text{portable}}), \quad D_{\text{import}}^{\text{portable}} \subseteq D_{\text{import}}^{\text{common}},$$

with

$$D_{\text{import}}^{\text{portable}} = \begin{cases} \text{DER-SPKI public keys,} \\ \text{DER-PrivateKeyInfo private keys,} \\ \text{OID = } \mathbf{rsaEncryption}, \\ \text{parameters} \in \{\text{absent}, \text{NULL}\}, \\ \text{two-prime RSA material.} \end{cases}$$

Equality holds dimension-by-dimension against the fully characterized common domain – container, OID, **parameters**, key shape – since no justification exists to admit less than what all three backends already demonstrably accept there:

$$D_{\text{params}}^{\text{portable}} = D_{\text{params}}^{\text{common}} = \{\text{absent}, \text{NULL}\}.$$

Encoding is the one dimension where the portable domain is a *strict, deliberate* restriction rather than an equality, because only a lower bound on the common domain is established there:

$$D_{\text{encoding}}^{\text{portable}} = \{\text{DER}\} \subseteq D_{\text{encoding}}^{\text{common}},$$

without asserting whether this inclusion is strict, since the full non-DER BER intersection remains uncharacterized.

Thus,

$$P_{\text{RSA-ser}}^{\text{portable}} \subseteq P_{\text{RSA-ser}}^{\text{common}}.$$

The portable profile preserves the complete demonstrated common domain in dimensions that are fully characterized, while deliberately restricting encoding to DER because no larger common BER admission domain has been established. Provider-specific acceptance of additional OIDs, AlgorithmIdentifier parameters, BER forms, or multi-prime structures does not widen the SDK contract and is handled as capability/validation behavior outside the portable profile.

The resulting policy is a *bounded common-ground reader and canonical writer*: the SDK accepts exactly the demonstrated portable representation domain – not merely whatever any single backend happens to accept (a genuine liberal-reader policy), and not merely the narrowest backend’s domain – and always emits one canonical DER representation with **rsaEncryption** and explicit ASN.1 NULL. Two equalities must not be conflated going forward: canonical export is *not* the same claim as complete common import domain, and portable import domain is *not* the same claim as the union of what providers happen to accept.

This is a genuine profile-selection decision, not inherited evidence, and is frozen as D-053 (Decision Log, §24) – mirroring the discipline already used for OAEP/PSS of dedicating a separate D-XXX to each intersection-forced finding (D-028, D-033) rather than folding it into surrounding derivation prose. Deliberately *not* included in D-053: the error model, trailing-byte admission behavior, and the admission-time-vs-use-time distinction – these remain open and would contaminate this decision with dimensions not yet frozen.

19.5 Admission-time validation semantics

The timing question, framed precisely. The three backend inventories show three genuinely different validation-timing architectures: WebCrypto performs no RSA mathematical-consistency check at any point on the confirmed import path (§19.1); Crypto++ performs none automatically, exposing a separate, manually-invoked, non-throwing `Validate()` (§19.2); Bouncy Castle performs an automatic, unconditional *partial* security check (odd modulus, no small prime factor) but no cross-field CRT consistency check (§19.3). **This is not resolved by requiring identical internal validation timing across three architecturally unrelated libraries** – that would not be a meaningful or enforceable contract clause. It is resolved by fixing what the *SDK*’s import operation must observably guarantee, independent of when or whether the underlying backend itself checks it:

$$\text{Import}_{\text{SDK},p}(S) \equiv \text{Import}_{\text{SDK},q}(S)$$

with respect to the frozen admission obligations below – not $\text{Validate}_p(K)$ occurs at the same internal point $\forall p$. This is the same principle already applied to GCM’s error model ($E_p^{\text{provider}} \rightarrow E^{\text{SDK}}$): what is compared is the adapter-mediated SDK-observable outcome, not the native backend’s internal architecture or exception name.

Why this must be a stable contractual obligation, not an empirical annotation (D-010).

If RSA mathematical-consistency validation were left to use-time, a private key could pass `import` successfully across all three backends and only fail later, inside whichever consuming operation (OAEP, PSS) happens to use it first – and it would fail differently in each, since none of the three backends’ use-time failure surfaces for this cause were shown to coincide. That failure is causally owned by RSA Key Serialization (the key itself is invalid), not by the operation that happened to expose it. Attributing it to the consuming operation would violate D-010, which requires dependent failures to be attributed to their owning clause/operation, and would let a consuming operation gain spurious diagnostic credit for a defect that is not its own.

What is deliberately excluded from this obligation: provider-specific security hardening.

Bouncy Castle’s automatic small-prime-factor screen (§19.3) is retained as documented provider-behavior evidence,

$$V^{\text{provider,BC}} \supset \{\text{odd modulus, no small-prime factor, } \dots\},$$

but is *not* promoted to $C_{pre}^{RSA-ser}$. Per D-021, a single backend’s capability or internal policy does not automatically widen the common contract; here the risk runs the other way – adopting it into C_{pre} would force WebCrypto’s and Crypto++’s adapters to fabricate a check neither backend offers natively, purely because one of the three providers happens to apply it. The distinction is causal, not incidental: RSA mathematical consistency is admitted into the SDK obligation because it defines what it means for `import` to have accepted a mathematically valid RSA key (a Key-Serialization-owned property, D-010-relevant); the small-prime screen is a security-hardening policy orthogonal to key validity as defined by RFC 8017, adopted unilaterally by one implementation. Formally,

$$V_{\text{SDK}}^{\text{admission}} = V_{\text{structural}} \cup V_{\text{profile}} \cup V_{\text{mathematical}}, \quad V_{\text{SDK}}^{\text{admission}} \neq V_{\text{structural}} \cup V_{\text{profile}} \cup V_{\text{mathematical}} \cup V_{\text{provider-security}}$$

RSA mathematical consistency, precisely enumerated (RFC 8017 §3.1–3.2, two-prime case, matching PrivateCRT2, D-053). Confirmed against the RFC’s literal text, not inferred from the mere presence of the ASN.1 fields. Validity requires both domain constraints (on individual components) and relational constraints (between components); RFC 8017 imposes both, and conflating them into only the relational half (e.g. $n = pq$ alone, without also requiring p, q to be distinct odd primes) would silently under-specify validity:

$$V_{\text{domain}}^{RSA,2} = \left\{ \begin{array}{l} p, q \text{ distinct odd primes,} \\ 3 \leq e \leq n - 1, \\ \gcd(e, \lambda(n)) = 1, \\ 0 < d < n, \quad 0 < d_P < p, \quad 0 < d_Q < q, \quad 0 < q_{inv} < p \end{array} \right\},$$

$$V_{rel}^{RSA,2} = \left\{ \begin{array}{l} n = p \cdot q, \\ e \cdot d \equiv 1 \pmod{\lambda(n)}, \\ e \cdot d_P \equiv 1 \pmod{(p-1)}, \\ e \cdot d_Q \equiv 1 \pmod{(q-1)}, \\ q \cdot q_{Inv} \equiv 1 \pmod{p} \end{array} \right\}, \quad \lambda(n) = \text{lcm}(p-1, q-1),$$

$$V_{mathematical}^{RSA,2} = V_{domain}^{RSA,2} \cup V_{rel}^{RSA,2}.$$

Note the exact form of the CRT relations: RFC 8017 defines d_P, d_Q in terms of e (“ $e \cdot d_P \equiv 1 \pmod{p-1}$ ”), not the common shorthand “ $d_P = d \pmod{p-1}$ ” – the two coincide under validity but only the e -based form is the RFC’s own defining relation, and is used here accordingly.

Cross-representation consistency ($d \leftrightarrow d_P, d_Q$) is a derived invariant, not an independent sixth obligation. `RSAPrivateKey` carries both representations of the same abstract key simultaneously, and RFC 8017 does not state an explicit cross-consistency clause for that co-presence – but one is not needed as a separate requirement, since it follows necessarily from $V_{rel}^{RSA,2}$ already stated: $(p-1) \mid \lambda(n)$ gives $e \cdot d \equiv 1 \pmod{p-1}$ from $e \cdot d \equiv 1 \pmod{\lambda(n)}$; combined with $e \cdot d_P \equiv 1 \pmod{p-1}$, this yields $e(d - d_P) \equiv 0 \pmod{p-1}$; and since $e \cdot d_P \equiv 1 \pmod{p-1}$ already certifies $\gcd(e, p-1) = 1$ (an element satisfying $ax \equiv 1 \pmod{m}$ necessarily has $\gcd(a, m) = 1$), e cancels, giving $d \equiv d_P \pmod{p-1}$. The identical argument gives $d \equiv d_Q \pmod{q-1}$. Adding a sixth, independent **CRT-CROSS-CONSISTENCY** obligation or causal class would therefore be redundant with $V_{rel}^{RSA,2}$ – a risk directly relevant to $\Gamma_0^{RSA-ser}$, where an independently-stated but logically-implied condition would artificially inflate coverage/diagnostic accounting through overlapping causal classes rather than genuine new content.

Resulting admission-time boundary, frozen as D-054. `SDK import` for RSA must observably reject any artifact failing structural validity (inherited from $D_{import}^{portable}$ ’s container/encoding requirements), profile validity (OID, `parameters`, key shape – D-053), or $V_{mathematical}^{RSA,2}$ – regardless of whether the underlying backend checks the latter natively. Adapters for backends that do not check $V_{mathematical}^{RSA,2}$ automatically (WebCrypto, Crypto++, and Bouncy Castle for the cross-field relations) must perform it themselves at the SDK’s `import` boundary. Provider-specific hardening beyond $V_{SDK}^{admission}$ (e.g. BC’s small-prime screen) remains documented capability/provider metadata, not a portable obligation, and is not silently promoted into $C_{pre}^{RSA-ser}$.

19.6 Precondition contract $C_{pre}^{RSA-ser}$

Built layer by layer, preserving the frozen Encoding/Validation/Error/Capability taxonomy, rather than as a flat list that would blur representation, mathematical validity, portable-profile membership, and error normalization into a single undifferentiated precondition:

$$C_{pre}^{RSA-ser} = C_{surface} \wedge C_{struct} \wedge C_{profile} \wedge C_{math}^{role}.$$

C_{error} is deliberately not part of C_{pre} : what must be rejected belongs here; with which SDK-observable error class that rejection surfaces is the separate, subsequent error-normalization step – the same separation already frozen between R_{val} and R_{err} as distinct relations.

$C_{surface}$ – **contractual call shape**. Only what is needed to determine which operation is being requested:

$$C_{surface} = \{\text{direction} \in \{\text{import}, \text{export}\}, \text{role} \in \{\text{public}, \text{private}\}, \text{public} \leftrightarrow \text{SPKI}, \text{private} \leftrightarrow \text{PrivateKeyInfo}\},$$

the container coupling already frozen in D-053, not two independent parameters – a `PrivateKeyInfo` artifact requested as `role=public` is a profile mismatch ($C_{profile}$), not malformed ASN.1 (C_{struct}).

C_{struct} – **well-formed representation, syntax/structure only**.

$$C_{struct}^{import}(S) \iff \text{ParseDER}(S) \wedge \text{ParseContainer}(S) \wedge \text{ParseInnerRSA}(S) \wedge \text{ExactConsumption}(S),$$

$$\text{ExactConsumption}(S) \iff \neg(\exists X, |X| > 0 : S = \text{DER}(K) \parallel X).$$

`ParseContainer(S)` holds when S parses as *either* of the two admitted structures for this operation, `SPKI` or `PrivateKeyInfo` – structural well-formedness only, deliberately not yet compared against the requested `role`: a syntactically valid `PrivateKeyInfo` presented where `role=public` was requested is not malformed ASN.1, it is a role/container mismatch, and belongs exclusively to C_{profile} below. Valid DER (sole owner of the DER-vs-non-DER distinction – not restated in C_{profile}), complete ASN.1 object, admitted container shape, mandatory fields present with correct ASN.1 types, and the inner BIT STRING/OCTET STRING decodable as the corresponding RSA structure. **Exact consumption is a deliberate contract decision, not a description of provider behavior:** `Crypto++`’s `Load()` and Bouncy Castle’s `InputStream` overload both tolerate trailing bytes natively (§19.2, §19.3), but the SDK’s contractual object is “a DER representation”, not “a DER prefix followed by arbitrary data” – adopted specifically to eliminate an already-confirmed observable divergence between backend surfaces, mirroring WebCrypto’s own `exactData=true` (§19.1).

C_{profile} – membership in the D-053 portable profile.

$$C_{\text{profile}}^{\text{import}}(S, \text{role}) \iff \text{OID} = \text{rsaEncryption} \wedge \text{parameters} \in \{\text{absent}, \text{NULL}\} \wedge \text{shape} = \text{two-prime} \wedge \text{RoleContainerMatch}$$

Causally distinct from C_{struct} even when both concern the same bytes: an `AlgorithmIdentifier.parameters` field encoding `INTEGER(5)` is perfectly well-formed ASN.1 ($C_{\text{struct}} = 1$) yet fails C_{profile} ($C_{\text{profile}} = 0$) – the same holds for a syntactically valid OID that simply is not `rsaEncryption`, or a syntactically valid `PrivateKeyInfo` presented as `role=public` (`RoleContainerMatch` fails, C_{struct} does not). Each obligation has exactly one owner: DER-vs-BER and container-shape well-formedness live only in C_{struct} ; OID, `parameters`, key shape, and role/container matching live only in C_{profile} – no property is checked, or can fail, in both simultaneously. This separation yields clean, non-overlapping `Encoding.ASN1` and `Validation.Semantic` mutation targets, directly useful for $\Gamma_0^{\text{RSA-ser}}$.

$C_{\text{math}}^{\text{role}}$ – mathematical validity, asymmetric by key role.

$$C_{\text{math}}^{\text{role}} = \begin{cases} C_{\text{math}}^{\text{public}}(n, e), & \text{role} = \text{public} \\ V_{\text{domain}}^{\text{RSA},2} \wedge V_{\text{rel}}^{\text{RSA},2}, & \text{role} = \text{private} \end{cases}$$

$C_{\text{math}}^{\text{private}}$ reuses D-054’s $V_{\text{domain}}^{\text{RSA},2}/V_{\text{rel}}^{\text{RSA},2}$ verbatim – full directly-checkable RSA validity, since the CRT material exposes p, q explicitly.

$C_{\text{math}}^{\text{public}}(n, e)$ cannot reuse this pattern: RFC 8017 §3.1 defines a valid public key in terms of n ’s factorization ($n = \prod r_i$, $u \geq 2$ distinct odd primes) and $\lambda(n)$, both of which require factoring n – intractable by RSA’s own security assumption, and the RFC itself acknowledges this scope limit (§6: “these schemes... do not include steps for... validating the key”). Two normatively necessary properties are, however, directly observable from (n, e) alone without factoring, since they follow structurally for *any* valid factorization rather than requiring the actual one: n must be composite (a stronger, equally cheap consequence of $u \geq 2$ distinct odd prime factors than merely $n > 1$: the smallest possible case is $n = 3 \cdot 5 = 15$, so $n \geq 15$), and n odd (each r_i odd); similarly e must be odd, since each $r_i - 1$ is even, making $\lambda(n)$ even, and $\gcd(e, \lambda(n)) = 1$ forces e to share no factor of 2 with it. Full compositeness of n (not merely $n \geq 15$) and $\gcd(e, \lambda(n)) = 1$ in its complete form remain normatively necessary but are not directly checkable from (n, e) alone without factoring or an additional selected technique (e.g. a primality test on n).

The normative/enforceable distinction, kept explicit and not conflated with D-054’s provider-hardening exclusion. Let $V^{\text{public}, \text{normative}}(n, e)$ denote the full RFC 8017 validity condition and $C_{\text{math}}^{\text{public}}(n, e)$ the admission-time subset the SDK selects to enforce:

$$V^{\text{public}, \text{normative}}(n, e) \implies n = \prod_{i=1}^u r_i, u \geq 2, r_i \text{ distinct odd primes}, 3 \leq e \leq n-1, \gcd(e, \lambda(n)) = 1,$$

$$C_{\text{math}}^{\text{public}}(n, e) = (n \in \mathbb{Z}) \wedge (n \geq 15) \wedge (n \equiv 1 \pmod{2}) \wedge (3 \leq e \leq n-1) \wedge (e \equiv 1 \pmod{2}).$$

Compositeness of n is deliberately *not* adopted as a mandatory portable check (e.g. via a primality test), even though it is normatively necessary and directly testable, since (i) an efficient primality test in general use (Miller–Rabin) is probabilistic – finding a witness certifies compositeness, but finding none only yields “probably prime”, while deterministic polynomial-time alternatives exist but would constitute a further, currently unneeded implementation-technique decision; and (ii) doing so would mix a trivially observable necessary condition with a selected additional detection mechanism for only a fraction of invalid public keys, while the conditions that actually matter most ($n = \prod r_i$ with distinct r_i ; full $\gcd(e, \lambda(n)) = 1$) remain unverifiable regardless. **This exclusion is categorically distinct from C_{math}^{public} from D-054’s exclusion of BC’s small-prime-factor screen:** rejecting a demonstrably prime n has direct normative grounding in RFC 8017 (such an n cannot be a valid RSA modulus), whereas BC’s screen is an additional unilateral hardening policy with no such grounding – the reason for excluding both from the portable contract may coincide (not currently needed), but their evidentiary status does not.

It would also overstate the case to say RSA’s security makes a public key unverifiable in general – some invalidity is directly provable (evenness, primality of n , wrong range). What RSA’s hardness assumption specifically blocks is reconstructing, from (n, e) alone, the factorization-dependent information (r_i ’s, $\lambda(n)$) that the full RFC 8017 definition is stated in terms of.

Resulting asymmetry, frozen as D-055.

$C_{math}^{private}$ = full directly-checkable RSA validity for exposed CRT material, C_{math}^{public} = directly observable necessary

This is not an inconsistency to reconcile – it is a direct structural consequence of what RSA private vs. public material exposes, itself part of why the algorithm is secure. $C_{pre}^{RSA-ser} = C_{surface} \wedge C_{struct} \wedge C_{profile} \wedge C_{math}^{role}$ is hereby frozen in full, including **ExactConsumption** within C_{struct} .

19.7 SDK error semantics

No common native exception exists across the three backends, confirmed by the closed inventories. Crypto++ concentrates structural failures in `BERDecodeErr` but leaves mathematical inconsistency and trailing bytes silent at `Load()`; Bouncy Castle mixes `IOException`, `IllegalArgumentException`, and silent admission depending on cause (§19.2, §19.3). Following the precedent already frozen for GCM, the adapter classifies causally *before* invoking or interpreting the provider; the native exception does not itself define the SDK class.

Import: four ordered classes, evaluated by logical dependency, not arbitrary priority.

$$E_{RSA-ser}^{SDK} = \{\text{unsupported}, \text{malformed_artifact}, \text{invalid_parameter}, \text{invalid_key}\},$$

$$\text{Classify}_{RSA-ser}^{import}(S, p) = \begin{cases} \text{unsupported}, & \neg A_p^{pre}(\text{RSA-ser}, \text{import}, \text{role}) \\ \text{malformed_artifact}, & \neg C_{struct}(S) \\ \text{invalid_parameter}, & C_{struct}(S) \wedge \neg C_{profile}(S, \text{role}) \\ \text{invalid_key}, & C_{struct}(S) \wedge C_{profile}(S, \text{role}) \wedge \neg C_{math}^{role}(S) \\ \text{success/delegate}, & \text{otherwise.} \end{cases}$$

The order $C_{struct} \rightarrow C_{profile} \rightarrow C_{math}$ is evaluability dependency, not preference: `RoleContainerMatch`/`OID`/`parameter` cannot be asked of a structure not yet successfully parsed, and RSA mathematical semantics cannot be evaluated until the artifact is first established as the portable RSA material the profile claims it to be.

SDK error class is determined by contractual cause, not by native exception – confirmed as a genuine three-way pattern, not a hypothetical:

- *Provider rejects, adapter normalizes* – e.g. Crypto++’s `BERDecodeErr` on malformed ASN.1 maps to `malformed_artifact`.

- *Provider accepts, adapter actively rejects* – BC natively admits `id-RSASSA-PSS` and `id-ea-rsa` for RSA import (§19.3), but these lie outside the D-053 portable profile; the adapter must reject an artifact its own underlying backend would accept, producing `invalid_parameter` with no native exception to draw on.
- *Provider performs no check, adapter validates and rejects* – $V_{\text{mathematical}}^{\text{RSA},2}$'s cross-field CRT relations are checked automatically by none of the three backends in full (§19.2, §19.3); the adapter itself must implement and enforce D-054's obligation to produce `invalid_key`.

This is not a symmetric convenience: for `invalid_key` specifically, the adapter typically cannot delegate to the backend at all (WebCrypto: never checks; Crypto++: only via a manual, non-automatic `Validate()` call the adapter would have to invoke itself; BC: covers only the modulus-security subset, never the CRT cross-relations) – confirming that D-054's admission-time obligation is doing real work here, not restating something backends already provide.

Export is not the mirror image of import, and modeling it as one would be a category error. Import classifies an untrusted artifact S against $C_{\text{struct}}/C_{\text{profile}}/C_{\text{math}}$, which are properties of that artifact. Export starts from a key object the SDK has already admitted – via import (already satisfying $C_{\text{pre}}^{\text{RSA-ser}}$) or via SDK-side generation (expected to satisfy the same key-material contract by construction. For export, representation properties are *postconditions*, not preconditions:

$$\text{Export}_{\text{SDK},p}(K) = S \implies C_{\text{ser}}^{\text{export}}(S).$$

Re-deriving `malformed_artifact` (an input-side notion for a value export does not consume as untrusted bytes), `invalid_parameter` (OID/parameters are writer-determined output, not caller input to reject), or `invalid_key` (the key was already admitted valid) as *legal export error outcomes* would be a category error: if an adapter's writer produces a malformed SPKI/PrivateKeyInfo from a valid key, that is an R_{ser} conformance defect in the implementation, not a legal error the SDK contract defines for the caller.

$$\text{Classify}_{\text{RSA-ser}}^{\text{export}}(K, p) = \begin{cases} \text{unsupported}, & \neg A_p^{\text{pre}}(\text{RSA-ser}, \text{export}, \text{role}) \\ \text{success}, & \text{otherwise,} \end{cases}$$

under the precondition that K is an already-admitted SDK key. **No catch-all contractual provider error.** If, after capability and precondition checks pass, the backend fails unexpectedly during export (e.g. an `OperationError` with no contractual cause in this model), the adapter does not invent an `internal_error` class to absorb it – such a failure is a realization/conformance defect of I_p , exactly the kind of divergence the experimental design exists to surface, not to normalize away.

Directional applicability, not forced symmetry:

	import	export
<code>unsupported</code>	✓	✓
<code>malformed_artifact</code>	✓	–
<code>invalid_parameter</code>	✓	–
<code>invalid_key</code>	✓	–

The four classes remain the common operation-level error vocabulary, but only `unsupported` is a legal export outcome – a directional error model, mirroring the directional $P_{\text{RSA-ser}}^{\text{common}}/\text{portable}$ already frozen in D-053 rather than a flat exception bag forced into artificial symmetry.

extractable/exportability: confirmed excluded from the portable contract, not merely deferred. WebCrypto's `exportKey()` can throw `InvalidAccessError` when `[[extractable]]`=`false` (§19.1), but `extractable` is object-local `CryptoKey` state, not part of the serialized bytes – already established when the representation/object-policy boundary was drawn. Promoting it into $E_{\text{RSA-ser}}^{\text{SDK}}$

as an exportability class would reintroduce, through the error model, a property D-053 deliberately excluded from the representation contract. The obligation frozen here is observable, not implementational: *provider-local exportability state must not leak into the RSA serialization contract*. How an adapter guarantees this (e.g. constructing internal `CryptoKey` objects with `extractable=true`) is a later implementation decision, not part of D-056.

Provisional normative fiche summary:

Dimension	Normative basis
RSA public material	RFC 8017 A.1.1
RSA private material	RFC 8017 A.1.2
Public container	RFC 5280
Private container	RFC 5208 (obsoleted) / RFC 5958
Algorithm OID	RFC 8017
Algorithm parameters (export)	RFC 8017/X.509 convention
Algorithm parameters (import, RSA)	WebCrypto Import Key steps
Binary encoding	ASN.1 BER/DER
Textual encoding	RFC 7468
Multi-prime RSA	RFC 8017 permits it

20 RSA Key Serialization – frozen contract clauses

Thirteen clauses, each a stable contractual obligation derived directly from $C_{pre}^{RSA-ser}/E_{RSA-ser}^{SDK}$ (D-053–D-056, §19.4–§19.7), frozen as D-057.

Audited before freezing: single-owner causal chain, no double-violation and no ownerless gap. An earlier draft paired `public-container/private-container` against `role-container` and risked a single stimulus (a well-formed `PrivateKeyInfo` presented as `role=public`) violating two clauses at once. Restricting `*-container`’s applicability to “only once the container type is already identified” fixed that case but opened a new one: malformed DER, or DER that parses but instantiates neither admitted container shape, had no owning clause at all – a genuine gap against D-055/D-056, which already classify that defect as `malformed_artifact`. The corrected chain, ordered by evaluability dependency exactly as D-056 already establishes at the C_{pre} level:

`der-syntax` → `exact-consumption` → `container` → `role-container` → `algorithm-id` → `algorithm-params`

with every case below owned by exactly one clause: malformed DER → `der-syntax`; valid DER with trailing bytes → `exact-consumption`; valid DER instantiating neither container shape → `container`; a correctly-typed container for the wrong role → `role-container`; a correctly-typed, correctly-roled container with the wrong OID → `algorithm-id`; correct OID but out-of-profile parameters → `algorithm-params`.

public-container/private-container merged into a single container clause. Both shared identical K_1/K_2 (Encoding.ASN1), the same contractual nature, and were correlated branches of one obligation rather than distinct semantic categories; separating them by role added no independent causal content once role-matching itself is owned exclusively by `role-container`.

public-material/private-material redefined as semantic preservation, not field presence. `public-material` : $(n, e)_{\text{after}} = (n, e)_{\text{contract}}$; `private-material` : $(n, e, d, p, q, d_P, d_Q, q_{\text{Inv}})_{\text{after}} = (\dots)_{\text{contract}}$. This separates two genuinely distinct defects that field-presence framing would have conflated: a serialization that silently alters e while still producing a mathematically valid key violates `public-material` (a value-preservation defect), not `public-validity` (a validity defect); a key with every field faithfully preserved but internally incoherent violates `private-relations`, not `private-material`.

No `rsa-ser.role`, `rsa-ser.output`, `rsa-ser.roundtrip`, or `rsa-ser.interop`. `role` is part of the contractual call shape (C_{surface}) – a selector determining which clauses apply, not an independently violable representation obligation; every observable obligation it could carry is already covered by `public-material/private-material/role-container`. `output` would duplicate the same obligations already stated by `container/algorithm-id/algorithm-params` under a different name – RSA Key Serialization has no Input/Output-shaped transform the way HKDF or GCM do, and the shared K_1 taxonomy does not require every operation to instantiate every kind. $R_{\text{ser}}/R_{\text{interop}}$ are relations that *observe* clauses (e.g. `container` via R_{ser} , a producer/consumer mismatch via R_{interop}); they are not themselves clause IDs, mirroring the discipline already applied when relation columns were deliberately omitted from the PSS clause table (§16).

ID	K1	K2	Frozen contract clause
<code>rsa-ser.public-material</code>	K_1	—	A portable public key’s material is exactly <code>Public(n, e): $(n, e)_{\text{after}} = (n, e)_{\text{contract}}$</code> across import/export.

ID	K1	K2	Frozen contract clause
<code>rsa-ser.private-material</code>	Material	—	A portable private key's material is exactly <code>PrivateCRT2($n, e, d, p, q, d_P, d_Q, q_{Inv}$)</code> : all eight components preserved exactly across import/export; multi-prime material is outside the portable profile (D-053).
<code>rsa-ser.der-syntax</code>	Encoding	ASN1	The serialized artifact must be a syntactically valid DER encoding of one complete ASN.1 object. Scope is parsing alone – it does not determine which object.
<code>rsa-ser.container</code>	Encoding	ASN1	After successful DER parsing, the artifact must instantiate one of the two portable RSA container structures: <code>RSAPublicKey</code> , or RFC 5208-compatible <code>PrivateKeyInfo</code> containing a two-prime <code>RSAPrivateKey</code> .
<code>rsa-ser.exact-consumption</code>	Validation	Syntactic	Import consumes exactly one complete DER artifact; trailing bytes ($S = \text{DER}(K) \ X$, $ X > 0$) are rejected regardless of native backend tolerance (D-055).
<code>rsa-ser.role-container</code>	Validation	Semantic	SPKI satisfies only <code>role=public</code> ; <code>PrivateKeyInfo</code> satisfies only <code>role=private</code> . A syntactically valid, correctly-typed container presented for the wrong role is rejected. Evaluated only once a container type has been identified by <code>rsa-ser.container</code> .
<code>rsa-ser.algorithm-id</code>	Parameter	AlgorithmChoice	<code>AlgorithmIdentifier.algorithm</code> must equal <code>rsaEncryption</code> (D-053); wider provider-specific OID acceptance (e.g. BC's <code>id-RSASSA-PSS/id-ea-rsa</code>) does not widen the portable profile.
<code>rsa-ser.algorithm-parameters</code>	Encoding	ASN1	<code>AlgorithmIdentifier.parameters</code> portably admits <code>absent</code> or ASN.1 NULL on import; canonical export always emits explicit NULL (D-053). Classified as encoded form, not algorithm choice, since the field carries no cryptographic degrees of freedom for <code>rsaEncryption</code> .
<code>rsa-ser.public-validation</code>	Validation	Semantic	Public-key admission enforces $C_{\text{math}}^{\text{public}}$ (D-055): $n \geq 15$, n odd, $3 \leq e \leq n-1$, e odd – the necessary conditions directly observable from (n, e) without factoring.

ID	K1	K2	Frozen contract clause
<code>rsa-ser.private-domain</code>	Validation	Semantic	Private-key admission enforces $V_{domain}^{RSA,2}$ (D-054): p, q distinct odd primes; range conditions on e, d, d_P, d_Q, q_{Inv} .
<code>rsa-ser.private-relational</code>	Validation	Semantic	Private-key admission enforces $V_{rel}^{RSA,2}$ (D-054): $n = pq$; $ed \equiv 1 \pmod{\lambda(n)}$; $ed_P \equiv 1 \pmod{p-1}$; $ed_Q \equiv 1 \pmod{q-1}$; $qq_{Inv} \equiv 1 \pmod{p}$. Cross-representation consistency ($d \leftrightarrow d_P, d_Q$) is a proven consequence of these relations, not a separate clause.
<code>rsa-ser.error</code>	Error	—	Observable rejections follow D-056’s directional classifier: <code>unsupported</code> , <code>malformed_artifact</code> , <code>invalid_parameter</code> , <code>invalid_key</code> for import; only <code>unsupported</code> is a legal export outcome.
<code>rsa-ser.cap</code>	Capability	—	RSA Key Serialization import/export and the requested role/direction must be present in the backend’s pre-registered capability manifest.

Relation columns deliberately omitted, following the same discipline already frozen for PSS (§16): fixed clause-to-relation assignments would prematurely bind $R_{cap}/R_{val}/R_{ser}/R_{interop}$ to clauses rather than deriving them per causal mutation class once $\Gamma_0^{RSA-ser}$ is constructed.

21 RSA Key Serialization – frozen $\Gamma_0^{RSA-ser}$

Seventeen causal classes across five families, frozen jointly with their Γ_0 assignment as D-058 – mirroring the GCM/PSS precedent (D-049) of freezing the class set and its causal ground truth as a single decision rather than two, since a class is not fully characterized until its Γ_0 is known, and D-007 requires Γ_0 pre-registration before mutation execution.

Import/export asymmetry (D-056) propagates into the causal classes. A reader admitting an out-of-contract artifact and a writer producing one can implicate the same clause but are causally distinct mechanisms triggering distinct relations (R_{val} for reader-side bypass, R_{ser} for writer-side divergence); four clauses (`der-syntax`, `container`, `algorithm-id`, `algorithm-params`) are accordingly observed by a paired import/export class rather than one, while `exact-consumption` has no export equivalent (D-055 defines it as an import-only obligation) and `public-material/private-material` are role-exclusive by construction (D-053’s role/container coupling), never co-applicable to the same artifact.

Deliberately not created: a fifth Family-D split into per-equation classes, and a catch-all provider-error class. `RSA-SER-PRIVATE-RELATIONAL-BYPASS` covers all of $V_{rel}^{RSA,2}$ ’s five relations ($n = pq$; $ed \equiv 1$; $ed_P \equiv 1$; $ed_Q \equiv 1$; $qq_{Inv} \equiv 1$) as one class with multiple independent stimuli, not five classes – each stimulus attacks the same stable contractual obligation and is expected to resolve to the same D-056 outcome (`invalid_key`); splitting them would approach the mutation_{*i*} $\leftrightarrow$ clause_{*i*} degeneracy D-016 exists to prevent. Similarly no `UNEXPECTED-PROVIDER-ERROR` class: D-056 already ruled an unexpected post-precondition backend failure a conformance defect of I_p , not a contractual

cause; inventing a catch-all causal class here would reintroduce through Γ_0 exactly what D-056 excluded from the contract.

Coverage and boundary audit, performed before freezing. All thirteen `rsa-ser.*` clauses (D-057) are covered by at least one class, with no clause left ownerless and no class without a clause – four clauses (`der-syntax`, `container`, `algorithm-id`, `algorithm-params`) map to two classes each via the import/export split, the remaining nine map to exactly one. All seventeen classes were checked for genuine (not merely textual) double-violation risk by constructing a minimal isolating stimulus for each: a non-canonical-but-decodable BER artifact isolates `RSA-SER-DER-IMPORT-BYPASS` without implicating `container` (the decoded content can still conform to SPKI/RSAPublicKey); a validly-DER-encoded artifact of neither admitted shape isolates `RSA-SER-CONTAINER-IMPORT-BYPASS` without implicating `role-container` (no identified container type exists yet to compare against role, mirroring the same evaluability-dependency reasoning already applied when `der-syntax` was split out in D-057); and a single-variable OID or `parameters` mutation, holding all other fields canonical, isolates `RSA-SER-OID-IMPORT-PROFILE-BYPASS` from `RSA-SER-PARAMETERS-IMPORT-PROFILE-BYPASS` and vice versa. No genuine double-violation was found; apparent overlap risk in all cases resolves to a single-variable stimulus-construction discipline (already required by D-016) rather than a clause-boundary defect.

Family A – Semantic material preservation (R_{ser} is the primary detector; role-exclusive, never co-applicable to the same artifact per D-053):

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
RSA-SER-PUBLIC-MATERIAL-DIVERGENCE	public-material	Key	R_{ser} ; $R_{interop}$ when a cross-provider export/import chain surfaces the divergence. Isolated by substituting the entire valid (n, e) tuple for a different valid one – not corrupting a single field, which would instead implicate public-validity.

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
RSA-SER-PRIVATE-MATERIAL-DIVERGENCE	private-material	Key	R_{ser} ; $R_{interop}$. Isolated by substituting the entire valid CRT octuple for a different valid one, keeping every component internally consistent – corrupting one component would instead implicate private-domain/private-r

Family B – Representation: reader vs. writer (R_{val} for import-side bypass; R_{ser} for export-side divergence):

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
RSA-SER-DER-IMPORT-BYPASS	der-syntax	Encoding	R_{val} . Reader admits non-DER/syntactically invalid ASN.1, e.g. decodable-but-non-canonical BER, contradicting the deliberate ExactConsumption-adjacent DER-only decision (D-055).
RSA-SER-DER-EXPORT-DIVERGENCE	der-syntax	Encoding	R_{ser} . Writer produces output that does not satisfy DER, independent of der-syntax's import-side reader obligation.
RSA-SER-CONTAINER-IMPORT-BYPASS	container	Encoding	R_{val} . Reader admits valid DER instantiating neither SPKI nor portable PrivateKeyInfo.

Mutation class	$\Gamma_0(c)$	$\mathbf{Kinds}_0(c)$	Expected relations
RSA-SER-CONTAINER-EXPORT-DIVERGENCE	container	Encoding	R_{ser} . Writer produces a container structure outside the two frozen forms.
RSA-SER-TRAILING-DATA-BYPASS	exact-consumption	Validation	R_{val} . Import accepts $S = \text{DER}(K) \ X$, $ X > 0$; no export equivalent exists since <code>ExactConsumption</code> is explicitly import-only (D-055). Isolated by keeping K itself fully valid, so only trailing-byte tolerance is under test.

Family C – Portable profile (R_{val} for import-side profile bypass; R_{ser} for export-side canonicalization divergence):

Mutation class	$\Gamma_0(c)$	$\mathbf{Kinds}_0(c)$	Expected relations
RSA-SER-ROLE-CONTAINER-BYPASS	role-container	Validation	R_{val} . A correctly-typed, otherwise-canonical container is admitted under the wrong requested role.
RSA-SER-OID-IMPORT-PROFILE-BYPASS	algorithm-id	Parameter	R_{val} . Import admits an OID other than <code>rsaEncryption</code> that a specific backend accepts natively (e.g. BC's <code>id-RSASSA-PSS/id-ea-rsa</code>), which the adapter must actively reject.

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
RSA-SER- OID-EXPORT- DIVERGENCE	algorithm-id	Parameter	R_{ser} . Export produces an OID other than <code>rsaEncryption</code> .
RSA-SER- PARAMETERS- IMPORT-PROFILE- BYPASS	algorithm-params	Encoding	R_{val} . Import admits <code>AlgorithmIdentifier.params</code> outside $\{\text{absent}, \text{NULL}\}$.
RSA-SER- PARAMETERS- EXPORT- CANONICALIZATION- DIVERGENCE	algorithm-params	Encoding	R_{ser} . Export fails to produce the canonical explicit NULL.

Family D – Mathematical validity of the key (import-only; R_{val} throughout, deliberately not deferred to a consuming operation’s $R_{interop}$, per D-054/D-010):

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
RSA-SER-PUBLIC- VALIDITY-BYPASS	public-validity	Validation	R_{val} . Import admits (n, e) violating any of D-055’s selected C_{math}^{public} conditions ($n \geq 15$, n odd, $3 \leq e \leq n - 1$, e odd).
RSA-SER-PRIVATE- DOMAIN-BYPASS	private-domain	Validation	R_{val} . Import admits a private key violating $V_{domain}^{RSA,2}$ (D-054): non-prime/non-distinct p, q , or a component out of its required range.

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
RSA-SER-PRIVATE-RELATIONAL-BYPASS	private-relations	Validation	R_{val} . Import admits a private key violating $V_{rel}^{RSA,2}$ (D-054); one class, exercised by multiple independent stimuli ($n \neq pq$; $ed \neq 1$; $ed_P \neq 1$; $ed_Q \neq 1$; $qqInv \neq 1$), all expected to resolve to <code>invalid_key</code> (D-056).

Family E – Error and capability (mirroring the pattern already established for OAEP/PSS):

Mutation class	$\Gamma_0(c)$	$\text{Kinds}_0(c)$	Expected relations
RSA-SER-ERROR-MISCLASSIFICATION	error	Error	R_{err} . The SDK correctly rejects but returns the wrong D-056 class (e.g. <code>malformed_artifact</code> where <code>invalid_parameter</code> was contractually due).
RSA-SER-PROVIDER-CAPABILITY-MISREPORT	cap	Capability	R_{cap} . Pre-registered capability manifest and observed backend behavior diverge for RSA Key Serialization import/export.

21.1 RSA Key Serialization: accumulated audit and closure

A logical (not re-empirical) cumulative audit was performed across D-053–D-058 before closing this operation. The primary-source verification (RFC 8017, W3C Editor’s Draft, Crypto++/Bouncy Castle source) is not repeated here – it is already complete in §19.1–§21. Seven cross-decision invariants were checked directly against the frozen text:

1. **D-053** $\leftrightarrow$ **D-055**. Everything $P_{RSA-ser}^{portable}$ admits is expressible via $C_{pre}^{RSA-ser}$ (C_{struct} covers the DER/container requirement; $C_{profile}$ restates `OID/parameters/shape` exactly), and C_{pre}

introduces no obligation contradicting the portable profile – C_{math}^{role} adds mathematical validity beyond what D-053 states, but D-053 is a representation-only profile and does not preclude additional admission obligations. Consistent.

2. **D-055 ↔ D-056.** The triage $\neg C_{struct} \rightarrow \text{malformed_artifact}$, $C_{struct} \wedge \neg C_{profile} \rightarrow \text{invalid_parameter}$, $C_{struct} \wedge C_{profile} \wedge \neg C_{math}^{role} \rightarrow \text{invalid_key}$ matches D-056’s frozen classifier verbatim. Consistent.
3. **D-055 ↔ D-057.** Every $C_{pre}^{RSA-ser}$ obligation has exactly one owning clause among the thirteen, confirmed by direct mapping (surface→role/container coupling; struct→der-syntax+container+exact-con profile→role-container+algorithm-id+algorithm-params; math→public-validity/private-domain). No duplication survived the two D-057 audit passes. Consistent.
4. **D-057 ↔ D-058.** All thirteen clauses are covered by at least one of the seventeen classes (four by two, via the import/export split; nine by exactly one); no class depends on a nonexistent clause; no class was created merely to reproduce a stimulus (Family D’s coarse private-relations grouping is the clearest evidence of this discipline being actively applied, not merely stated). Consistent.
5. **Import/export asymmetry.** No malformed_artifact/invalid_parameter/invalid_key-flavored class exists on the export side; all four export-side classes (DER-EXPORT-DIVERGENCE, CONTAINER-EXPORT-DIVERGENCE, OID-EXPORT-DIVERGENCE, PARAMETERS-EXPORT-CANONICALIZATION-DIVERGENCE) carry R_{ser} as their expected relation, matching D-056’s principle that export representation defects are conformance failures, not legal SDK errors. Consistent.
6. **Expected-relations columns are predictions, not covert Γ_0 .** Each cell names a relation already defined transversally in the framework (R_{val} , R_{ser} , R_{err} , R_{cap} , $R_{interop}$) and states what that relation would detect if the class’s Γ_0 clause were violated; none of the seventeen cells restates $\Gamma_0(c)$ itself or defines a relation ad hoc for this operation. No circularity found.
7. **D-010 and dependency attribution.** By construction, D-054 moved RSA mathematical-consistency checking to admission time specifically so that a badly-admitted key’s eventual failure inside OAEP/PSS is never the first point of detection; Family D’s three classes (D-058) all carry R_{val} , evaluated at RSA Key Serialization’s own admission boundary, not deferred to a consuming operation’s $R_{interop}$. Consistent with D-010.

Stale-text sweep. A document-wide search for residual pending/left open/candidate language tied to RSA Key Serialization surfaced five remaining fragments predating D-053–D-058 (the Validation.Structural/Semantic split question, resolved by D-057’s Encoding/Validation.Syntactic/Validation.Semantic split; the multi-prime portable-profile question, resolved by D-053; the WebCrypto-section closing sentence and the corresponding Open items entry, both still phrased as now-pending for admission-time-vs-use-time, resolved by D-054; and two normative-fiche summary-table rows for attributes/publicKey still marked pending provider audit, resolved by D-053’s frozen container shape) – all corrected in place.

RSA Key Serialization = CLOSED. The full chain – normative fiche → three backend inventories → $P_{RSA-ser}^{common}/P_{RSA-ser}^{portable}$ (D-053) → admission-time validation semantics (D-054) → $C_{pre}^{RSA-ser}$ (D-055) → directional SDK error semantics (D-056) → thirteen clauses (D-057) → seventeen causal classes with $\Gamma_0^{RSA-ser}$ (D-058) – passes accumulated audit with no open transversal contradiction. This operation is not reopened during EC P-256 construction unless EC evidence reveals a genuine cross-operation contradiction, not merely a stylistic difference in how EC’s own key material is shaped.

22 EC P-256 Key Serialization – normative fiche

Scope, fixed from the start. EC P-256 key import/export/validation only – not ECDSA, not ECDH. Four things must be characterized before any provider is examined: key material, container, point encoding, and mathematical membership/validity. Following the RSA-ser precedent (§19–§21.1), but deliberately not mirroring it structurally where EC’s own obligations differ – point representation, curve membership, and private/public consistency are genuinely new dimensions with no RSA analogue, which is precisely why EC was retained as a separate candidate (D-015) rather than treated as “RSA serialization with different ASN.1.”

Discipline for this fiche, adopted explicitly to avoid repeating RSA-ser’s own stale-text lesson. Three states are kept visually and textually distinct throughout: *normatively established* (fixed by RFC/FIPS/SEC text, independent of any backend), *provider-dependent* (requires the three-backend inventory before any claim is made), and *portable-profile decision* (requires provider intersection, per D-021, before being frozen). Nothing below is asserted as portable-profile content; that begins only once WebCrypto/Crypto++/Bouncy Castle evidence exists.

22.1 Normatively established

Curve identity. P-256 = secp256r1 = prime256v1, confirmed via RFC 5480’s own normative note: “[FIPS186-3] refers to... secp256r1 as P-256”. OID secp256r1 = 1.2.840.10045.3.1.7 (confirmed, OID repository and RFC 5480’s ECPParameters registration). NIST retains P-256 among its recommended curves; domain parameters are now normatively centralized in SP 800-186, complementary to FIPS 186-5.

Fixed mathematical domain. $E/\mathbb{F}_p : y^2 = x^3 + ax + b \pmod{p}$, with fixed public parameters (p, a, b, G, n, h) , per FIPS 186-5’s EC domain parameter description (field, coefficients, generator G , order n , cofactor h). For secp256r1, $h = 1$ – confirmed directly as a domain parameter (not requiring a separate RFC citation: it is the same kind of fixed constant as p, a, b, G, n , verifiable from the curve’s own parameter set).

Contractual material candidates.

PublicEC(Q), $Q = (x, y)$; PrivateEC(d, Q), $1 \leq d \leq n-1$, and $Q = dG$ when Q is present/materialized.

A genuine departure from RSA-ser: EC carries a natural mathematical membership obligation on Q that is not merely ASN.1 validation, and a distinct private/public *consistency* question ($Q = dG$) that RSA-ser has no analogue of.

Public container – RFC 5480. SubjectPublicKeyInfo = (AlgorithmIdentifier, subjectPublicKey). AlgorithmIdentifier.algorithm = id-ecPublicKey = 1.2.840.10045.2.1 (confirmed, RFC 5480 literal ASN.1 module text). **Sharp divergence from RSA-ser:** RFC 5480 requires parameters to be *present* and non-NULL – “The parameter for id-ecPublicKey is as follows and *MUST* always be present: ECPParameters ::= CHOICE {namedCurve OBJECT IDENTIFIER, implicitCurve NULL, specifiedCurve SpecifiedECDomain}” – with “implicitCurve and specifiedCurve *MUST NOT* be used in PKIX” (confirmed verbatim). For this profile, the natural candidate is therefore

$SPKI_{P-256} = (\text{AlgorithmIdentifier}(\text{id-ecPublicKey}, \text{namedCurve}(\text{secp256r1})), \text{BIT STRING}(\text{ECPoint}(Q)))$,

with *algorithm OID* and *curve OID* kept as two independent dimensions from the outset – not fused into one, mirroring the discipline that separated *algorithm-id/algorithm-params* in RSA-ser (D-057), but here for a structurally different reason: RSA had one obligatory value (NULL) split from OID identity; EC has two independently-varying OIDs (algorithm identity vs. curve identity) inside the same field.

Point encoding – SEC 1/X9.62, referenced by RFC 5480. Uncompressed form *MUST* be supported; compressed form *MAY* be supported; hybrid form *MUST NOT* be used; any leading

octet outside the admitted values must be rejected (normatively established encoding rule, not yet a portable-profile claim about which forms this SDK admits). For P-256, uncompressed is $04\|X_{32}\|Y_{32}$, each coordinate exactly 32 big-endian octets with left zero-padding as needed – 65 bytes total; independently confirmed as the representation TLS 1.3 uses for secp256r1. Compressed is $02\|X$ or $03\|X$.

Private container – RFC 5915.

$\text{ECPrivateKey} ::= \text{SEQUENCE}\{\text{version}, \text{privateKey}, \text{parameters} [0] \text{ OPTIONAL}, \text{publicKey} [1] \text{ OPTIONAL}\}$,

with $\text{version} = 1$ fixed, and privateKey the private integer as an OCTET STRING of length $\lceil \log_2 n/8 \rceil = 32$ bytes for P-256. For PKCS#8 distribution, RFC 5915 itself describes $\text{PrivateKeyInfo} \supset \text{privateKeyAlgorithm}(\text{id-ecPublicKey}, \text{namedCurve}) + \text{OCTET STRING}(\text{ECPrivateKey})$, giving the RSA-ser-comparable candidate

$$\text{PKCS8}_{P-256} = \text{PrivateKeyInfo}(\text{id-ecPublicKey}, \text{secp256r1}, \text{ECPrivateKey}(d, \dots)).$$

Basic mathematical conditions, established. Scalar range $1 \leq d < n$. Membership of Q on the curve (full statement in §22.1 below). $h = 1$ is a domain property of P-256 specifically – no separate subgroup-membership obligation is provisionally created beyond curve membership, since for $h = 1$ a valid finite curve point already lies in the correct (only) subgroup. This is stated as a property of *this curve*, not a general EC framework simplification; it would not transfer to a curve with $h > 1$ without separate justification.

Public-key membership, established as a mathematical fact independent of any provider.

$$V_{\text{membership}}^{P-256}(Q) = (Q \neq \mathcal{O}) \wedge (x, y \in \mathbb{F}_p) \wedge (y^2 \equiv x^3 + ax + b \pmod{p}),$$

per SEC 1’s EC public-key validation procedure, restricted to P-256’s own domain parameters. This gives a genuine, non-inflated $K_1 = \text{Membership}$ instantiation – distinct from ASN.1/DER validation and distinct from RSA-ser’s C_{math} , since RSA has no comparable curve-membership structure.

22.2 Explicitly OPEN – provider inventory required, not decided here

- Compressed-point acceptance on import, and compressed-point production on export.
- Exact DER-vs-BER tolerance for the outer container and the inner **ECPoint**.
- Whether any backend tolerates explicit EC parameters (**specifiedCurve**) despite RFC 5480’s PKIX-context prohibition – a candidate we do not project onto EC generally, only note as a question, mirroring how RSA-ser did not project the WICG Secure Curves **parameters-forbidden** rule onto RSA without direct evidence.
- Treatment of **ECPrivateKey.parameters** [0]: presence/absence tolerance, and whether a present-but-mismatched **parameters** [0] is rejected, ignored, or used.
- Treatment of **ECPrivateKey.publicKey** [1]: presence/absence tolerance, and whether it is validated against **privateKey**, silently accepted, or silently regenerated.
- Whether any backend reconstructs Q from d when **publicKey** [1] is absent, and under what conditions.
- Whether $Q = dG$ is actively checked by any backend when both components are present in a single artifact.
- What curve/point membership validation each backend performs automatically at import (admission-time vs. use-time, mirroring the axis already resolved for RSA-ser as D-054 – not assumed to resolve the same way here without evidence).

- What scalar-range validation ($1 \leq d < n$) each backend performs automatically.
- Error/exception surface for each rejection cause, per backend.
- Divergence between multiple surfaces of the same provider (e.g. lightweight vs. JCA for Bouncy Castle, already confirmed as a real axis of divergence for RSA-ser, §19.3) – not assumed absent for EC.

Deliberately not yet frozen: $Q = dG$ as a portable admission obligation. It is mathematically the correct relation between the two components of one key pair, but which private representations in our compared universe contain/preserve Q at all, which derive it, and what the SDK’s contractual boundary will be must first be established from provider evidence. Recorded here as a *consistency property to investigate*, not a precondition – keeping separate three things that must not be conflated: mathematical truth, serialized-field requirement, and SDK admission obligation (the same three-way separation D-055 relied on for RSA’s $V_{rel}^{RSA,2}$, arrived at only after normative grounding, not assumed a priori).

No new Decision Log entry. This section records normative evidence and open research questions, not a change to C_{pre} , Γ_0 , a metric, or the portable profile – the same discipline already applied when WebCrypto’s RSA inventory opened without a D-0XX (§19.1).

22.3 Backend inventory: WebCrypto EC P-256 (closed)

Evidence basis and its status. Taken from Web Cryptography API Level 2’s ECDSA/ECDH normative algorithms, currently a First Public Working Draft – recorded here explicitly as such, not as Recommendation status; the 2017 Recommendation remains the only W3C Recommendation-status WebCrypto text. Already substantially more closed at inventory-open time than RSA-ser’s WebCrypto inventory was, since the RSA-ser methodology (structural/profile/mathematical separation, exact-consumption discipline, capability = provider+version+API surface) transferred directly.

Surface exposed. `EcKeyImportParams` has a mandatory `namedCurve`; the specification recognizes P-256, P-384, P-521 directly – `namedCurve` is an explicit API-level dimension, not something inferred from the container. ECDSA supports import/export of `spki` (public), `pkcs8` (private), `jwk`, and `raw` (public only). **raw is retained in the inventory but not assumed part of P_{EC-ser}^{common}** until Crypto++/BC evidence exists.

SPKI import – substantially stricter than RSA-ser’s WebCrypto surface. Generic ASN.1 parsing is DER with `exactData=true` (exact consumption, trailing bytes → `DataError`, inherited from the same parsing primitive as RSA). Then: `AlgorithmIdentifier.algorithm = id-ecPublicKey`, else `DataError`. **Sharp contrast with RSA:** `parameters` is mandatorily inspected – absent → `DataError`; present but not a `namedCurve` selection → `DataError`; and the selected curve OID must equal the `namedCurve` requested via `EcKeyImportParams`, else `DataError`. No RSA-style {absent, NULL} tolerance exists for EC.

Point encoding – import is implementation-dependent for compressed, export is uniformly uncompressed. `subjectPublicKey` is decoded via SEC 1: uncompressed MUST be supported; compressed MAY be supported (implementation choice); hybrid and any other leading octet → `DataError`; decode failure or the identity point → `DataError`. Confirmed: $D_{point, WebCrypto}^{import} \supseteq \{\text{uncompressed}\}$, with compressed acceptance not attributable to “WebCrypto” uniformly – capability must be qualified as provider+version+API surface here exactly as already established for RSA-ser.

Membership – explicitly validated, substantially stronger evidence than RSA-ser’s mathematical-validation finding. After point decoding, WebCrypto requires the decoded value

to be “a valid point on the Elliptic Curve”; the identity point is separately rejected during conversion. Confirmed:

$$\text{WebCrypto Import}_{EC\text{-}public} \models \text{PointDecode} \wedge Q \neq \mathcal{O} \wedge \text{ValidPoint}(Q, P\text{-}256).$$

This is a design-history-confirmed choice, not incidental: a 2014 WG mailing-list thread (Richard Barnes, “How to handle invalid EC public keys”) explicitly weighed rejecting at import (`DataError`) versus rejecting at `sign()/deriveBits()` time, with Ryan Sleevi and others endorsing reject-at-import, citing X9.62’s own requirement. A separate historical bug (#25466) documents that curve/`namedCurve` consistency checking was itself once a genuine specification gap, later closed – both pieces of history corroborate that the membership and curve-matching obligations now in the text were deliberate, debated additions, not oversights.

PKCS#8 import – exact consumption at two nested layers. Outer container parses as RFC 5208 `PrivateKeyInfo`, DER with exact consumption; requires `privateKeyAlgorithm.algorithm = id-ecPublicKey` and `parameters` present as `namedCurve` (= `secp256r1` for this profile). The inner `privateKey` content is then parsed explicitly as RFC 5915 `ECPrivateKey`, *also* with `exactData=true` – exact consumption applies independently at both the outer container and the inner octet-string content, not merely once.

`ECPrivateKey.parameters[0]` – resolved for WebCrypto. If present, must be `namedCurve` and must equal the outer `PrivateKeyAlgorithmIdentifier.parameters` OID exactly, else `DataError`; the algorithm text is explicitly conditional (“if the `parameters` field... is present”), so absence is tolerated. `ECPrivateKey.parameters[0]` : absent accepted; present $\Rightarrow$ must match outer `namedCurve`.

`ECPrivateKey.publicKey[1]` and $Q = dG$ – confirmed import/export asymmetric, and the import side is a genuine open specification ambiguity, not resolved by interpretation. On *export*, WebCrypto’s own algorithm is unambiguous and strict: the constructed `ECPrivateKey` MUST have both `parameters[0]` present (equal to the outer `PrivateKeyInfo.privateKeyAlgorithm.parameters`) and `publicKey[1]` present (representing the public key associated with the private scalar) – confirmed not optional in WebCrypto’s canonical output, despite both being optional in RFC 5915’s grammar:

$$\text{WebCrypto PKCS8 export} \Rightarrow \text{ECPrivateKey}(d, \text{parameters} = P\text{-}256, \text{publicKey} = Q).$$

On *import*, the evidence is genuinely inconclusive, and is recorded as such rather than resolved by plausible interpretation. The current spec text states, after constructing the private key, that a `DataError` is raised “if the private key value is not a valid point on the Elliptic Curve” – textually strange, since RFC 5915 represents the private value as a scalar d , not a point; “valid point” properties apply to Q , not d . **This is confirmed as a live, unresolved specification ambiguity, not an editorial imperfection safely reinterpreted away:** W3C issue `w3c/webcrypto#356` (“Unspecified importKey behavior for PKCS8-encoded raw private keys”, opened 2023, still open) documents, with a reproducible code example, that Chrome and other browsers throw `DataError` for a PKCS#8-encoded EC private key lacking `publicKey[1]`, while Node.js and Bun accepted the same artifact and returned a valid `CryptoKey` – confirmed directly from the issue’s own text and a reproducible Node REPL transcript. Node.js subsequently addressed this in `nodejs/node#50234` (“ensure valid point on elliptic curve in `SubtleCrypto.importKey`”), citing the spec’s ECDSA/ECDH Import Key algorithms and OpenSSL’s `EVP_PKEY_check`, with a contributor comment framing missing point validation as a genuine security issue given known invalid-curve attacks. **Classification adopted:** $Q = dG$ admission validation in PKCS#8 is **SPECIFICATION-AMBIGUOUS / IMPLEMENTATION-DIVERGENT** – neither “WebCrypto requires $Q = dG$ ” nor “WebCrypto does not require $Q = dG$ ” is asserted as a normative property; the security-motivated reading (the check targets a reconstructed/embedded Q , not d itself, consistent with X9.62-style validation and the invalid-curve-attack motivation cited in the Node PR) is plausible but not textually unambiguous, and is not adopted as evidence on that plausibility alone. Two distinct questions are kept separate, since the issue evidence distinguishes them: (i) whether `publicKey[1]` presence is required (empirically divergent: some runtimes reject its absence, others accept), and (ii) whether $Q = dG$ consistency

is actively checked when both are present (not demonstrated by this evidence either way) – an implementation could require `publicKey[1]` while checking only Q ’s curve membership, or derive Q from d when absent, and these are different behaviors that must not be merged into one finding. Observed 2023 behavior is not projected onto future versions of the same runtimes without further evidence.

Temporal update, confirmed against Chromium’s current main source (`components/webcrypto/algorit`), not merely the 2023 issue snapshot. Both `ImportKeySpki` and `ImportKeyPkcs8` route through a single shared function, `VerifyEcKeyAfterSpkiOrPkcs8Import`, which calls BoringSSL’s `EC_KEY_check_key`. Its own comment states directly: “*When importing an ECPrivateKey, the public key is optional. If it was omitted then the public key will be calculated by BoringSSL and added into the EC_KEY.*” **Current Chromium main therefore tolerates `publicKey[1]` absence and derives $Q := dG$ automatically** – the opposite of the `DataError`-on-absence behavior #356 documented for Chrome in October 2023. Whether this represents a deliberate fix made after #356 or a longer-standing divergence between the reported version and current `main` is not established here; only that current `main`’s behavior differs from the 2023 snapshot, and this document does not retain an unqualified, unversioned claim that “Chromium rejects absent `[1]`.” **This does not weaken D-059’s exclusion of `[1]`-absent from the common/portable profile – it strengthens the case for it:** D-059 rests on the claim that absence does not constitute a stable WebCrypto capability across the evidence and versions considered, not on any single version’s specific behavior; a documented flip from reject to accept across time is precisely the kind of instability D-059 already anticipated, not a counterexample to it. **Crucially, this new evidence concerns only Q_{derived} (computed by BoringSSL when `[1]` is absent), which trivially satisfies $Q = dG$ by construction, not Q_{supplied} (an embedded point present in the artifact).** No evidence was found, in this pass or the prior one, that Chromium’s `EC_KEY_check_key` verifies $Q_{\text{supplied}} \stackrel{?}{=} dG$ when `publicKey[1]` is present and *could* diverge from d – `EC_KEY_check_key` is documented by OpenSSL only generically as performing “various sanity checks... to confirm that it is valid,” without an itemized enumeration confirming scalar-range or pairwise-consistency checks specifically. For $Q_{\text{supplied}} \stackrel{?}{=} dG$, WebCrypto (via this Chromium realization) remains classified **not characterized**, not resolved by the auto-derivation finding.

Canonical export – confirmed uncompressed-only, a clean accepted-vs-canonical divergence case. Public: `DER(SPKI(id-ecPublicKey,secp256r1,Q))`, with export *always* using the uncompressed point form – even where an implementation accepts compressed on import, `compressedimport → CryptoKey → uncompressedexport`, a direct EC-side instance of the accepted-representation-vs-canonical-output-representation split already established for RSA-ser’s `parameters` field (D-053). Private: RFC 5208 `PrivateKeyInfo`, `version = 0`, `id-ecPublicKey`, `namedCurve(secp256r1)`, containing `DER ECPrivateKey (RFC 5915)` with `parameters` and `publicKey` both present as just established. WebCrypto’s EC surface is confirmed a canonical DER writer, matching the RSA-ser finding for the same API.

raw format – retained in inventory, deferred from profile decisions. Public-only; import requires at least uncompressed, permits compressed if supported, rejects decode failure/identity; export always uncompressed: $\text{raw}_{\text{export}}^{P-256} = 04\|X_{32}\|Y_{32}$. Whether **raw** belongs to $P_{\text{EC-ser}}^{\text{common}}$ awaits `Crypto++/BC` evidence, not decided here.

Observable error surface, preliminary (not yet mapped to an EC error model – that follows the three-backend pattern, as with RSA-ser).

Cause	WebCrypto
Malformed ASN.1/DER; trailing bytes	<code>DataError</code>
<code>OID ≠ id-ecPublicKey</code>	<code>DataError</code>
Outer <code>parameters</code> absent, or present but not <code>namedCurve</code>	<code>DataError</code>
Curve <code>OID ≠ requested namedCurve</code>	<code>DataError</code>
Point decode failure; identity point; point not on curve	<code>DataError</code>
Inner <code>parameters</code> present but mismatched	<code>DataError</code>
Incompatible <code>keyUsages</code>	<code>SyntaxError</code>
Unsupported format	<code>NotSupportedError</code>
Export role/format incompatible	<code>InvalidAccessError</code>
Underlying material inaccessible on export	<code>OperationError</code>

WebCrypto EC P-256 backend inventory closed. Point membership, curve/OID matching, exact consumption at two nested layers, `parameters` [0] resolution, and canonical uncompressed-DER export are all confirmed with strong evidence. `publicKey` [1]/ $Q = dG$ admission is confirmed as a genuinely characterized uncertainty (specification-ambiguous, implementation-divergent), not an unresolved gap in this inventory – knowing precisely where WebCrypto’s own evidence ends is itself the closure condition here, mirroring how RSA-ser’s Layer-2 consumption question was closed as “plausible, architecturally derived, not independently verified” rather than forced to a false certainty. Compressed-point import/export intersection, exact DER-vs-BER tolerance, `raw`’s portable-profile status, and the EC error model remain open pending Crypto++/Bouncy Castle inventories.

22.4 Backend inventory: Crypto++ EC P-256 (closed)

Comparable surface, same discipline as RSA-ser. Save/Load for SPKI/PKCS#8 interoperability, confirmed by official documentation as operating on `SubjectPublicKeyInfo` and `PrivateKeyInfo`, writing DER and reading BER; `DEREncode*`/`BERDecode*` retained only as internal-layer evidence, not the comparison surface, mirroring §19.2.

Curve and AlgorithmIdentifier – confirmed, with a real divergence from RFC 5480/WebCrypto. `DL_GroupParameters_EC<EC>::GetAlgorithmID()` returns `id-ecPublicKey`; the group can be initialized by `OID` via `Initialize(oid)`, with recommended curve parameters including `secp256r1` built in, and unknown curve `OIDs` rejected with `UnknownOID`. **Divergence:** `DL_GroupParameters_EC::` dispatches on the first ASN.1 tag – an `OBJECT IDENTIFIER` loads a named curve, anything else attempts to decode a full explicit `ECPParameters` sequence. Confirmed: $D_{ECparams, Crypto++}^{import} \supseteq \{\text{namedCurve}, \text{explicit parameters}\}$ – a real divergence from the RFC 5480/WebCrypto profile, which prohibits explicit curves in this context (§22.1, §22.3).

Public SPKI – point materialized, not treated as opaque. `DL_PublicKey_EC<EC>::BERDecodePublicKey` takes the `BIT STRING` content, calls `GetCurve().DecodePoint(P, ...)`, and raises `BERDecodeError` on failure; the decoded point is then stored as the public element. **Precision maintained:** a successful `DecodePoint()` is not by itself evidence that `Load()` automatically runs the full `ValidateElement(level, ...)` logic – that explicit validation exists as a separate routine (below).

Compressed/uncompressed – both supported, and this is a genuine representation-dependent membership-enforcement finding, source-confirmed. Export calls `EncodePoint(..., GetPointCompression())` – output form is a configurable group property (`m_compress/GetPointCompression()`) unlike WebCrypto’s uniformly-uncompressed canonical export (§22.3): $Crypto++_{export} \rightarrow \{\text{uncompressed}, \text{compressed}\}$ by configuration. Import via `ECP::DecodePoint` was traced to its literal body (`ecp.cpp`, current master):

```
case 2: case 3: {                                     // compressed
```

```

...
P.y = ((P.x*P.x+m_a)*P.x+m_b) % p;
if (Jacobi(P.y, p) != 1) return false;          // curve equation, inherent
P.y = ModularSquareRoot(P.y, p);
...
}
case 4: {                                       // uncompressed
    P.x.Decode(bt, len);
    P.y.Decode(bt, len);
    return true;                               // no equation check at all
}

```

Confirmed, not inferred: for compressed points, reconstructing y from x necessarily requires evaluating $y^2 = x^3 + ax + b \pmod{p}$ and testing that it is a quadratic residue (`Jacobi(...)` $\neq 1$ rejects otherwise) – curve-equation validity is an unavoidable byproduct of decompression itself, not a deliberate `VerifyPoint()` invocation. For uncompressed points, both coordinates are read directly with *no* equation evaluation and *no* call to `VerifyPoint()` anywhere in the branch – confirmed absent by literal source, not merely undemonstrated. `ECP::VerifyPoint` itself (confirmed body: range checks on x, y plus the curve equation) exists as a fully separate, explicitly-invoked routine, matching `ValidateElement`’s architecture below.

$\text{DecodePoint}_{\text{compressed}} \Rightarrow \text{curve-equation validity (inherent, source-confirmed)}, \quad \text{DecodePoint}_{\text{uncompressed}} \not\Rightarrow \text{VerifyPoint}$

This yields a genuine, non-hypothetical instance of *admission validation as a function of provider, surface, and representation* – not merely provider: the same backend, same operation, same API surface offers materially different membership guarantees depending on which point encoding the caller supplies at import.

Membership – ValidateElement, confirmed separate from Load. `ValidateElement(level, g, ...)` first requires $g \neq \mathcal{O}$, then `GetCurve().VerifyPoint(g)`; at level ≥ 2 , additionally checks $[q]g = \mathcal{O}$ for subgroup order q . As validation capability: `Crypto++ Validate` $\models Q \neq \mathcal{O} \wedge Q \in E$, and at higher level $[n]Q = \mathcal{O}$ – for P-256 with $h = 1$ this adds no independent mathematical class beyond full-group membership (§22.1), but confirms `Crypto++` carries general subgroup-validation infrastructure regardless. As with RSA-ser, `Load` $\not\Rightarrow$ `Validate` is adopted at high-confidence architectural evidence – the same `ASN1CryptoMaterial` $\rightarrow$ `X509PublicKey`/`PKCS8PrivateKey` hierarchy already traced for RSA (§19.2) governs EC keys, not an independent line-by-line re-confirmation for EC specifically.

PKCS#8 private – richest divergence from WebCrypto. `DL_PrivateKey_EC<EC>::BERDecodePrivateKey` parses a literal RFC-5915-like structure (`SEQUENCE{version=1, privateKey OCTET STRING, [0] parameters OPTIONAL, [1] publicKey OPTIONAL}`); the scalar decodes as integer x , stored via `SetPrivateExponent(x)`.

parameters[0] – two admitted sources, divergent from WebCrypto’s single-source model. If `parameters` came neither from the outer `AlgorithmIdentifier` nor from inner `[0]`, `BERDecodeError`. If `[0]` is present, `Crypto++` calls `AccessGroupParameters().BERDecode(parameters)` again – reusing the same `DL_GroupParameters_EC::BERDecode` already confirmed to admit *either* `namedCurve` *or* explicit parameters. Confirmed: `Crypto++` admits parameters from *either* the outer `AlgorithmIdentifier` *or* the inner `ECPrivateKey.parameters[0]`, with the inner source itself able to carry explicit (non-`namedCurve`) parameters – a real divergence from WebCrypto’s single-source, `namedCurve`-only model (§22.3).

publicKey[1] – parsed, then discarded, and no $Q = dG$ check found. If `[1]` is present, `Crypto++` decodes the `BIT STRING`, requires `unusedBits==0`, calls `DecodePoint(Q, ...)`, and fails with `BERDecodeError` if undecodable – but the decoded Q is not retained; the routine’s own comment marks the public element as skipped, and the routine ends by storing only `SetPrivateExponent(x)`.

Confirmed: Crypto++ PKCS8 import : `publicKey[1]` may be parsed, but is not preserved as private-key material. **no evidence found of a $Q \stackrel{?}{=} dG$ comparison** – an embedded Q must be decodable as a point (subject to the compressed/uncompressed asymmetry just established), but this routine never compares it against dG . This is a clean, informative contrast with WebCrypto’s #356 ambiguity (§22.3): where WebCrypto’s own text is textually unclear about what exactly is checked, Crypto++’s routine is clear and the check simply is not present for this specific comparison.

Canonical private export – minimal, omitting both optional RFC 5915 fields. `DEREncodePrivateKey` writes only `SEQUENCE{version = 1, privateKey = d}` – neither `parameters[0]` nor `publicKey[1]` is emitted in the inner `ECPrivateKey`; curve parameters live only in the outer wrapper. The scalar is serialized at subgroup-order byte length (32 bytes for P-256, matching expectation). Confirmed:

Crypto++ inner `ECPrivateKeyexport` = {version = 1, privateKey = d_{32} }.

Sharp canonical-writer divergence from WebCrypto (§22.3, which mandates both fields present on export):

	WebCrypto export	Crypto++ export
<code>parameters[0]</code>	present	absent
<code>publicKey[1]</code>	present	absent

Central evidence for the eventual P_{EC-ser}^{common} derivation: semantic representation may remain interoperable (both encode the same key pair) even where canonical export bytes diverge structurally between backends.

$Q = dG$ – **Crypto++ substantially clarifies, without resolving, the cross-backend picture.** Private-key material is essentially the scalar d ; an associated public point can be generated mathematically via DL key-generation operations, but an *imported* `publicKey[1]` is not retained in `DL_PrivateKey_EC`. With current evidence: **Crypto++ import does not demonstrate admission-time enforcement of embedded $Q = dG$** – if [1] is absent, there is no serialized Q to compare; if present, it is decoded as a point then discarded, with no comparison against dG found. This does not establish that Crypto++ accepts every inconsistent (d, Q) pair (a higher-layer caller could still validate independently), only that the RFC-5915-specific decode routine itself does not.

Scalar validity – appears separate from Load, not yet fully closed. Import concludes at `SetPrivateExponent(x)` with no explicit $1 \leq d < n$ test visible in that routine. Crypto++ has general `Validate()` infrastructure for DL keys, but whether `Load()` invokes it automatically follows the same open pattern as the public case – not assumed resolved by symmetry with the already-confirmed `Load` $\nrightarrow$ `Validate` finding without being named explicitly here.

BER reader / DER writer; trailing-buffer behavior – inherited from the shared architecture, high-confidence, not independently re-traced line-by-line for EC. Same “write DER, read BER” philosophy already documented for RSA-ser, confirmed for EC’s own `Save/Load` by the same official documentation. $\{\text{DER}\} \subset D_{\text{Crypto++}}^{\text{import}}$, with non-canonical BER plausibly admitted by the same generic `BufferedTransformation`-based reader already traced for RSA – and, as with RSA, no evidence that `Load()` enforces exhaustion of the outer buffer. This is carried over as *shared-architecture trace, not an independent EC-specific line-by-line confirmation*, avoiding redundant re-proof of a superclass-level behavior already established once.

Crypto++ EC P-256 backend inventory closed. All items resolved to the epistemic standard the RSA-ser precedent set: literal source where obtained (compressed/uncompressed `DecodePoint`, `parameters[0]`/`publicKey[1]` handling, canonical export), high-confidence architectural inheritance where re-tracing would be redundant (`Load/Validate` separation, trailing-buffer tolerance), and explicit absence-of-evidence framing where genuinely undemonstrated ($Q = dG$ comparison, scalar-range checking on `Load`). Explicit EC parameters as an admitted import source (beyond `namedCurve`) is retained as a confirmed capability, not yet a portable-profile question – that awaits Bouncy Castle.

22.5 Backend inventory: Bouncy Castle EC P-256 (closed)

Target version and surface, same discipline as RSA-ser. Bouncy Castle Java 1.77, lightweight surface (PublicKeyFactory/PrivateKeyFactory for import, SubjectPublicKeyInfoFactory/PrivateKeyInfoFactory for export) as primary comparison surface, contrasted against JCA (KeyFactory("EC","BC")) only where it could change an observable conclusion, mirroring §19.3. PEM infrastructure exists as a separate BC layer and is excluded from this SPKI/PKCS#8 core, consistent with the comparison surface fixed for RSA.

Private export – literal evidence, and a striking result: BC computes $Q := dG$ explicitly during export. PrivateKeyInfoFactory’s EC branch: for ECNamedDomainParameters, constructs X962Parameters(curveOID), sizes the scalar via orderBitLength=n.bitLength() (32 bytes for P-256), then:

```
ECPoint q = new FixedPointCombMultiplier().multiply(domainParams.getG(), priv.getD());
DERBitString publicKey = new DERBitString(q.getEncoded(false));
```

Confirmed literal, not inferred: $Q := dG$ is computed explicitly during private-key serialization, encoded uncompressed (getEncoded(false)), and both parameters[0] and publicKey[1] are populated in the resulting ECPrivateKey:

$BC_{private-export} \Rightarrow ECPrivateKey(d, parameters[0] = secp256r1, publicKey[1] = Q:=dG)$ within PrivateKeyInfoFactory

Three-way canonical-writer comparison for the inner RFC 5915 structure – a clean, well-evidenced divergence.

Inner field	WebCrypto	Crypto++	BC lightweight
parameters[0]	present	absent	present
publicKey[1]	present	absent	present
Q exported	associated	—	computed dG
Point form	uncompressed	configurable	uncompressed (TODO notes planned compression support)

WebCrypto and BC converge structurally on including both optional RFC 5915 fields; Crypto++ omits both. This suggests $WebCrypto_{canonical} \approx BC_{canonical} \neq Crypto++_{canonical}$ in inner structure, though byte-identity across WebCrypto and BC is not established here and is not assumed.

BC’s EC-parameters model is the widest of the three, structurally closer to Crypto++ than to WebCrypto. The same export routine distinguishes three cases: null domain parameters (implicit CA/NULL), ECNamedDomainParameters (OID), or other ECDomainParameters (explicit X9ECParameters). Confirmed: $D_{ECparams,BC}^{model} \supseteq \{\text{implicitCA}, \text{namedCurve}, \text{explicitParameters}\}$ – export/model support confirmed directly; import acceptance of each variant through the eventual P-256 import path is a separate question, not automatically implied.

Exact consumption – same byte[]/InputStream asymmetry already confirmed for RSA-ser, reappears identically for EC. createKey(byte[]) routes through ASN1Primitive.fromByteArray(...), exact outer consumption; createKey(InputStream) uses new ASN1InputStream(inStr).readObject() directly, one object read with no inherent stream-exhaustion check. Reinforces that capability must be qualified by API surface, not backend name alone (§19.3).

Public import membership – confirmed by current literal source, and the cleanest cross-backend contrast in this entire EC inventory. ECCurve.decodePoint() (BC main, class org.bouncycastle.math.ec.ECCurve):

```
case 0x02: case 0x03: { // compressed
```

```

...
p = decompressPoint(yTilde, X);
if (!p.implIsValid(true, true))
    throw new IllegalArgumentException("Invalid point");
break;
}
case 0x04: {                                // uncompressed
    BigInteger X = ...; BigInteger Y = ...;
    p = validatePoint(X, Y);                // explicit membership check
    break;
}

```

where `validatePoint(x,y)` calls `createPoint(x,y)` then throws unless `p.isValid()`. **Confirmed: BC validates membership explicitly for *both* compressed and uncompressed encodings** – direct source evidence from current `main`; high-confidence applicability to 1.77, not version-pinned. Since `PublicKeyFactory`'s EC converter calls `curve.decodePoint(x9Encoding)` directly (§22.5, confirmed prior turn), this validation is inherited automatically at the `PublicKeyFactory` layer without further tracing needed.

The cross-backend contrast, stated precisely without overgeneralizing "BC is more secure."

	compressed	uncompressed
Crypto++	validity inherent in decompression	no membership validation (source-confirmed absence)
BC	explicit <code>implIsValid</code> check	explicit <code>isValid</code> check

For Crypto++, membership enforcement is representation-dependent: compressed enforces it as an unavoidable mathematical byproduct, uncompressed does not enforce it at all. **For BC, membership enforcement is representation-independent across the two SEC1 forms examined** – both branches enforce explicitly and deliberately, not as a side effect of one encoding's mathematics. This is the opposite conclusion from Crypto++, not a stronger version of the same one, and is stated as such rather than folded into a single "BC is stricter" generalization.

Private import – traced to the literal EC branch of `PrivateKeyFactory`, resolving P1, P2, P4, P5 in one pass, and P3 by strong indirect evidence. Literal source, `core/crypto/util/PrivateKeyFactory.java`, current `main`:

```

else if (algOID.equals(X9ObjectIdentifiers.id_ecPublicKey))
{
    /*
     * TODO Consistency checks in case parameters and/or public key are specified at both the
     * PrivateKeyInfo and ECPrivateKey levels?
     */
    ECPrivateKey ecPrivateKey = ECPrivateKey.getInstance(keyInfo.parsePrivateKey());
    X962Parameters parameters = X962Parameters.getInstance(algId.getParameters().toASN1Primitive());
    ECDomainParameters domainParams;
    if (parameters.isNamedCurve())
        domainParams = ECNamedDomainParameters.lookup(
            ASN1ObjectIdentifier.getInstance(parameters.getParameters()));
    else
        domainParams = new ECDomainParameters(X9ECParameters.getInstance(parameters.getParameters()));
    BigInteger d = ecPrivateKey.getKey();
    return new ECPrivateKeyParameters(d, domainParams);
}

```


The routine’s own TODO comment is the strongest evidence found anywhere in this inventory: BC’s own implementers explicitly flag, unresolved, the exact absence under investigation – not an inference from missing calls, but the maintainers’ own acknowledgment that inter-level consistency checks are not performed.

- **P1 (parameters [0]):** domainParams is built exclusively from `algId.getParameters()` – the *outer* AlgorithmIdentifier. The inner `ecPrivateKey`’s own parameters accessor is never consulted in this routine. Confirmed: `outer AlgorithmIdentifier.parameters` is authoritative; inner [0] neither determines nor is cross-checked against it.
- **P2 (publicKey [1]):** only `ecPrivateKey.getKey()` is called; `ecPrivateKey.getPublicKey()` is never invoked in this routine. Precisely stated (not overclaiming that ASN.1 parsing itself skipped the field, which `ECPrivateKey.getInstance(...)` may have structurally interpreted): *publicKey [1] is not consulted, materialized as key material, validated, or compared by PrivateKeyFactory*. The embedded Q , if present, plays no role in this route’s semantics.
- **P4/P5 (membership of embedded Q ; $Q = dG$):** not merely undemonstrated but structurally inapplicable – since Q is never read, no comparison against dG or curve membership of an embedded point can occur in this routine.
- **P3 (scalar range $1 \leq d < n$):** reopened and corrected after the initial closure. `ECDomainParameters` exposes `validatePrivateScalar(BigInteger d)`, confirmed literal (current main): rejects outside $[1, n - 1]$ with `IllegalArgumentException("Scalar is not in the interval [1, n - 1]")`, and has been part of the public API since at least BC 1.66 (confirmed via that version’s published Javadoc), well before 1.77. A real-world Java (not C#) production stack trace (a [web3j](#) issue report, 2021) independently confirms the wiring: `ECPrivateKeyParameters.<init>` appears directly above `ECDomainParameters.validatePrivateScalar` in the trace, throwing exactly that message. **Corrected conclusion: `ECPrivateKeyParameters`’s constructor enforces $1 \leq d < n$ via `validatePrivateScalar`** – confirmed by a genuine Java stack trace plus the method’s confirmed pre-1.77 public presence; high confidence for 1.77 specifically (a later, stable release unlikely to have removed the check), but not a literal line-by-line read of the exact 1.77-tagged constructor body. The initial closure of this item (“no scalar-range enforcement demonstrated”) is retracted – it rested on signature/usage-pattern inference that a more targeted search overturned, illustrating precisely why indirect absence-of-evidence conclusions are held to a lower confidence tier than literal or stack-trace-confirmed findings in this design.

Resulting directional asymmetry, confirmed at both ends.

$\text{Import}(d, Q_{\text{embedded}}) \longrightarrow$ materializes d , ignores Q_{embedded} semantically, $\text{Export}(d) \longrightarrow Q' := dG$, serialized.

A consequence worth flagging for the eventual R_{ser} treatment: $Q_{\text{embedded}} \neq dG$ on a private-key artifact does not necessarily survive an import→export round-trip, since the writer reconstructs $Q' := dG$ regardless of what was embedded on import – an inconsistent input could be silently canonicalized to a consistent output without the original inconsistency ever being observable post-round-trip. This is not claimed as “BC accepts every inconsistent (d, Q) pair” – only that this specific decode routine does not use $Q = dG$ consistency as a materialization condition; a higher-layer caller could still validate independently, and this routine’s silence does not preclude that.

Bouncy Castle EC P-256 backend inventory closed (v0.5.21, correcting one item from the initial v0.5.20 closure). Public-side membership is the strongest, most cleanly source-confirmed finding in the entire EC inventory to date (explicit validation, both point forms, current main source). Private-side import via `crypto.util.PrivateKeyFactory` is the most minimal of the three backends examined regarding `parameters [0]/publicKey [1]/ $Q = dG$` – narrower

even than Crypto++, which at least parses and discards the embedded point – but this minimality does *not* extend to scalar-range admission: $1 \leq d < n$ is enforced at construction via `ECDomainParameters.validatePrivateScalar`, confirmed by a genuine Java stack trace, correcting the initial closure’s opposite conclusion. The public/private asymmetry (rigorous membership validation on the public path; scalar-range enforced but embedded- Q /cross-level consistency entirely unchecked on the private path) is itself a genuine, well-evidenced finding, not an artifact of incomplete tracing.

22.6 Three-backend EC P-256 common intersection P_{EC-ser}^{common}

Modeled directionally, following the same discipline as RSA-ser (D-053, §19.4): $P_{EC-ser}^{common} = (P_{export}^{common}, D_{import}^{common})$, kept separate because “what all three produce” and “everything all three can read” are not the same claim and conflating them would reintroduce a false symmetry. Not modeled as a free Cartesian product either: role, container, algorithm identifier, curve parameters, point representation, and inner RFC 5915 fields are structurally correlated, not independent axes.

Common semantic material.

$$M_{EC}^{common} = \text{Public}(Q) \uplus \text{Private}(d), \quad Q = (x, y) \in E(\mathbb{F}_p), \quad 1 \leq d < n.$$

$Q = dG$ holds whenever an associated public component exists, but Q is deliberately *not* made an obligatory component of common private material: Crypto++ does not retain an imported embedded Q , and BC’s lightweight private import does not use it semantically (§22.4, §22.5). Hence $\text{Private}(d) \neq \text{Private}(d, Q)$ in M^{common} .

Common export.

$$P_{export}^{common} = \begin{cases} \text{Public}(Q) \mapsto \text{DER-SPKI}(\text{id-ecPublicKey}, \text{secp256r1}, Q_{uncompressed}), \\ \text{Private}(d) \mapsto \text{DER-PrivateKeyInfo}(\text{id-ecPublicKey}, \text{secp256r1}, \text{ECPrivateKey}(\text{version}=1, d, \dots)), \end{cases}$$

with private invariants confirmed common: $\text{version} = 1$, $|d|_{enc} = 32$ bytes – without yet requiring uniform presence of `[0]`/`[1]` at the export level, since the three writers diverge there (§22.4, §22.5): WebCrypto and BC both emit `parameters[0]` and `publicKey[1]` present; Crypto++ emits neither. No single byte-level canonical inner `ECPrivateKey` template is common to all three writers – this is stated explicitly as a structural finding relevant to R_{ser} , not glossed over.

Point encoding, export. $P_{point-export}^{common} = \{\text{uncompressed}\}$: WebCrypto always exports uncompressed; BC computes and exports $Q := dG$ uncompressed (`getEncoded(false)`); Crypto++’s point-compression flag defaults to `false` (uncompressed), confirmed as the constructor default, not merely an achievable configuration. $\text{ECPPoint}(Q) = 04\|X_{32}\|Y_{32}$, 65 bytes – a clean case of $P_{common}^{common} \subsetneq P_{Crypto++}^{common}$ (compressed remains a Crypto++-specific capability) without treating that extra capability as a conformance failure.

Algorithm identifier and curve parameters.

$$P_{ECparams}^{common} = (\text{id-ecPublicKey}, \text{namedCurve}(\text{secp256r1})).$$

Confirmed common to all three: WebCrypto mandates `namedCurve`, rejecting absence or non-`namedCurve` forms; Crypto++ and BC both accept and can produce the named-curve form. Explicit parameters and `implicitCA` are outside this intersection – WebCrypto’s profile excludes them entirely (§22.3), while Crypto++ and BC support them as additional, non-common capability (§22.4, §22.5).

Encoding. $P_{encoding-export}^{common} = \text{DER}$; $\text{DER} \subseteq D_{encoding-import}^{common,p}$ for each p (WebCrypto: DER-only on the fixed surfaces; Crypto++: writes DER, reads BER, DER a strict subset of its read domain; BC: writes canonical DER, `byte[]`-surface reads at least DER via exact-consumption parsing).

$D_{\text{encoding-import}}^{\text{common}} = \{\text{DER}\}$ – non-canonical BER is not included in the common domain, since WebCrypto’s fixed surfaces do not admit it; Crypto++/BC’s wider read tolerance remains provider-specific capability, not promoted into the intersection.

Private inner ECPrivateKey import domain – the one dimension requiring resolution beyond direct grammar inspection, closed here, not deferred.

$\text{ECPrivateKey}_{\text{compatible}} = \text{ECPrivateKey}(\text{version}=1, d, \text{parameters}[0] \in \{\text{absent}, \text{present-matching}\}, \text{publicKey}[$

$\text{parameters}[0]$ optional: confirmed no backend requires its presence when the outer `AlgorithmIdentifier.parameters` (already common, above) is present – WebCrypto’s own algorithm explicitly states “*if present*” for the inner check (§22.3); Crypto++ falls back to the outer source when $[0]$ is absent (§22.4); BC’s `PrivateKeyFactory` never consults $[0]$ at all (§22.5). If present, must be consistent with P-256 for the common domain. `publicKey[1]` required present: not a portable-profile choice but a consequence forced by already-collected evidence – W3C issue #356 demonstrates that its *absence* does not belong stably to any single, well-defined WebCrypto realization (Chromium/Firefox/WebKit reject it; Node/Bun accept it), so by the same intersection discipline already applied throughout this design (e.g. RSA-ser’s multi-prime exclusion, D-053), an artifact lacking $[1]$ cannot be asserted common. *Presence*, by contrast, is uncontested across every realization examined on all three backends, including both sides of #356’s divergence – confirmed by audit: Crypto++ accepts (parses, then discards) an imported $[1]$; BC’s factory does not reject its presence (simply does not consult it); WebCrypto natively produces and accepts it. No contradiction found against any closed inventory finding (§22.3–§22.5).

Deliberately excluded from $P_{\text{EC-ser}}^{\text{common}}$: membership and $Q = dG$ consistency. That WebCrypto, Crypto++, and BC validate curve membership differently (§22.3–§22.5) does not make “membership behavior” a coordinate of the representation-common surface, exactly mirroring the discipline already applied to RSA’s mathematical validity (D-054, D-055): Q itself belongs to the material/import domain; the obligation $Q \in E(\mathbb{F}_p)$ belongs to C_{math} and an eventual `Membership.CurveMembership` clause. The same applies to $Q = dG$ consistency. Both are deferred to $C_{\text{pre}}^{\text{EC-ser}}$ construction, not decided here.

Consolidated statement.

$$P_{\text{EC-ser}}^{\text{common}} = (P_{\text{export}}^{\text{common}}, D_{\text{import}}^{\text{common}}),$$

$$P_{\text{export}}^{\text{common}} : \begin{cases} \text{Public} \leftrightarrow \text{DER-SPKI}(\text{id-ecPublicKey}, \text{secp256r1}, Q_{\text{uncompressed}}), \\ \text{Private} \leftrightarrow \text{DER-PrivateKeyInfo}(\text{id-ecPublicKey}, \text{secp256r1}, \text{ECPrivateKey}(\text{version}=1, d_{32})), \end{cases}$$

$$D_{\text{import}}^{\text{common}} \supseteq \left\{ \begin{array}{l} \text{DER-SPKI}(\text{id-ecPublicKey}, \text{secp256r1}, Q_{\text{uncompressed}}), \\ \text{DER-PrivateKeyInfo}(\text{id-ecPublicKey}, \text{secp256r1}, \text{ECPrivateKey}_{\text{compatible}}) \end{array} \right\}.$$

No new Decision Log entry at this point in the derivation – this is evidence-derived common surface, not yet a profile-selection decision; $P_{\text{EC-ser}}^{\text{portable}}$ is derived next (§22.7), after which both are frozen jointly as D-059 (§24), mirroring D-053’s joint common+portable freeze for RSA-ser.

22.7 EC P-256 portable profile $P_{\text{EC-ser}}^{\text{portable}}$

Derived without new provider evidence: $P_{\text{EC-ser}}^{\text{common}}$ (§22.6) is already demonstrated; this step selects, deliberately, a single representation within that intersection rather than discovering capability. Goal: unambiguous $\wedge$ cross-provider importable $\wedge$ canonically reproducible – a representation the SDK can *produce* independent of backend, not merely something all three happen to accept. $P_{\text{EC-ser}}^{\text{portable}} \subseteq P_{\text{EC-ser}}^{\text{common}}$, mirroring D-053’s discipline for RSA-ser.

Material. $M_{\text{EC}}^{\text{portable}} = \text{Public}(Q) \uplus \text{Private}(d)$ – unchanged from $M_{\text{EC}}^{\text{common}}$, deliberately *not* written as $\text{Private}(d, Q)$: doing so would prejudge the still-undecided $Q = dG$ obligation. The inner

`publicKey[1]` field included in the portable private serialization is representational material required by the portable container, not a redefinition of the private key’s abstract semantics.

Public portable – coincides with common; no reduction needed.

$$\text{SPKI}_{\text{portable}}(Q) = \text{DER-SPKI}(\text{AlgorithmIdentifier}(\text{id-ecPublicKey}, \text{secp256r1}), 04\|X_{32}\|Y_{32}).$$

Dimension	Portable
Curve	<code>secp256r1</code> / P-256
Algorithm	<code>id-ecPublicKey</code>
Container	SPKI
Encoding	DER
Algorithm parameters	named-curve OID <code>secp256r1</code>
Point form	uncompressed
Point size	65 bytes
Coordinates	X_{32}, Y_{32} fixed-width

No compressed/hybrid/explicit-parameters/JWK – not because any is necessarily invalid for some provider, but because none belongs to the selected portable profile.

Private portable – the genuine profile-selection decision.

$$\text{PKCS8}_{\text{portable}}(d, Q) = \text{DER-PrivateKeyInfo}(\text{id-ecPublicKey}, \text{secp256r1}, \text{ECPrivateKey}_{\text{portable}}),$$

$$\text{ECPrivateKey}_{\text{portable}} = \text{ECPrivateKey}(\text{version}=1, d_{32}, \text{parameters}[0]=\text{secp256r1}, \text{publicKey}[1]=Q_{\text{uncompressed}}).$$

Portable selects `parameters[0]=present` and `publicKey[1]=present` – but the two presences are justified by different reasons, and conflating them would misstate what kind of decision each is:

- **`publicKey[1] = present`**: not a profile choice with real freedom – it is inherited directly from $D_{\text{import}}^{\text{common}}$ (§22.6), where its absence was already excluded by the intersection itself (W3C #356’s divergence, not a portable-level decision). Portable simply carries that already-forced restriction forward.
- **`parameters[0] = present-matching`**: a genuine canonicalization choice. $D_{\text{import}}^{\text{common}}$ permits {absent, present-matching}; portable selects the single present-matching case because (i) it lies within $D_{\text{import}}^{\text{common}}$, (ii) WebCrypto and BC already produce it natively, and (iii) it removes an otherwise-harmless ASN.1 variation, yielding one unique portable representation. This is exactly the kind of {absent, present} $\rightarrow$ {present} reduction P_{portable} exists to perform – [0] is not normatively required by RFC 5915 nor by the common intersection; it is obligatory only within the selected portable profile.

Native export $\neq$ portable export – a genuine, methodologically important consequence, not glossed over. For public keys, $\text{Export}_p^{\text{native}} \approx \text{Export}^{\text{portable}}$ across all three backends on the relevant coordinates. For private keys, $\text{Export}_{\text{Crypto++}}^{\text{native}} \notin P_{\text{EC-ser}}^{\text{portable}}$, since Crypto++’s native writer omits the optional fields now made canonical (§22.4). **This does not mean Crypto++ fails conformance**: the adapter transforms $\text{Private}(d) \rightarrow \text{PKCS8}_{\text{portable}}(d, Q)$ using the material available to it (Q computable from d via the group generator, exactly as BC’s own native writer already does, §22.5), consistent with $I_p = \text{API}_p \circ \text{Adapter}_p$. The experimental obligation is on $\text{Export}_p^{\text{SDK}}(k)$, not $\text{Export}_p^{\text{native}}(k)$ – directly connected to the adapter-induced-divergence threat already carried throughout this design.

Import portable – deliberately narrower than common, not a restatement of it.

$$D_{\text{public-import}}^{\text{portable}} = \{\text{SPKI}_{\text{portable}}(Q)\}, \quad D_{\text{private-import}}^{\text{portable}} = \{\text{PKCS8}_{\text{portable}}(d, Q)\}, \quad D_{\text{import}}^{\text{portable}} \subsetneq D_{\text{import}}^{\text{common}}.$$

That a backend additionally accepts [0] absent, compressed points, non-canonical BER, explicit parameters, or other representations is provider-specific capability, not part of the portable obligation – an artifact can be outside $P_{EC-ser}^{portable}$ without being “invalid” for a given provider. Three distinct states, not two: portable $\subseteq$ common $\subseteq$ provider-specific accepted.

Consolidated statement.

$$P_{EC-ser}^{portable} = (P_{export}^{portable}, D_{import}^{portable}), \quad D_{import}^{portable} = P_{export}^{portable}$$

(at the level of contractual representation: what the SDK portably produces is exactly the representation used as portable canonical input) – this does *not* assert $D_{import}^{common} = D_{import}^{portable}$; the strict-subset relation established above is retained explicitly, not blurred by the coincidence of $D_{import}^{portable}$ and $P_{export}^{portable}$.

$$P_{export}^{portable} = \begin{cases} \text{Public}(Q) \leftrightarrow \text{DER-SPKI}(\text{id-ecPublicKey}, \text{secp256r1}, 04\|X_{32}\|Y_{32}), \\ \text{Private}(d) \leftrightarrow \text{DER-PKCS8}(\text{id-ecPublicKey}, \text{secp256r1}, \text{ECPrivateKey}(1, d_{32}, [0]=\text{secp256r1}, \end{cases}$$

Deliberately not introduced here: $Q = dG$ as an obligation. The portable representation requires [1] present, but whether the SDK imposes any contractual consistency requirement between the embedded d and Q is not decided by that representational fact alone – it is resolved separately, in the forthcoming Membership/ $Q = dG$ construction (§22.8), where whether it becomes a Key, Membership, Validation, or carefully-scoped combination obligation is decided consciously rather than smuggled in through the representation choice made here.

P_{EC-ser}^{common} and $P_{EC-ser}^{portable}$ are now consolidated and frozen jointly as **D-059** (§24), mirroring D-053’s joint treatment for RSA-ser.

22.8 EC P-256 mathematical validity: Membership and $Q = dG$

Three distinct mathematical properties are separated here, not two – collapsing any pair would either lose a real causal distinction or misclassify an obligation under the wrong K_1 .

$$V_{scalar} : 1 \leq d < n, \quad V_{curve} : Q \in E(\mathbb{F}_p), \quad V_{pair} : Q = dG \text{ when supplied } Q \text{ and } d \text{ coexist in a contractual a}$$

V_{scalar} – private scalar admissibility. A property of the private key material alone, independent of any public component. Confirmed BC-specific evidence already exists (D-054’s RSA precedent for the pattern; here, BC’s `ECDomainParameters.validatePrivateScalar`, §22.5, corrected in v0.5.21). Provisionally classified `Key.PrivateScalar` – not `Membership`, since d is a scalar, not a point – with final K_1/K_2 assignment deferred to joint clause construction (following C_{pre}^{EC-ser} , §22.9), consistent with the discipline already applied to RSA’s analogous open classification.

V_{curve} – public point curve membership. $Q = (x, y): y^2 \equiv x^3 + ax + b \pmod{p}$, together with the coordinate/representation conditions already established (§22.1). This is exactly the property that justified keeping `Membership` independent of `Validation` in the first place: a point can be correctly serialized (well-formed DER, correct encoding, correct length) yet mathematically foreign to the curve. Classified `Membership.CurveMembership`.

No independent `Membership.SubgroupMembership` for this operation. A general curve would require distinguishing $Q \in E(\mathbb{F}_p)$ from $[n]Q = \mathcal{O}$ (subgroup membership) as two separable properties. P-256’s cofactor $h = 1$ (§22.1) means a valid finite curve point already lies in the unique subgroup of interest – there is no second experimentally useful membership dimension here. The K_2 subtype remains available in the shared taxonomy for curves where $h > 1$; it is simply not instantiated by an EC-P256 clause, per the standing rule that new K_2 values are introduced only when a real clause needs them, not speculatively.

V_{pair} – **not curve membership**. Consider $d_1 \neq d_2$, both valid scalars ($1 \leq d_1, d_2 < n$), with the artifact constructed as $d = d_1$, $Q = d_2G$. Then $Q \in E(\mathbb{F}_p)$ holds (it is a perfectly valid point – indeed d_2G for a valid d_2) while $Q \neq d_1G$. This single stimulus cleanly separates the two properties: it preserves DER validity, PKCS#8 structure, RFC 5915 fields, P-256 OIDs, point encoding, and curve membership, breaking only the coherence between the private and its associated public component. V_{curve} answers “does this object belong to a valid mathematical structure?”; V_{pair} answers “are two related components of the same cryptographic material mutually consistent?” – a *key-consistency* question, not a membership question. Classified **Key.PublicPrivateConsistency**, confirmed distinct from **Membership.CurveMembership** by the (d_1, d_2G) construction above, which is deliberately preserved as the canonical isolating stimulus for the eventual causal class (post- C_{pre}^{EC-ser} clause/ Γ_0 construction, still pending).

V_{pair} is an obligation of the composite portable artifact, not a redefinition of abstract private-key material. $M_{EC}^{common}/M_{EC}^{portable}$ (§22.6, §22.7) already froze $Private(d)$, deliberately not $Private(d, Q)$. That decision stands unchanged: V_{pair} does not reopen it. But $PKCS8_{portable}(d, Q)$ (§22.7) contractually includes both d and $publicKey[1]=Q$ – once both are included in the SDK’s own portable format, an artifact with $Q \neq dG$ is internally self-contradictory by the contract’s own terms. The obligation is therefore stated as:

$$PKCS8_{portable}(d, Q) \text{ is contractually valid} \implies Q = dG,$$

a property of the composite artifact the SDK itself defines, not a claim about what a bare private scalar must satisfy in the abstract.

Admission semantics, audited across the three closed backend inventories, with explicit epistemic states – not collapsed into a single verdict. Four states are used throughout, not a binary enforced/not-enforced: **enforced** (positive evidence of an automatic check), **not enforced** (positive evidence of its absence), **not characterized** (evidence insufficient either way), **representation-dependent** (enforcement varies by encoding form within the same backend).

Property	WebCrypto (Chromium)	Crypto++	BC
V_{scalar} ($1 \leq d < n$)	not characterized	not enforced	enforced
V_{curve} (embedded/derived Q)	plausible, not itemized-confirmed	representation-dependent	not applicable ([1])
V_{pair} ($Q_{supplied} = dG$)	not characterized	not enforced	not enforced

V_{scalar} : BC’s enforcement is the only positive finding (§22.5); Crypto++’s `BERDecodePrivateKey` shows no visible check (§22.4); WebCrypto/Chromium calls `EC_KEY_check_key`, but its exact scope is not itemized by OpenSSL’s own documentation beyond “various sanity checks,” so it is not promoted to **enforced** on that generic description alone.

V_{curve} : WebCrypto’s public-import path is independently confirmed explicit (§22.3); for the private-import path specifically, current Chromium’s auto-derivation of Q when `[1]` is absent guarantees a valid point trivially (it is computed by the group operation itself), which is a different claim from confirming that a *supplied* Q is checked for membership when `[1]` is present – kept as **plausible, not itemized-confirmed** rather than promoted to **enforced**. Crypto++ is confirmed representation-dependent (compressed decode inherently validates via the decompression equation; uncompressed decode does not, §22.4). BC’s private-import route never reads `[1]` at all (§22.5), so the question does not apply on that route – distinct from “not enforced,” which would imply the check was attempted and absent.

V_{pair} , **the decisive row**. **Crypto++**: confirmed **not enforced** – `[1]` is parsed then discarded, no comparison against dG found (§22.4). **BC**: confirmed **not enforced** – `[1]` is never read at all by `PrivateKeyFactory` (§22.5). **WebCrypto**: **not characterized** for $Q_{supplied}$ specifically – the

auto-derivation behavior just confirmed for current Chromium concerns only $Q_{derived}$, which satisfies $Q = dG$ trivially by construction and says nothing about whether a *supplied*, potentially-inconsistent Q would be checked against d when [1] is present. The precise, evidence-respecting statement is therefore

No common native admission guarantee for $Q_{supplied} = dG$ has been established across the three backends,

not the stronger, unsupported claim that no backend checks it – Crypto++ and BC’s absence is positively confirmed; WebCrypto’s is not.

Consolidated statement, frozen here.

$$C_{math}^{EC} = \left\{ \begin{array}{ll} V_{scalar} : & 1 \leq d < n, \\ V_{curve} : & Q \in E(\mathbb{F}_p), \\ V_{pair} : & Q = dG \text{ when supplied } Q \text{ and } d \text{ coexist in a contractual artifact,} \end{array} \right\} \quad V_{curve} \neq V_{pair}.$$

$V_{curve} \rightarrow \text{Membership.CurveMembership}$; $V_{pair} \rightarrow \text{Key.PublicPrivateConsistency}$; $V_{scalar} \rightarrow \text{Key.PrivateScalar}$ (provisional). No $\text{Membership.SubgroupMembership}$ clause for this operation.

Conclusion, connecting directly to the structural finding already established for RSA-ser (D-054/D-010). Native admission semantics are heterogeneous and do not themselves constitute the mathematical contract. In particular, no common native admission guarantee has been established for supplied public/private consistency $Q = dG$; therefore, if this property is required by the contractual portable artifact, it must be enforced uniformly by the SDK/adaptor layer rather than delegated to provider-specific parsers. This is the same structural conclusion RSA-ser reached independently for $V_{rel}^{RSA,2}$ – two unrelated operations converging on the same design principle (no cross-field consistency check can be assumed from provider intersection; the SDK must build it) is evidence of a transversal pattern in this design, not a coincidence specific to one operation.

Short audit against D-059 before proceeding. D-059 froze $\text{publicKey}[1]=\text{present}$ in both D_{import}^{common} and $P_{export}^{portable}$, on the grounds that [1]-absence does not constitute a stable WebCrypto capability across the evidence considered (§22.6, §22.7). The Chromium temporal update recorded above (§22.3) does not contradict this: D-059’s claim was never version-specific (“current Chromium accepts/rejects [1]-absence”), only that absence has not been shown stable across the evidence and versions considered – and a confirmed flip from reject (Oct. 2023) to accept (current main) is additional evidence of exactly that instability, reinforcing D-059 rather than undermining it. No contradiction found; D-059 is not reopened. C_{math}^{EC} as frozen here is new content, not yet reflected in a Decision Log entry – it becomes part of C_{pre}^{EC-ser} ’s construction, the next step.

22.9 Precondition contract C_{pre}^{EC-ser}

Built from five blocks, deliberately excluding error categories, mutations, Γ_0 , native per-backend behavior, and any obligation on compressed points, BER, explicit parameters, JWK, subgroup membership, or RNG – consistent with the norm/evidence $\rightarrow$ contract $\rightarrow$ clauses $\rightarrow$ mutations $\rightarrow$ Γ_0 ordering already followed throughout.

$$C_{pre}^{EC-ser} = \{C_{role}, C_{curve}, C_{pub}, C_{priv}, C_{math}, C_{direction}\}.$$

Governing principle: $P_{portable} \subseteq P_{common}$ (D-059) – the experimental contract uses the portable profile as canonical representation, while common delimits what cross-provider obligation can be justified at all.

Public contract. For $\text{Public}(Q)$: $Q = (x, y)$, $Q \in E(\mathbb{F}_p)$, canonical representation $\text{DER-SPKI}(\text{id-ecPublicKey}, s$

$$\text{Export}_p^{SDK}(\text{Public}(Q)) = \text{SPKI}_{portable}(Q), \quad \text{Import}_q^{SDK}(\text{Export}_p^{SDK}(\text{Public}(Q))) \rightarrow \text{Public}(Q) \quad \forall p, q.$$

Two obligations kept distinct here, not fused: **Encoding** and **Membership.CurveMembership**.

Private contract. Semantic material remains $\text{Private}(d)$, $1 \leq d < n$ – *not* reopened by what follows. The portable contractual representation contains (d, Q) because $[0] = \text{secp256r1}$ and $[1] = 04\|X_{32}\|Y_{32}$ were selected (D-059):

$$\text{Export}_p^{SDK}(\text{Private}(d)) = \text{PKCS8}_{\text{portable}}(d, Q), \quad Q = dG.$$

This does not change $\text{Private}(d) \neq \text{Private}(d, Q)$: the pair describes the composite representation the writer produces, not a redefinition of the abstract private material.

Admission contract – the central equation for this operation. For a contractual private artifact $A = \text{PKCS8}_{\text{portable}}(d, Q)$, conformant admission requires simultaneously $1 \leq d < n$, $Q \in E(\mathbb{F}_p)$, $Q = dG$ – independent of which validations any given backend happens to perform natively:

$$\text{Accept}_C(A) \iff \text{SyntacticValid}(A) \wedge \text{RepresentationValid}(A) \wedge \text{ScalarValid}(d) \wedge \text{CurveMember}(Q) \wedge \text{PairConsistent}(A)$$

Implementation consequence: $\text{Accept}_p^{SDK}(A) = \text{Accept}_C(A)$, even though $\text{Accept}_p^{\text{native}}(A) \neq \text{Accept}_C(A)$ in general (§22.8: no backend demonstrates all five conjuncts natively) – formalizing the adapter’s role without depending on any parser’s accidental semantics.

Directional contract. Two obligations kept explicit, plus one cross-provider obligation:

$$\text{Export}_p^{SDK}(k) \in P_{EC\text{-}ser}^{\text{portable}}, \quad \text{Import}_p^{SDK}(A) \text{ accepts iff } A \text{ satisfies the contractual portable admission domain,}$$

$$\text{Import}_q^{SDK}(\text{Export}_p^{SDK}(k)) \simeq k \quad \forall p, q.$$

$\simeq$ is deliberately narrow: $\text{Public}(Q') \simeq \text{Public}(Q) \iff Q' = Q$; $\text{Private}(d') \simeq \text{Private}(d) \iff d' = d$ – no accidental equality of internal objects, flags, or provider-specific structure is required, and private equivalence is stated over d alone, consistent with $\text{Private}(d)$ (not the pair) remaining the semantic material.

Deliberately left open for the next step, not resolved here. Whether $\text{Import}_p^{SDK}(A)$ accepts iff $A \in P_{\text{portable}}$ *without qualification* is not yet decided: the SDK could strictly reject any valid-but-non-canonical common representation, or could accept certain common representations and normalize them to portable. This affects $R_{\text{val}}/R_{\text{err}}$ directly and belongs to Error semantics, not here – $C_{\text{pre}}^{EC\text{-}ser}$ freezes canonical export and contractual admission requirements, deliberately not the exact reject/normalize boundary.

Frozen principle, connecting directly to RSA-ser’s structural finding, and frozen jointly with the five blocks above as D-060 (§24). *Provider-native admission behavior is evidence about each realization, not the definition of the pre-registered contract. Where a contractual validity obligation is not guaranteed uniformly by native import paths, the SDK adapter is responsible for enforcing it before provider consumption.*

22.10 EC P-256 SDK error semantics and common→portable normalization

Resolves the reject-vs-normalize boundary $C_{\text{pre}}^{EC\text{-}ser}$ (D-060) deliberately left open, since what must be rejected and how a rejection or acceptance surfaces observably are different questions.

Neither extreme is adopted. Strict-portable-only rejection would treat deliberate SDK canonicalization choices as if they were cryptographic validity properties – the clean counterexample is **parameters** [0]: D-059 already froze $[0] \in \{\text{absent}, \text{present-matching}\}$ as common, with portable selecting only present-matching as canonical. Rejecting [0]-absent as an **invalid_key** would conflate *noncanonical* with *invalid*, damaging exactly the diagnostic capability this framework exists to preserve. Unrestricted normalization is equally rejected: normalization is bounded by the already-derived common profile, not by whatever any single backend happens to tolerate – “if some backend can swallow it, normalize it” is explicitly not the rule.

$\text{Normalize}(A) \iff A \in D_{\text{import}}^{\text{common}} \wedge A \notin P_{\text{EC-ser}}^{\text{portable}}, \quad \text{Reject}(A) \iff A \notin D_{\text{import}}^{\text{common}} \text{ or } \neg C_{\text{math}}^{\text{EC}}(A).$

This preserves $P^{\text{portable}} \subseteq P^{\text{common}} \subseteq P_p$ (D-059) without ever promoting a single provider’s extended acceptance into the cross-provider contract.

Four SDK error categories, provider-independent, not exposing native exceptions as contract – kept distinct from unsupported, which is a capability-gate outcome evaluated prior to and separately from these content-level categories (corrected into the formula below, §22.11).

- E_{syntax} : artifact not structurally interpretable – broken DER/ASN.1/container, truncation, incoherent lengths.
- $E_{\text{representation}}$: structurally interpretable but incompatible with the contractual profile – wrong algorithm/curve identifier, unadmitted point form, contradictory parameters.
- E_{key} : internally invalid cryptographic material – $d \notin [1, n-1]$ or $Q \neq dG$ when d, Q contractually coexist. Both V_{scalar} and V_{pair} (§22.8) map to this single observable class – a coarser grouping at the error-category level than at the clause level, mirroring the RSA-ser precedent where multiple distinct clauses share one observable class without losing their causal distinction internally.
- $E_{\text{membership}}$: $Q \notin E(\mathbb{F}_p)$, kept deliberately separate from E_{key} . Collapsing it into E_{key} would undo the methodological effort already invested in keeping $\text{Membership} \neq \text{Key}$ (§22.8) – if both collapsed to a single `invalid_key` at the observable layer, R_{err} would lose exactly the diagnostic value this separation was built to measure.

Obligation, validation mechanism, and error category kept as three distinct layers, not conflated. Example: $Q \notin E(\mathbb{F}_p)$ attacks the clause `Membership.CurveMembership`; the check that discovers it may be classified `Validation.Semantic`; the observable result is $E_{\text{membership}}$. obligation $\neq$ validation mechanism $\neq$ error category – three layers, each already established separately in this design (D-057’s clause taxonomy, D-060’s $C_{\text{math}}^{\text{EC}}$, this section’s error classes), and this section does not merge them.

$Q = dG$ specifically: strict, never normalized, by construction of the formula above. An artifact ($d_1, Q = d_2G$) with $d_1 \neq d_2$ is not normalizable. Even though Q could technically be discarded and $Q' := d_1G$ reconstructed, doing so would destroy exactly the evidence that the supplied artifact violated `Key.PublicPrivateConsistency`:

$Q_{\text{supplied}} \neq dG \implies E_{\text{key}}$, never normalization; $Q \notin E(\mathbb{F}_p) \implies E_{\text{membership}}$, never normalization.

This is not an independent stipulation – it follows directly from the $\text{Import}_p^{\text{SDK}}$ formula below, since normalization is conditioned on $C_{\text{math}}^{\text{EC}}(A)$ holding; a mathematically inconsistent artifact never reaches the normalization branch. Normalization applies to equivalent representations, not to invalid semantics – and this matters experimentally: reconstructing Q would let the adapter silently “repair” a future causal mutation before it could be observed, undermining the mutation’s purpose.

Consolidated contractual boundary – corrected during the clause-audit pass (§22.11) to add an explicit capability gate, absent from the original formulation. An initial version of this formula omitted the `unsupported` branch entirely, leaving `ec-ser.cap` without textual grounding here – exactly the kind of gap this design’s standing discipline catches before freezing (RSA-ser’s D-055/D-057 double-violation and gap corrections are the direct precedent), not after:

$$\text{Import}_p^{\text{SDK}}(A) = \begin{cases} \text{Reject}(\text{unsupported}), & \neg A_p^{\text{pre}}(\text{EC-ser}, \text{import}, \text{role}), \\ \text{Accept}(A), & A \in P_{\text{EC-ser}}^{\text{portable}} \wedge C_{\text{math}}^{\text{EC}}(A), \\ \text{Accept}(N(A)), & A \in D_{\text{import}}^{\text{common}} \setminus P_{\text{EC-ser}}^{\text{portable}} \wedge C_{\text{math}}^{\text{EC}}(A), \\ \text{Reject}(E_{\kappa}), & \text{otherwise,} \end{cases}$$

mirroring RSA-ser’s D-056 evaluability-dependency ordering: capability must be checked before any representation/admission property can be meaningfully evaluated at all. where N is a deterministic, semantics-preserving normalization: $\text{Sem}(N(A)) = \text{Sem}(A)$, required idempotent ($N(N(A)) = N(A)$) with canonical output ($N(A) \in P_{EC-ser}^{\text{portable}}$). These two properties make “normalization” a precise contractual operation, not a generic excuse to repair arbitrary input.

No requirement that native error surfaces coincide across backends. $\text{Error}_{\text{WebCrypto}}^{\text{native}} = \text{Error}_{\text{Crypto++}}^{\text{native}} = \text{Error}_{\text{BC}}^{\text{native}}$ would be an absurd requirement given the confirmed heterogeneity of these APIs (§22.3–§22.5). The obligation is instead

$$\text{Map}_p(\text{Error}_p^{\text{native}}) \rightarrow E_\kappa,$$

applied when the provider itself surfaces the defect, or adapter-side pre-detection when the contract requires a guarantee the provider does not natively offer (§22.8: no backend demonstrates all five conjuncts of Accept_C natively). R_{err} thereby compares the contractual category, not native strings, exception types, or codes.

Frozen as D-061, following the sequence D-059 (profiles) $\rightarrow$ D-060 (C_{pre}) $\rightarrow$ D-061 (admission/error semantics): portable determines canonical output form; common bounds admissible normalization; C_{math}^{EC} determines validity; the four E_κ categories determine the contractual outcome of rejection.

22.11 EC P-256 frozen contract clauses

Thirteen clauses, each a stable contractual obligation, not a mutation or a test – same discipline as HKDF/RSA-ser (D-057). **Membership** kept separate from **Validation**, since that separation was frozen precisely to preserve mathematical structural validity as an independent concept, not folded back in here for convenience.

Bidirectional audit performed before freezing, not after. Forward ($\forall g \in G_{EC-ser}$, g must have direct textual support in D-059/D-060/D-061 or C_{math}^{EC}): confirmed for twelve of thirteen directly; **ec-ser.cap** initially had no grounding, since D-061’s original formula lacked an explicit **unsupported** branch – corrected in D-061 itself (§22.10) before this clause set was frozen, following the same before-not-after discipline as RSA-ser’s D-055/D-057 corrections. Reverse (every C_{pre}^{EC-ser} obligation must be represented by at least one clause): confirmed, with one nuance made explicit rather than left implicit – C_{pub} ’s membership obligation ($Q \in E(\mathbb{F}_p)$ for the public point) has no dedicated public-specific clause; it is covered by **ec-ser.curveMembership**, stated generically over “every contractually supplied Q ,” covering both the public point and any private-embedded point under one clause rather than two role-specific duplicates.

ID	K1	K2	Frozen contract clause
ec-ser.key.role	Key	—	The contract distinguishes $\text{Public}(Q)$ from $\text{Private}(d)$ and preserves role across import/export.
ec-ser.curve	Parameter	DomainParameter	The contractual curve is P-256/ secp256r1 ; no implicit domain substitution.
ec-ser.public.asn1Encoding		ASN1	The portable public representation is DER-SPKI with id-ecPublicKey and namedCurve(secp256r1) .
ec-ser.public.pointEncoding		KeyPoint	Q is encoded only as $04\ X_{32}\ Y_{32}$ – a K_2 subtype genuinely distinct from Encoding.ASN1 , not exercised by RSA-ser, directly relevant to marginal K_2 coverage.

ID	K1	K2	Frozen contract clause
ec-ser.private.asn1	Encoding	ASN1	The portable private representation is DER-PKCS#8/RFC 5915 with version = 1, d_{32} , $[0] = \text{secp256r1}$, $[1] = Q_{uncompressed}$.
ec-ser.private.scalar	Key	PrivateScalar	The private scalar satisfies $1 \leq d < n$.
ec-ser.curveMembership	Membership	CurveMembership	Every contractually supplied Q satisfies $Q \in E(\mathbb{F}_p)$ – covers both public and private-embedded occurrences of Q under one clause.
ec-ser.pairConsistency	Key	PublicPrivateConsistency	When d and a supplied Q coexist in a contractual artifact, $Q_{supplied} = dG$.
ec-ser.export	Output	—	Export _p ^{SDK} produces the canonical portable representation, preserving the abstract semantic material.
ec-ser.validation.syntax	Syntax	Syntactic	Malformed DER/ASN.1/container structures are rejected before semantic consumption.
ec-ser.validation.semantic	Semantic	Semantic	Admission applies C_{math}^{EC} and the D-061 boundary: portable $\rightarrow$ accept; common-non-portable and mathematically valid $\rightarrow$ normalize; outside common or mathematically invalid $\rightarrow$ reject.
ec-ser.error	Error	—	Rejections are exposed via the contractual taxonomy $\{E_{syntax}, E_{representation}, E_{key}, E_{membership}\}$, independent of the native error.
ec-ser.cap	Capability	—	The pre-registered manifest distinguishes what each provider supports from the common/portable contractual surface.

Three granularity decisions, deliberately not collapsed further.

1. **The two public Encoding clauses are not merged.** SPKI/AlgorithmIdentifier/DER and $04\|X_{32}\|Y_{32}$ are causally distinct phenomena sharing $K_1 = \text{Encoding}$ but differing in K_2 (ASN1 vs. KeyPoint). This matters directly for Marginal_{K_2} : RSA-ser already exercises ASN.1 but never EC point representation – exactly the case the necessity-analysis design intends, where a genuinely distinct K_2 subtype prevents an empty marginal K_1 contribution from eliminating an operation.
2. **private.scalar and pairConsistency remain separate**, even though both observably resolve to E_{key} : their causes are distinct ($d \notin [1, n-1]$ versus $d \in [1, n-1] \wedge Q \in E(\mathbb{F}_p) \wedge Q \neq dG$), and sharing an observable error category never justifies merging the causal obligations behind it – the same principle already applied when D-061 grouped V_{scalar}/V_{pair} under one E_{κ} without collapsing their clauses.
3. **curveMembership does not also appear inside validation.semantic.** The former states which mathematical property must hold; the latter states what the SDK layer must do about acceptance/rejection/normalization. Exactly obligation $\neq$ validation mechanism $\neq$ observable error, already established in §22.10.

No `ec-ser.inputArtifact` clause, added only to populate $K_1 = \text{Input}$. The imported artifact is obviously an API input, but no independent input semantics exists here beyond what **Encoding**, **Validation**, **Key**, and **Membership** already express – a clause created solely to inflate coverage would violate the standing methodology. **Authentication** does not apply to this operation either. The real K_1 coverage for EC-ser is therefore **{Key, Parameter, Output, Encoding, Validation, Membership, Error, Capabil** – 8 of 10 K_1 values, with **Input** and **Authentication** legitimately absent, not missing.

One `ec-ser.cap`, not two. AES-GCM’s split between provider and portable capability was justified by a genuinely rich parameterized boundary (tag lengths, IV lengths). $P^{\text{portable}} \subseteq P^{\text{common}} \subseteq P_p$ already represents the analogous boundary for EC-ser explicitly; no second, independent capability obligation has been identified. If a genuine causal difference between **Supported(EC-serialization)** and **Supported(EC-serialization, θ)** emerges during mutation construction, the clause can still be split before Γ_0 is frozen – not preemptively here.

Most novel clause, transversally. `ec-ser.curveMembership` (Membership.CurveMembership**):** AES-GCM covered 9 of 10 K_1 values, with **Membership** the one systematic absence – one of the original methodological reasons EC import/export/validation was retained as a candidate operation without requiring full ECDSA (D-015). The second most interesting is `ec-ser.pairConsistency` (**Key.PublicPrivateConsistency**), whose marginal K_2 value against RSA-ser is not yet decided – whether RSA-ser’s own public/private relationship classifies under the identical **PublicPrivateConsistency** subtype or a distinct one is deliberately left to the transversal audit (§23, not yet performed for this pair), not resolved here.

Formally, $G_{EC-ser} = \{g_1, \dots, g_{13}\}$ with $\text{kind}_1(g) \in K_1$, $\text{kind}_2(g) \in K_2 \cup \{-\}$, maintaining strictly $C_{pre}^{EC-ser} \rightarrow G_{EC-ser} \rightarrow \text{Mut}(EC-ser) \rightarrow \Gamma_0^{EC-ser}$. **Frozen as D-062.**

22.12 EC P-256 causal classes and Γ_0^{EC-ser}

Fourteen causal classes, not a mechanical one-per-clause correspondence: the mutation defines the known cause; observable relations are the detectors evaluated afterward, never the other way around. Γ_0 is not built by asking which relation we want to justify.

Microaudit performed against the frozen text before writing this table, not assumed.

Three points checked directly: (i) **unsupported** is confirmed a standalone outcome (**Reject(unsupported)**, gated by $\neg A_p^{\text{pre}}$), not a member of E_κ – **EC-CAPABILITY-MANIFEST-MISMATCH** accordingly targets only `ec-ser.cap`, inventing no fifth error category D-061 never created. (ii) K_2 labels confirmed exactly **PrivateScalar** and **PublicPrivateConsistency**, matching D-062 verbatim. (iii) `ec-ser.private.asn1`’s frozen text (§22.11) describes only the shape of the canonical portable representation, not an obligation to reconstruct it from a common-but-non-portable input – it does not cover the normalization obligation, and is accordingly *excluded* from $\Gamma_0(\text{EC-PRIVATE-PARAMS-ABSENT})$, corrected from an initial draft that included it.

Causal class	Controlled stimulus	$\Gamma_0(c)$
EC-ROLE-CONTAINER-MISMATCH	Public material contractually directed as private, or vice versa, base material held constant	<code>ec-ser.key.role</code> , <code>ec-ser.validation.semantic</code> , <code>ec-ser.error</code>
EC-CURVE-SUBSTITUTION	Substitute <code>secp256r1</code> for another domain/OID, remainder structurally valid	<code>ec-ser.curve</code> , <code>ec-ser.public.asn1</code> or <code>ec-ser.private.asn1</code> (by role), <code>ec-ser.validation.semantic</code> , <code>ec-ser.error</code>
EC-PUBLIC-POINT-ENCODING	Substitute <code>04 X₃₂ Y₃₂</code> for compressed/hybrid form, same Q	<code>ec-ser.public.point</code> , <code>ec-ser.validation.semantic</code> , <code>ec-ser.error</code>

Causal class	Controlled stimulus	$\Gamma_0(c)$
EC-PUBLIC-OFF-CURVE	Q' well-encoded, uncompressed, but $Q' \notin E(\mathbb{F}_p)$	ec-ser.curveMembership, ec-ser.validation.semantic, ec-ser.error
EC-PRIVATE-SCALAR-RANGE	$d = 0$ or $d \geq n$, PKCS#8/DER otherwise correct	ec-ser.private.scalar, ec-ser.validation.semantic, ec-ser.error
EC-PRIVATE-PAIR-MISMATCH	$Q = d_1G$, $Q = d_2G$, $d_1 \neq d_2$, both individually valid	ec-ser.pairConsistency, ec-ser.validation.semantic, ec-ser.error
EC-PUBLIC-DER-MALFORMED	Purely syntactic corruption of SPKI/DER, not converted into another valid semantic artifact	ec-ser.public.asn1, ec-ser.validation.syntax, ec-ser.error
EC-PRIVATE-DER-MALFORMED	Purely syntactic corruption of PKCS#8/RFC 5915	ec-ser.private.asn1, ec-ser.validation.syntax, ec-ser.error
EC-PRIVATE-PARAMS-ABSENT	Remove [0] parameters, keep [1], outer curve, and math valid	ec-ser.validation.semantic, ec-ser.export
EC-PRIVATE-PARAMS-MISMATCH	[0] present but incompatible with outer secp256r1	ec-ser.curve, ec-ser.private.asn1, ec-ser.validation.semantic, ec-ser.error
EC-PRIVATE-PUBKEY-ABSENT	Remove [1] publicKey, keep d valid	ec-ser.private.asn1, ec-ser.validation.semantic, ec-ser.error
EC-NATIVE-EXPORT-LEAK	Adapter exposes $\text{Export}^{\text{native}}$ directly instead of constructing/normalizing portable $\text{Export}^{\text{SDK}}$	ec-ser.export, ec-ser.public.asn1 or ec-ser.private.asn1 (by role)
EC-ERROR-MAP-SWAP	Correct rejection, but the contractual category is deliberately swapped (e.g. $E_{\text{membership}} \rightarrow E_{\text{key}}$)	ec-ser.error
EC-CAPABILITY-MANIFEST-MISMATCH	Discrepancy introduced between A_p^{pre} and effective behavior/unsupported branch	ec-ser.cap

Off-curve vs. pair-mismatch: the cleanest experimental pair in this Γ_0 . $c_{\text{offcurve}} : Q' \notin E(\mathbb{F}_p)$ versus $c_{\text{pair}} : d = d_1, Q = d_2G, d_1 \neq d_2$ (where $Q \in E(\mathbb{F}_p)$ still holds) must remain fully separated: $\text{ec-ser.curveMembership} \in \Gamma_0(c_{\text{offcurve}})$ but $\text{ec-ser.pairConsistency} \notin \Gamma_0(c_{\text{offcurve}})$; symmetrically $\text{ec-ser.pairConsistency} \in \Gamma_0(c_{\text{pair}})$ but $\text{ec-ser.curveMembership} \notin \Gamma_0(c_{\text{pair}})$. This yields genuine causal isolation, and Membership is exactly the K_1 AES-GCM’s coverage leaves empty – real marginal potential.

[0]-absent and [1]-absent must not be merged – this follows directly from D-059/D-061, not from stylistic preference. For [0]-absent: $A \in D_{\text{import}}^{\text{common}} \setminus P^{\text{portable}}$, and the contractually correct outcome is $\text{Accept}(N(A))$ – **ec-ser.error** is therefore *not* attributed to this class, since the contract does not demand an error here. For [1]-absent, under the profile already frozen, $A \notin D_{\text{import}}^{\text{common}}$, and the contractual outcome is rejection. This pair directly attacks the exact boundary between common-non-portable $\rightarrow$ normalize and outside common $\rightarrow$ reject – plausibly the single most informative stimulus pair in this Γ_0 , since it exercises the D-061 boundary formula at its precise edge rather than deep inside either region.

[0]-absent’s Γ_0 , **precisely justified**. `ec-ser.private.asn1` is excluded (confirmed by the microaudit above: its frozen text does not cover a reconstruction obligation). The class attacks `ec-ser.export`’s canonical-representation obligation – specifically, the requirement that $N(A) \in P_{\text{portable}}$ once normalization is triggered – while `ec-ser.validation.semantic` covers the obligation to admit-and-normalize rather than reject. *Non-portable is not malformed*: this must be stated literally so it is never conflated by a future reader.

Why EC-ERROR-MAP-SWAP is necessary. Suppose an off-curve artifact is correctly rejected ($R_{\text{val}} = \checkmark$), but the adapter returns E_{key} instead of $E_{\text{membership}}$: then $R_{\text{err}} = \times$ while acceptance/rejection remains correct. $\Gamma_0(c_{\text{err-map}}) = \{\text{ec-ser.error}\}$ provides an isolating stimulus for whether R_{err} carries information independent of R_{val} – precisely what must not be prejudged before the framework’s necessity analysis (§23) is run.

Why EC-CAPABILITY-MANIFEST-MISMATCH is scoped as narrowly as possible. It should disturb only $A_p^{\text{pre}}(\text{EC-ser})$ – e.g. falsifying the pre-registered manifest to `false` while the realization genuinely satisfies the operation, or the controlled inverse – while leaving cryptographic behavior intact, so that R_{cap} has a genuine experimental opportunity to demonstrate independent value rather than remaining permanently correlated with validation/error.

Resulting coverage.

$$\text{Coverage}_{\text{local}}(\text{EC-ser}) = \bigcup_{c \in \text{Mut}(\text{EC-ser})} \Gamma_0(c) = G_{\text{EC-ser}}$$

(all thirteen D-062 clauses covered; verified directly against the table above, not assumed). K_1 coverage: `{Key, Parameter, Output, Encoding, Validation, Membership, Error, Capability}` – 8/10, as anticipated in D-062, with `Input/Authentication` legitimately, not artificially, absent. K_2 coverage includes at minimum `{Parameter.DomainParameter, Encoding.ASN1, Encoding.KeyPoint, Validation.Syntactic}` using the exact D-062 subtype labels confirmed in the microaudit.

Frozen as D-063.

22.13 Cross-operation audit: RSA key serialization vs. EC P-256 key serialization

Establishes $\text{Marginal}(\text{EC-ser} \mid \text{RSA-ser})$ specifically – *not* yet the global $\text{Marginal}(\text{EC-ser})$ against the full operation set O , which requires comparison against every other candidate operation and is deferred to the formal Coverage/Marginal computation (§23, not yet performed). This is an intermediate scientific result, not a working note: it demonstrates *why* EC-ser is not redundant with RSA-ser before that formal computation runs, and belongs to the traceability record for eventual O_{min} selection.

K_1/K_2 -by-dimension comparison, six results, each stated explicitly.

<code>Encoding.ASN1</code>	shared	RSA-ser: der-syntax, container, algorithm-params. EC-ser: p
<code>Encoding.KeyPoint</code>	EC-specific	No RSA-ser analogue exists – RSA has no notion of a curve
<code>Membership.CurveMembership</code>	EC-specific	No RSA-ser clause uses $K_1 = \text{Membership}$ at all – the full K_1
<code>Key.PublicPrivateConsistency</code>	EC-specific	New K_2 within an already-covered $K_1 = \text{Key}$; structural, not
<code>Validation.Syntactic/Semantic</code>	shared	Same K_2 subtypes as RSA-ser; no new coverage here.
<code>Capability</code>	shared	Same K_1 , no subtype in either operation, established before

Three distinct marginal contributions, not two – a correction made during this audit, not glossed over. An initial draft of this analysis grouped `Key.PublicPrivateConsistency` together with `Membership` as if both were manifestations of the same K_1 -level novelty. They are not: D-062 froze `ec-ser.pairConsistency` under $K_1 = \text{Key}$, $K_2 = \text{PublicPrivateConsistency}$, while `ec-ser.curveMembership` sits under the separate $K_1 = \text{Membership}$, $K_2 = \text{CurveMembership}$. Conflating them would misstate the nature of two genuinely different kinds of marginal contribution:

$$\text{Marginal}_{K_1}(\text{EC-ser} \mid \text{RSA-ser}) \supseteq \{\text{Membership}\}, \quad \text{Marginal}_{K_2}(\text{EC-ser} \mid \text{RSA-ser}) \supseteq \{\text{Encoding.KeyPoint, Mer}\}$$

Membership contributes an entire empty K_1 (the strongest kind of marginal result, directly confirming D-015’s original ground for retaining EC serialization as a candidate operation without requiring full ECDSA); **Encoding.KeyPoint** and **Key.PublicPrivateConsistency** each contribute a genuinely new K_2 within a K_1 RSA-ser already covers – a real but categorically different kind of marginal value.

$V_{rel}^{RSA,2} \not\equiv V_{pair}^{EC}$: **the structural argument, preserved here because it is the substantive finding, not a labeling choice.** RFC 8017’s RSAPrivateKey (Appendix A.1.2) carries $n, e, d, p, q, d_P, d_Q, q_{Inv}$ all as components of one structure – there is no separate, optional, redundant public component that could independently diverge from the private material, because n, e (the public material) are embedded directly, not supplied as an independent field a caller could fill inconsistently. **private-relations** checks the *internal self-consistency of a single object*. RFC 5915 is structurally different: **publicKey**[1] is an *optional, redundant* field – d alone already determines $Q = dG$ mathematically, so [1] adds no new information when consistent, *but its existence as a separate field is exactly what opens the possibility that it could be populated with a diverging value*. That structural possibility (a redundant field that can diverge) simply does not exist in RFC 8017. **Key.PublicPrivateConsistency** is therefore genuinely new, not a re-labeling of something RSA-ser already covered – a consequence of RFC 5915 exposing an attack surface (an optional redundant field) that RFC 8017 does not, not a classification choice made for convenience.

$E_{membership}$ **noted as an observable-taxonomy novelty, deliberately not computed as K_1/K_2 marginal.** D-061’s error-category taxonomy gained a genuinely new class ($E_{membership}$, alongside $E_{syntax}/E_{representation}/E_{key}$, none of which RSA-ser’s four-class taxonomy {**unsupported, malformed_artifact, i** includes under that exact name), but this is an axis distinct from K_1/K_2 clause coverage and must not be folded into $Marginal_{K_1}/Marginal_{K_2}$ accounting, which concerns contractual clauses, not observable error-class richness.

Conclusion, frozen as D-064. EC-ser cannot be dismissed as a mere duplicate of RSA-ser: it contributes one fully novel K_1 (**Membership**) and two novel K_2 subtypes within otherwise-shared K_1 values (**Encoding.KeyPoint**, **Key.PublicPrivateConsistency**), the latter grounded in a genuine RFC 5915/RFC 8017 structural asymmetry rather than a terminological distinction. This result specifically establishes $Marginal(EC-ser \mid RSA-ser)$; the formal, global $Marginal(EC-ser)$ against the complete operation set O is computed next, in §23, and may narrow or confirm this pairwise finding once every other candidate operation is included.

23 Necessity decision rule

Operations and relations are not retained by attachment or symmetry. Necessity is evaluated in cascade: (1) marginal K_1 coverage, (2) marginal K_2 subtype coverage, (3) detection uniqueness/gain, and (4) diagnostic gain. An operation with no contribution on all four dimensions is removed from O_{min} .

This rule is especially important for RSA key serialization versus EC key serialization/validation, and for deciding whether R_val and R_err remain separate after empirical evaluation.

23.1 Formal Coverage/Marginal computation, stages 1–2

$O = \{HKDF, AES-GCM, RSA-OAEP, RSA-PSS, RSA-ser, EC-ser\}$. Only stages (1)–(2) are computed here: Γ_0 and the frozen clause tables are pre-registered and available now, so $Coverage_{K_1}/Coverage_{K_2}$ can be closed; **detection gain and diagnostic gain (stages 3–4) require executed mutations and are deliberately not computed here** – consistent with the necessity rule as already frozen, not a simplification introduced for convenience.

Mechanical verification, not recollection. Every entry below was read directly from each

operation’s frozen clause table (§13, §16, §20, §22.11, and the HKDF/AES-GCM tables), not reconstructed from memory of prior discussion – a provisional K-1/K-2 sketch is exactly the kind of claim this design’s standing discipline (D-036, D-038, D-050, D-057) requires checking against source before freezing.

K_1 coverage, verified per operation.

HKDF	{Input, Parameter, Output, Validation, Error, Capability}
AES-GCM	{Key, Input, Parameter, Output, Authentication, Encoding, Validation, Error, Capability}
RSA-OAEP	{Key, Parameter, Input, Output, Validation, Error, Capability}
RSA-PSS	{Key, Parameter, Input, Output, Authentication, Validation, Error, Capability}
RSA-ser	{Key, Parameter, Encoding, Validation, Error, Capability}
EC-ser	{Key, Parameter, Output, Encoding, Validation, Membership, Error, Capability}

AES-GCM $\cup$ EC-ser alone already covers all ten K_1 kinds. Consequently:

$$\text{Marginal}_{K_1}(\text{EC-ser}) = \{\text{Membership}\}, \quad \text{Marginal}_{K_1}(o) = \emptyset \quad \forall o \neq \text{EC-ser}.$$

No other operation’s clause table instantiates $K_1 = \text{Membership}$ anywhere; since AES-GCM alone already reaches 9/10 (missing only **Membership**), no combination of the remaining operations could produce a second empty- K_1 marginal without one of them freezing a **Membership** clause, which none do.

K_2 coverage, verified per operation, with cross-operation sharing made explicit rather than assumed.

HKDF	{Parameter.AlgorithmChoice, Parameter.Length, Validation.Semantic}
AES-GCM	{Parameter.ExternalRandomnessControl, Parameter.Length, Authentication.AEADTag, Encoding.AEADArtifact}
RSA-OAEP	{Parameter.Length, Parameter.AlgorithmChoice, Parameter.DomainParameter, Parameter.ExternalRandomnessControl}
RSA-PSS	{Parameter.Length, Parameter.AlgorithmChoice, Parameter.DomainParameter, Parameter.ExternalRandomnessControl}
RSA-ser	{Encoding.ASN1, Validation.Syntactic, Validation.Semantic, Parameter.AlgorithmChoice}
EC-ser	{Encoding.ASN1, Encoding.KeyPoint, Validation.Syntactic, Validation.Semantic, Membership.CurveMembership, KeyPair}

A genuine finding surfaced only by this cross-check, not visible from the RSA-ser/EC-ser pairwise audit alone (D-064): **Parameter.DomainParameter** is shared between **oaep.mgfCoupling** and **pss.mgfCoupling** (RSAOAEP= $H_{\text{MGF1-coupling}}$ / PSS= $H_{\text{MGF1-coupling}}$ respectively) – confirming it was never a viable marginal candidate for EC-ser’s **ec-ser.curve**, though D-064’s pairwise comparison against RSA-ser alone (which has no **DomainParameter** clause) could not have revealed this either way, since RSA-ser simply never instantiated the subtype. **Encoding.ASN1** and **Validation.Syntactic** are shared exclusively between RSA-ser and EC-ser (no other operation instantiates either), so neither contributes to either operation’s marginal at the global level despite each being introduced first by RSA-ser chronologically – marginal status is a set-difference property, not a precedence claim. **Parameter.ExternalRandomnessControl** is shared across AES-GCM, RSA-OAEP, and RSA-PSS (the IV, the (absent) OAEP seed/RNG, and the (absent) PSS RNG/salt-bytes controls respectively) – confirmed not marginal for any of the three. **Parameter.AlgorithmChoice** and **Parameter.Length** are the most widely shared subtypes across the full set.

$$\text{Marginal}_{K_2}(\text{HKDF}) = \emptyset, \quad \text{Marginal}_{K_2}(\text{AES-GCM}) = \{\text{Authentication.AEADTag, Encoding.AEADArtifact}\},$$

$$\text{Marginal}_{K_2}(\text{RSA-OAEP}) = \emptyset, \quad \text{Marginal}_{K_2}(\text{RSA-PSS}) = \{\text{Authentication.SignatureVerification}\},$$

$$\text{Marginal}_{K_2}(\text{RSA-ser}) = \emptyset, \quad \text{Marginal}_{K_2}(\text{EC-ser}) = \{\text{Encoding.KeyPoint, Membership.CurveMembership, KeyPair}\}.$$

Resulting stage-1/2 classification, frozen as D-065. Structurally necessary by Coverage alone, before any mutation is executed:

$$O_{\text{structurally necessary}} \supseteq \{\text{AES-GCM, RSA-PSS, EC-ser}\}.$$

Three operations whose survival in $O_{\min}$ depends entirely on stages 3–4, not yet decided:

$$\{\text{HKDF}, \text{RSA-OAEP}, \text{RSA-ser}\}, \quad \text{Marginal}_{K_1} = \text{Marginal}_{K_2} = \emptyset \text{ for each.}$$

This is precisely what the necessity methodology was designed to produce: not every operation invested in survives automatically, and Coverage alone already separates the six candidates into two groups with different evidentiary burdens going forward. HKDF’s candidate role for stages 3–4 is its clean deterministic R_{byte} baseline (also reachable via AES-GCM, so not obviously sufficient on its own); RSA-OAEP’s is R_{interop} under probabilistic encryption ($\text{Enc}_p(m) \neq \text{Enc}_q(m)$ while $\text{Dec}_q(\text{Enc}_p(m)) = m$ must hold); RSA-ser’s stands entirely on detection/diagnostic gain with zero Coverage contribution, the sharpest test case for stages 3–4 of any operation in O .

23.2 Stages 3–4 protocol: detection and diagnostic gain, pre-registered

Registers the *measure*, not an empirical result. Stages 3–4 require executed mutations (§23.1); what is frozen here is how DetectionGain and DiagnosticGain will be computed once that execution happens, and the three specific hypotheses each zero-Coverage-marginal candidate must clear to survive in $O_{\min}$.

Detection gain. For each observable relation $R_j \in \{R_{\text{byte}}, R_{\text{interop}}, R_{\text{ser}}, R_{\text{val}}, R_{\text{err}}, R_{\text{cap}}\}$, let $D(o)$ be the set of divergence patterns causally detected by some relation when executing $\text{Mut}(o)$. Then

$$\text{DetectionGain}(o) = D(o) \setminus \bigcup_{o' \neq o} D(o').$$

Diagnostic gain, corrected during the cumulative audit (D-069) – the original formulation below is superseded, not merely restated. The initial definition here computed $|\Pi_R(O)| - |\Pi_R(O \setminus \{o\})|$ over raw observable-vector equivalence. Independent verification during the cumulative audit confirmed a genuine partial conflation: since each operation’s Γ_0 draws exclusively from its own local clause namespace, a single mutation with an observable vector $r(c)$ not replicated by any other operation is enough to make that difference positive – but this only demonstrates a unique detectable pattern, exactly what DetectionGain already measures, not that the mutation resolves any genuine causal ambiguity. Confirmed non-trivial but real by direct comparison of two already-frozen classes: GCM-PROVIDER-CAPABILITY-MISMATCH (expected relation R_{cap} alone, D-038) versus HKDF-CAPABILITY-BOUNDARY-MISMATCH (expected $R_{\text{cap}}; R_{\text{val}}; R_{\text{err}}$) – not universally degenerate, but the flaw is triggered by any single uniquely-shaped vector, which is common enough to compromise stages 3 and 4 as cleanly separate tests.

Corrected construct. First, the *semantic causal signature* of a mutation, over the shared cross-operation taxonomy, not operation-local clause IDs:

$$\sigma(c) = \{\text{kind}_{K_1/K_2}(g) : g \in \Gamma_0(c)\}.$$

Because $\sigma(c)$ is defined over K_1/K_2 kinds shared across all six operations, two mutations from *different* operations can share σ – unlike raw clause IDs, which never overlap across operations by construction, and unlike the raw vector $r(c)$ used in the superseded formula. A diagnostic distinction realized by operation o is a pair of genuinely distinct semantic signatures that o ’s own mutations demonstrate are empirically separable:

$$(\sigma_i, \sigma_j) \in Q(o) \iff \exists c_i, c_j \in \text{Mut}(o) : \sigma(c_i) = \sigma_i, \sigma(c_j) = \sigma_j, \sigma_i \neq \sigma_j, r(c_i) \neq r(c_j).$$

$$\text{DiagnosticGain}(o) = \left| Q(o) \setminus \bigcup_{o' \neq o} Q(o') \right|.$$

This decouples cleanly from DetectionGain: a single uniquely-shaped $r(c)$ contributes nothing to $Q(o)$ by itself – a contribution requires *two* mutations with distinct semantic signatures whose

empirical separability, as a *pairing*, is not reproduced by any other operation; neither individual vector need itself be unique. $Q(o)$ remains an empirical, stage-4 quantity (requires executed $r(c)$), computable only once mutations run, exactly matching this design’s execute-before-conclude discipline (D-007). $\text{Keep}(o) \iff \text{DetectionGain}(o) > 0 \vee \text{DiagnosticGain}(o) > 0$ is unchanged; only the diagnostic term’s definition is corrected.

D-052 remains the finer-grained, within-operation quantity, untouched by this correction. $H(C \mid R, o)$ or equivalently $I(C; R \mid o) = H(C \mid o) - H(C \mid R, o)$ can still measure how much diagnostic information an operation’s relations provide at the entropy level; $Q(o)$ answers the coarser, comparative question of whether o contributes a marginal cross-operation distinction no other operation also provides. The two are complementary, not competing, measures.

Auxiliary quantities, to be computed per candidate once mutations run.

$$U_D(o) = \{r(c) : c \in \text{Mut}(o)\} \setminus \bigcup_{o' \neq o} \{r(c') : c' \in \text{Mut}(o')\}$$

(non-empty indicates detection-pattern uniqueness); recall of detected causes, $\text{Recall}_R(o) = |\{c : \exists R_j, R_j \text{ detects } c\}|/|\text{Mut}(o)|$; and $\Pi_R(o) = \text{Mut}(o)/\sim_R$ as the operation-local partition, whose fineness is a proxy for diagnostic richness before the global DiagnosticGain subtraction is computed.

Keep rule, no exceptions.

$$\text{Keep}(o) \iff \text{DetectionGain}(o) > 0 \vee \text{DiagnosticGain}(o) > 0.$$

Combined with D-065: if $\text{Marginal}_{K_1}(o) = \text{Marginal}_{K_2}(o) = \emptyset$ and $\text{DetectionGain}(o) = \text{DiagnosticGain}(o) = 0$, then $o \notin O_{\min}$, without exception for effort already invested.

Three specific hypotheses, pre-registered for the three zero-Coverage-marginal candidates – stated as what must be demonstrated, not asserted in advance.

H_{HKDF} : the pure-deterministic case contributes independent detection or diagnostic value for R_{byte} . This is a genuinely open question, not rhetorical: `gcm.ciphertext` already states determinism explicitly (“*Equal K, IV, AAD and M produce identical ciphertext bytes*”), and multiple already-frozen AES-GCM classes (`GCM-PLAINTEXT-NORMALIZATION`, `GCM-IV-INTERPRETATION-DIVERGENCE`, `GCM-CIPHERTEXT-COMPUTATION-DIVERGENCE`) already expect R_{byte} as their detector. HKDF’s hypothesis is not merely “HKDF uses R_{byte} ” – that alone is insufficient given AES-GCM’s own deterministic surface. If AES-GCM detects and diagnoses the same byte-level divergence patterns HKDF would, $\text{HKDF} \notin O_{\min}$ follows, and this would be a legitimate, not a disappointing, outcome: being useful for designing the framework (HKDF’s documented pilot role, §23.1) does not entail being necessary for validating it.

H_{OAEP} : OAEP contributes divergences detectable by R_{interop} that cannot be correctly evaluated via R_{byte} . Precisely stated as an *applicability* property, not yet empirical detection gain: $\text{Applicability}(R_{\text{byte}}, \text{OAEP}) = 0$, $\text{Applicability}(R_{\text{interop}}, \text{OAEP}) = 1$, since $\text{Enc}_p(m) \neq \text{Enc}_q(m)$ can both be correct under probabilistic encryption while $\text{Dec}_q(\text{Enc}_p(m)) = m$ is the actual contractual obligation – exactly OAEP’s original design motivation. This already proves the framework needs both relations conceptually, but OAEP’s survival in $O_{\min}$ requires the stronger empirical claim: $\exists c \in \text{Mut}(\text{OAEP})$ detected by R_{interop} , with no remaining operation providing an equivalent causal stimulus. Judged the strongest candidate for surviving stage 3 among the three, but not decided here.

$H_{\text{RSA-ser}}$: RSA/ASN.1-specific structure and mutations contribute detection or diagnostic gain that EC-ser does not reproduce. RSA-ser’s hypothesis cannot rest on “a serialization operation is needed” – EC-ser already satisfies that role (`Encoding, Validation.Syntactic/Semantic`, import/export, R_{ser} , errors, capability, per D-065). The more interesting candidate source of gain is not `Encoding.ASN1` itself but RSA’s internal causal spectrum ($n, p, q, d, d_P, d_Q, q_{\text{Inv}}$ relations) versus EC’s ($d, Q, Q = dG$) – but D-065 already establishes that causal difference does not automatically

imply taxonomic marginality: RSA-ser survives only if its mutations produce an observable vector pattern $(R_{\text{ser}}, R_{\text{val}}, R_{\text{err}}, R_{\text{cap}})$ that EC-ser’s mutations do not, or improve causal disambiguation where EC-ser’s vectors collapse distinct causes together. If RSA-ser’s mutations resolve to the same observable patterns EC-ser already produces, RSA-ser $\notin O_{\text{min}}$ – and prior investment (RSA-ser is the single largest normative/contractual construction in this design, D-053–D-058) is explicitly not a defense against this outcome (D-065).

Pre-registered expectation, explicitly not converted into a conclusion. If forced to hypothesize before execution: RSA-OAEP has the strongest documented justification via the $R_{\text{byte}} \not\sim R_{\text{interop}}$ separation; HKDF could legitimately disappear from O_{min} since AES-GCM already supplies a deterministic R_{byte} case; RSA-ser is the most threatened by redundancy with EC-ser and must earn its place through causal diagnosis, not Coverage. None of these three expectations is frozen as a result – only the protocol and the three hypotheses are frozen here, as **D-066**. O_{min} ’s exact final composition awaits executed mutations; fabricating that result now would violate the same execute-before-conclude discipline this design has maintained throughout (D-007).

23.3 The O_{min} decision boundary, closed preexperimentally

O_{min} itself is not closed as a set – but the region of uncertainty around it is completely closed preexperimentally. This distinction is the substantive result of this subsection, not a caveat appended to it.

Three unambiguous names, deliberately not two. O_{fixed} – membership already decided; O_{pending} – membership contingent on stages 3–4; O_{min} – the final result after empirical execution. $O_{\text{min}}^{\text{pre}}$ is deliberately not used for O_{fixed} : that name would suggest a provisional minimal set already close to final, when what is actually being asserted is a strict lower bound with an explicit, bounded region of remaining uncertainty – a different and stronger kind of claim.

$$O_{\text{fixed}} = \{\text{AES-GCM, RSA-PSS, EC-ser}\}, \quad O_{\text{pending}} = \{\text{HKDF, RSA-OAEP, RSA-ser}\}.$$

$O_{\text{fixed}} \subseteq O_{\text{min}}$ holds without depending on any future result: $\text{Marginal}_{K_1}(\text{EC-ser}) \neq \emptyset$ and $\text{Marginal}_{K_2}(\text{AES-GCM}), \text{Marginal}_{K_2}(\text{RSA-PSS}), \text{Marginal}_{K_2}(\text{EC-ser}) \neq \emptyset$ (D-065) already satisfy the necessity rule’s retention condition regardless of stages 3–4. For each $o \in O_{\text{pending}}$, $\text{Marginal}_{K_1}(o) = \text{Marginal}_{K_2}(o) = \emptyset$ (D-065), so membership is determined exclusively by D-066’s rule:

$$O_{\text{min}} = O_{\text{fixed}} \cup O_{\text{emp}}, \quad O_{\text{emp}} = \{o \in O_{\text{pending}} : \text{DetectionGain}(o) > 0 \vee \text{DiagnosticGain}(o) > 0\} \subseteq O_{\text{pending}}.$$

Immediate bounds:

$$O_{\text{fixed}} \subseteq O_{\text{min}} \subseteq O_{\text{fixed}} \cup O_{\text{pending}} = O, \quad \{\text{AES-GCM, RSA-PSS, EC-ser}\} \subseteq O_{\text{min}} \subseteq \{\text{HKDF, AES-GCM, RSA-OAEP, RSA-ser}\}.$$

What is closed now, definitively, four items. (i) The candidate universe O itself. (ii) O_{fixed} , the three structurally-necessary operations. (iii) O_{pending} , the exact three operations whose necessity remains empirically open – no fourth or fifth candidate is in play. (iv) The exact criterion (D-066) that will resolve each of the three, fixed before the harness runs. After D-066, there is no remaining freedom to look at results and retroactively decide what “useful” means – consistent with this design’s pre-registered character throughout.

What must not be closed now. Neither $O_{\text{min}} = \{\text{AES-GCM, RSA-PSS, EC-ser}\}$ nor $O_{\text{min}} = O$ may be asserted – both are unexecuted empirical claims. Nor may OAEP be declared to “survive” on the strength of its conceptual $R_{\text{interop}} \neq R_{\text{byte}}$ separation alone: D-066 correctly converts that argument into a pre-registered hypothesis, not evidence. Symmetrically, HKDF must not be eliminated merely because AES-GCM already has determinism, nor RSA-ser merely because EC-ser already has R_{ser} – the data, not the argument’s plausibility, must be what rescues or eliminates each of the three via $\text{DetectionGain} > 0$ or $\text{DiagnosticGain} > 0$.

The remaining space is enumerable. With exactly three separately-evaluated binary membership decisions – not three statistically or empirically *independent* ones; their eventual empirical outcomes could well be correlated, and nothing in this design requires or assumes otherwise – $O_{\min} = O_{\text{fixed}} \cup S$, $S \subseteq O_{\text{pending}}$, gives exactly $2^3 = 8$ possible final outcomes, ranging from the absolute minimum {AES-GCM, RSA-PSS, EC-ser} to the full six-operation set. This is precisely the right way to express how far the design has progressed: which of the eight candidate sets $O_{\min}$ will turn out to be is not yet known, but exactly why and how the choice among those eight will be made is now fully specified.

Frozen as D-067. *The composition of $O_{\min}$ is not frozen preexperimentally. What is frozen is the complete decision boundary: three operations are structurally necessary, and only three specified operations remain contingent on the pre-registered empirical stages 3–4.*

Consequence for what “sealing v0.6” means. v0.6 does not require executing the mutation harness to be sealed, provided v0.6 remains the closure of the preexperimental research design/evidence base, not of the experiment itself. The methodologically correct closing state is: O_{fixed} closed, O_{pending} closed, the decision rule closed, $O_{\min}$ explicitly pending empirical execution. This is not a debt carried by the design – it is exactly what an honest pre-registration is supposed to look like. Resolving $O_{\min}$ belongs to the implementation/mutation/data/analysis phase that follows, not to a retroactive redefinition of the experiment already specified – mirroring the separation this design’s own history already drew between closing Γ_0 /selection and the subsequent execution phase.

24 Decision log

ID	Decision	Reason
D-001	Iberbox is motivation, not empirical subject.	Avoids unsupported claims about proprietary code and preserves paper independence.
D-002	Empirical validation uses a reference SDK/framework.	Makes the work reproducible and owned by the research.
D-003	R_round removed; cross-round-trip absorbed into bidirectional R_interop.	Keeps all R relations genuinely cross-provider.
D-004	R_interop is directional $p \rightarrow q$ and $q \rightarrow p$.	Allows asymmetric interoperability to be observed.
D-005	Measure specificity/FPR in addition to mutation detection/recall.	Prevents an always-fail oracle from appearing strong.
D-006	Capability manifests are pre-registered/frozen from documentation before execution.	Prevents circular capability claims.
D-007	Gamma_0 is pre-registered before mutation execution.	Prevents post-hoc causal explanations.
D-008	Diagnostic inference uses the full relation-result spectrum.	Preserves joint diagnostic information; SBFL-inspired.
D-009	Dep separates experimental dependency from relation applicability.	Removes ambiguous ‘indirect R_ser’ cells.
D-010	Dependency-caused failures are attributed to the owning clause/operation.	Prevents inflated value for dependent operations.

ID	Decision	Reason
D-011	Shared hierarchical K1/K2 taxonomy used for necessity.	Fixes disjoint-local-namespace Coverage/Marginal failure.
D-012	Membership remains a K1 kind separate from Validation.	Preserves mathematical structural validity as an independent concept.
D-013	Parameter.ExternalRandomness Control excludes provider-internal RNG policy.	Maintains the declared scope and ecological-validity limitation.
D-014	AES-GCM has its own R_ser only because the SDK contract defines a portable AEADArtifact.	Separates SDK serialization from provider API formatting.
D-015	EC import/export/validation remains a candidate without full ECDSA.	Preserves point encoding/membership/capability cases at lower experimental cost.
D-016	A contract clause is a stable semantic obligation, not a mutation/test.	Prevents Gamma_0 degenerating into mutation_i→clause_i identity.
D-017	HKDF is the pilot operation.	Cheaply validates the methodology before richer operations.
D-018	HKDF absent salt follows RFC 5869; empty salt is distinct.	Normative semantics replace an arbitrary framework convention.
D-019	HKDF length perturbation is split into within-domain and boundary-crossing classes.	Separates output correctness from validation/error/capability regimes.
D-020	HKDF L upper bound follows RFC 5869; lower-bound L=0 semantics remain an explicit profile decision.	Distinguishes normative rule from genuine design freedom.
D-021	Common contract parameters defined as $P_o^{\text{common}} = \bigcap_p P_{p,o}$ (intersection, not union).	Backend-specific extra control becomes declared capability, never an equivalence failure.
D-022	AES-GCM common profile fixes $ IV = 96$, $t = 128$; IV/tag coupling and short-tag deprecation cited from SP 800-38D and its active revision.	Aligns profile choice with normative trend, not arbitrary preference; yields a free normatively-grounded mutation class.
D-023	RQ2–RQ4 and H1–H3 with five explicit refutation criteria are pre-registered as part of the research design.	Keeps the project genuinely falsifiable before implementation begins.

ID	Decision	Reason
D-024	AES-GCM C_{pre}^{GCM} frozen: AES-256-GCM, $ IV = 96$ bits, $ T = 128$ bits, concrete AEADArtifact = version $\ IV_{12}\ C\ T_{16}$, four-class SDK error model with pre-registered adapter classification order, $P_{GCM}^{portable} \subseteq P_{GCM}^{common} \subseteq P_{p,GCM}$ formalized.	All three backends verified at source/primary-specification level; no further normative or backend variable justified delaying the freeze; enables Γ_0^{GCM} construction.
D-025	gcm.cap split into gcm.cap.provider (observed matches A_p^{pre}) and gcm.cap.portable (no provider extension leaks into $P^{portable}$); explicit disambiguation rule frozen: an out-of-portable-profile but syntactically valid parameter (e.g. $t = 80$) is <i>invalid_parameter</i> , not <i>unsupported</i> .	Provider-only capability legitimately supported and correctly declared is not an R_{cap} failure; without the disambiguation rule, Γ_0^{GCM} 's boundary-bypass mutations would be ambiguous between two error classes.
D-026	Γ_0^{GCM} pre-registered as 12 causal mutation classes organized into four families: computation/semantic, authentication/adversarial, representation/validation, and capability/error. Mutation classes are formally separated from their concrete stimuli. Redundant or detection-driven classes were removed or refined, and independent causal coverage of gcm.ciphertext was restored.	Corrects cases where Γ_0 had been assigned partly to make relations experimentally distinguishable rather than purely from causal analysis, enforcing the pre-execution Γ_0 discipline of D-007 and eliminating redundant causal classes.
D-027	$E_{OAEP}^{SDK} =$ { <i>unsupported</i> , <i>invalid_key</i> , <i>invalid_parameter</i> , <i>invalid_key</i> , <i>invalid_parameter</i> } frozen, with a four-stage pre-registered adapter classification order (manifest $\rightarrow$ key contract $\rightarrow$ portable parameter $\rightarrow$ RSAES-OAEP-DECRYPT domain, the last collapsing unconditionally per RFC 8017 §7.1.2's own pseudocode). <i>invalid_key</i> added after auditing key-role enforcement across all three backends (WebCrypto: runtime; Crypto++: compile-time; Bouncy Castle low-level: unenforced, adapter-responsible).	The obligation is retained because it is conceptually stable and pre-OAEP-checkable, not because providers share a common error surface – they explicitly do not, which is the argument <i>for</i> an abstract SDK-level class rather than against one. Demonstrates ObservedConformance = Provider + Adapter concretely.

ID	Decision	Reason
D-028	P_{OAEP}^{common} and $P_{OAEP}^{portable}$ require $H_{OAEP} = H_{MGF1}$. Bouncy Castle’s independent-digest capability remains valid but lies outside both; a portable request for decoupled digests classifies as <i>invalid_parameter</i> , not <i>unsupported</i> .	Direct consequence of the capability-intersection rule (D-021): WebCrypto and Crypto++ structurally cannot realize decoupled digests, so no choice of profile digest can bring independence into P_{OAEP}^{common} . Closable now, independent of which digest is later selected for the portable profile.
D-029	The general rule $P_o^{portable} \subseteq P_o^{common}$ (§11) is extended with two named categories of legitimate narrowing – profile-selection (options fully covered by the governing spec; narrowed for reproducibility) and normative-scope (technically common capability excluded because the governing spec disclaims scope over it) – plus a safeguard requiring an exact scope citation for the latter.	Prevents ad hoc invocation of “normative reasons” for narrowing without traceable justification; makes explicit that two structurally different kinds of $P^{portable} \subsetneq P^{common}$ exist before OAEP’s label decision relies on the distinction.
D-030	$P_{OAEP}^{portable}.label = \{\emptyset\}$, citing RFC 8017’s explicit disclaimer that non-empty label use is outside the scope of the document (Appendix A.2.1’s <code>pSourceAlgorithm</code> syntax notwithstanding). Non-empty labels remain valid P_{OAEP}^{common} capability but are excluded portably; requests for them classify <i>invalid_parameter</i> .	First applied instance of the normative-scope category (D-029); satisfies the safeguard via verbatim RFC citation rather than inference from provider silence.
D-031	$P_{OAEP}^{common}.hash = \{SHA-1, SHA-256, SHA-384, SHA-512\}$, confirmed across all three backends. $P_{OAEP}^{portable}.hash = \{SHA-256\}$, classified as profile-selection (D-029), not normative-scope. SHA-1/384/512 remain valid common capability, excluded portably; requests classify <i>invalid_parameter</i> .	Second applied instance of the D-029 narrowing categories, this time profile-selection: all four candidates are equally covered by RFC 8017; SHA-256 chosen for reproducibility and to avoid legacy SHA-1 without needing SHA-384/512 to exercise any additional framework dimension.

ID	Decision	Reason
D-032	$P_{OAEP}^{portable}.\text{modulus} = \{3072 \text{ bits}\}$ (profile-selection, D-029), aligning RSA’s ~ 128 -bit security strength with SHA-256’s, per NIST SP 800-57 and the forward-looking SP 800-131Ar3 128-bit transition. RSA-2048/4096 remain valid common capability, excluded portably. Derived consequence: $mLen_{\max} = 384 - 64 - 2 = 318$ bytes.	Third profile-selection instance under D-029; mirrors AES-GCM’s forward-alignment argument (D-022) rather than choosing the bare regulatory floor. $mLen_{\max}$ is derived, not chosen, from D-031+D-032, preserving the same discipline used for AEADArtifact’s derivation from frozen GCM parameters.
D-033	The portable RSA-OAEP profile exposes no seed/RNG parameter; external randomness control (available in Crypto++ and Bouncy Castle, absent in WebCrypto) lies outside P_{OAEP}^{common} and $P_{OAEP}^{portable}$ by the intersection rule. OAEP encryption remains probabilistic portably, so R_{byte} does not apply to the ciphertext; R_{interop} is the operative relation. Deterministic RNG use is permitted only as backend-internal instrumentation, never as cross-provider portable evidence.	Direct application of D-021’s intersection rule to a capability held by two of three backends – confirms the portable contract cannot admit majority-but-not-unanimous capability. Reaffirms the ecological-validity threat’s scope boundary (CSPRNG quality/policy remains out of scope for every backend).
D-034	RSA-OAEP’s portable ciphertext is the raw, fixed-length RSAES-OAEP output ($ C = 384$ bytes under RSA-3072), with no SDK-level wrapper. No <code>oaep.artifact</code> clause, no <code>Encoding-kind</code> coverage from OAEP itself; R_{ser} is not applicable to this operation. Ciphertext length mismatch at decrypt reuses the existing E_{OAEP}^{SDK} stage-4 collapse (D-027), not a new error class. C_{pre}^{OAEP} is now fully frozen (D-027, D-028, D-030 through D-034).	RFC 8017’s I2OSP fixes ciphertext encoding unambiguously, unlike GCM’s genuine $C T$ representational choice (D-014); no representational degree of freedom exists for R_{ser} to capture. Avoids fabricating an Encoding obligation purely to have “an artifact,” which would have inflated O_{cand} ’s coverage artificially.

ID	Decision	Reason
D-035	Twelve <code>oaep.*</code> clauses frozen (§13). <code>oaep.key/oaep.modulus</code> and <code>oaep.hash/oaep.mgfCoupling</code> kept as separate clause pairs rather than merged; <code>oaep.label</code> classified Input (not Parameter) for structural comparability with <code>gcm.aad</code> ; <code>oaep.validation</code> restricted to accept/reject of pre-OAEP requests, with decrypt-failure collapse left exclusively to <code>oaep.error</code> . No <code>oaep.messageLength</code> clause; the 318-octet bound stays inside <code>oaep.message</code> , documented as a deliberate distinction from HKDF’s independently-selected output length. Coverage _{K₁} (OAEP) = {Key, Parameter, Input, Output, Validation, Error, Capability}.	Each merge/split decision traced to a genuine causal-independence argument, not convenience: unmerged pairs preserve future diagnostic precision (mirrors the GCM AAD-divergence/AAD-ignored split); <code>oaep.label</code> ’s reclassification keeps cross-operation Coverage _K comparisons meaningful; Authentication/Encoding/Membership absence is contractual inapplicability, not omission.
D-036	Retroactive correction to already-frozen Γ_0^{GCM} : <code>gcm.error</code> removed from the Γ_0 of GCM-KEY-PROFILE-BOUNDARY-BYPASS , GCM-IV-PROFILE-BOUNDARY-BYPASS , and GCM-TAGLENGTH-PROFILE-BOUNDARY-BYPASS (each reduced from 4 to 3 clauses; R_{err} dropped from their expected-relations column). No new mutation classes introduced; GCM-ERROR-MISCLASSIFICATION (unchanged, $\Gamma_0 = \{gcm.error\}$) already exclusively and transversally covers the “correctly rejected but misclassified” defect these three classes had redundantly duplicated. GCM’s 12 causal classes, backends, profile, and C_{pre}^{GCM} are otherwise untouched.	Found by cross-auditing OAEP’s OAEP-LABEL-PROFILE-BYPASS against the already-frozen GCM classes: the three GCM classes conflated two causally defects (silent acceptance of an out-of-profile value vs. correct rejection with the wrong error class) under one Γ_0 , contradicting the very discipline (D-026) that had already separated these concerns via GCM-ERROR-MISCLASSIFICATION . Demonstrates the pre-registration mechanism catching a genuine regression before any experimental execution, not weakening the design but validating the audit process.

ID	Decision	Reason
D-037	Retroactive correction to already-frozen <code>oaep.*</code> clause table (D-035): <code>oaep.ciphertext</code> narrowed to cryptographic correctness only (“produces a valid RSAES-OAEP ciphertext for the declared plaintext, key, and frozen OAEP parameters”); a new clause <code>oaep.ciphertextLength</code> (Output, no K2) added for the I2OSP-encoded length obligation (“exactly $k = 384$ octets”). Twelve clauses become thirteen. $\text{Coverage}_{K_1}(\text{OAEP})$ unchanged (both clauses are kind <code>Output</code>).	Found while auditing <code>OAEP-CIPHERTEXT-COMPUTATION-DIVERGENCE</code> the original single clause conflated two independently-failable obligations – a mathematically correct RSA integer can be I2OSP-encoded with missing zero-padding (wrong length, correct value), and a structurally correct 384-octet string can carry an incorrect ciphertext (correct length, wrong value). Splitting keeps $\Gamma_0(\text{OAEP-CIPHERTEXT-COMPUTATION})$ <code>{oaep.ciphertext}</code> from silently depending on an ambiguous reading of the original clause. <code>oaep.ciphertextLength</code> is <code>Output</code>, not <code>Parameter.Length</code>, for the same reason <code>oaep.messageLength</code> was never created: it is not a caller-selected parameter.

ID	Decision	Reason
D-038	Second retroactive correction to already-frozen Γ_0^{GCM}: <code>gcm.error</code> removed from <code>GCM-PROVIDER-CAPABILITY-MISMATCH</code> Γ_0 (reduced from <code>{gcm.cap.provider, gcm.error}</code> to <code>{gcm.cap.provider}</code>); “possibly R_{err} ” removed from its expected-relations column, leaving only R_{cap} . GCM’s other 11 causal classes untouched.	Found by cross-auditing OAEP’s <code>OAEP-PROVIDER-CAPABILITY-MISREPORT</code>: <code>GCM-PROVIDER-CAPABILITY-MISMATCH</code> is manifest/observed-behavior mismatch is detectable by comparing declared vs. observed capability alone, without executing any operation – $R_{cap} = 0 \wedge R_{val} = R_{err} = R_{interop} = 1$ is a coherent, demonstrable state. The original “possibly R_{err}” hedge was exactly the soft, unproven inclusion D-036 had already taught this design to distrust; no causal path from capability misreport to error misclassification survives scrutiny. GCM and OAEP’s provider-capability-mismatch classes are now structurally identical ($\Gamma_0 = \{\text{cap.provider}\}$ in both), evidence that OAEP is stress-testing the framework as an independent second operation, not merely copying GCM’s machinery.

ID	Decision	Reason
D-039	Γ_0^{OAEP} frozen: 12 causal classes across 5 families (A: computation/semantic; B: profile/capability leakage bypass; C: intrinsic/derived or semantic-role boundary bypass; D: interface capability leak; E: error/capability). Class-count reconciliation: 12 original candidates -1 (OAEP-PORTABLE-CAPABILITY-LEAK, eliminated as causally redundant with five more specific classes) $+1$ (OAEP-CIPHERTEXT-LENGTH-DIVERGENCE, added to cover <code>oaep.ciphertextLength</code>, the only one of thirteen clauses with no prior Γ_0 coverage) = 12. All thirteen <code>oaep.*</code> clauses now have Γ_0 coverage. Two methodological notes recorded: (1) a three-way taxonomy determining when <code>cap.portable</code> belongs in a boundary-bypass class (profile/capability leakage vs. intrinsic/derived boundary vs. semantic-role boundary), orthogonal to why a value is portably excluded (D-029's profile-selection/normative-scope vs. D-028's capability-forced); (2) stimulus executability can be backend-specific without altering class identity or Γ_0.	Final pre-freeze audit checked minimality, inter-class non-redundancy, mutation/stimulus separation, and full clause coverage – the same four-point discipline that had already caught two GCM regressions (D-036, D-038) during this same cross-auditing process. <code>oaep.ciphertextLength</code>'s missing coverage was found precisely because it was introduced by D-037 after the original 12 candidates were drafted, and coverage was checked explicitly rather than assumed.
D-040	RSA-PSS digest coupling: $H_{PSS} = H_{MGF1}$ in $P_{PSS}^{portable}$, intersection-forced by WebCrypto and Crypto++ against Bouncy Castle's decoupling capability.	Structural parallel to D-028 (OAEP), arising independently from a second, differently-implemented operation (<code>PSSSigner</code> vs. <code>OAPEncoding</code>) – not a copy-forward, a second confirming instance.

ID	Decision	Reason
D-041	RSA-PSS salt-length intersection: $sLen = hLen$ in $P_{PSS}^{portable}$, intersection-forced by Crypto++’s audited standard surface (PSSR_MEM’s compile-time SALT_LEN).	Not a security preference; a consequence of one backend’s concrete API surface. Kept as a separate decision from D-042 despite both being intersection-forced, since <i>saltLength</i> geometry (<code>Parameter.Length</code>) and randomness-control capability (<code>Parameter.ExternalRandomnessControl</code>) are different contractual dimensions expected to generate different Γ_0^{PSS} mutation classes.
D-042	RSA-PSS portable randomness-control surface: neither external RNG injection nor explicit salt bytes are exposed by $P_{PSS}^{portable}$, excluded by the intersection rule since WebCrypto offers neither. Both remain distinct, valid provider-only capabilities (recorded, not discarded).	True parallel to OAEP’s D-033. Deliberately not merged with D-041 for the reason given there – premature merging of causally distinct capability dimensions was exactly the error corrected in D-036/D-038 for GCM.
D-043	RSA-PSS portable digest selection: $H_{PSS} = H_{MGF1} = SHA-256$, via profile-selection (D-029), not intersection-forcing. SHA-1/384/512 remain valid common capability, excluded portably.	Interoperability, elimination of legacy SHA-1, parsimony, alignment with the rest of the frozen profile – independently justified for PSS, not inherited from OAEP’s D-031 merely because the value coincides.
D-044	RSA-PSS portable modulus selection: $modBits = 3072$, via profile-selection, aligned with RSA-OAEP’s D-032 (NIST SP 800-57/SP 800-131Ar3 128-bit alignment). Derived consequence (not a third decision): $sLen = 32$ bytes follows from D-041+D-043. Verification: $emBits = 3071$, $emLen = 384 \geq hLen + sLen + 2 = 66$, comfortably satisfied. $P_{PSS}^{portable}$ is now fully frozen.	Completes $P_{PSS}^{portable}$ across all six components (D-040 through D-044). Key role, message semantics, and the error/true/false model deliberately excluded from $P_{PSS}^{portable}$ – they belong to C_{pre}^{PSS} ’s validation/error layers, not the cryptographic parameter profile.

ID	Decision	Reason
D-045	$C_{pre}^{PSS} =$ $C_{manifest} \wedge C_{key} \wedge C_{profile} \wedge C_{request}$ provisionally frozen (§15.7), with $C_{profile} =$ $C_{profile-values} \wedge C_{realizable}$. Encoding-realizability guard confirmed with exact exception names on both remaining backends: Crypto++ throws PK_SignatureScheme::KeyTooShort (shared by TF_SignerBase/TF_VerifierBase); Bouncy Castle throws IllegalArgumentException("key too small..." in PSSSigner.init(). $ \sigma = 384$ initially left unclassified pending further evidence. D-045 remains in force in full ($C_{manifest}, C_{key},$ $C_{profile}/C_{realizable}$, the four-stage precedence) except for the treatment of signature length, which is amended by D-046: subsequent normative review found RFC 8017 already classifies $ \sigma \neq k$ as part of Verify's result domain, not as a C_{pre}^{PSS} precondition.	Preserves the causal-layer separation established while auditing this question: verifier/signer <i>configuration</i> realizability (confirmed, in $C_{realizable}$) is a different condition from a specific <i>signature's</i> length. The initial deferral to further search, (Other than to normative text, was itself the gap D-046 corrects – a narrow amendment to one component, not a replacement of the whole decision.

ID	Decision	Reason
D-046	Correction to an initial misclassification. $\sigma \neq k$ is <i>not</i> part of $C_{pre}^{PSS}/C_{request}$, contrary to an earlier draft of this decision. RFC 8017 §8.1.2 (RSASSA-PSS-VERIFY), Step 1, defines signature-length mismatch as producing “invalid signature” – part of Verify’s own two-valued output domain $\{\text{valid signature, invalid signature}\}$, textually alongside RSAVP1’s “signature representative out of range” and EMSA-PSS inconsistency. The adapter normalizes $\sigma \neq k$ to false; this is a direct consequence of the RFC’s own text, not an adapter design preference. Crypto++’s exact native InputSignature() behavior remains empirically unconfirmed, now recorded purely as an adapter-induced-divergence-threat data point (§7.2), no longer as a contractual unknown.	An initial draft treated this the same way as Bouncy Castle’s unenforced key role (D-027) – “<i>ObservedConformance = Provider + Adapter, so the adapter decides</i>” – but that principle applies to obligations the SDK contract itself imposes, not to outcomes a normative source already classifies. Misapplying it here would have repeated, inside PSS, the exact causal-layer conflation already corrected twice for GCM (D-036, D-038). Caught by direct verification against RFC 8017 primary text rather than accepted on assertion.
D-047	$E_{PSS}^{SDK} = \{\text{unsupported, invalid_key, invalid_frozen} - \text{three classes, not OAEP’s four. Classification order inherited from } C_{pre}^{PSS}\text{’s four-stage precedence (D-045), with } C_{request} \text{ now correctly restricted to message structural validity only (D-046). No decryption_error-equivalent class. false comprises, once } C_{pre}^{PSS} \text{ is satisfied: } \sigma \neq k; \text{ RSAVP1 out-of-range representative; EMSA-PSS-inconsistent encoding; and a cryptographically-correct-but-non-matching signature} - \text{all folded by RFC 8017 §8.1.2 into Verify’s own result domain.}$	Primary justification, corrected: RFC 8017 §8.1.2 already defines RSASSA-PSS-VERIFY’s entire output as $\{\text{valid signature, invalid signature}\}$ – this normative text, not an argument from the absence of a Bleichenbacher/Manger-style rationale, is the reason no fourth class exists. The OAEP security-rationale contrast (§15) remains a valid secondary explanation for <i>why</i> the two operations differ in kind, but was incorrectly treated as the primary justification in an earlier draft.

ID	Decision	Reason
D-048	Fifteen <code>pss.*</code> clauses frozen (§16), with two structural decisions: (1) <code>pss.rngControl</code> and <code>pss.saltBytes</code> kept as separate local clauses despite sharing $K_2 = \text{ExternalRandomnessControl}$ – genuinely distinct capabilities, local granularity preserved independent of shared-taxonomy projection; (2) <code>pss.verification</code> materializes <code>Authentication.SignatureVerification</code> for the first time in any operation. Relation columns omitted, matching the historical format – a draft proposing fixed clause-to-relation assignments was rejected. Coverage_{K_1}(RSA-PSS) = {Key, Parameter, Input, Output, Authentication, Validation, Error, Capability} 8 of 10 kinds.	The clause-to-relation rejection prevents a recurrence of the D-036/D-038 error class: R_{cap} is not an intrinsic property of a parameter clause, it activates only on declared-vs-observed divergence for a specific causal class, determined downstream via $\Gamma_0(c)$. Four causally distinct layers now clearly separated: cryptographic output correctness (<code>pss.signature</code>) $\neq$ structural output obligation (<code>pss.signatureLength</code>) $\neq$ verification semantics (<code>pss.verification</code>) $\neq$ authentication validation (<code>pss.validation</code>) $\neq$ error normalization (<code>pss.error</code>).

ID	Decision	Reason
D-049	Γ_0^{PSS} frozen: 15 causal classes across 6 families (A: computation/semantic; B: profile/capability leakage bypass; C: intrinsic/semantic-role boundary bypass; D: interface capability leak; E: verification semantics, new to PSS; F: error/capability), derived from the pss.* clauses directly, not copied from GCM/OAEP. Three methodological decisions: (1) message-normalization split into ($\Gamma_0 = \{\text{pss.message}, \text{pss.signature}\}$) and PSS-VERIFY-MESSAGE-NORMALIZATION ($\Gamma_0 = \{\text{pss.message}, \text{pss.verification}\}$) – causally distinct once pss.signature/pss.verification exist as separate clauses; (2) PSS-VERIFICATION-FALSE-ACCEPT/-FALSE-REJECT kept separate despite sharing $\Gamma_0 = \{\text{pss.verification}\}$ – opposite semantic defects, with asymmetric expected relations (R_{val} only vs. $R_{\text{interop}} + R_{\text{val}}$); (3) a diagnostic-separability note recorded for two class pairs with potentially overlapping observable symptoms but distinct Γ_0, explicitly distinguishing limited diagnostic power of the current relation family from causal equivalence. All fifteen clauses covered; no fifth profile-realizability class needed (redundant with the three profile-bypass classes).	Grounded in the same four-point audit discipline used for GCM/OAEP (minimality, non-redundancy, mutation$\neq$stimulus, full clause coverage), applied to genuinely new PSS structure (two randomness capabilities, verification semantics) rather than mechanically inherited class names. The diagnostic-separability note operationalizes $\Delta_j^{\text{diag}}(c)$ (§6, PSS-SIGN-MESSAGE-NORMALIZATIOND-052) correctly: shared $r(c)$ across two classes reflects on the relation family’s discriminating power, not on whether the classes represent the same cause.

ID	Decision	Reason
D-050	Cross-operation causal consistency audit correction. (i) <code>gcm.error</code> removed from GCM-AUTHENTICATION-BYPASS (Γ_0 reduced to <code>{gcm.authentication, gcm.validation}</code> and GCM-ARTIFACT-STRUCTURE-CORRUPTION (Γ_0 reduced to <code>{gcm.artifact, gcm.validation}</code>)); error classification remains exclusively attributable to GCM-ERROR-MISCLASSIFICATION. Expected R_{err} removed from both; the hedged “potentially R_{interop}” in GCM-AUTHENTICATION-BYPASS replaced with a precise conditional (R_{interop} applicable specifically when the stimulus is instantiated through cross-provider authenticated decryption). <code>gcm.validation</code> left untouched in both rows – its boundary was not challenged by this finding and any question about it would require a separate, symmetric audit against Validation.Semantic’s definition. (ii) OAEP-MESSAGE-NORMALIZATION amended from $\Gamma_0 = \{\text{oaep.message}\}$ to $\Gamma_0 = \{\text{oaep.message}, \text{oaep.ciphertext}\}$, since D-037 gave <code>oaep.ciphertext</code> an explicit cryptographic-correctness obligation relative to the declared plaintext (“for the declared plaintext”) that did not exist when this class was originally built against the pre-split clause text. R_{interop} retained as the sole expected relation; no R_{val} (message remains a valid input; the defect is misinterpretation, not out-of-domain acceptance).	Found via the systematic cross-audit of PSS against HKDF/GCM/OAEP across five axes (causal-layer consistency, input/output consistency, profile-bypass consistency, relation consistency, granularity consistency). Both corrections propagate methodological rules already established elsewhere (D-036’s error/validation separation; D-037’s ciphertext correctness/length split) to rows that predated or escaped those refinements – they do not change any portable profile or introduce new causal classes. Confirms D-036 was directionally correct but incomplete in its original application scope, and that a clause-text amendment (D-037) can silently invalidate a previously-correct Γ_0 assignment elsewhere if not propagated.

ID	Decision	Reason
D-051	Relation-spectrum diagnostic equivalence (§18). $r(c)$ is formally not injective over causal mutation classes; $c_1 \sim_r c_2 \iff r(c_1) = r(c_2)$ defines diagnostic-equivalence classes $[c]_r$. PSS’s full 15-class audit yields a $4 + 4 + 2 + 3 + 1 + 1$ partition (only 2/15 classes individually identifiable by $r(\cdot)$ alone). A directed four-hypothesis cross-operation audit confirms the phenomenon is structural, not PSS-specific: profile-bypass collapse under $\{R_{\text{val}}, R_{\text{cap}}\}$ confirmed in GCM (3-way, post-D-036), OAEP (4-way), and PSS (4-way); R_{val}-only collapse confirmed within OAEP itself (OAEP-MESSAGE-BOUNDARY-BYPASS $\sim_r$ OAEP-KEY-ROLE-BYPASS) and again in PSS; computation/input-semantics collapse confirmed under R_{byte} in GCM (3-way) and R_{interop} in OAEP (3-way) and PSS (4-way), tracking R_{byte}’s applicability (D-033). Only the R_{cap}-only 3-way cluster is genuinely PSS-specific, arising from PSS’s unique dual randomness-control-surface structure (D-048) – GCM/OAEP each have only one pure-R_{cap} class, with nothing to collide against (OAEP-PORTABLE-CAPABILITY-LEAK was already eliminated at design time, D-039). Frozen result: $r(c_1) = r(c_2) \Rightarrow \Gamma_0(c_1) = \Gamma_0(c_2)$ – equal relation vectors reflect the current relation family’s limited diagnostic power, not causal equivalence; no class is merged or removed. Explicitly not decided: whether a seventh relation, coarser diagnostic partitioning, or a richer pre-registered diagnostic mechanism is warranted – deferred to the next phase.	PSS’s exhaustive 15-class applicability/detection audit surfaced the first fully-worked example; four hypotheses generated directly from that partition were then tested against already-frozen GCM/OAEP content rather than assumed, confirming three as transversal framework properties (present since GCM’s original design, simply never previously surfaced) and correctly refuting the fourth as PSS-specific with a precise structural explanation. RSA-PSS is the operation where non-injectivity was first identified through exhaustive partitioning, not where it originates – the phenomenon predates PSS by construction of the profile-bypass and computation-divergence patterns in GCM.

ID	Decision	Reason
D-052	Diagnostic gain redefined over causal-class hypothesis reduction (§6). $\Gamma_0(c)$ remains the pre-registered contractual ground truth for causal attribution and coverage (unchanged) – it is no longer used as the diagnostic hypothesis space, since during the mutation campaign c is known by construction and $\Gamma_0(c)$'s elements are jointly-violated consequences of that one known cause, not competing hypotheses r could adjudicate between. Diagnostic inference instead operates over the pre-registered causal-class set M_o: $H(c \mid r) = \{c' \in M_o : r(c') = r(c)\}$, which is exactly $[c]_r$ (D-051). Marginal diagnostic contribution: $\Delta_j^{\text{diag}}(c) = H(c \mid r_{-j}) - H(c \mid r)$; normalized $DG_j(c) = 1 - H(c \mid r) / H(c \mid r_{-j}) \in [0, 1]$. $H(c \mid r) = 1$ denotes diagnostic identifiability; larger sets denote residual diagnostic equivalence. Superseded formulas $(\Gamma(c \mid r) \subseteq \Gamma_0(c);$ $DG_j(c) = \Delta_j(c) / \Gamma_0(c))$ are retired; residual references elsewhere in the document updated to the new notation.	D-051's concrete construction of $[c]_r$ across all fifteen PSS classes exposed that the original formula answered a question with no coherent operational procedure (narrowing among jointly-true clauses of an already-known cause), while the question D-051 actually needed to ask – and answered – was hypothesis reduction over candidate causal classes. Discovered and corrected before any experimental implementation or necessity computation depended on the old definition, consistent with the project's standing preference for pre-execution correction over post-hoc rescue (D-007, D-036, D-038, D-050).

ID	Decision	Reason
D-053	RSA key serialization: three-backend intersection and portable profile frozen (§19.4). $P_{\text{RSA-ser}}^{\text{common}}$ modeled directionally, not as a flat Cartesian product: tagged-union material domain $M_{\text{RSA}}^{\text{common}} = \text{Public}(n, e) \uplus \text{PrivateCRT2}(n, e, d, p, q, d_P, d_Q, qInv)$ structurally coupled to SPKI/PrivateKeyInfo respectively; common export $= \text{DER}(\text{rsaEncryption}, \text{NULL})$ for both key roles; common import fully characterized on OID $(= \text{rsaEncryption})$, parameters $(\in \{\text{absent}, \text{NULL}\})$, and key shape (two-prime only, multi-prime excluded since Crypto++ cannot represent it and BC discards it on materialization); encoding known only as a lower bound, $\{\text{DER}\} \subseteq D_{\text{encoding}}^{\text{common, import}}$, non-DER BER intersection not characterized. $P_{\text{RSA-ser}}^{\text{portable}} \subseteq P_{\text{RSA-ser}}^{\text{common}}$ freezes: DER-SPKI public / DER-PrivateKeyInfo private, rsaEncryption, parameters $\in \{\text{absent}, \text{NULL}\}$ (equality with the common domain, since it is fully characterized there), DER-only portable import (a deliberate restriction, since the common domain is only a demonstrated lower bound there), two-prime RSA, and canonical DER export with explicit NULL. Resulting SDK policy: bounded common-ground reader and canonical writer – not a liberal-reader policy (does not accept whatever any single backend accepts) and not the narrowest-backend policy either.	Separates empirically common provider capability from the deliberately offered SDK contract, per D-021. Preserves every fully characterized common admission choice while avoiding unsupported assumptions about the intersection of non-DER BER acceptance – absence of characterization is recorded as unestablished, not as a negative property of P^{common}. Provider-specific wider OID (<code>id-RSASSA-PSS</code>, <code>id-ea-rsa</code>, <code>id-RSAES-OAEP</code> on BC), parameter admission, BER dialect, or multi-prime acceptance cannot silently widen the portable SDK surface. Error model, trailing-byte admission, and admission-time-vs-use-time validation deliberately excluded from this decision – not yet frozen, would contaminate this profile-selection decision with unresolved dimensions.

ID	Decision	Reason
D-054	RSA admission-time validation semantics frozen (§19.5). Provider validation timing is empirical metadata (WebCrypto: none; Crypto++: manual/separate <code>Validate()</code>; BC: automatic/partial modulus screen, no CRT cross-check); SDK admission outcome is a stable contractual obligation: $\text{Import}_{\text{SDK},p}(S) \equiv \text{Import}_{\text{SDK},q}(S)$, not identical internal validation timing across backends. $V_{\text{SDK}}^{\text{admission}} = V_{\text{structural}} \cup V_{\text{profile}} \cup V_{\text{mathematical}}$, explicitly excluding provider-specific security hardening ($V^{\text{provider-security-policy}}$, e.g. BC's small-prime-factor screen), which remains documented capability/provider metadata, not a $C_{\text{pre}}^{\text{RSA-ser}}$ obligation. $V_{\text{mathematical}}^{\text{RSA},2}$ enumerated from RFC 8017 §3.1–3.2 (two-prime case) as domain constraints (p, q distinct odd primes; $e, d, d_P, d_Q, q_{\text{Inv}}$ range conditions) union relational constraints ($n = pq$; $ed \equiv 1 \pmod{\lambda(n)}$; $ed_P \equiv 1 \pmod{p-1}$; $ed_Q \equiv 1 \pmod{q-1}$; $qq_{\text{Inv}} \equiv 1 \pmod{p}$), using the RFC's own e-based definition of d_P, d_Q rather than the common $d \bmod (p-1)$ shorthand. Cross-representation consistency ($d \equiv d_P \pmod{p-1}$, $d \equiv d_Q \pmod{q-1}$) proved as a necessary consequence of the relational constraints already stated (via $(p-1) \mid \lambda(n)$ and cancellation of e, licensed by $\gcd(e, p-1) = 1$ following from $ed_P \equiv 1 \pmod{p-1}$) – not adopted as an independent sixth obligation.	D-010 requires dependent failures to be attributed to their owning clause/operation; leaving RSA mathematical consistency to use-time would let a key pass import across all three backends and fail later, differently, inside whichever consuming operation (OAEP, PSS) used it first – misattributing a Key-Serialization-owned defect and letting a consuming operation gain spurious diagnostic credit. Per D-021, BC's small-prime screen is retained as provider evidence but not promoted to C_{pre}, since doing so would force WebCrypto/Crypto++ adapters to fabricate a check neither backend offers, on the strength of one provider's unilateral policy. Treating the derived cross-representation consistency as an independent causal class would artificially inflate $\Gamma_0^{\text{RSA-ser}}$ coverage/diagnostic accounting through logically-overlapping classes – verified by direct derivation rather than assumed, consistent with the project's standing discipline of proving rather than asserting redundancy (D-039, D-050).

ID	Decision	Reason
D-055	$C_{pre}^{RSA-ser}$ frozen (§19.6), layered as $C_{surface} \wedge C_{struct} \wedge C_{profile} \wedge C_{math}^{role}$, preserving the Encoding/Validation/Error/Capability taxonomy rather than a flat precondition list; C_{error} deliberately excluded (rejection content vs. SDK error-class normalization are separate steps, mirroring the R_{val}/R_{err} split). $C_{surface}$ inherits D-053's role/container coupling (public$\leftrightarrow$SPKI, private$\leftrightarrow$PrivateKeyInfo) as non-independent. C_{struct} gains ExactConsumption: $S = \text{DER}(K) \ X$ with $X > 0$ is rejected, a deliberate contract choice (mirroring WebCrypto's exactData=true) overriding Crypto++'s Load() and BC's InputStream native tolerance of trailing bytes; C_{struct} is sole owner of DER-vs-BER validity and admitted container <i>shape</i> (SPKI or PrivateKeyInfo, without yet comparing against the requested role). $C_{profile}$ restates D-053's portable-profile membership (OID, parameters, <i>shape</i>, and RoleContainerMatch against the requested role), causally distinct from C_{struct}: a well-formed but non-rsaEncryption OID, non-NULL parameters, or a syntactically valid PrivateKeyInfo requested as role=public all pass C_{struct} but fail $C_{profile}$. Each obligation is checked in exactly one of $C_{struct}/C_{profile}$, never both – an initial draft of this decision briefly duplicated DER-vs-encoding and role/container matching across both layers, caught and corrected before freezing (§19.6). C_{math}^{role} is asymmetric by construction: private reuses D-054's $V_{domain}^{RSA,2} \wedge V_{rel}^{RSA,2}$ in full (CRT material exposes p, q directly); public is restricted to $(n \geq 15) \wedge (n \equiv 1 \pmod{2}) \wedge (3 \leq e \leq n-1) \wedge (e \equiv 1 \pmod{2})$ – necessary conditions directly	Layering prevents collapsing representation, portable-profile membership, and mathematical validity into one undifferentiated precondition, and keeps C_{pre} decisions traceable to the specific D-05x finding that grounds each ($C_{math}^{private}$ to D-054; profile fields to D-053). Single-owner assignment of every obligation (rather than allowing structural and profile layers to both check the same property) is required so that a single mutation cannot fail two blocks by definition and contaminate $\Gamma_0^{RSA-ser}$'s causal-class boundaries with artificial overlap. ExactConsumption is adopted despite contradicting two of three backends' native behavior because the SDK's contractual artifact is a DER representation, not a DER prefix, eliminating an already-confirmed cross-backend divergence outright rather than deferring it to R_{val}. The public/private C_{math} asymmetry is not an inconsistency but a direct consequence of what RSA key material exposes by design; forcing symmetry would either weaken $C_{math}^{private}$ unjustifiably or fabricate an unenforceable C_{math}^{public}. Primality testing of n was considered and explicitly not adopted, preserving the normative-necessity/admission-time-enforcement distinction rather than conflating a cheap partial detector with the (unverifiable) condition that actually matters, and rather than silently reproducing BC's precedent of unilaterally adding a technique-specific check.

ID	Decision	Reason
D-056	RSA Key Serialization SDK error semantics frozen as directional (§19.7). Import uses the four-class ordered adapter classifier $E_{RSA-ser}^{SDK} = \{\text{unsupported}, \text{malformed_artifact}, \text{invalid_parameter}, \text{invalid_key}\}$, evaluated $\neg A_p^{pre} \rightarrow \neg C_{struct} \rightarrow (C_{struct} \wedge \neg C_{profile}) \rightarrow (C_{struct} \wedge C_{profile} \wedge \neg C_{math}^{role})$ – an evaluability-dependency order (cannot check profile membership before structure parses; cannot check mathematical semantics before profile membership establishes the artifact as the claimed portable RSA material), not an arbitrary priority. Classification follows contractual cause, not native exception, confirmed as a genuine three-way pattern across the closed backend inventories: provider rejects/adapter normalizes (Crypto++ <code>BERDecodeErr</code> $\rightarrow$ <code>malformed_artifact</code>); provider accepts/adapter actively rejects (BC natively admits <code>id-RSASSA-PSS/id-ea-rsa</code>, outside D-053’s profile, forcing <code>invalid_parameter</code> with no native exception to draw on); provider performs no check/adapter validates and rejects (none of the three backends check $V_{mathematical}^{RSA,2}$’s cross-field CRT relations in full, so the adapter itself must implement D-054’s obligation to produce <code>invalid_key</code>). Export is not import’s mirror: representation properties are export postconditions ($\text{Export}_{SDK,p}(K) = S \implies C_{ser}^{export}(S)$) on an already-admitted key, not preconditions on untrusted input, so <code>malformed_artifact/invalid_parameter/invalid_key</code> are not legal export outcomes; export’s classifier has only <code>\{unsupported, success\}</code>. No catch-all contractual provider error: an unexpected backend failure after satisfied preconditions is a realization/conformance defect of	Mirrors the precedent already frozen for GCM: the adapter classifies causally before invoking or interpreting the provider, so the native <code>invalid_key</code> defines the SDK class. The directional import/export asymmetry mirrors D-053’s directional $P_{RSA-ser}^{common/portable}$ rather than forcing artificial symmetry between two operations with structurally different obligations (untrusted-artifact classification vs. postcondition verification on already-trusted material). Re-deriving input-side error classes for export would be a category error – a writer producing malformed output from a valid key is an R_{ser} conformance defect the experimental design should surface, not a legal error the contract defines for the caller to handle, and inventing a catch-all provider-error class for the same reason would normalize away exactly the divergences this design exists to detect. <code>extractable</code>’s exclusion is required by consistency with D-053, which already drew the representation/object-policy boundary; admitting it into the error model would reintroduce, through a different channel, a property already deliberately excluded from the representation contract.

ID	Decision	Reason
D-057	Thirteen rsa-ser.* clauses frozen (§20), derived directly from D-053–D-056. Corrected through two audit passes before freezing: (i) an initial public-container/private-container split risked double-violation against role-container on a single stimulus (a well-formed PrivateKeyInfo presented as role=public); restricting *-container to apply only post-type-identification fixed that case but left malformed/unidentifiable DER with no owning clause, contradicting D-055/D-056’s own malformed_artifact classification – resolved by introducing der-syntax (DER parsing alone, object-agnostic) and merging public-container/private-container into a single container clause (identical K_1/K_2, correlated branches of one obligation, not independent semantic categories), yielding the ordered, gap-free, single-owner chain der-syntax → exact-consumption → container → role-container → algorithm-id → algorithm-params → {public-validity private-domain ∧ private-relations}; (ii) public-material/private-material redefined as value-preservation clauses $((n, e)_{\text{after}} = (n, e)_{\text{contract}}$, etc.), not field-presence clauses, separating a silent-value-alteration defect from a validity defect that field-presence framing would have conflated. rsa-ser.role, rsa-ser.output, rsa-ser.roundtrip, rsa-ser.interop explicitly not created: role is a C_{surface} selector, not an independently violable obligation; output would duplicate container/algorithm-id/algorithm-params under a different name; $R_{\text{ser}}/R_{\text{interop}}$ are relations that	Preserves $C_{\text{pre}} \rightarrow \text{Clauses} \rightarrow \Gamma_0$ correspondence exactly: every $C_{\text{pre}}^{\text{RSA-ser}}$ obligation from D-055 must have exactly one owning clause, or later causal-class construction and $\Gamma_0^{\text{RSA-ser}}$ inherit an ownerless gap or an artificial double-violation. Both audit passes were caught by systematically walking the full causal chain against concrete boundary stimuli (malformed DER; valid DER of unidentified structure; correctly-typed wrong-role container; correct role with wrong OID; correct OID with out-of-profile parameters) rather than inspecting the clause list in isolation – the same discipline that has repeatedly caught overlap/gap errors before Γ_0 construction in prior operations (D-036, D-038, D-050) and was applied here before, not after, freezing.

ID	Decision	Reason
D-058	Seventeen $\Gamma_0^{RSA-ser}$ causal classes frozen jointly with their Γ_0 assignment (§21), across five families: (A) semantic material preservation, 2 classes, role-exclusive; (B) representation reader-vs-writer, 5 classes (der-syntax/container each split import/export, plus import-only exact-consumption); (C) portable profile, 5 classes (algorithm-id/algorithm-params each split import/export, plus role-container); (D) mathematical validity, 3 classes, import-only, deliberately kept coarse (RSA-SER-PRIVATE-RELATIONAL-BYPASS covers all five $V_{rel}^{RSA,2}$ relations as one class with multiple stimuli, not five classes); (E) error/capability, 2 classes, mirroring the OAEP/PSS pattern. D-056's import/export asymmetry propagates into the causal classes: reader-bypass classes expect R_{val}, writer-divergence classes expect R_{ser}, both implicating the same clause via causally distinct mechanisms. All thirteen D-057 clauses are covered (four by two classes via the import/export split, nine by exactly one); no class was created without a clause, no clause was left without a class. No fifth Family-D per-equation split and no catch-all provider-error class were created, both deliberately, per D-016 and D-056 respectively. All seventeen classes audited for genuine double-violation risk via minimal isolating stimuli before freezing; none found – apparent overlap resolves to single-variable stimulus-construction discipline (D-016), not a clause-boundary defect.	Follows the GCM/PSS precedent (D-049) of freezing the class set and its Γ_0 ground truth as one decision, since a causal class is not fully characterized until its Γ_0 is known, and D-007 requires Γ_0 pre-registration before mutation execution; splitting class-naming and Γ_0-assignment into two decisions risks an artificial intermediate state (classes frozen, ground truth still revisable) that could force reopening an already-frozen decision for a problem belonging to the same conceptual object. Coarse Family-D granularity avoids the mutation_i ↔ clause_i degeneracy D-016 exists to prevent – each $V_{rel}^{RSA,2}$ stimulus attacks the same stable obligation and is expected to resolve to the same D-056 outcome (invalid_key), so per-equation classes would add stimulus count without adding causal distinctness. The boundary audit was performed by constructing concrete minimal stimuli for each of the six import-side profile/encoding classes rather than inspecting the class list in isolation, the same discipline already applied when auditing D-057's clauses and consistent with the project's standing preference for pre-execution correction (D-007, D-036, D-038, D-050).

ID	Decision	Reason
D-059	EC P-256 key serialization frozen as a two-level profile (§22.6, §22.7): $P_{EC-ser}^{portable} \subseteq P_{EC-ser}^{common}$, one decision, not two – common and portable are two faces of the same methodological choice (P^{common} = evidence-derived intersection; $P^{portable}$ = canonical selection within that intersection), not independent commitments. The common profile is the evidence-derived three-backend intersection over DER SPKI/PKCS#8, <code>id-ecPublicKey</code>, <code>secp256r1</code>, uncompressed public-point representation, and the shared private-import family; modeled directionally as $P_{EC-ser}^{common} = (P_{export}^{common}, D_{import}^{common})$, not a flat Cartesian product – role, container, algorithm identifier, curve parameters, point representation, and inner RFC 5915 fields remain structurally correlated. In RFC 5915 private import, <code>parameters</code>[0] may be absent or present-matching, whereas <code>publicKey</code>[1] is required by the common admission intersection itself (W3C #356’s divergence excludes its absence, not a portable-level choice). The portable profile selects a single canonical representation with both <code>parameters</code>[0] and <code>publicKey</code>[1] present, the latter uncompressed – but the two presences are asymmetric in kind, not degree: [0]=present is a genuine canonicalization choice made <i>within</i> the common-permitted {absent, present-matching} range, while [1]=present is inherited directly from the common intersection, where its absence was already excluded by evidence, not selected by portable-level preference. Native provider export need not itself equal the portable representation (Export_p^{native} ≠ Export_p^{SDK} in general – Crypto++ is the	Separates evidence-derived interoperability from deliberate SDK canonicalization, avoids treating provider-specific serialization choices as equivalence failures, preserves directional export/import obligations (so this decision cannot later be misread as “all three providers produce this exact PKCS#8”), and makes R_{ser} reproducible without prematurely imposing curve-membership or $Q = dG$ semantics. Freezing common and portable in one entry, rather than two, prevents the misleading impression that they were chosen independently, when in fact the portable selection is entirely a canonicalization act performed inside an already-fixed common intersection – mirroring D-053’s joint treatment for RSA-ser. Keeping the [0]/[1] asymmetry explicit (choice vs. inherited consequence) prevents a generic future reading such as “portable requires both optional fields” from obscuring that one is a decision and the other is forced by already-collected evidence, which matters directly for how Γ_0^{EC-ser} will later attribute causal responsibility for a violation of each. Explicitly deferring $Q = dG$ preserves the next investigation as a conscious decision (Key, Membership, Validation, or a carefully scoped combination) rather than one smuggled in through the representation choice made here.

ID	Decision	Reason
D-060	C_{pre}^{EC-ser} frozen (§22.9), five blocks: $C_{role}, C_{curve}, C_{pub}, C_{priv}, C_{math}, C_{direction}$ (six named, five conceptual: role and direction are call-shape selectors, mirroring $C_{surface}$'s treatment in RSA-ser). Public contract: canonical DER-SPKI(id-ecPublicKey, sec256for04 X Y Z), requiring with Encoding and Membership. CurveMembership kept distinct. Private contract: semantic material remains Private(d) (not reopened); portable export is PKCS8_{portable}(d, Q) with $Q = dG$, the composite representation, not a redefinition of abstract material. Admission contract, the central equation: $\text{Accept}_C(A) \iff \text{SyntacticValid}(A) \wedge \text{RepresentationValid}(A) \wedge \text{ScalarValid}(d) \wedge \text{CurveMember}(Q) \wedge \text{PairConsistent}(d, Q)$, with $\text{Accept}_p^{SDK}(A) = \text{Accept}_C(A)$ enforced regardless of $\text{Accept}_p^{native}(A)$ (§22.8: no backend demonstrates all five conjuncts natively). Directional contract: $\text{Export}_p^{SDK}(k) \in P_{EC-ser}^{portable}$; $\text{Import}_p^{SDK}(A)$ accepts iff A satisfies the contractual portable admission domain; cross-provider $\text{Import}_q^{SDK}(\text{Export}_p^{SDK}(k)) \simeq k$ with $\simeq$ narrowly defined over Q (public) or d alone (private, not the pair). Deliberately excluded: error categories, mutations, Γ_0, native per-backend behavior, compressed points, BER, explicit parameters, JWK, subgroup membership, RNG obligations, and – most importantly – the exact strict-reject-vs-normalize boundary for valid-but-non-portable common representations, left for Error semantics.	Mirrors RSA-ser's D-055 precedent: C_{pre} gets its own decision, separate from Error semantics (D-056's RSA analogue), since what must be rejected and how a rejection surfaces as an SDK-observable class are different evidence. The admission-contract equation formalizes the adapter's role established in §22.8 without depending on any single backend's parser semantics, directly connecting EC-ser to RSA-ser's D-010/D-054 structural finding: native admission behavior is evidence about a realization, not the definition of the contract. Narrowing $\simeq$ to Q or d alone (not internal object state, not the (d, Q) pair for private material) prevents the equivalence relation from silently reintroducing Private(d, Q) after $M_{EC}^{common/portable}$ deliberately froze Private(d). Leaving the reject-vs-normalize boundary open here, rather than resolving it now, avoids contaminating C_{pre} with an R_{val}/R_{err}-scoped decision before the evidence and framework needed to make it are in place.

ID	Decision	Reason
D-061	EC P-256 SDK error semantics and common→portable normalization boundary frozen (§22.10), resolving what D-060 left open, and corrected during the subsequent clause-audit pass (D-062, §22.11) to add an explicit unsupported/capability-gate branch, absent from the initial formulation and caught only because ec-ser.cap had no textual grounding to point to. Neither strict-portable-only rejection nor unrestricted normalization: $\text{Normalize}(A) \iff A \in D_{import}^{common} \wedge A \notin P_{EC-ser}^{portable};$ $\text{Reject}(A) \iff A \notin D_{import}^{common} \text{ or } \neg C_{math}^{EC}(A) - \text{bounded strictly by the already-derived common profile, never by provider-specific extended tolerance. Four provider-independent error categories: } E_{syntax} \text{ (not structurally interpretable), } E_{representation} \text{ (structurally interpretable, profile-incompatible: wrong algorithm/curve OID, unadmitted point form, contradictory parameters), } E_{key} \text{ (} d \notin [1, n-1] \text{ or } Q \neq dG - V_{scalar} \text{ and } V_{pair} \text{ share this one observable class, a coarser grouping than the clause level, mirroring RSA-ser's precedent of multiple clauses sharing one observable class), } E_{membership} \text{ (} Q \notin E(\mathbb{F}_p) \text{, deliberately kept separate from } E_{key} \text{) - kept distinct from unsupported, a capability-gate outcome evaluated prior to and separately from these four content-level categories. Obligation, validation mechanism, and error category kept as three distinct layers (e.g. Membership.CurveMembership clause} \rightarrow \text{Validation.Semantic mechanism} \rightarrow E_{membership} \text{ observable), never conflated. } Q_{supplied} \neq dG \stackrel{149/162}{=} E_{key} \text{, never normalization - not an independent stipulation but a}$	The [0]-absent counterexample (common per D-059, non-canonical per portable) shows strict-portable-only rejection would conflate noncanonical with invalid, damaging the diagnostic capability the framework exists to measure; unrestricted normalization would let provider-specific tolerance silently widen the cross-provider contract, violating $P_{portable} \subseteq P_{common} \subseteq P_p$ (D-059). Keeping $E_{membership}$ separate from E_{key} preserves, at the observable layer, the same Membership ≠ Key distinction already fought for at the clause layer (§22.8) – collapsing them would erase exactly the diagnostic value that separation was built to produce, even though E_{key} itself already coarsens two distinct clauses (V_{scalar}, V_{pair}) into one class, showing the granularity choice is deliberate per-pair, not accidental uniformity. Refusing to normalize $Q \neq dG$ or off-curve Q is required so a future causal mutation targeting these properties is observed, not silently repaired by the adapter before it can be measured – normalizing away the very defect a mutation is designed to test would nullify the experiment's diagnostic purpose. The unsupported-branch correction follows the same discipline already applied to RSA-ser's D-055/D-057 gap corrections: caught by systematically checking that every clause has textual grounding in a frozen decision, before freezing the clause set itself, not after.

ID	Decision	Reason
D-062	Thirteen ec-ser.* clauses frozen (§22.11), derived from D-059–D-061 and C_{math}^{EC}. Bidirectional audit performed before freezing: forward audit found one gap (ec-ser.cap initially ungrounded, since D-061’s formula lacked an unsupported branch), fixed by amending D-061 itself before this clause set was frozen; reverse audit confirmed no orphaned C_{pre}^{EC-ser} obligation, with C_{pub}’s membership component covered by the role-agnostic ec-ser.curveMembership rather than a dedicated public-only clause. Three granularity decisions made explicit: the two public Encoding clauses (ASN1 vs. KeyPoint) are not merged, since RSA-ser exercises the former but never the latter – directly relevant to future $Marginal_{K_2}$; private.scalar and pairConsistency remain separate despite sharing the observable class E_{key}, since sharing an error category never justifies merging distinct causes; curveMembership is not restated inside validation.semantic, preserving obligation $\neq$ validation mechanism $\neq$ observable error. No ec-ser.inputArtifact clause (would exist only to populate $K_1 = \text{Input}$, with no independent semantics); Authentication does not apply. Real K_1 coverage: $\{\text{Key, Parameter, Output, Encoding, Validation, Membership, Error, Capabilities}\}$. 8 of 10. Single ec-ser.cap, not split provider/portable, since no genuinely rich parameterized boundary comparable to AES-GCM’s tag/IV-length split has been identified for EC-ser; $P_{portable} \subseteq P_{common} \subseteq P_p$ already represents the relevant boundary explicitly.	Mirrors RSA-ser’s D-057 clause-construction discipline exactly: one obligation per clause, not a mutation or test; bidirectional audit performed before, not after, freezing, since a C_{pre} obligation left without a clause or a clause without contractual grounding would propagate as a gap into Γ_0^{EC-ser} construction. The Encoding.ASN1/Encoding.KeyPoint split is preserved specifically because it is expected to matter for the necessity-analysis cascade (§23): RSA-ser’s prior coverage of ASN1 must not be allowed to mask EC-ser’s genuinely novel KeyPoint contribution when marginal K_2 coverage is computed. curveMembership is this design’s most methodologically significant EC-ser clause, since Membership was the one K_1 value AES-GCM’s coverage left systematically empty and one of the original grounds for retaining EC serialization as a candidate operation (D-015) rather than requiring full ECDSA; pairConsistency’s marginal value against RSA-ser’s own public/private relationship is deliberately deferred to the transversal audit rather than asserted

ID	Decision	Reason
D-063	Fourteen Γ_0^{EC-ser} causal classes frozen (§22.12), not a mechanical one-per-clause correspondence. A three-point microaudit against the frozen text preceded freezing: unsupported confirmed a standalone outcome distinct from E_κ (so EC-CAPABILITY-MANIFEST-MISMATCH targets only ec-ser.cap, no invented fifth error category); K_2 labels confirmed exactly PrivateScalar/PublicPrivateConsistency; ec-ser.private.asn1's frozen text confirmed to describe only the canonical portable shape, not a reconstruction obligation – corrected out of Γ_0(EC-PRIVATE-PARAMS-ABSENT) from an initial draft that had wrongly included it, leaving that class's Γ_0 as $\{\text{ec-ser.validation.semantic, ec-ser.private.asn1}\}$; EC-PUBLIC-OFF-CURVE and EC-PRIVATE-PAIR-MISMATCH are held fully disjoint in Γ_0 (curveMembership in one, pairConsistency in the other, never both), the cleanest isolating pair in this set. [0]-absent and [1]-absent are deliberately not merged: the former is contractually a normalize case (no ec-ser.error), the latter a reject case, directly exercising the D-061 common-vs-portable boundary at its precise edge. EC-ERROR-MAP-SWAP ($\Gamma_0 = \{\text{ec-ser.error}\}$) isolates whether R_{err} carries information independent of R_{val}; EC-CAPABILITY-MANIFEST-MISMATCH ($\Gamma_0 = \{\text{ec-ser.cap}\}$) analogously isolates R_{cap}. Coverage confirmed: all thirteen D-062 clauses covered; K_1 coverage 8/10 as anticipated; K_2 coverage includes at least eight confirmed subtypes.	The mutation defines the known cause; relations are detectors evaluated afterward – Γ_0 is never built by asking which relation should be justified, and the pre-freeze microaudit exists specifically to prevent smuggling an intextualized obligation into a clause's Γ_0 by assumption rather than by confirmed grounding, mirroring the distinction already applied to RSA-ser's own pre-Γ_0 audits (D-036, D-038, D-050, D-057, D-058). Disjoint off-curve/pair-mismatch Γ_0 sets give Membership – the one K_1 AES-GCM's coverage leaves empty – genuine, isolable marginal potential rather than a diluted signal shared explicitly. Not merging [0]-absent with [1]-absent is required precisely because their contractual outcomes differ (normalize vs. reject); collapsing them into one class would erase the single most informative boundary-edge stimulus available in this Γ_0. EC-ERROR-MAP-SWAP and EC-CAPABILITY-MANIFEST-MISMATCH exist to keep R_{err}'s and R_{cap}'s eventual necessity determination (§23) genuinely open rather than prejudged by a Γ_0 that never gave either relation a class where it could independently succeed or fail.

ID	Decision	Reason
D-064	Cross-operation audit, RSA-ser vs. EC-ser, frozen (§22.13): establishes $\text{Marginal}(\text{EC-ser} \mid \text{RSA-ser})$ specifically, not yet the global $\text{Marginal}(\text{EC-ser})$ against the full operation set O. Six dimensions compared: Encoding.ASN1 shared; Encoding.KeyPoint EC-specific (RSA has no curve-point concept); Membership.CurveMembership EC-specific (no RSA-ser clause uses $K_1 = \text{Membership}$ at all); Key.PublicPrivateConsistency EC-specific, a new K_2 within an already-covered K_1; Validation.Syntactic/Semantic shared; Capability shared. Corrected during this audit, not glossed over: an initial draft grouped Key.PublicPrivateConsistency with Membership as if both were the same kind of K_1-level novelty – they are not (D-062 froze the former under $K_1 = \text{Key}$, the latter under the separate $K_1 = \text{Membership}$), yielding three distinct marginal contributions, not two: $\text{Marginal}_{K_1}(\text{EC-ser} \mid \text{RSA-ser}) \supseteq \{\text{Membership}\}$; $\text{Marginal}_{K_2}(\text{EC-ser} \mid \text{RSA-ser}) \supseteq \{\text{Encoding.KeyPoint}, \text{Membership}\}$. $V_{rel}^{\text{RSA},2} \neq V_{pair}^{\text{EC}}$ preserved as the substantive structural argument: RFC 8017’s RSAPrivateKey embeds n, e directly with no separate, optional, redundant public field that could diverge, checking only internal self-consistency of one object; RFC 5915’s publicKey[1] is optional and redundant (d alone determines $Q = dG$), and it is exactly that redundant-field structure that opens the possibility of divergence RFC 8017 structurally cannot exhibit. $E_{membership}$ noted as a genuinely new observable-error-taxonomy class but deliberately not counted toward K_1/K_2 marginal accounting, which concerns contractual clauses, not error-class richness.	Establishes, before the formal global Coverage/Marginal computation runs, that EC-ser is not a mere duplicate of RSA-ser – required traceability for eventual O_{min} selection, not a working note. Separating the three marginal contributions (one full K_1, two K_2-within-shared-K_1) rather than collapsing them into a generic “EC contributes Membership-related value” preserves the categorically different strength of each: a fully empty K_1 is the strongest possible marginal result and directly confirms D-015’s original ground for retaining EC serialization without full ECDSA, while a new K_2 within shared K_1 is real but weaker and must not be inflated to look like the same kind of contribution. The RFC 5915/RFC 8017 structural argument is preserved verbatim (not merely its conclusion) because it is the actual evidence for Key.PublicPrivateConsistency’s novelty – a format exposing a CurveMembership, Key.PublicPrivateConsistency independently-populatable field is a structural fact about the two encodings, not an assertion resting on classification convenience. Keeping $E_{membership}$’s novelty out of K_1/K_2 marginal accounting prevents conflating two genuinely different axes (contractual clause coverage vs. observable error-taxonomy richness) that this design has kept separate throughout (§22.10).

ID	Decision	Reason
D-065	Formal Coverage_{K₁}/Coverage_{K₂} computed for the full operation set $O =$ {HKDF, AES-GCM, RSA-OAEP, RSA-PSS, RSA-ser, EC-ser}, stages 1–2 of the necessity cascade frozen (§23.1); stages 3–4 (detection/diagnostic gain) explicitly deferred, requiring executed mutations. Every entry read mechanically from each operation’s frozen clause table, not reconstructed from memory. Marginal_{K₁}(EC-ser) = {Membership}; Marginal_{K₁}(o) = $\emptyset$ for all other five, confirmed exactly as anticipated pre-mechanically. Marginal_{K₂}(AES-GCM) = {Authentication.AEADTag, Encoding.AEADType, Validation.AEADType, Validation.AEADTag}; Marginal_{K₂}(RSA-PSS) = {Authentication.SignatureVerification}; Marginal_{K₂}(EC-ser) = {Encoding.KeyPoint, Membership.ChromeMembership, Key.PublicPrivateConsistency}; Marginal_{K₂}(HKDF) = Marginal_{K₂}(RSA-OAEP) = Marginal_{K₂}(RSA-ser) = $\emptyset$. New finding surfaced only by this global cross-check, invisible to D-064’s RSA-ser-only pairwise comparison: Parameter.DomainParameter is shared between oaep.mgfCoupling and pss.mgfCoupling; Encoding.ASN1/Validation.Syntax are shared exclusively between RSA-ser and EC-ser (contributing to neither’s marginal, regardless of which operation instantiated the subtype chronologically first); Parameter.ExternalRandomnessComputation is shared across AES-GCM/RSA- OAEP/RSA-PSS. $O_{\text{structurally necessary}} \supseteq$ {AES-GCM, RSA-PSS, EC-ser}; {HKDF, RSA-OAEP, RSA-ser} carry zero Coverage marginal and their O_{min} membership depends entirely on stages 3–4.	Coverage/Marginal is computable now precisely because it depends only on pre-registered Γ_0/clause tables, and not on the computing it before mutation execution is the methodology’s own design, not an expedient. Marginal is a set-difference property over the full operation set, not a pairwise or chronological claim: a subtype’s “first” appearance confers no special status once a second operation is confirmed to share it, which is exactly why the mechanical full-set check (rather than trusting D-064’s pairwise result) was required before any global marginal claim. Explicitly separating structurally-necessary operations from the three zero-Coverage-marginal candidates, before a single mutation runs, operationalizes the necessity rule’s own stated purpose: operations are not retained by attachment to the amount of prior work invested in them, and RSA-ser – despite being the single largest normative/contractual construction in this entire design (D-053–D-058) – has zero Coverage-stage marginal contribution, the sharpest possible demonstration that Coverage/Marginal and effort invested are genuinely independent axes.

ID	Decision	Reason
D-066	Stages 3–4 protocol (detection gain, diagnostic gain) frozen as a measure, not an empirical result (§23.2). DetectionGain(o) = $D(o) \setminus \bigcup_{o' \neq o} D(o')$, $D(o)$ the set of divergence patterns causally detected by some relation over $\text{Mut}(o)$ – unaffected by the D-069 correction below. The diagnostic-gain formula originally frozen here, DiagnosticGain(o) = $\Pi_R(O) - \Pi_R(O \setminus \{o\})$ over raw observable-vector partitioning, is superseded by D-069’s $\sigma(c)/Q(o)$ construct, which corrects a genuine partial conflation with DetectionGain found during the cumulative audit; see D-069 for the corrected definition. Keep rule unchanged: $\text{Keep}(o) \iff \text{DetectionGain}(o) > 0 \vee \text{DiagnosticGain}(o) > 0$; combined with D-065, zero Coverage marginal and zero on both gains means $o \notin O_{\min}$, no exception for effort invested. Three hypotheses pre-registered for the zero-Coverage-marginal candidates: H_{HKDF} (pure-deterministic case contributes independent R_{byte} value, genuinely contestable since <code>gcm.ciphertext</code> already states determinism and three AES-GCM classes already expect R_{byte}); H_{OAEP} (contributes R_{interop}-detectable divergence unreachable via R_{byte}, backed now by a precise <i>applicability</i> distinction, $\text{Applicability}(R_{\text{byte}}, \text{OAEP}) = 0 \neq \text{Applicability}(R_{\text{interop}}, \text{OAEP}) = 1$, not yet empirical detection gain); $H_{\text{RSA-ser}}$ (RSA/ASN.1-specific causal structure contributes gain EC-ser’s own mutations do not reproduce, explicitly not defensible by “a serialization operation is needed” since EC-ser already fills that role). A pre-registered expectation (OAEP	Separating measure definition from empirical result is required by D-007’s execute-before-conclude discipline; freezing the protocol now, before mutations run, prevents the measure itself from being silently shaped by whichever outcome later seems convenient. H_{HKDF} is stated as genuinely contestable, not rhetorical, specifically because AES-GCM’s own frozen clauses and classes already claim deterministic R_{byte} coverage – an unchallenged assumption here would have prejudged stage 3 before it runs, the same failure mode this design’s necessity rule exists to prevent. $H_{\text{RSA-ser}}$ explicitly rules out “a serialization operation is needed” as sufficient justification, since D-065 already demonstrated EC-ser satisfies that role, forcing RSA-ser’s defense onto genuine causal-diagnostic grounds rather than a generic appeal to representation-layer necessity. Distinguishing DiagnosticGain(o) (whole-operation, O-level) from D-052’s $DG_j(c)$ (single-clause, within-operation) prevents conflating two legitimately different diagnostic questions the framework needs to keep separate: whether an entire operation is needed, versus whether one obligation within an already-retained operation is needed.

ID	Decision	Reason
D-067	$O_{\min}$ decision boundary frozen (§23.3), closing the preexperimental selection problem without closing $O_{\min}$ itself. $O_{\text{fixed}} =$ {AES-GCM, RSA-PSS, EC-ser} (structurally necessary regardless of stages 3–4, per D-065); $O_{\text{pending}} =$ {HKDF, RSA-OAEP, RSA-ser} (zero Coverage marginal, membership determined exclusively by D-066’s Keep rule). $O_{\min} = O_{\text{fixed}} \cup O_{\text{emp}},$ $O_{\text{emp}} = \{o \in O_{\text{pending}} : \text{DetectionGain}(o) > 0 \vee \text{DiagnosticGain}(o) > 0\} \subseteq O_{\text{pending}},$ giving $O_{\text{fixed}} \subseteq O_{\min} \subseteq O_{\text{fixed}} \cup O_{\text{pending}} = O$ and exactly $2^3 = 8$ enumerable possible final outcomes. Three unambiguous names adopted – O_{fixed} (decided), O_{pending} (contingent), $O_{\min}$ (final, post-execution) – deliberately not $O_{\min}^{\text{pre}}$ for O_{fixed}, since that name would misleadingly suggest a provisional near-final set rather than a strict, bounded lower bound. <i>The composition of $O_{\min}$ is not frozen preexperimentally. What is frozen is the complete decision boundary: three operations are structurally necessary, and only three specified operations remain contingent on the pre-registered empirical stages 3–4.</i> Consequence for v0.6: sealing does not require executing the mutation harness, provided v0.6 remains the closure of the preexperimental design/evidence base – the correct closing state is O_{fixed} closed, O_{pending} closed, the decision rule closed, $O_{\min}$ explicitly pending empirical execution.	Formalizes a direct, dependency-free logical consequence of D-065+D-066 rather than a new empirical claim – $O_{\text{fixed}} \subseteq O_{\min}$ requires no future result, since the necessity rule’s retention condition is already satisfied by already-frozen Γ_0/clause evidence. Naming O_{fixed} precisely (not $O_{\min}^{\text{pre}}$) prevents a future reader from conflating a proven lower bound with a near-final guess. Explicitly listing what must <i>not</i> be closed now (neither $O_{\min} = O_{\text{fixed}}$ nor $O_{\min} = O$, nor any individual pending operation’s survival/elimination by argument plausibility alone) prevents the same silent prejudgment risk D-066 already guarded against, applied here at the level of the boundary statement itself rather than the measure definition. Recognizing that v0.6 can seal without executed mutations reframes the remaining empirical work as the honest, expected next phase of a pre-registered design, not a deficiency of the design closed here – mirroring the historical separation already drawn between closing Γ_0/selection and the subsequent execution phase.

ID	Decision	Reason
D-068	HKDF $L > 0$ lower-bound debt closed, resolving (not contradicting) D-020's pending decision: $C_{pre}^{HKDF} : 1 \leq L \leq 255 \cdot \text{HashLen}$, i.e. $1 \leq L \leq 8160$ octets for HKDF-SHA-256. RFC 5869 supplies the upper bound but leaves the zero-length case unspecified as an explicit lower-bound requirement; the portable SDK profile excludes the degenerate zero-length output. $\text{Accept}_C(L) \iff 1 \leq L \leq 8160$; $L = 0$ and $L > 8160$ both resolve to <code>invalid_parameter</code>, not <code>unsupported</code> – a syntactically valid but out-of-portable-profile length is a parameter-contract violation, not a backend capability gap, exactly the disambiguation rule already applied to AES-GCM's tag-length/IV-length boundaries (D-025) and RSA-ser/EC-ser's profile boundaries. Retroactive correction to HKDF-LENGTH-BOUNDARY-CROSSING: <code>hkdf.cap</code> removed from Γ_0 (reduced from four clauses to three, R_{cap} dropped from expected relations), since a plain profile-boundary crossing does not itself attack the capability-manifest obligation – that remains exclusively HKDF-CAPABILITY-BOUNDARY-MISMATCH's role. HKDF-LENGTH-WITHIN-DOMAIN-PERTURBATION is untouched. $\text{Coverage}_{K_1}(\text{HKDF})/\text{Coverage}_{K_2}(\text{HKDF})$ unchanged (still <code>Parameter.Length</code>); D-065, D-066, D-067 unaffected. D-020 is preserved verbatim as historical traceability, not rewritten – it was correct at the time it was frozen.	RFC 5869's silence on $L = 0$ must not be read as permission by default: $L = 0$ is a degenerate case for this design's own experimental purposes ($OKM = \epsilon$ trivially satisfies $OKM_p = OKM_q$ without exercising any materially useful derivation, making it a poor R_{byte} instance), and $P_{\text{portable}} \subseteq P_{\text{common}}$ already establishes that restricting the profile below what ungoverned grammar would tolerate is legitimate. The <code>hkdf.cap</code> correction applies retroactively the same cause-equals-detector discipline matured through RSA-ser/EC-ser's own Γ_0 audits (D-036, D-038, D-050, D-057, D-058, D-063) back onto the earliest-built operation in this design, closing a debt inherited from before that discipline existed rather than leaving HKDF's Γ_0 frozen at an earlier, less rigorous standard than every operation built after it.

ID	Decision	Reason
D-069	Cumulative preexperimental design audit: four corrections across two independent audit passes – one substantive formula correction, one minor wording fix, and two stale-text updates – all verified before freezing, none left as a known defect. (i) D-066’s DiagnosticGain(o) formula corrected, superseding $\Pi_R(O) - \Pi_R(O \setminus \{o\})$ over raw observable-vector partitioning: since each operation’s Γ_0 draws exclusively from its own local clause namespace, a single mutation with a not-elsewhere-replicated $r(c)$ suffices to make that difference positive, demonstrating only unique detectability – exactly what DetectionGain already measures – not genuine causal disambiguation. Confirmed real but partial (not universally degenerate) by direct comparison of GCM-PROVIDER-CAPABILITY-MISMATCH (R_{cap} alone) against HKDF-CAPABILITY-BOUNDARY-MISMATCH ($R_{\text{cap}}; R_{\text{val}}; R_{\text{err}}$) – genuinely different vectors, so the conflation is triggered by any single uniquely-shaped vector, not by systemic collapse. Corrected construct: semantic causal signature $\sigma(c) = \{\text{kind}_{K_1/K_2}(g) : g \in \Gamma_0(c)\}$, defined over the shared cross-operation K_1/K_2 taxonomy (not operation-local clause IDs, which never overlap across operations by construction); $(\sigma_i, \sigma_j) \in Q(o)$ iff $\exists c_i, c_j \in \text{Mut}(o)$ with $\sigma(c_i) = \sigma_i \neq \sigma_j = \sigma(c_j)$ and $r(c_i) \neq r(c_j)$; $\text{DiagnosticGain}(o) = Q(o) \setminus \bigcup_{o' \neq o} Q(o')$. Decouples cleanly from DetectionGain: a lone unique vector contributes nothing to $Q(o)$ without a paired, distinctly-signatured, empirically-separable partner, and the uniqueness required is of the <i>pairing</i>, not of either vector individually. D-052’s	Both corrections were verified independently before being accepted, not applied on the strength of a single review pass: the DiagnosticGain flaw was confirmed via a concrete comparison against already-frozen classes rather than accepted as plausible-sounding, and the corrected $\sigma(c)/Q(o)$ construct was checked to genuinely decouple from DetectionGain (not merely rename the same quantity) before being adopted. Marking D-066’s original formula <i>superseded</i> rather than silently replacing it preserves the same before-not-after correction discipline already established for D-036/D-038/D-050 (retroactive Γ_0 corrections) and D-052 (an analogous within-operation diagnostic-gain correction) – this is the same class of error this design has repeatedly caught and fixed, now applied to its own necessity-stage machinery rather than to a single operation’s contract. The independence-wording correction is minor but real: an unqualified “independent” could later be misread as a claim this design does not make and does not need, about the statistical relationship between three empirical outcomes rather than the combinatorics of three separately-evaluated flags. A second, independent audit pass over the physical document (rather than the conceptual chain of decisions alone) confirmed the two formula/wording corrections and additionally located two stale-text items neither prior pass had caught, illustrating why a genuinely independent re-check of the artifact itself – not just of the reasoning that

ID	Decision	Reason
D-070	Version 0.6 frozen as the preexperimental research design and evidence base (§28), the final entry of this Decision Log. Frozen: scientific question, formal framework (six relations, K_1/K_2 taxonomy, adapter-vs-provider threat model, $P_{\text{portable}} \subseteq P_{\text{common}} \subseteq P_p$), all six operations’ normative evidence/C_{pre}/clauses/Γ_0, D-064’s transversal audit, D-065’s Coverage/Marginal, D-066/D-069’s detection/diagnostic-gain protocol, D-067’s $O_{\text{fixed}}/O_{\text{pending}}$ boundary, D-068’s HKDF closure. Explicitly, not implicitly, left open: O_{min} pending empirical execution, bounded $\{\text{AES-GCM, RSA-PSS, EC-ser}\} \subseteq O_{\text{min}} \subseteq O$; R_{err}’s necessity likewise pending. Post-freeze amendment policy: a frozen obligation found technically unrealizable during implementation is not silently edited here – it is recorded as an explicit deviation in the successor <i>Experimental Evidence Base</i> document, referencing this document’s frozen state as baseline. Preexperimental design audit status at freeze time: PASS (D-069).	Sealing the document is a documentation act, not a further scientific decision – the freeze itself introduces no new claim, only records that prior claims (D-001–D-069) are now stable and future changes require logged amendment rather than silent revision. This is required precisely because the design’s own methodology (D-007’s execute-before-conclude discipline, applied throughout to every operation and to the necessity machinery itself) only has evidentiary value if the boundary between preexperimental and empirical work is itself unambiguous and dated – an undated, continuously-editable design document could not later support a Threats to Validity argument that the protocol was fixed before data existed. Separating this document from the forthcoming experimental-evidence document preserves exactly that boundary, and explicitly refusing to mark O_{min} closed prematurely (D-067) is what makes this freeze an honest one rather than a convenient one.

25 Evidence and reference register

This register is intentionally a working bibliography/evidence list. Exact bibliographic metadata and final citation style will be normalized when the manuscript bibliography is built.

- RFC 5869 – HMAC-based Extract-and-Expand Key Derivation Function (HKDF). Normative source for absent-salt semantics and $L \leq 255 \cdot \text{HashLen}$.
- NIST SP 800-38D – Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. Normative source for GCM parameters, IV guidance, tag lengths, IV/tag coupling, and authenticated-decryption semantics. Note: under active revision (2024–2026) to remove sub-96-bit tags.

- Web Cryptography API Level 2 (W3C Editor’s Draft, w3c.github.io/webcrypto) – normative source, confirmed against algorithm-steps text, for: AES-GCM `encrypt/decrypt` output as `ciphertext||tag` with tag as the final `tagLength` bits; `additionalData` absent-equals-empty semantics. Cross-corroborated by W3C Bug 22570 (*RESOLVED FIXED*). Must still be tied to a concrete browser/runtime version for the empirical implementation phase.
- Crypto++ documentation and source ([weidai11/cryptopp](https://github.com/weidai11/cryptopp), official GitHub) for HKDF, AES-GCM, RSA-OAEP/PSS and serialization-related APIs. AES-GCM confirmed against the official Wiki (output format default, wiki-recommended tag set) and directly against source: `gcm.h` (`IVSize()`=12, `MinIVLength()`=1, and `MaxIVLength()`=`UINT_MAX`); `cryptlib.cpp`’s `HashTransformation::ThrowIfInvalidTruncatedSize` (only constraint is $t \leq 16$ bytes – no GCM-specific lower bound enforced generically, a materially wider capability than the Wiki’s recommended $\{32, \dots, 128\}$ set); the distinct `InvalidKeyLength/HashVerificationFailed` exception types. AAD absent-vs-empty equivalence is confirmed architecturally from the source-level channel-routing design: AAD is supplied through `AAD_CHANNEL`, while absence of AAD results in no bytes being routed to that channel, making it equivalent to an explicitly empty AAD sequence for the authenticated-data computation.
- Bouncy Castle documentation and source (`bc-java`, `GCMBlockCipher.java`, main branch) for HKDF, GCM, RSA and key import/export. AES-GCM confirmed directly against source: output format (`C||T`); `InvalidCipherTextException` (“mac check in GCM failed” / “data too short”); `OutputLengthException`; nonce length ≥ 1 byte with an optimized 12-byte path; AAD `null`→empty-array normalization; tag-length validation $32 \leq t \leq 128$, $t \bmod 8 = 0$ (wider than WebCrypto’s discrete set – a genuine capability divergence). Historical versions/forks of `GCMBlockCipher.init()` have enforced different minimum tag-length floors (32, 64, and 96 bits observed across different releases/forks), reinforcing the mandatory version-pinning requirement. See also CVE-2026-8149 (GCM chunking bad-tag issue, BC-LTS 2.73.0–2.73.10), cited as a threats-to-validity data point, not as an in-scope mutation.
- WasmChecker – semantic preservation neighbour, closest to our gap and requiring explicit differentiation.
- Cryptofuzz / CryptoFuzz++ / CLFuzz – differential cryptographic testing neighbours.
- EverCrypt / Project Everest – verified-provider contrast.
- FHE backend/scheme mutation work (“HERTA”) – cross-backend metamorphic-testing precedent in a different cryptographic domain.
- Rameshan & Messmer (2026), Intent-Based Cryptographic API Design for Cryptographic Agility, and companion Assessment Framework paper – provider abstraction/agility neighbour.
- 2026 ACVP/ML-KEM cross-implementation conformance work – neighbouring work on capability boundaries and conformance.
- X.509 post-quantum linting work – isolated encode/decode-identity conformance rule, primitive-level.
- Google `package:webcrypto` (Dart/Flutter) – production cross-backend compatibility testing, documented discrepancies; best real-world motivating evidence.
- RustCrypto/KDFs issue documenting absent-vs-empty HKDF salt confusion – real-world bug evidence.
- dint-dev/cryptography issue documenting the same HKDF salt class and cross-checking against Botan/CryptoKit – independent real-world evidence.

26 Explicit non-claims / out of scope

- No verification or audit claim about the real Iberbox SDK.
- No claim of full security equivalence between providers.
- No browser-engine isolation or memory-safety equivalence analysis.
- No side-channel equivalence analysis.
- No evaluation of provider-internal CSPRNG quality or nonce-generation policy.
- No claim that seeded mutations exhaust real-world divergence classes.
- No assumption that WebCrypto is one abstract backend independent of browser/runtime version.

27 Open items – pick up here next session

1. **[Closed]** Verify WebCrypto AES-GCM output format against primary spec text. Confirmed: $C||T$, tag as final `tagLength` bits, `additionalData` absent-equals-empty. See the AES-GCM normative-grounding paragraph above and the evidence register.
2. **[Pending empirical execution]** Evaluate the necessity of R_{err} under the frozen AES-GCM SDK error model – not a design debt, but an instance of the same execute-before-conclude discipline that governs O_{min} itself (D-067). Backend error behaviour has been verified across all three providers: Crypto++ distinguishes `InvalidKeyLength` from `HashVerificationFailed`; Bouncy Castle distinguishes authentication failure, malformed artifact, and output-length problems; and WebCrypto exposes a coarser `OperationError`. The provider-exception-to-SDK-error-class normalization rules and their classification order are already frozen in $C_{\text{pre}}^{\text{GCM}}$ (D-024). Remaining: execute the pre-registered necessity analysis (D-066/D-069’s corrected `DetectionGain/DiagnosticGain`) to determine whether R_{err} provides independent diagnostic value.
3. **[Closed]** Freeze $C_{\text{pre}}^{\text{GCM}}$ and construct Γ_0^{GCM} : profile frozen (D-024), `gcm.cap` split into `gcm.cap.provider/gcm.cap.portable` (D-025), unsupported-vs-invalid_parameter disambiguation rule frozen (D-025), 12-class Γ_0^{GCM} pre-registered and final circularity/redundancy audit completed (D-026).
4. **[Closed, v0.5.3]** WebCrypto-side normative inventory for RSA Key Serialization admission/object semantics: `structure/OID/DER` (v0.5.2), `extractable`, `keyUsages` (three rejection layers), RSA mathematical-consistency validation (confirmed not normatively mandated, four converging lines of evidence), two-prime/multi-prime (JWK explicit support vs. PKCS#8 structural allowance with documented design intent), `AlgorithmIdentifier.parameters` on import (confirmed not inspected, by contrast with X25519/ML-KEM in the same specification family). Admission-time vs. use-time validation, once an open WebCrypto-side item here, was subsequently resolved transversally as D-054 (§19.5); RSA Key Serialization’s full normative/contractual construction (D-053–D-058) is now complete, pending only the accumulated-audit checkpoint before EC P-256 begins.
5. **[Closed, v0.5.4]** Crypto++ RSA Key Serialization backend inventory: container surface (`Save/Load`), `OID`, `AlgorithmIdentifier.parameters` on import (diverges from WebCrypto: absent tolerated, present must be `NULL`) and export (always `NULL`, confirmed unconditional), `Load` vs. `Validate` separation (triple-sourced), trailing-data non-enforcement at the outer

buffer (diverges from WebCrypto’s `exactData`), two-prime-only internal representation (multi-prime excluded from $P_{RSA-ser}^{common}$ by intersection, not prohibition), observable error surface (`BERDecodeErr` for structural failure, silence for semantic/completeness failure), and round-trip/canonicalization (output rule confirmed by code; byte-stability architecturally plausible, not executed – no build environment available this session).

6. **[Closed, v0.5.5]** Bouncy Castle RSA Key Serialization backend inventory: lightweight vs. JCA surface distinction, export (`rsaEncryption+NULL`, converges with WebCrypto/Crypto++), OID domain (confirmed asymmetric across public-lightweight/JCA/private-lightweight, not assumed symmetric – three vs. four OIDs), `parameters` on import (not inspected for any confirmed OID, including `id-RSASSA-PSS`’s own structured parameters), trailing-data admission (asymmetric by `byte[]/InputStream` overload, not just by backend), admission-time validation (confirmed partial – automatic modulus security check, no cross-field CRT consistency check), two-prime/multi-prime (parse-but-collapse: ASN.1 layer can hold `otherPrimeInfos`, materialization layer discards it – a third category distinct from WebCrypto’s structural allowance and Crypto++’s inability to parse), and observable error surface (heterogeneous: checked `IOException` for dispatch failures, unchecked `IllegalArgumentException` for both security validation and malformed-structure failures, with a confirmed API-design hazard where the unchecked exceptions escape the method’s declared `throws IOException`).
7. **[Closed, v0.5.6]** Mini-audit of three previously-compressed BC items: lightweight-vs-JCA contrast (confirmed as architecturally distinct – a third OID-dispatch mechanism, `BouncyCastleProvider.getPu` identified and left as explicit remaining scope rather than silent gap), round-trip/canonicalization (closed by analogy to confirmed export behavior), internal ASN.1 consumption Layers 1/2 (retained explicitly as low priority, not dropped). Going forward, the working checklist for this research line is kept at the granularity the person specified, independent of how compressed any given `.tex` Open-items paragraph is – macro-items in the `.tex` do not imply sub-item loss from the actual plan.
8. Three-backend intersection: derive $P_{RSA-ser}^{common}$ and $P_{RSA-ser}^{portable}$ from the now-closed WebCrypto/Crypto++/Bouncy Castle inventories; resolve the admission-time-vs-use-time distinction as either a stable framework dimension or an empirical annotation, using the three now-contrasting behaviors (WebCrypto: none; Crypto++: manual/separate; BC: automatic/partial) as the deciding evidence; build $C_{pre}^{RSA-ser}$ and normalized SDK error categories from the three now-closed error-surface inventories.
9. **[Closed, D-057/D-058/D-062/D-063]** Build clause tables and Γ_0 for RSA Key Serialization and EC Key Serialization, following the same pattern as HKDF/AES-GCM/RSA-OAEP/RSA-PSS (normative check $\rightarrow$ clauses $\rightarrow \Gamma_0$, splitting causal regimes where needed). RSA-ser: thirteen clauses (D-057), seventeen causal classes and $\Gamma_0^{RSA-ser}$ (D-058). EC-ser: thirteen clauses (D-062), fourteen causal classes and Γ_0^{EC-ser} (D-063).
10. **[Closed, D-065]** Recompute $\text{Coverage}_\kappa/\text{Marginal}_\kappa$ across all six operations once all Γ_0 tables exist. Computed mechanically against every operation’s frozen clause table; $O_{\text{fixed}} = \{\text{AES-GCM, RSA-PSS, EC-ser}\}$ structurally necessary, $O_{\text{pending}} = \{\text{HKDF, RSA-OAEP, RSA-ser}\}$ deferred to stages 3–4 (D-066/D-069, corrected). O_{min} itself remains pending empirical execution (D-067) – this item closes the Coverage/Marginal computation, not O_{min} ’s final composition.
11. **[Closed, D-068]** HKDF’s $L > 0$ lower bound frozen explicitly as a C_{pre} portable-profile design decision (RFC 5869 leaves it open): $1 \leq L \leq 8160$ octets for HKDF-SHA-256, with $\Gamma_0(\text{HKDF-LENGTH-BOUNDARY-CROSSING})$ corrected accordingly.
12. Journal selection – deferred; revisit once the design and realistic implementation effort are fully scoped.

13. Once $O_{\min}$ is closed, begin drafting manuscript sections that are already stable (Related Work positioning, Formal Model) while implementation/data collection proceeds, per the two-layer workflow agreed for this project.

28 Preexperimental design freeze

This document, Version 0.6, is hereby frozen as the preexperimental research design and evidence base. Frozen means: the scientific question, the formal framework (six relations, K_1/K_2 taxonomy, adapter-vs-provider threat model, common/portable profile discipline), the six operations’ normative evidence and contracts (C_{pre}), their clause tables and pre-registered Γ_0 s, the RSA-ser $\leftrightarrow$ EC-ser transversal audit (D-064), the Coverage/Marginal computation (D-065), the detection/diagnostic-gain protocol (D-066/D-069), the $O_{\text{fixed}}/O_{\text{pending}}$ decision boundary (D-067), and HKDF’s closed contractual debt (D-068) do not change after this point except through an explicitly logged post-freeze amendment (below) – not through silent editing.

Explicitly, not implicitly, left open.

$O_{\min}$ is *pending empirical execution* : $\{\text{AES-GCM, RSA-PSS, EC-ser}\} \subseteq O_{\min} \subseteq \{\text{HKDF, AES-GCM, RSA-C}$

resolved only once $\text{DetectionGain}(o)$ and $\text{DiagnosticGain}(o)$ are computed from executed mutations for $o \in \{\text{HKDF, RSA-OAEP, RSA-ser}\}$ (D-066/D-069). R_{err} ’s necessity under the AES-GCM error model (§23, Open items) is likewise pending the same empirical machinery. Neither is a design debt; both are the intended, honest output of a pre-registered design at this stage (D-067).

Post-freeze amendment policy. If implementation or execution reveals that a frozen obligation is technically unobservable on some backend, or otherwise cannot be realized as specified, **this document is not silently edited**. The deviation is recorded in the successor experimental-evidence document (§28) as an explicit amendment: what the frozen protocol specified, what occurred during implementation, why, and what experimental treatment was applied as a consequence. This is deliberate scientific practice, not a workaround: it preserves this document’s value as a record of what was decided *before* any data existed, and it becomes direct evidence for the eventual manuscript’s Threats to Validity discussion.

Relationship to the next document. Implementation, adapters, the mutation harness, execution logs, raw-results provenance, and the empirical resolution of $O_{\min}$ belong to a separate, subsequent document – provisionally *Paper 4 – Experimental Evidence Base*, versioned independently from this one (e.g. starting at its own v0.1), and explicitly referencing this document’s frozen state (version, date, and repository tag/hash once established) as its baseline. This document does not grow further to absorb that material.

Cumulative audit trail into this freeze. Two independent audit passes (conceptual chain-of-decisions and physical document re-check) converged on the same two substantive findings and jointly surfaced two additional stale-text items, all corrected in D-069 before this freeze – not after. No cryptographic contract, no operation’s C_{pre} /clauses/ Γ_0 , and no D-059–D-065 decision was reopened by either pass. The preexperimental design audit result standing at freeze time is **PASS**.

Frozen as D-070, the final entry of this document’s Decision Log: *Version 0.6 is the preexperimental design freeze. $O_{\min}$ and R_{err} ’s necessity are explicitly pending empirical execution, not open design debt. Post-freeze changes to any frozen decision require a logged amendment in the successor experimental-evidence document, never a silent edit to this one.*